Kinematics Puzzle Game

Table of Contents

Analysis 6
Introduction to the Problem 6
Justification of Amenability by Computational Methods 6
Stakeholders 7
Stakeholder 1: Nikodem Grabowski 7
Stakeholder 2: Mr McLennan 8
Persona: Beginner Student 9
Researching Existing Solutions 9
Existing Solution 1: SimPop 9
Existing Solution 2: GeoGebra 13
Existing Solution 3: PhET Simulations 15
Stakeholder Additional Input 19
Features of Proposed Solution 19
Limitations of the Proposed Solution 26
Hardware and Software Requirements Specifications 28
Success Criteria 28
Design and Development 34
Basic Project Design 34
Decomposition Chart 34
Justification 37
Data Structures 39
Object Oriented Programming (OOP) 40
Testing 40
Prototype 1 – Projectile Sandbox 43
Class Diagrams 44
Database 45
Algorithms 47
Usability Features 50
Testing 52
Development 53
Testing 66
Remedial Action 69
Validation 69
Review 69
Prototype 2 – Torque Sandbox 70
Class Diagrams 70

Algorithms 71
Usability Features 74
Key Variables 74
Testing 75
Development 76
Testing 94
Remedial Action 96
Validation 96
Review 96
Prototype 3 –Puzzles 96
Class Diagrams: 97
Database 98
Algorithms 99
Usability Features 102
Key Variables 102
Testing 103
Development 104
Testing 138
Remedial Action 142
Validation 142
Review 142
Prototype 4 – Leaderboard 143
Class Diagrams 143
Database 143
Key Algorithms 144
Usability Features 144
Key Variables 145
Testing 145
Development 145
Testing 148
Remedial Action 150
Validation 150
Review 150
Stakeholder Analysis 150
Remedial Action 152
Prototype 5 – Usability Features 154
Show Answers 154
Font and Sizing 154

Colours 154
Graph Customisability 154
Account Settings 155
Key Variables 155
Testing 155
Development 156
Testing 164
Validation 167
Remedial Action 168
Review 168
Prototype 6 – Levels 169
Class Diagram 169
Database 170
Algorithms 171
Usability Features 173
Key Variables 173
Development 173
Validation 181
Review 181
Prototype 7 – Save Feature 181
Database 182
Algorithms 183
Key Variables 184
Validation 184
Testing 184
Development 185
Testing 190
Review 192
Testing & Evaluation 192
Function Testing 192
Usability Features 212
Feature 1: Light and Dark Mode 212
Feature 2: Music 212
Feature 3: Leaderboard Colours and Sections 212
Feature 4: Reset Account 213
Feature 5: Change username and password 214
Feature 6: Save and Load Setups 214
Feature 7: Graph Themes 214

Feature 8: Graph Thickness 215
Feature 9: Graph 215
Feature 10: Menu 215
Feature 11: Font 216
Feature 12: Navigation 216
Feature 13: Tutorial 217
Feature 14: Attractive UI 218
Feature 15: Notes 218
Overall Score 219
Robustness 219
Success Criteria Evaluation 224
Addressing Partially and Unmet Criteria 234
Usability Features and Robustness 234
Success Criteria 235
Stakeholder Feedback 238
Maintenance, Limitations & Future Improvements 239
Future Maintenance 239
Limitations 242
Future Development 244
Conclusion 244
Bibliography 245

Analysis

Introduction to the Problem

Mechanics is a core component of both the GCSE / A Level Physics and A Level Maths curriculum; however, many students find it difficult to understand through traditional teaching methods such as diagrams. This can limit students understanding as they don't allow the student to interact with the system. This is why I plan to make a kinematics puzzle game to help students learn mechanics in an engaging manner. I aim to create a program that will allow the students to apply their understanding and thus gain a deeper insight into mechanics principles. In order to ensure this the program will feature two main types of puzzles: balance puzzles and projectile motion puzzles which will be procedurally generated. In the balance puzzles, the player must achieve equilibrium by placing weights on a beam, which will allow the player to gain a deeper understanding of moments. In the projectile motion puzzles, the player must launch a projectile at a target adjusting certain parameters such as launch angle, velocity. Each puzzle will have randomized target placement, obstacles and adjustable environmental factors e.g. gravity. To enhance the educational value, the game will also include an AI solver to demonstrate the optimal projectile solutions as well as a leaderboard system to store and compare user performance to introduce an element of gamification

Justification of Amenability by Computational Methods

Feature	Justification
Real Time Calculations	▪ This program uses many physics calculations such as SUVAT equations and M = F x D, that will be inefficient to do by hand. ▪ Due to the extensive number of calculations necessary to accurately animate the motion of the object and to present the results instantaneously, it would be impractical, if not impossible, for a human to perform these computations manually within a reasonable timeframe. ▪ A modern computer can execute these calculations quickly with precision, allowing for real-time processing and continuous animation. This efficiency not only ensures accuracy but also enables the system to respond dynamically to changing inputs, thereby enhancing the overall user experience and functionality of the program. ▪ Since the results appear immediately, users can experiment and change parameters, seeing the effects right away. Without this capability, users would have to wait for slow, manual calculations or static results, which would reduce engagement and limit experimentation
UI Interactivity	▪ A computer can instantly respond to user inputs like dragging and clicking which without a computer would take more time to display this information ▪ Visual feedback like colour changes, motion animation makes it easier for the user to understand what changes that they are making

	▪ By using a computer-based solution, it allows for the abstraction of other elements such as air resistance to make it easier for the user to understand and be able to use the key concepts of mechanics, something that would not be possible with a physical demonstration.
Storing User Data	▪ A computer can track and store large amounts of data across multiple users which would be inefficient to do by hand as sorting and displaying leaderboards requires frequent updates ▪ Allows persistent storage via CSV files for data lookup. Doing this on paper would require manual lookup which is slower and less efficient ▪ Statistical calculations can be performed quickly and accurately compared to doing it without automation ▪ By using a computer-based solution the user does not have to manage their own records as the program will save the data automatically.
AI Solver	▪ A computer can try a vast number of different inputs in a very short amount of time to find a feasible solution, something that would take a human many hours to do via trial and error ▪ The Solver can generate hints and solutions to help the user learn where they went wrong – something no worksheet or example can do
Procedural Puzzle Generation	▪ Without a computer generating puzzles, designing many puzzles would be incredibly inefficient as there are many constraints to meet to ensure that the puzzle is solvable that a human would struggle to check all the constraints ▪ It can incorporate randomness with solvability to ensure educational value ▪ Automated puzzle generation ensures users always have new puzzles to solve without requiring a developer to manually create each one.
Undo / Redo Functionality	▪ A computer can store previous states using stack data structures which would be difficult to track manually ▪ By using a computer program with undo / redo functionality, players can revert to a previous state if they make a mistake without resetting the entire puzzle

Stakeholders

Stakeholder 1: Nikodem Grabowski

This stakeholder is a student at Bournemouth School preparing for their A Levels in Maths and Physics. He understands the content but struggles to stay focused in traditional lessons. He prefers learning through trial

and error and is more engaged when the experience feels fun and interactive rather than simply answering textbook questions. He has not tried using interactive tools or simulations to aid his revision.

How they will use the solution

He will use it as a revision tool to reinforce what he has learnt during class by attempting puzzles, which will check his understanding and reinforce concepts in a more hands-on way. By using the solution, Nikodem will gain greater problem-solving skills, which will help him during his exams. The visual animations will help him check whether his calculations are correct, and if not, how far off they were, giving him a clearer sense of where he went wrong.

How it is appropriate to their needs

Since Nik enjoys gaming, the interactive and game-like elements of the solutions will appeal to him and make him more engaged, since the puzzles will give him a sense of progress and challenge. As he is a hands-on learner, the ability to test scenarios and get instant feedback as well as being able to 'play' with the physics will help gain a deeper understanding of the concepts.

Requirements

- Smooth, clear animations
- Customisable UI
- Personalised hints to help him finish the problem
- Engaging and Fun Puzzles
- Match puzzle complexity to the stage of learning
- Option to annotate and save puzzles so that they can be revisited
- Scoring system with leaderboards
- Use of drag and drop rather than buttons

Stakeholder 2: Mr McLennan

This stakeholder is a teacher of Physics at Bournemouth School. He teaches both GCSE and A Level classes the mechanics section of the Physics course and is looking for ways to help students gain a deeper understanding so that they are more confident during exams

How they will use the solution
He will use the program as a classroom resource to demonstrate key kinematic principles in a visual, interactive format. It would be particularly helpful when introducing new concepts like SUVAT equations as he can use the solution to generate puzzles that he can give to his students to practice. He may also direct students to use the solution independently to help with their revision

How it is appropriate to their needs

The solution will support his teachings by helping students visualise the abstract ideas that he is explaining without him having to draw many diagrams manually on the board, since students often learn better through seeing and interacting, the solution will encourage more engagement during lessons

Requirements

- Teacher dashboard for tracking student performance
- Graph generation of the puzzles
- Ability to change animation speed
- Ability to input custom scenarios / experiment in a sandbox environment

- Able to run on school computers
- Ability to display underlying maths

Persona: Beginner Student

This stakeholder is a student preparing for their GCSEs. They have been introduced to basic mechanics topics, but they struggle to understand what is happening from textbook diagrams. They prefer learning through visual interactive means. They have used other simulation programs before but find them too abstract.

How they will they use the solution

The student will use the program to explore the kinematic concepts in a hands-on way by experimenting and learning through trial and error to gain an intuitive understanding of the physics. They will use the solution to learn from their mistakes, and They may also use the program to revise before exams, as it allows them to revisit tricky topics in a more hands-on, visual way than standard revision methods.

How it is appropriate to their needs

As they prefer to learn visually, the interactive puzzles allow them to learn by doing, which is much more effective than reading or watching. In addition, real-time feedback helps them quickly see what works and what doesn't, reinforcing concepts and highlighting mistakes. As well as that, undo/redo options allow the user to explore the physics and allow them to gain a deeper understanding

Requirements (Excluding Previous)

- Simple and clear tutorials that explain mechanics principles
- Instant visual feedback for actions and results
- Non-competitive practice mode
- Undo/redo system for experimentation
- A large bank of questions so that there is always a new question to complete

Researching Existing Solutions

Existing Solution 1: SimPop

(SimPop, 2018)

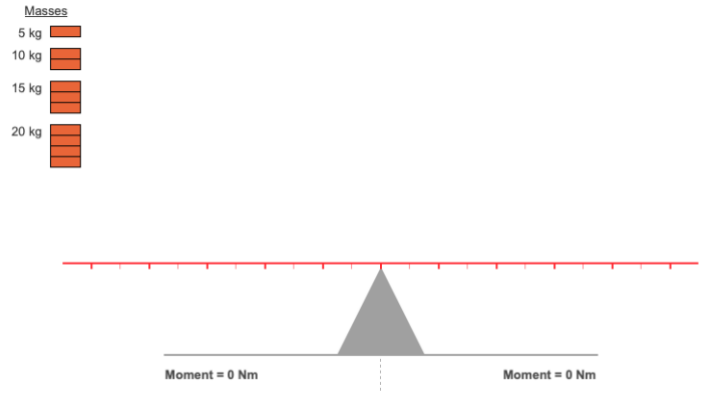


Figure 1. 1

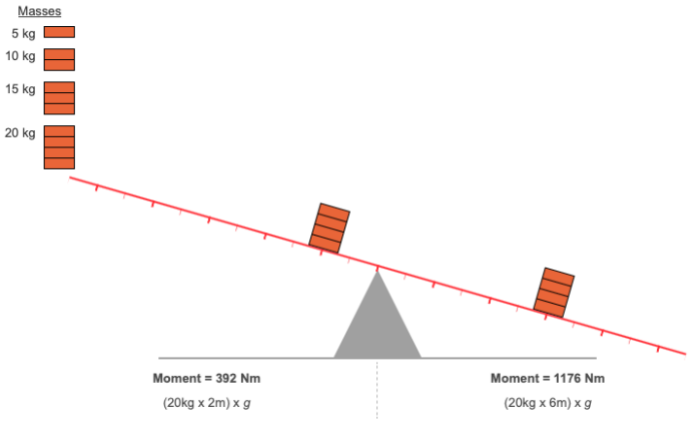


Figure 1. 2

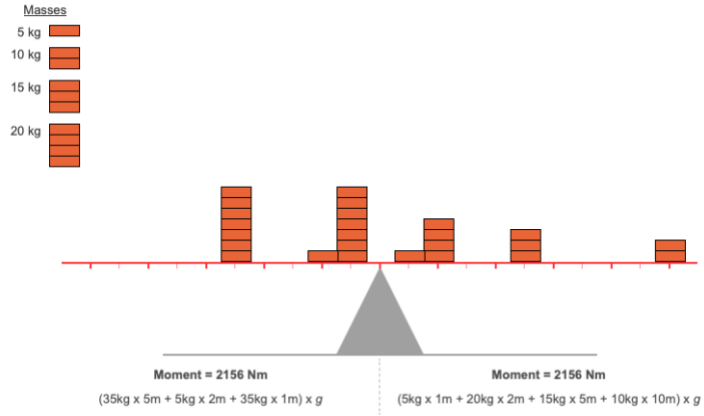


Figure 1. 3

This is a simple simulation of the principle of torque that allows the user to place masses of different weights on a beam, which has a pivot point in the centre. It is designed to be both educational and interactive, allowing users to explore the concept of rotational equilibrium in a hands-on, visual manner. The simulation allows you to place these masses in certain intervals along the beam using a drag-and-drop interface, where you can drag the masses from the side and place them on the beam. Once you have placed a mass it will then show you how that will impact the equilibrium of the beam as well as how much it will rotate as seen in *Figure 1.2*.

The solution allows you to stack masses on top of each other which can be seen in *Figure 1.3*. This allows you to have a large number of possible combinations to arrange the masses and thus allows a greater degree of freedom to experiment without being constrained by the fixed masses. The simulation will allow you to move the blocks of mass that you have placed on the beam, however you will not be able to remove it from the beam which can cause some problems for the user if that is not the mass they intended to place. Additionally, once the masses have been stacked on top of each other, they will no longer be able to be separated unless you restart the program and start from scratch.

This simulation uses fluid animations and is very responsive to changes made by the user making it an enjoyable experience for the user. The simulation will show you how the beams position is changing rather than just displaying what will happen to the beam. The developers have also animated the rotation of the masses and not just the rotation of the beam, resulting in a more immersive and intuitive user experience.

This simulation also shows you both the clockwise and anticlockwise moment as well as the calculation it is performing as seen in *Figures 1.1 – 1.3*. These values are calculated each time a block is placed or moved on the beam, and they are calculated using the fundamental moments formula which is Moment = Force * Distance from the pivot. This allows users to see both what is happening visually but also why it is happening mathematically.

Features of this solution	Approach for my solution	Justification
Animated Movement of Beam	I will implement smooth, real-time animation that visually rotate the beam based on the moment calculation	▪ Animations that show how the torque force is causing the beam to rotate will make the physics clear and engaging as it will help users visualise what is happening ▪ Animation and a responsive UI will keep the experience more immersive and intuitive which will encourage more experimentation with the solution
Drag and Drop Interface to place masses	My solution will have a drag and drop system for users to place masses along set intervals on the beam	▪ Drag and drop makes the simulation interactive and easier for users to use as users can use the solution without having to use complex commands or menus which will reduce the learning curve
Display Clockwise and Anticlockwise moments	I will display the calculations for both clockwise and anticlockwise moments and update them each	▪ By displaying the underlying maths, it will help students using the solution to gain a better

	time the forces on the beam change	understanding of the principle of moments which will allow them to gain a deeper understanding that will aid them during formal schooling ▪ By displaying both the clockwise and anticlockwise moments, the user can see the opposing forces and understand what makes the beam stable and unstable
Stacking Masses	I will be implementing this however with certain constraints as my solution will be presenting this as a puzzle rather than a sandbox simulation	▪ By allowing users to stack masses it will increase the number of possible setups making the simulation more versatile
Ability to choose from different masses	I will be implementing this, however unlike this existing solution there will not be infinite masses to select from	▪ Allows users to compare how light and heavy masses affect the torque force which mirrors real physics practicals thus strengthening the educational value of the simulation
Vector diagram	This feature will be implemented if I have time	▪ The primary aim of this solution is to help users understand projectile motion which can be achieved without vector diagrams ▪ The key features are already displayed numerically which means that the vector diagrams are not necessary

Existing Solution 2: GeoGebra

(GeoGebra, n.d.)

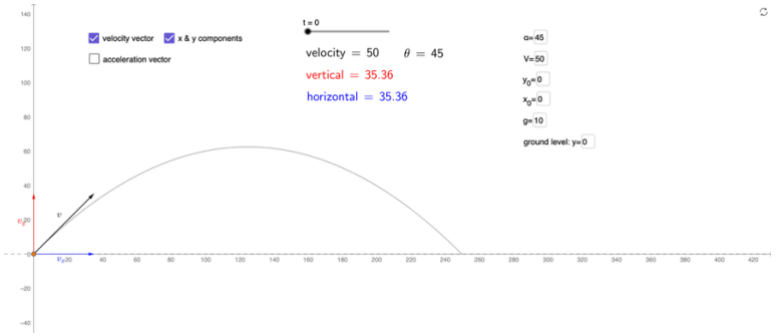


Figure 2. 1

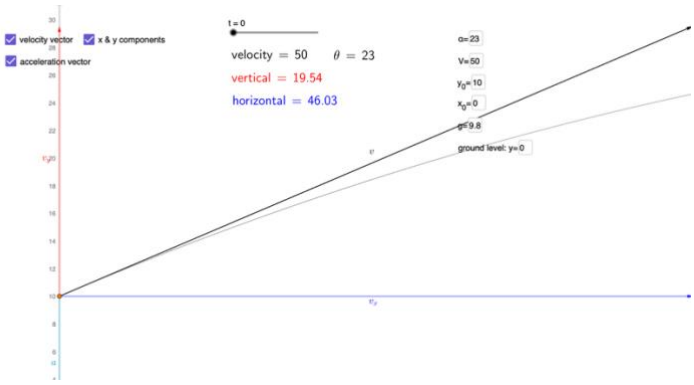


Figure 2. 2

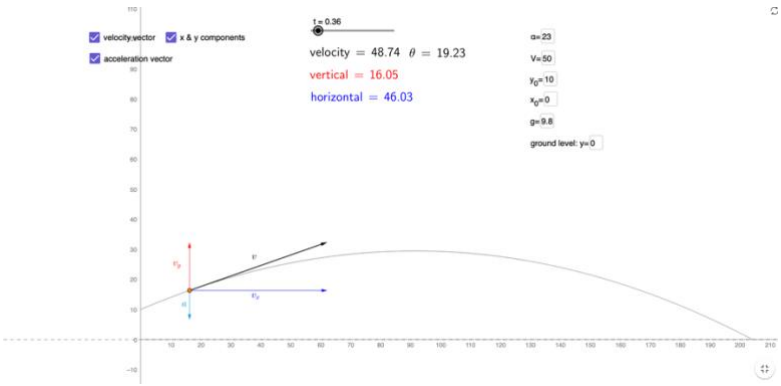


Figure 2. 3

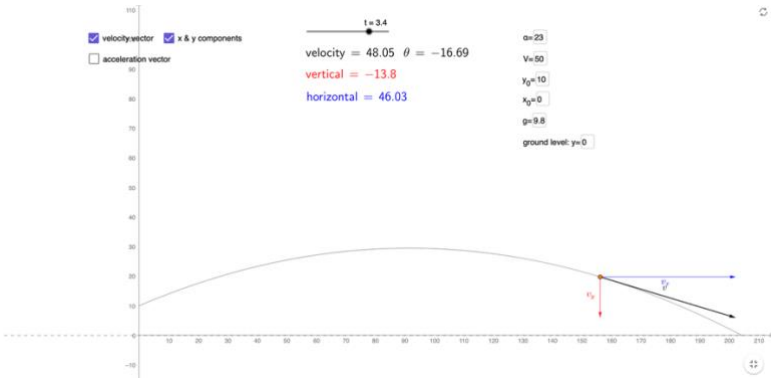


Figure 2. 4

This solution provides a graph of the displacement against time for projectile motion offering a clear representation of its motion dynamics. It allows you to select your desired setup through the textboxes

displayed on the top right of the screen, as seen in *Figures 2.1 – 2.4* allowing them to customize the simulation to their requirements. This solution will then produce a graph of the particles displacement and using a slider you can see how the forces interacting on the particle change over time and its corresponding place on the graph with key values displayed above. This solution also allows you to zoom in and out of the graph as seen in *Figure 2.2* so that users can see specific sections of the graph in greater detail. It also allows for users to toggle which vector diagrams they would like to see allowing them to tailor the visual information to their needs and deepen their understanding.

Features of this solution	Approach for my solution	Justification
Displaying graph of projectile motion	I will include this	▪ Many students find the displacement-time graphs for projectile motion difficult to understand so by including it alongside the visualisation it will increase the educational value of the solution
Vector diagram	This feature will be implemented if I have time	▪ The primary aim of this solution is to help users understand projectile motion which can be achieved without vector diagrams ▪ The key features are already displayed numerically which means that the vector diagrams are not necessary
Showing particle position on graph	I will not be implementing this	▪ Adding a moving particle to the graph would require recalculating the animation to the graph's scaling which adds significant development complexity for limited benefit ▪ Due to time constraints, it would be more effective to prioritise features that have a greater impact on the solution
Zoom Functionality	I will be implementing this	▪ It will allow users can see specific sections of the graph in greater detail which will increase the educational value as they can use the graph zoom

		functionality to find key values from the graph
Customisable Setup	I will be implementing this	▪ Justified in Existing Solution 1

Existing Solution 3: PhET Simulations

(PhET Interactive Simulations, n.d.)

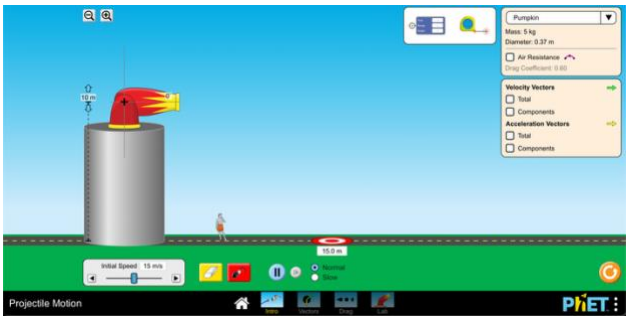


Figure 3. 1



Figure 3. 2

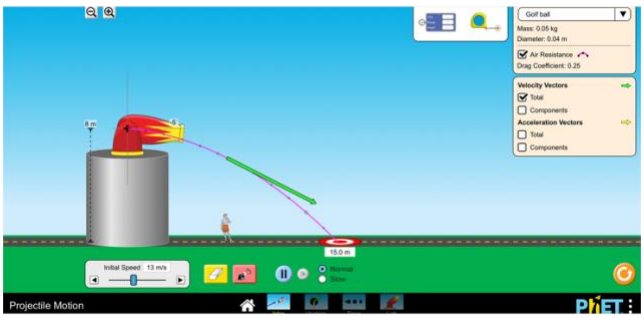


Figure 3. 3

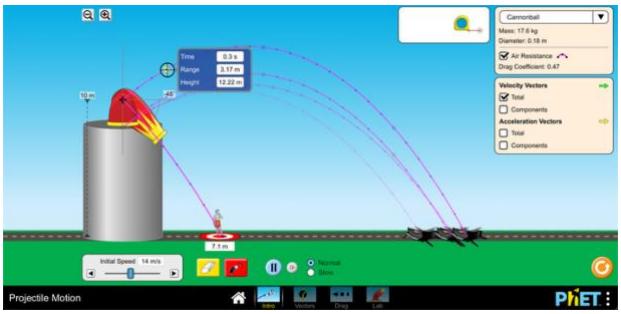


Figure 3. 4

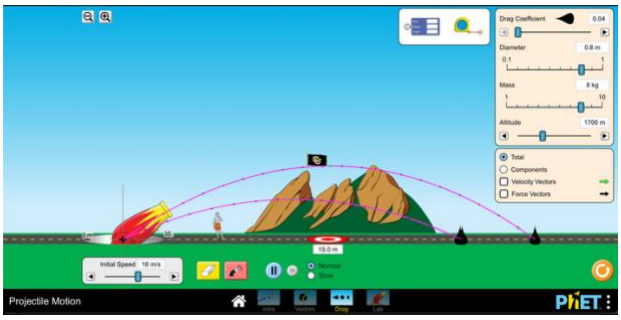


Figure 3. 5

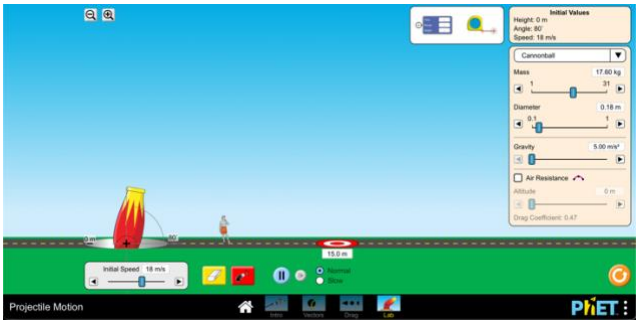


Figure 3. 6

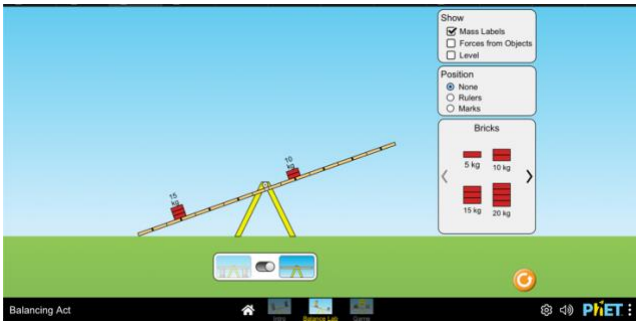


Figure 3. 7

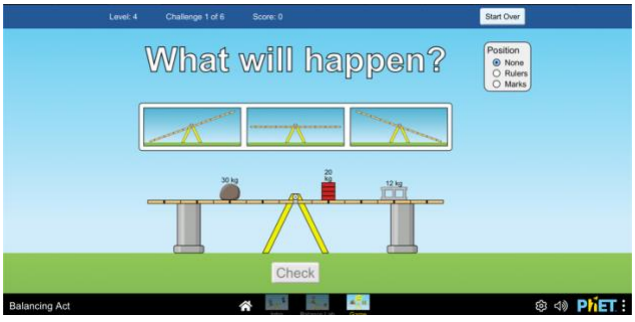


Figure 3. 8

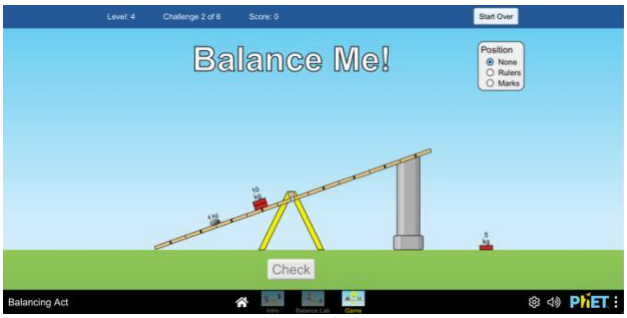


Figure 3. 9



Figure 3. 10

This solution has multiple different parts to it, a projectile motion game / simulation where you can set up the environment to your requirements. It allows you to change the angle and height of release *(Figure 3.1, Figure 3.2*), as well as the object that you are releasing *(Figure 3.3)*. Alongside that, the simulation also allows you to select whether you want to see the effects of air resistance, gravitational forces and drag *(Figures 3.4 – 3.6)*.

Each part of the simulation is completely customisable, as it allows you to choose from a set of projectiles to release which each have a different mass. It also allows you to adjust the target that you are trying to hit and when you hit it, the program will make a small animation to acknowledge it which makes the simulation feel more fun and more game-like *(Figure 3.2)*. Additionally, the use of abnormal projectiles such as human, piano and car increases engagement and enjoyment for the user *(Figure 3.3)*. However, for a student revising projectile motion, this simulation offers visual clarity but lacks worded examples and guided learning prompts.

This solution also allows the user to be able to choose their preferred playback speed of the simulation, 'Normal' or 'Slow', which will help tailor the simulation to the user's needs *(Figure 3.2)*. This solution uses a mixture of dragging elements as well as button clicks and sliders/dropdown menus, which can be a little unintuitive as the buttons are not clearly labelled *(Figure 3.1 – 3.10)* and some users may expect either everything to be drag-and-drop or everything to be sliders, buttons and dropdown menus.

The simulation does display several key values useful for calculation, such as speed or displacement, but it does not show all relevant forces or calculations — a limitation shared with Existing Solution 2. Unlike Existing Solution 1, it does not explicitly show the mathematical calculations or numerical values of opposing forces, which could limit its usefulness for deeper understanding.

In the balancing act simulation, it can be a bit difficult to work out how to play the simulation as the simulation requires you to switch between two different views *(Figure 3.7)*. It also includes some basic puzzles on the principle of moments which A Level Students are unlikely to find challenging but would be useful for GCSE students who are first learning about the principle of moments *(Figure 3.8 – Figure 3.9)*

This solution also includes a 'data lab' mode *(Figure 3.10)* which is more limited compared to the other modes as it only has 4 preset orientations rather than the full 180 degrees of movement given earlier (*Figure 3.1 –*

Figure 3.6). This mode allows you to project an object multiple times, with each time the environmental factors changing slightly and then plots a histogram of the displacement of the object.

Features of this solution	Approach for my solution	Justification
Customisable angle, height and velocity	I will include this	▪ Let's users explore how initial conditions affect trajectory ▪ Justified Customisability in Existing Solution 1
Air Resistance Toggle	I will include this	▪ Air Resistance will make calculations more difficult and is not required for either GCSE / A-Level, therefore by making it toggleable only those looking to extend themselves will need to turn it on ▪ Shows difference between ideal physics and real-world complexity ▪ Allows users to understand the effects of resistive forces on projectile motion
Multiple Projectile Types	I will be including this but only by considering it as a particle with varying masses	▪ Due to the complexity of animating many different objects and considering how air resistance will impact its motion, I will only be considering my projectile as a particle ▪ By allowing the user to change the mass, the user will be able to see the effect of mass when there is air resistance
Graphing Functionality	I will include this	▪ Justified in Existing Solution 2
Puzzles	I will include this	▪ My aim with my solution was to build a puzzle-based kinematics program to allow users to learn in a more engaging way ▪ Providing varied difficulty levels caters to both new

		students and advanced students

Stakeholder Additional Input

After presenting some of the features of my solution to my stakeholder, I asked them to identify any features that they felt should be included

Essential Feature	Justification
Ability to have multiple pivots	▪ Having multiple pivots will allow users to analyse and understand more complicated problems where they would have to understand what forces would cause the beam to topple for example
Ability to change the height of launch point for projectile puzzles	▪ Allows users to have a greater degree of freedom in what they can change in the simulation and thus gain a deeper understanding of projectile motion
Ability to see key information at certain points along the graph	▪ Allows the user to see key values such as velocity, acceleration and displacement at certain points along the graph. Being able to inspect key data points encourages learners to connect numerical values to graphs

Features of Proposed Solution

After researching similar existing solutions and obtaining the needs and requirements of my stakeholders, I have formed a table to show the essential features of my solution below.

Essential Feature	Description of Feature	Justification	Link to stakeholder (If Applicable)
User Interface and Smooth Animations	User interface that is pleasant for the user and is easy to use through the use of sliders, buttons and dropdown menus	▪ Having a user interface that is easy for the user to use and looks good will ensure a better user experience and will be	Nikodem had requested a pleasant User Interface and smooth animations.

		more likely to use the solution. ▪ By using sliders, buttons and dropdown menus, the data inputted from the user does not have to be validated and will not produce an error as all the possible inputs will be valid / in range. In addition, it will also result in a more lightweight and simpler UI. ▪ With a physics simulation, it is important to have a solution where the animations are clear so that the user can understand what the underlying physics in a simpler manner.	Mr McLennan has asked for software that can run on school computers which will be possible by having a more lightweight UI. The beginner student requested instant visual feedback which would only be possible with smooth animations.
Engaging Puzzles	Puzzles that are not too simple to solve or too difficult	▪ It will keep users engaged as it will prevent boredom or them finding the puzzle too hard and wanting to quit ▪ By making the puzzles engaging the solution will feel more game-like rather than revision tasks which will incentivise users to continue working on the problems ▪ The use of puzzles force users to apply their knowledge which strengthens their understanding of the topic ▪ Solving puzzles will provide motivation that will allow users to keep learning the mechanics topics without the need for constant external validation.	Nikodem requested Engaging and Fun Puzzles. In addition, he is also a keen gamer so by making the solution more game-like, he is more incentivised to use the solution which will assist him with his A-Levels.

Match Puzzle Complexity to the stage of learning of the user	The solution will assess the users' skills by collecting data such as how many attempts they required to solve a question of a certain level. Using this information, the next puzzle can be of a similar difficulty, easier or harder.	▪ This will allow the user to ensure that they are challenging themselves enough to make the greatest use of the educational value of this solution ▪ It will keep users engaged as it will prevent boredom or them finding the puzzle too hard and wanting to quit ▪ By introducing, levels of difficulty the solution will feel more structured and give the user an idea of what to expect rather than each question being a random difficulty which could reduce user engagement.	Nikodem requested to match puzzle complexity to the stage of learning**.** In addition, both Nikodem and the beginner student are using this solution to learn/revisit these topics for exams, so they would prefer a solution with greater structure
Leaderboard system	The number of questions a student solves along with the level of that question and number of attempts will be used to form a score for each user. That score will then be used to place the user on a local leaderboard if they opt into it which displays their name and score in a ranked order	▪ The use of a leaderboard will make the solution feel more game-like rather than revision tasks which will incentivise users to continue working on the problems as they would wish to beat their friends on the leaderboard and be able to compare stats. ▪ By making the leaderboard optional, it means that students who wish to not participate in the leaderboard will still be able to complete questions without having to compare themselves to other students as well. ▪ By giving each user a score, they can use it to compare themselves to others and will also allow stakeholders such as Mr McLennan to monitor how students are doing.	Nikodem requested that the solution includes a leaderboard system with scoring. Mr McLennan requested a teacher dashboard and whilst that isn't being fully implemented it is being implemented in part through the use of scores which he can use to monitor student progress The beginner student had requested that there to be a non-competitive practice mode so by ensuring that the leaderboard is

			optional the student is able to practice without his stats being visible to other users.
Graph Generation	The solution will display a graph of the motion of the projectile for projectile motion questions	▪ Many students find the displacement-time graphs for projectile motion difficult to understand so by including it alongside the visualisation it will increase the educational value of the solution ▪ By displaying the graph of motion, it will allow users to gain a deeper understanding into questions in GCSE / A-Level which are regarding the motion graphs as they will have visually seen what factors will cause the graph to change.	Mr McLennan requested graph generation for each of the puzzles
Customisation	This solution will allow users to customise the initial setup as well as giving the user customisability in terms of UI through the use of a settings menu	▪ Having a setting menu, makes it easier for the user to make all the changes to the UI that they want in one place rather than having to go to multiple different places ▪ By giving the user, a greater degree of control in terms of the initial setup of the simulation. By giving the user, a greater degree of control in terms of the initial setup of the simulation. It allows users to experiment more and promotes independent thinking. ▪ By making the solution customisable, it will cater for a larger range of	Nikodem requested customisable UI Mr McLennan requested ability to change animation speed

		users as they will all have slightly different requirements and needs. ▪ By allowing for more customisation, the solution becomes more interesting as there are many different combinations that you can try ▪ By allowing the user to customise the UI, they can focus on what they want to see. For example, A-Level students may want to see more of the calculations and graphs whereas GCSE students would rather just focus on the visual side of the solution	
Sandbox Mode	A part of the solution where the user can experiment without trying to solve puzzles.	▪ The puzzles are generated randomly and cannot give every possible combination thus if it was just a puzzle mode the user would not be able to see every possible combination, which they could in a sandbox mode. ▪ Having a sandbox mode allows users who wish to try certain permutations to experiment with those instead of waiting for it to appear as a puzzle. ▪ For teachers using the solution, they will be able to show certain examples to a class and modify it exactly to their needs.	Mr McLennan will benefit from this when he is teaching lessons on projectile motion and moments and he has also requested the ability to input custom scenarios / experiment in a sandbox environment The beginner student persona requested a non-competitive practice mode and by having a sandbox mode, will allow the user to practice and experiment with values of their own choice.
Tutorials	An interactive tutorial that walks through how to use	▪ An interactive tutorial will help a user to use the program as fast as	Nikodem and Mr McLennan has not used physics

	the solution that you can access at any point	possible as it will teach them all the basics rather than making them read through a help page	simulations before so it may help for him to have a tutorial to help them understand how to use the program.
Undo / Redo Functionality	Allows the user to go back to a previous state in their workings for a puzzle or during the sandbox mode	▪ Having undo/redo functionality will allow the user to not have to restart the program each time that they make a move or try to go back to the previous state manually. This will allow the users to experiment more freely.	Beginner Student requested undo / redo functionality
Puzzle Generation	Each puzzle will be procedurally generated depending on the user's level	▪ It allows the file size to be smaller as it does not have to store a large number of predefined puzzles which would also take significant development time ▪ Each puzzle will be suited to the right level of difficulty for each user making it more educationally enriching ▪ It increases 'replayability', as there is always a new puzzle to solve as you cannot exhaust the number of puzzles encouraging users to return. ▪ It supports personalisation as users can be given new puzzles based on their previous mistakes or performance. ▪ Allows stakeholders such as Mr McLennan to be able to use this as homework as each puzzle will be tailored to each student's needs and each student will	Helps Mr McLennan when teaching this topic set questions for his students to do Ensures that there are always more puzzles to solve for Stakeholder beginner player persona

		have to work independently on their puzzles as they will each be different.	
Drag and Drop Interface	Allows users to interact with objects by dragging and placing them using the mouse or touchscreen.	▪ Drag and drop makes the simulation interactive and easier for users to use as users can use the solution without having to use complex commands or menus which will reduce the learning curve ▪ Using drag and drop will lead to less input errors as users will not be misclicking buttons or accidently using the wrong keyboard shortcut	Nikodem requested a drag and drop interface
Display of calculations occurring	Shows the calculations for equations such as SUVAT and Moment = F * D as well as the values being used for the calculation	▪ By displaying the underlying maths, it will help students using the solution to gain a better understanding of the principle of moments which will allow them to gain a deeper understanding that will aid them during formal schooling ▪ By displaying both the clockwise and anticlockwise moments, the user can see the opposing forces and understand what makes the beam stable and unstable ▪ By displaying the key values for projectile motion the users will be able to calculate value and be able to see what causes maximum and minimum distance travelled in a more mathematical sense By displaying the key values	Mr McLennan requested for the calculations and values to be displayed

		for projectile motion, the users will be able to calculate value and be able to see what causes maximum and minimum distance travelled in a more mathematical sense	
Sound Effects	Sound effects that will play at certain points when you are using the solution	▪ The use of sound effects will enhance the aesthetic appeal of the game, and it will provide a more fun experience	Nikodem requested sound and music.
Save Feature	This feature will allow users to save setups that either they or the solution has generated with a unique name so that they can reload it into the solution in the future to look back over it.	▪ This feature is useful as it will allow the user to be able to save setups that they wish to look over in the future to aid their revision ▪ If a student wishes to receive help about a problem, they can save the setup to show the teacher later whilst still being able to complete other puzzles.	Nikodem requested a feature to allow users to save setups

Limitations of the Proposed Solution

These are the limitations of the solution, highlighting features seen in examples as well as features requested by stakeholders, but not used in my solution

Identified Limitation	Explanation and Justification
Programming Language / UI Framework used	I am using C# with Windows Forms which isn't the optimal UI Framework for this solution as it is not a game engine so rendering animations and precise physics will be inefficient which could be handled better by game engines such as Unity. In addition, Windows Forms is an old framework that hasn't kept up with modern UX / UI standards so the UI can feel slightly outdated. However, due to time constraints, I will use Windows Forms in my solution as it is a framework that I am familiar with. Using Windows Forms will also allow me to control each element directly which will help with building the solution. In addition, there is a large number of libraries and resources available for Windows Forms in .NET which means that I can integrate useful tools – such as graphing tools without having to build them from scratch.
Windows Only	Since I am using Windows Forms, my solution will only run natively on Windows making them inaccessible to students

	on. Linux, Chromebooks and MacOS, unless you use virtual machines. Although I could have rewritten the program for multiple OS, however this would have been rather time consuming. Additionally, I could have compiled the solution using .NET core, however that would require more embedded resources which may not be downloaded on every computer. Since most people, including all my stakeholders, have access to Windows devices, I have decided to leave my solution as Windows only.
Annotations	Some users would benefit from being able to visually annotate the simulation such as by drawing on forces and marking angles. This would help them reinforce concepts and make it easier for them to solve puzzles. However, implementing this functionality in Windows Forms would require a custom drawing UI designed over the simulation as there are no native tools. This would take a lot of development time and can introduce bugs. Thus, due to time and complexity constraints, I will not be introducing this feature. However, I will add a text box feature to allow the users to jot down their ideas instead.
No custom feedback	Custom feedback is useful to help the user understand why they didn't get a problem right, without feedback users may keep guessing without learning the underlying physics thus encouraging trial-and-error rather than giving the users a conceptual understanding of kinematics. However, providing specific feedback such as "your angle was too high" would demand either a pre-defined database of all the possible input combinations and the errors that they would produce which would be infeasible due to the near-infinite range of inputs. Alternatively, the solution would have to use machine learning / AI reasoning models to infer what the user did wrong which would require advanced knowledge of AI frameworks and probabilistic modelling
No online features	The core features of this solution do not require any online features. Whilst features like leaderboards and teacher dashboard would require an online element. As well as that for students who are working on different devices it will allow for their details to be saved to the cloud and then allow them to pick up from where they left off rather than starting from scratch on each new device. However, implementing online features would require me to learn how to implement the networking protocols required for this to work which would take more time and give me less time to focus on the core functionality.
Limited Features due to Time Constraints	Time constraints have also meant, along with the limitations shown above, more general features have also been removed to focus on features that the clients requested. This

	is because at the core of this solution there is no need to implement features such as advanced graphics and extensive tutorial modes.

Hardware and Software Requirements Specifications

Hardware / Software Element	Requirement for Solution	Justification
Operating System	Windows OS	This solution was built using the Windows Forms Framework which only runs natively on Windows
CPU	1.8 GHz clock speed with at least dual core	The solution will perform many calculations to accurately display what is occurring, so a minimum of 1.8Ghz clock speed will be required. In addition, if there are multiple cores then calculations can take place in parallel to optimise the solution
RAM	Minimum 4GB	4GB is the minimum RAM required for .NET desktop apps to run without freezing. This solution will use minimal RAM due to the limited visual complexity
GPU	Integrated Graphics	This solution uses limited visual complexity so it should be able to run on the most basic software
Hard Disk Storage	500 MB – 1 GB	The solution uses very little storage as only the .exe file and CSV files will have to be stored
Input Devices	Keyboard and Mouse	The keyboard will be needed for the user to enter information. A mouse will be required for the user to interact with the drag-and-drop interface as well as allowing them to select buttons
Sound	Headphones / Speaker (Optional)	This solution includes sound effects

Success Criteria

Feature		Identified Success Criteria	Justification for Success Criteria	Method of Measuring Success Criteria
1. Projectile Motion	1	Does the projectile launch when a button is pressed	The solution should progress as expected	GUI test
	2	Is the starting height of the launch point adjustable	The user should be able to change these values in	GUI test

	3	Is the angle of launch adjustable	order to solve the problems. They may not all be adjustable for each problem	GUI test
	4	Is the starting velocity of the projectile variable		GUI test
	5	Is the projectile mass variable for required questions		GUI test
	6	Is the target displayed with distances shown	Having distances shown allows the user to use the skills that they have learnt to solve the problems rather than just using trial and error.	GUI test
	7	Are the puzzles unique	The puzzles should be unique so that they can solve new problems without redoing the same problems	GUI test
	8	Do the puzzles stated difficulty match the actual difficulty	Ensures that the user is solving problems that are suitable for the level that they are at	Stakeholder questionnaire
	9	Is a graph shown when a button is pressed	The solution should progress as expected	GUI test
	10	Does the graph match the motion of the projectile	The graph should match the motion so that it can be used to assist the user	GUI test
	11	Are the key values on the graph labelled	Being able to inspect key data points encourages learners to connect motion to graphs	GUI test
	12	Is the motion of the projectile animated	Makes the solution more visually appealing as well as emphasising what is happening in the game	GUI test
	13	Does the simulation accurately show what would happen with the setup	Essential to ensure the simulation's accuracy as it serves as the user's tool for understanding the concepts.	GUI Test
	14	Does the program show you the calculations required to solve the problem	Helps the user understand why the simulation has produced the result it has	GUI Test
	15	Can the graph be displayed when the button is clicked	By displaying the graph of motion, it will allow users to gain a deeper understanding into questions in GCSE / A-	GUI Test

			Level which are regarding the motion graphs as they will have visually seen what factors will cause the graph to change.	
	16	Can you zoom into the graph	It will allow users can see specific sections of the graph in greater detail which will increase the educational value as they can use the graph zoom functionality to find key values from the graph	GUI Test
	17	Does the graph show the vector setup	Helps the user understand the vector formation as well as the graphs.	GUI Test
	18	Is the graph customisable	This allows the user to make the graph easier to read for themselves	
	19	Does the undo/redo button take you to the previous and following state respectively.	When users are experimenting, they can undo moves without restarting the entire solution which is especially useful for puzzles that require trial and error	GUI test
	20	Is the correct answer displayed if the user requires it	Gives the user feedback that allows them to gain understanding on where they went wrong	GUI Test
2. Torque Problems	1	Does the solution animate the result of placing masses / moving the pivot point	Makes the solution more visually appealing as well as emphasising what is happening in the game	GUI test
	2	Are there puzzles with multiple pivot points	Allows the user to apply their knowledge to more complex systems that will challenge more advanced users	GUI test
	3	Is there an option to stack masses	Gives the user more control in the total mass placed on certain points	GUI test
	4	Does the undo/redo button take you to the previous and following state respectively.	When users are experimenting, they can undo moves without	GUI test

			restarting the entire solution which is especially useful for puzzles that require trial and error	
	5	Are the puzzles unique	The puzzles should be unique so that they can solve new problems without redoing the same problems	GUI test
	6	Do the puzzles stated difficulty match the actual difficulty	Ensures that the user is solving problems that are suitable for the level that they are at	Stakeholder questionnaire
	7	Does the program show the calculation	By displaying the underlying maths, it will help students using the solution to gain a better understanding of the principle of moments which will allow them to gain a deeper understanding that will aid them during formal schooling	GUI Test
	8	Is the correct answer displayed if the user requires it	Gives the user feedback that allows them to gain understanding on where they went wrong	GUI Test
3. Leaderboard	1	Is the leaderboard automatically updated to show which player is in the lead	The leaderboard is needed to maintain competitiveness. If the leaderboard didn't update automatically, it would feel irrelevant to users thus reducing engagement.	GUI Test
	2	Does the leaderboard correctly show the score of the player		GUI Test
	3	Do users have the option to leave the leaderboard	Allows the user to decide whether their stats are visible to other users.	GUI Test
	4	Do users have the option to rejoin the leaderboard		GUI Test
4. Sandbox	1	Can the user change all the parameters such as height, velocity, target, projectile mass and air resistance	The sandbox allows for users to test certain scenarios rather than	GUI Test

	2	Can users set up balance problems with their required parameters	looking to just solve a problem	GUI Test
5. User	1	Can users enter a username and password	Needed so that users can create their own account so that progress is accurate and they don't overwrite other user's data	GUI Test
	2	Are the username and password checked against stored values to allow or deny entry	Needed to ensure that the user is accessing their own account	GUI Test
	3	Can users enter the sandbox mode without any details	Allows the user to play for short periods without having to sign in	GUI Test
	4	Does the solution allow for new users to be created	Allows new users to use the program with their data being saved	GUI Test
	5	If a returning user signs in, will the solution retrieve their current level and score	Allows for continuity between sessions and ensures that the solution is useful thus improving user retention	GUI Test
	6	Do users level match their stage of learning	To ensure that users get the most out of this solution, it will have to become more difficult as the user progresses so that they can learn how to do more difficult questions and better apply their knowledge.	Stakeholder questionnaire
	7	Are the puzzles generated right for that level		Stakeholder questionnaire
6. Tutorial	1	Does the tutorial appear when the solution is first executed	Allows the user to learn while they are becoming familiar to the program	GUI Test
	2	Can the tutorial be loaded from the main menu when necessary	In case the user forgets how to use the solution the tutorial can be loaded so that they can refresh their understanding	GUI Test

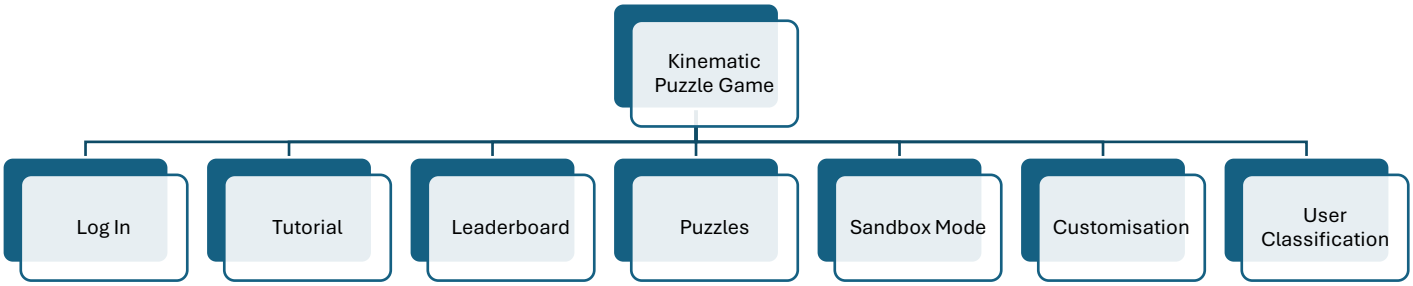
	3	Is the tutorial easy to understand	The tutorial should be easy to understand so that the user can understand how to use the program in as little time as possible	Stakeholder questionnaire
7. Save	1	Can a user save a puzzle setup	This feature is useful as it will allow the user to be able to save setups that they wish to look over in the future to aid their revision	GUI Test
	2	Can a user load a setup they previously created		GUI Test
8. UI	1	Are there multiple themes that the user can apply to the UI	Allows the program to be more visually appealing to the user	GUI Test
	2	Is the UI easy to navigate and use	People who have never used this program should be able to use the solution without problem from the very beginning thus increasing usability	Stakeholder questionnaire
	3	Is it a drag-and-drop interface	Users should be able to learn the controls quickly and be able to use the program with limited prior knowledge	GUI Test
	4	Can the music be toggled	Allows users to avoid distractions	GUI Test
	5	Can the user reset their account	Allows the user to delete all data stored on them	GUI Test
	6	Can the user change their username/password	Gives the user more control over their account	GUI Test
	7	Is there a place for users to write notes	Means that the user no longer has to do all the calculations in their head or find paper to do it on	GUI Test

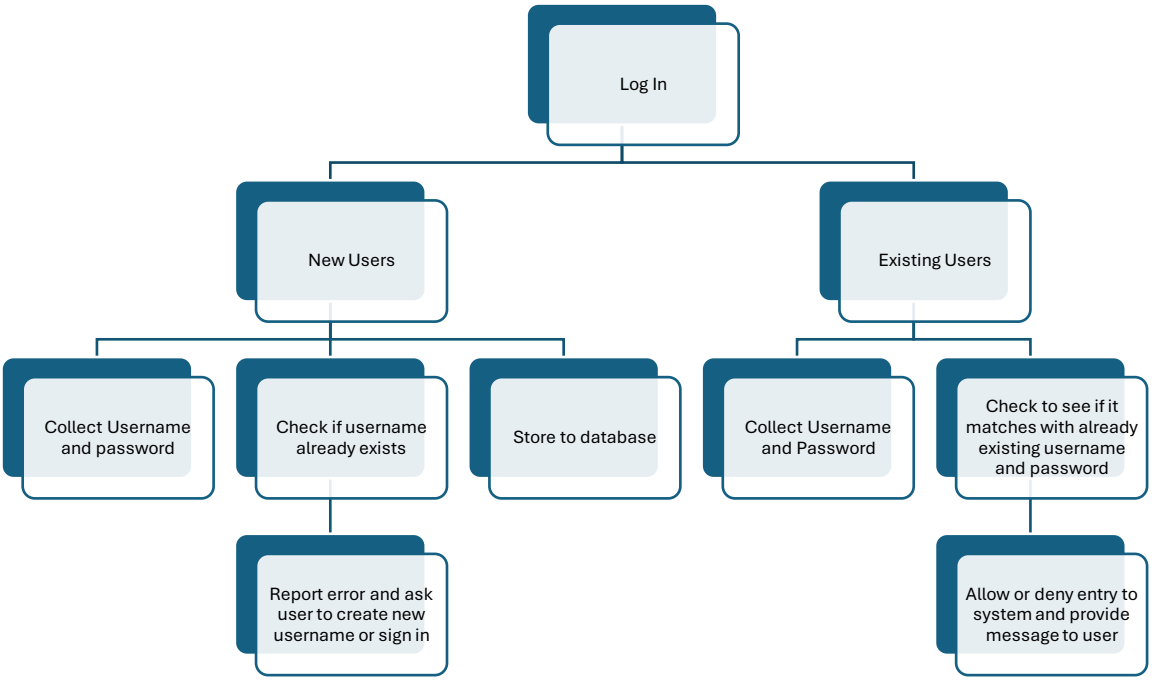
Design and Development

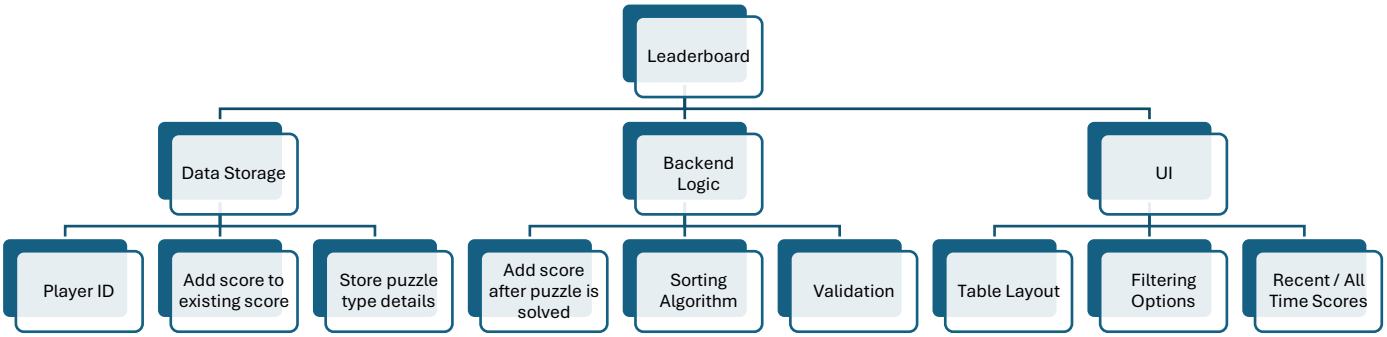
Basic Project Design

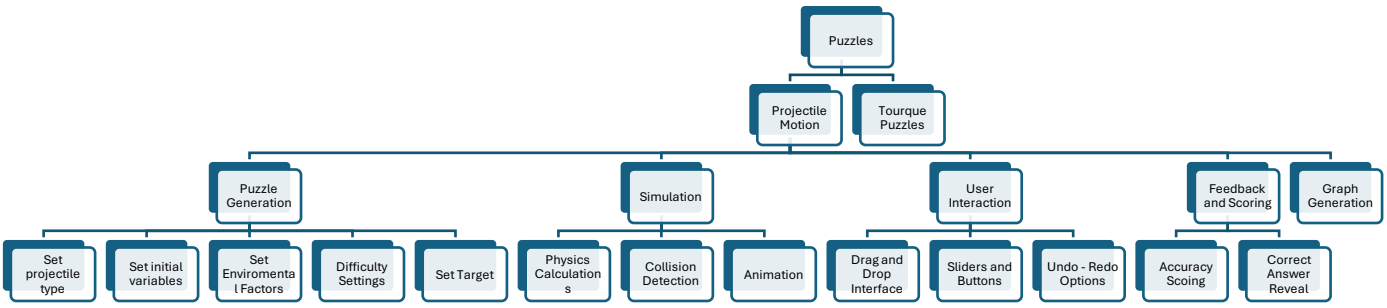
Decomposition Chart

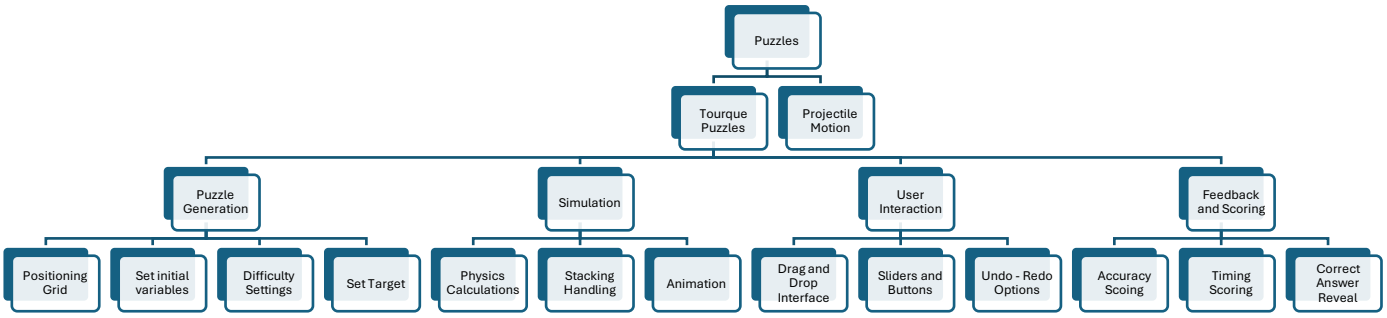
To ensure readability, the decomposition chart has been split into separate charts for each subproblem

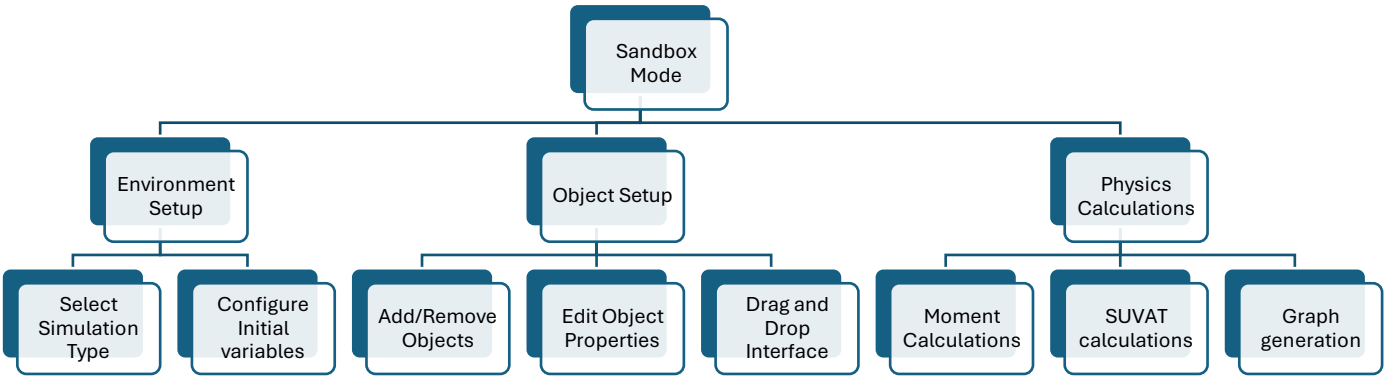


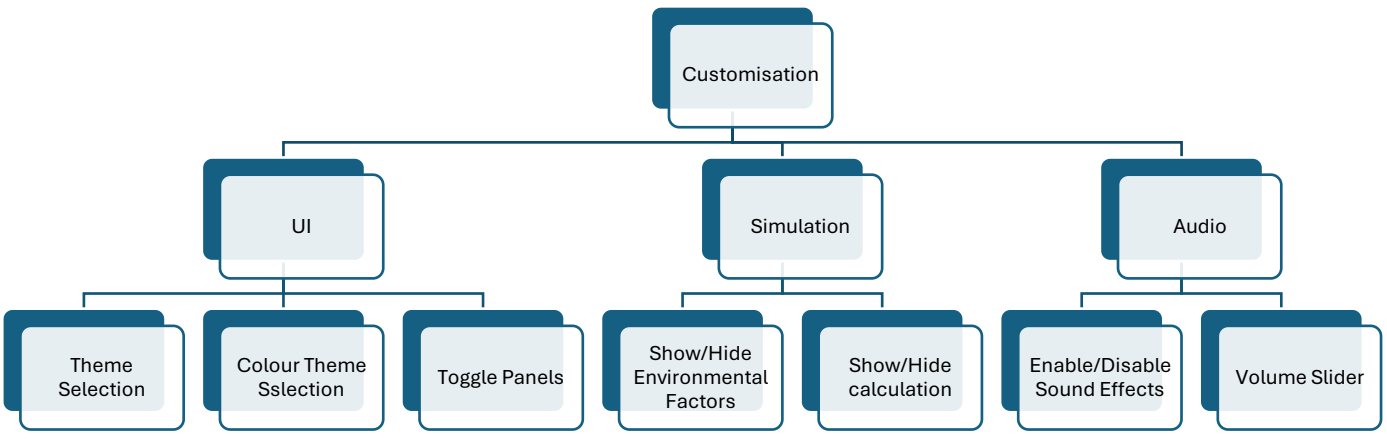


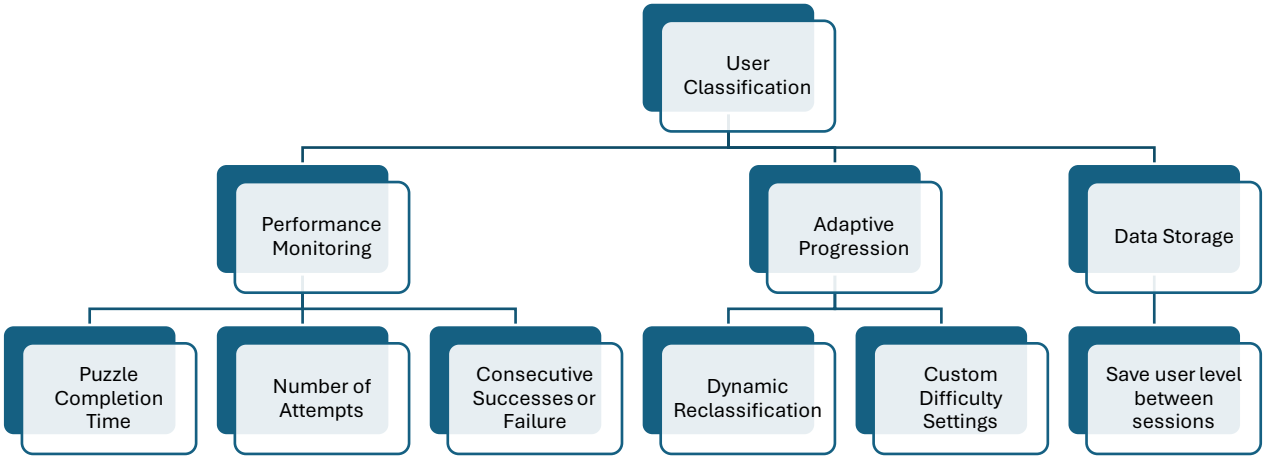

Leaderboard
Data Storage
Backend Logic
UI
Player ID
Add score to existing score
Store puzzle type details
Add score after puzzle is solved
Sorting Algorithm
Validation
Table Layout
Filtering Options
Recent / All Time Scores


Puzzles
Projectile Motion
Tourque Puzzles
Puzzle Generation
Simulation
User Interaction
Feedback and Scoring
Graph Generation
Set projectile type
Set initial variables
Set Enviromental Factors
Difficulty Settings
Set Target
Physics Calculations
Collision Detection
Animation
Drag and Drop Interface
Sliders and Buttons
Undo - Redo Options
Accuracy Scoing
Correct Answer Reveal


Puzzles
Tourque Puzzles
Projectile Motion
Puzzle Generation
Simulation
User Interaction
Feedback and Scoring
Positioning Grid
Set initial variables
Difficulty Settings
Set Target
Physics Calculations
Stacking Handling
Animation
Drag and Drop Interface
Sliders and Buttons
Undo - Redo Options
Accuracy Scoing
Timing Scoring
Correct Answer Reveal


Sandbox Mode
Environment Setup
Object Setup
Physics Calculations
Select Simulation Type
Configure Initial variables
Add/Remove Objects
Edit Object Properties
Drag and Drop Interface
Moment Calculations
SUVAT calculations
Graph generation


Customisation
UI
Simulation
Audio
Theme Selection
Colour Theme Sslection
Toggle Panels
Show/Hide Environmental Factors
Show/Hide calculation
Enable/Disable Sound Effects
Volume Slider



Justification

Sub Problem	Justification
Log In	Allows users to have their own profile to track their progress so that puzzles are generated at an appropriate level of difficulty. In addition, it also supports the leaderboard functionality. The log in system should ensure that each user has a unique username and also ensure that the user can only access their account.
Tutorial	Supports users to use the platform and makes it easier to use the platform. New users will not be familiar with the UI so the tutorial should display immediately with the option for users to skip if they feel it is not necessary. The user should also be able to reload the tutorial if they feel it is necessary, and they need help with how to use the UI.
Leaderboard	The data must be stored to ensure that all of the users' data is used to make an accurate leaderboard. This data should be linked to the user's ID and must be kept up to date for the leaderboard to have purpose. The leaderboard must implement a sorting algorithm to ensure that the data is shown in the correct order so that users can easily see where they are in comparison to other users. The data being entered into the leaderboard system should also be validated to ensure that the data is accurate. In addition, the user should have the ability to filter the table to compare themselves to similar users, recent and all-time statistics as this will give the user a greater degree of freedom.
Puzzle Generation	The system will set projectile properties using input ranges defined per difficulty levels which will allow for more tailored challenges and more accurate physics

	modelling. Similarly environmental factors when in use can be dynamically adjusted for difficulty. In addition, the program has to check that the targets are feasible and as challenging as required to optimise user engagement. As well as that by using
Simulation	The system must numerically calculate the position and velocity of the projectile over time using integration on Newton's Second Law. These forces are computed at each frame to update the projectile's trajectory. The simulation must detect collisions to properly deal with them so that the animations display realistic physics on where the projectile will end up
User Interaction	The system should have a way for the user to interact with it. This can include drag and drop as well as sliders and buttons. Drag and drop allows for users to directly manipulate objects on the screen as they would in the real world. In order to validate this system, the system will have to perform collision checks and snap-to-grid logic. Sliders also allow users to control variables that can be easily limited to ensure only valid data is passed into the system. In addition, the use of stacks of prior states stored as JSON files will allow the user to revert to previous state
Scoring	Scoring will require comparing user input to expected values and using error margins gives partial credit for close answers. Additionally, the predefined success criteria will be tested, and the score will be updated upon completion. In addition, timing scoring will use a stopwatch and compared to certain thresholds to determine the number of points to award for efficiency and for working under pressure
Sandbox	The sandbox should allow users to freely manipulate all simulation parameters without restrictions to explore concepts. For this to occur the user should be able to configure the environmental factors such as gravity and air resistance which will require real time updating of the physics calculations with validation to ensure that the data inputted is feasible. The system should allow for the user to select objects with different properties and support switching between objects without restarting the simulation.
Customisation	Allow users to modify UI elements for accessibility and personal comfort that won't break functionality. Allow users to toggle background music and sound effects as well as adjust volume levels
Performance Monitoring	Records the total time taken by a user to complete a puzzle to track efficiency and progression. In addition, it

	will also record how many attempts the user takes to solve a problem and track streaks of correct or incorrect puzzle completion. This data can be used to adjust puzzle difficulty dynamically and move users up and down levels.

Below the relationship between the tables in my database can be seen. PK stands for Primary Key and FK refers to the foreign key



Data Structures

Data Structure	Description	Justification
SQL Database	Used to store all user accounts, leaderboard scores, saved puzzles, and progress data in a relational database	The use of a SQL database allows for persistent storage of data across sessions. An alternative would have been to use CSV however CSV cannot store relational data and does not support atomic updates. In addition SQL allows for efficient retrieval of data even with large datasets. Furthermore it also allows data integrity and ACID compliance.
Lists	I will be using lists to store pivot positions and trajectory of projectiles	A list will allow me to have O(1) access speed which will allow for fast retrieval of data. In addition, it allows for easy iteration as well as built in LINQ support which will be useful for filtering and sorting. I am using a list over a static array as arrays are static but for my solution I will need a data structure where the number of elements are variable.
JSON	JSON serialisation can be used for complex puzzle data where multiple values must be saved in a single save slot	Through the use of JSON it makes the data more compact as a single JSON string can store multiple related values in one string, this will also help with debugging the code as JSON strings are human readable. Unlike CSV, JSON strings will maintain nested lists and their order making them more powerful to use. Furthermore, .NET offers built in serialisation using .JsonConvert making it simple to use within my code
Stack	The undo/redo functionality will use two stacks to store states	The undo/redo feature requires the most recent state to be reverted first which is LIFO behaviour i.e. how stacks

		operate. In addition, push and pop are O(1) which will ensure efficient operations.
Tuples	Tuples can be used when returning data from the database or other methods	The use of tuples allows for efficient data passing without adding unnecessary class structures for short term grouping of data

Object Oriented Programming (OOP)

For my solution, I have chosen to use OOP principles to allow for greater structure whilst also allowing for easier debugging. For my solution, I will decompose the program into logical sections so that each class can handle a single logical component, and changes to one class will not affect other classes and the solution as a whole. Furthermore, through the use of OOP it makes it easier to add new features such as creating new puzzle modes, improving the physics engine and adding more leaderboard types.

Encapsulation

Each component of the program will be handled by a class. For example, the physics class will perform the calculations that other classes can call if they wish to use. For my program, encapsulation ensures that internal details, such as the calculations, are hidden from other parts of the program and that the UI interacts with each other through the use of these methods. Furthermore, it will also make it easier to trace bugs as you will be aware of which class is responsible for which logical component.

Abstraction

Through the use of OOP, the code becomes more readable to anyone debugging or maintaining the code as the front-end code will only call a function that will deal with the complex calculations rather than in the UI.

Modularity

Each class is independent of each other, meaning that they can be edited without breaking the rest of the code. This modularity is essential for a project with multiple prototypes, where code will need to be frequently updated.

Reusability

Through the use of OOP, code does not need to be repeated in multiple forms as they can all reference the one class, meaning that if a change needs to be made, it only needs to be made in one method/class.

Inheritance

By allowing classes to inherit from each other allows code reusability. In addition, it makes each class more specific to its logical component rather than combining multiple components into one class. This will make it easier to debug and maintain

Testing

This test table was created to ensure that the final solution could be systematically verified against the identified success criteria.

Success Criteria	Test Category	Test Description	Test Type	Expected Outcome	Justification

1.1, 1.13, 1.14	Physics Accuracy	Test projectile motion using standard GCSE/A-Level scenarios without drag or restitution	Normal	Motion follows a parabolic trajectory consistent with equations	The core purpose of the system depends on the accuracy of the physics. By using standard scenarios, SUVAT equations can be used to ensure validity
1.1, 1.13	Physics Accuracy	Test projectile motion with drag and coefficient of restitution enabled	Normal	Motion reflects realistic energy loss and altered trajectory	Test to make sure that the model remains realistic and supports more advanced experimentation
2.1	Physics Accuracy	Test torque simulation with single and multiple pivot setups	Normal	System correctly balances moments	Accurate simulation is essential as it is the core purpose of the solution
1.2–1.5, 4.1	Parameter Manipulation	Verify that users can modify key variables such as height, angle, velocity, and mass	Normal	Changes are applied immediately and displayed correctly	Stakeholders requested the ability to experiment freely
1.2–1.5	Constraint Enforcement	Verify that puzzle mode restricts modification of preset values	Normal	Preset puzzle values remain locked	Prevents users for bypassing puzzles and maintains integrity
1.6, 8.3	UI Interaction	Test drag-and-drop interaction for targets and interactive elements	Usability	Objects move smoothly and display updated positions	Visual interaction will improve engagement and accessibility for users
1.7, 2.5	Puzzle Generation	Verify that procedurally generated puzzles do not repeat over multiple attempts	Normal	Each puzzle has a unique configuration	Repetition would reduce user engagement
1.8, 2.6, 5.6	Difficulty Scaling	Check that puzzle difficulty matches the user's current level	Usability	Difficulty aligns with user ability	Adaptive difficulty ensures that puzzles are at an appropriate

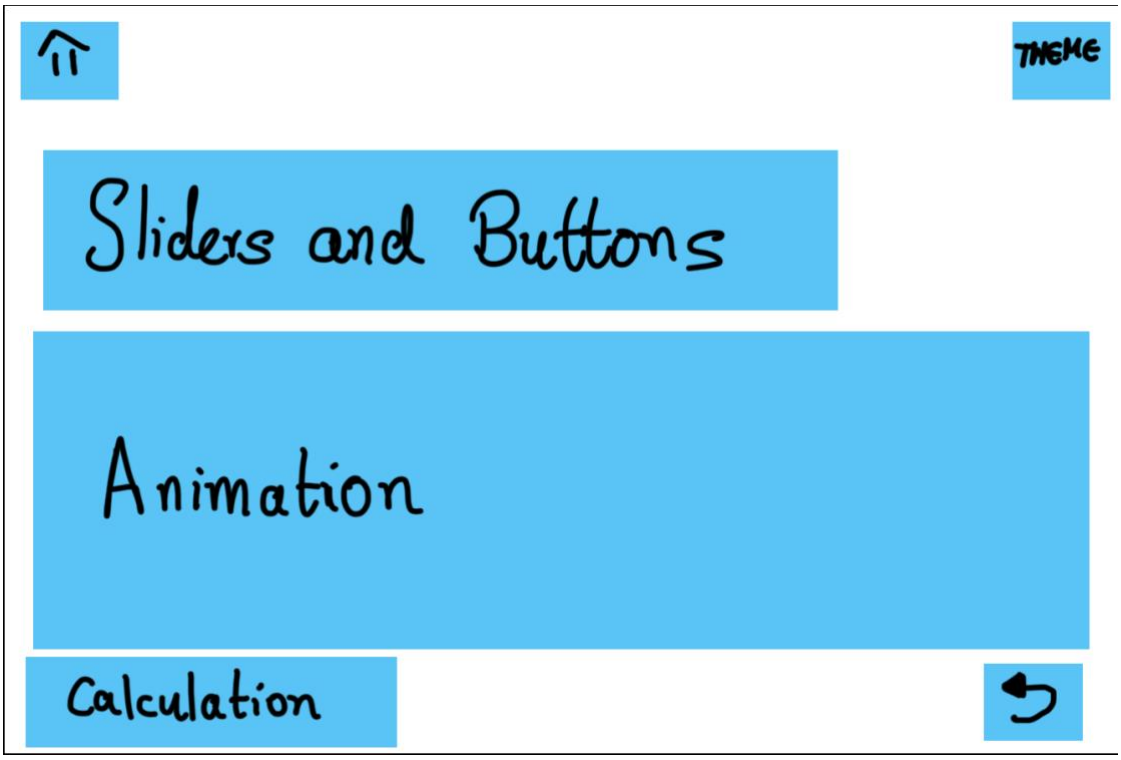
					level for the user
1.9, 1.10, 1.15	Data Visualisation	Test that graphs can be displayed when requested	Normal	Correct graphs are shown	Graph shows interpretation of the motion to reinforce understanding
1.11	Graph Detail	Verify that key values (e.g. maxima, intercepts) can be identified on graphs	Normal	Key values are clearly visible	Highlighting key values supports users when solving exam style problems
1.12	Animation	Test projectile animation at different simulation speeds	Normal	Animation remains smooth and accurate	Improves usability and accessibility
1.16	Graph Interaction	Verify users can zoom into specific regions of graphs	Usability	Graph scales correctly	Enables users to clearly read values from graph
1.17	Vector Representation	Verify that users can understand force and velocity components	Normal	Components are clearly represented	Supports users understanding of the components that make up the motion
1.18	Graph Customisation	Test visibility, colour, and thickness toggling of graph lines	Usability	Graph appearance updates correctly	Customisation improves clarity for users making it more accessible
1.19, 2.4	Undo/Redo	Test undo and redo functionality for setup changes	Robustness	Previous states are restored correctly	Undo/redo functionality supports experimentation
1.20, 2.8	Solution Reveal	Verify that correct puzzle solutions can be displayed on request	Normal	Correct solution is shown	Provides feedback to user
3.1–3.4	Leaderboard	Test automatic score updates and opt-in/opt-out functionality	Normal	Leaderboard updates accurately	Test that leaderboard works as expected
5.1–5.4	Authentication	Test account creation, login, and credential validation	Robustness	Invalid actions are rejected	Needed to work as expected for system security and data integrity

5.5	Persistence	Verify that user preferences, scores, and levels persist across sessions	Normal	Data is correctly retrieved	Essential for long term engagement
6.1–6.3	Tutorial	Test automatic and manual tutorial access	Usability	Tutorial loads correctly and is understandable	Supports new users
7.1–7.2	Save/Load	Test saving and loading of puzzle setups	Normal	Setups are restored accurately	Prevents loss of work
8.1	UI Themes	Test application of UI themes across forms	Normal	Theme applied consistently	Improves accessibility
8.2	Navigation	Evaluate overall ease of navigation	Usability	UI is intuitive	Poor navigation would reduce educational value, can be tested through stakeholder questionnaire
8.4	Audio Control	Test toggling of background music	Normal	Audio responds correctly	Should update to user preference
8.5–8.6	Account Management	Test account reset and credential modification	Robustness	Changes are applied correctly	Allows users to have control over their data

All referenced video files can be found in the folder *NEA Videos / Iterative Testing Videos*

Prototype 1 – Projectile Sandbox

The first prototype will focus on developing the projectile motion sandbox, as this is the foundation for testing the program's physics engine and interaction model. The prototype will include a login system, allowing each user to securely access their account. Within the sandbox, users will be able to experiment with projectile motion by adjusting parameters such as launch angle, initial velocity, and gravity, observing the resulting motion in real time. Focusing on the projectile motion sandbox ensures that the core physics calculations, collision detection, and animation systems are tested and validated early in development. This provides a stable framework upon which the puzzles can be built. Including the login system at this stage supports future features such as leaderboards, progress monitoring, and personalised puzzle generation. By developing this projectile sandbox first, the prototype directly demonstrates the program's educational value while keeping the scope manageable.



Class Diagrams

These are the key classes for this prototype. The physics engine will allow the program to carry out all the required calculation and the database class handles data logic and encapsulates all database operations, allowing for reusability and clean code.

PhysicsEngine
Key Attributes Double x,y *#current position* Double vx,vy *#current velocity* Double mass Double drag Double eFloor, eWall, eCeiling Double muFloor Double g Double targetLeft Double targetRight Double Epsilon List<(double t, double x, double y)> trajectory;

Key Methods
`Update(dt, width, height, currentTime … )`
`Reset()`
`CheckHit()`
`GetTrajectory()`

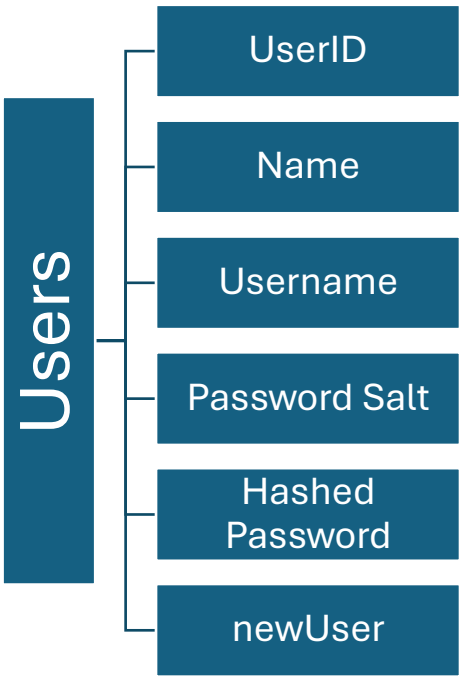
```
SetProperties(mass, drag, e …)
```

Database
Key Attributes `ConnectionString`

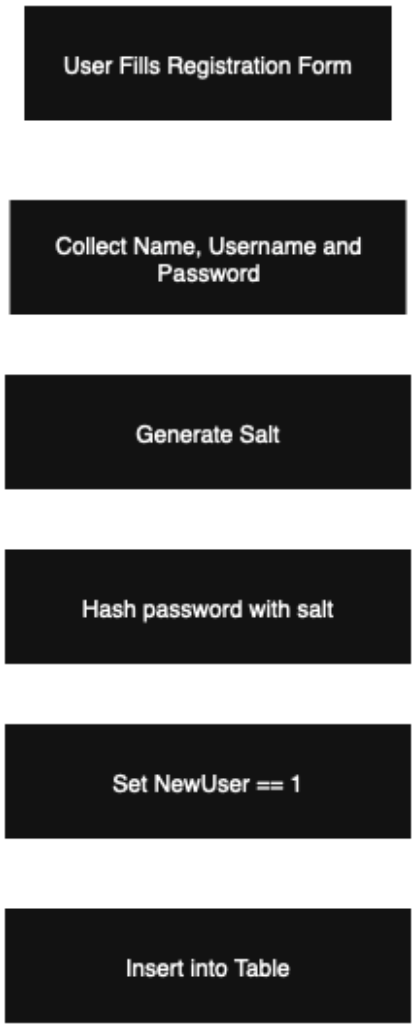
Key Methods
`RegisterUser()`
`ValidateLogin()`
`UsernameExists()`

Database

In this program, I am using SQLite to hold user data. The database contains a table with columns for UserID (an auto-incrementing primary key), Name, Username, Salt, HashedPassword, and IsNewUser. I used a CHECK constraint on IsNewUser to ensure it only contains valid Boolean equivalents (0 or 1). and all forms in the application connect to it using a consistent connection string, allowing multiple forms to access and update user data efficiently. In order to insert new user data into the SQLite database, I needed a method that could execute SQL commands that modify the database but do not return any data. I used ExecuteNonQuery() for this purpose. (Kalinda, 2025)



```
CREATE TABLE Users(
UserID INT PRIMARY KEY AUTOINCREMENT,
Name STR,
Username STR,
Salt STR,
HashedPassword STR,
IsNewUser INT CHECK(IsNewUser IN (0,1))
)
```



The flowchart above shows how the data will be stored in the database. When the data is being inserted into the table, it will be parameterised rather than directly embedding values into the SQL command as it will prevent SQL injection attacks. All input fields are added to the query as parameters, and the database automatically generates the UserID.

```
INSERT INTO Users(Name, Username, Salt, HashedPassword, IsNewUser)
VALUES (@name, @username, @salt, @hashedpassword, @isNewUser);
```

Algorithms

Log in

This algorithm will call the database class and retrieve the hashed password for the relevant username if it can be found and will then compare the inputted value to the stored value

```
LogIn(username, password)
	try
		stored = database.password(username)
	except
		print errorMessage
		return false
	else
		if password == stored
			return true
		else
			print errorMessage
			return false
```

Password Hashing

This algorithm will generate the salt if not already stored and hash the password. A salt is used to prevent rainbow table attacks. I am hashing the passwords to increase the security of my solution as even if they gain access to. The databases they cannot see the actual password as hashing is a one-way function

```
GenerateSalt()
	Byte array saltBytes = [16]
	For each index in saltBytes:
		saltBytes[index] = random byte
	return Base64Encode(saltBytes)

HashPassword(password, salt)
	Concatenate = password + salt
	Hash = SHA-256Hash(Concatenate)
	Return Base64Encode(Hash)
```

Undo

The undo feature will be implemented through the use of arrays and a stack. Each time the user makes a change, a variety in the form shown below will be pushed to the stack. The array contains the variables Coefficient of Friction (coF) and Coefficient of Restitution (coR). Delta Time (dt) is the timestep used in the simulation

```
Environment = [INT angle, INT height, INT initialVelocity, DOUBLE drag,
BOOL AirResistance, DOUBLE gravity, DOUBLE coF, DOUBLE coR, DOUBLE dt]
```

When the user clicks undo, the stack will pop the top array and apply those variables. In order to reduce the size of the program, the solution will keep the stack size to 5. This will be implemented through the use of the algorithm shown below.

```
If stack.size() == 5:
      topElement = stack.pop()
      stack.clear()
      stack.push(topElement)
```

Update

This function updates the state of a projectile by a timestep in the sandbox. It calculates the effects of gravity, air resistance and collisions and updates the projectiles position accordingly. I had to research how to model the effects of air resistance (Hamxa, 2023) and collisions (Rawshdeh, 2007). This algorithm uses Euler's method to repeatedly update the object's velocity and position under gravity and air resistance. I chose to do this because air resistance makes the equations of motion difficult to solve exactly, so a numerical method like Euler's provides a practical way to approximate velocity and displacement at small time intervals.

```
While true
      status = projectile.Update(dt, sandboxwidth, sandboxHeight, time)
      If statues == "hit" || status == "stopped"
            Break
      time += dt
```

```
Update (dt, width, height, currentTime)
       dragX = drag * vx #drag forces
      dragY = drag * vy
       ax = -dragX / mass #acceleration
      ay = -g - (dragY / mass)
       x += vx * dt #update position
       y += vy * dt

       if y <= 0 #projectile has reached the floor
              y = 0
              if CheckHit():
                     trajectory.Add(currentTime, x, y)
                     return “hit”
              vy = -vy * eFloor
              vx = vx * (1 - muFloor)

       if y >= height #projectile has reached the ceiling
              y = height
              vy = -vy * eCeiling

       if x <= 0 #projectile has reached the left wall
              x = 0
              vx = -vx * eWall

       if x >= width #projectile has reached the right wall
              x = width
              vx = -vx * eWall

       if (Math.Abs(vx) < epsilon && Maths.Abs(vy) < epsilon && y == 0)
              vx = 0
              vy = 0
              return “stopped”

       trajectory.Add(currentTime, x, y)
       return “moving”
```

Graph

This algorithm uses the trajectory values and dt to calculate the velocities at each point. These values will then be used to draw a graph where multiple lines can be placed and removed to allow for clear comparison.

```
For i in range (trajectory. Length – 1)
      Vx[i] = (trajectory[i + 1][1] – trajectory[i][1])/dt
      Vy[i] = (trajectory[i + 1][2] – trajectory[i][2])/dt
      V[i] = (Vx[i] ** 2 + Vy[i]) ** 0.5
For i in range (Vx – 1)
      Ax[i] = (Vx[i + 1] – Vx[i])/dt
      Ax[i] = (Vx[i + 1] – Vx[i])/dt
      A[i] = (Ax[i] ** 2 + Ay[i]) ** 0.5
      Time[i] = trajectory[i][0]


Series VxSeries = new Series("Vx")
vxSeries.ChartType = SeriesChartType.Line
for (int i = 0; i < time.Count; i++)


    vxSeries.Points.AddXY(time[i], vx[i])


chart1.Series.Add(vxSeries)
```

Usability Features

This solution will use sliders and buttons to allow the user to interact with the solution. The user should be able to change the height, angle and velocity of release as well as change environmental factors and projectile factors such as drag, coefficient of restitution etc... The solution should also allow the user to place a target of size 1m on the canvas. In addition, by clicking a button the user should be able to load a graph where they can select whether they would like to see a Velocity – time graph or acceleration-time graph with just x component, y component or magnitude.

Key Variable

Variable Name	Data Type	Description	Justification
x, y	double	Current position of the projectile	Essential to track projectile position at each timestep
VX, VY	double	Current horizontal and vertical velocities	Needed to update positions using Newtonian mechanics
Mass	Double	Mass of the projectile	Allows different projectile types and realistic physics
Drag	Double	Drag coefficient representing air resistance	Allows simulation of energy loss due to air resistance
eFloor, eWall, eCeiling	double	Coefficient of restitution	Controls how much energy is lost on collision and to model bounces realistically. Different values allow customization of bounciness.

muFloor	Double	Coefficient of friction	Reduces horizontal velocity after floor collisions to simulate energy loss due to friction. Adds realism to simulation.
Epsilon	double	Threshold to stop tiny movements	Prevents jitter when projectile velocities are very small, allowing simulation to stop cleanly.
Trajectory	List	Stores the projectile positions over time	Needed for animation and plotting graphs
StateStack	Stack	Stores the simulation variables at each change	Stack enables LIFO behaviour needed for undo functionality. Each change is pushed onto the stack so it can be reverted efficiently.
State	Array	An array storing all the variables that define the simulation state	Makes it easier to manage the undo operation
dt	Double	Delta Time - timestamp used for updating positions and velocities	Recording the timestep allows the undo/redo system to accurately revert motion calculations.
Vx, Vy, V **Ax, Ay, A**	List	Holds the velocities and accelerations at different time steps	Used for easy access to graph the curves.
targetLeft	Double	Holds the position of the left of the target	Used for the collision detection algorithm
Target	Panel	A visual element that represents the target that the projectile should hit.	Using a Panel allows easy manipulation of position, size, and colour, making it simple to detect collisions with the projectile. It provides instant visual feedback, which is essential for engagement and understanding in a sandbox simulation.
2DSpace	Panel	The main container representing the simulation area (the “world”) where the projectile moves.	Encapsulates all interactive elements (projectile, target, obstacles) and serves as a coordinate space for animation. Using a Panel simplifies drawing and positioning, making the simulation more

			interactive and visually clear.
Projectile	Panel	A small visual element representing the moving object that the user launches.	Using a Panel allows easy animation by updating its Location property, and it can change appearance dynamically (size, colour) depending on the projectile type. This approach is simpler than drawing graphics manually.

Testing

Input	Testing	Justification	Expected Result
Enter valid new username password	Log In / Registration	Tests account creation and storage which is essential for the rest of the solution	Account is created and saved; the user is let into the solution and can log in using this details next time
Enter Invalid Username/Password	Log In / Registration	Tests input validation	Error message shown; account not created
Launch Projectile from height 0m Angle 45° Velocity 10ms^{-1} Drag = 0 E = 0 Gravity = 9.81ms^{-2}	Projectile Motion	Test basic 2d projectile motion to ensure it follows physics accurately	Parabolic trajectory. Lands at distance of ≈ 10.2m
Launch Projectile from height 5m Angle 30° Velocity 15ms^{-1} Drag = 0.1 E = 0 Gravity = 9.81ms^{-2}	Projectile Motion	Tests projectile motion with initial height and air resistance to show effect of air resistance	Projectile slows slightly and will land just short of ≈26m
Launch projectiles with varying coefficient of restitution (e)	Projectile Motion	To test bounce behaviour and simulate energy loss	Projectile bounces with height = previous height × coR; stops eventually if coR < 1
Move Target	Projectile Motion	Test the system recognises where the target is	Visual UI change as well as the system recognising if the target has been hit
Undo Last Action	Projectile Motion	Tests undo stack	Projectile state reverts to previous state with variables restored

Test graphing capability with Vx, Vy and $A_{magnitude}$	Projectile Motion	Tests that position values are recorded and plotted over time	Multiple lines in different colours should appear on the graph

Development

I had decided to program my solution in Windows Forms as this was a framework that I was most familiar with. As I was building this prototype, I had to research how to implement the hashing algorithm, which I decided to place in the database class as the methods GenerateSalt(), HashPassword(). In order to build this, I implemented the algorithm I found on Stack Exchange (Exchange, 2020). To begin with, I built the Log In and Registration system, where I had to experiment with how to manipulate the data returned from the SQL queries so that I could use them.

```
5 references
internal class Database
{
    private string connectionString;
    2 references
    public Database(string dbPath = "\"C:\\Users\\pranavbibin\\source\\repos\\Kinematics Puzzle Game - NEA\\mydb.db\"")
    {
        connectionString = $"Data Source = {dbPath}; Version=3;";
    }
    1 reference
    public static string GenerateSalt()
    {
        var rng = new RNGCryptoServiceProvider();
        byte[] saltBytes = new byte[16];
        rng.GetBytes(saltBytes); //filling the array with random bytes taken from a secure random number generator
        return Convert.ToBase64String(saltBytes); // converting the array to a string so that it can be easily stored
    }

    2 references
    public static string HashPassword(string password, string salt)
    {
        byte[] bytes = Encoding.UTF8.GetBytes(password + salt); // converts concanated string into a byte array
        var hashManager = new SHA256Managed();
        byte[] hash = hashManager.ComputeHash(bytes); //returns 32 byte array, the hashed output

        return Convert.ToBase64String(hash);
    }
    1 reference
    public void ExecuteNonQuery(string sql, params SQLiteParameter[] parameters)
    {
        using (var conn = new SQLiteConnection(connectionString)) //creates connection to the database
        {
            conn.Open(); //opens connection
            using (var cmd = new SQLiteCommand(sql, conn))
            {
                if (parameters != null)
                    cmd.Parameters.AddRange(parameters); //adds paramaters to the command

                cmd.ExecuteNonQuery(); //executes the command
            }
        }
    }
```

```
public DataTable ExecuteQuery(string sql, params SQLiteParameter[] parameters)
{
    using (var conn = new SQLiteConnection(connectionString)) //creates connection to the database
    {
        conn.Open(); //opens connection
        using (var cmd = new SQLiteCommand(sql, conn))
        {
            if (parameters != null)
                cmd.Parameters.AddRange(parameters); //adds paramaters to the command

            using (var adapter = new SQLiteDataAdapter(cmd)) //creates a data adapter wihich executes the command and fills a table with the results
            {
                var dt = new DataTable();
                adapter.Fill(dt);
                return dt;
            }
        }
    }
}


1 reference
public void RegisterUser(string name, string username, string password)
{
    string salt = GenerateSalt();
    string hashedPassword = HashPassword(password, salt);

    string sql = @"
        INSERT INTO Users (Name, Username, Salt, HashedPassword, IsNewUser)
        VALUES (@name, @username, @salt, @hashedPassword, 1);
        ";
    ExecuteNonQuery(sql,
        new SQLiteParameter("@name", name),
        new SQLiteParameter("@username", username),
        new SQLiteParameter("@salt", salt),
        new SQLiteParameter("@hashedPassword", hashedPassword)
        );

}
```

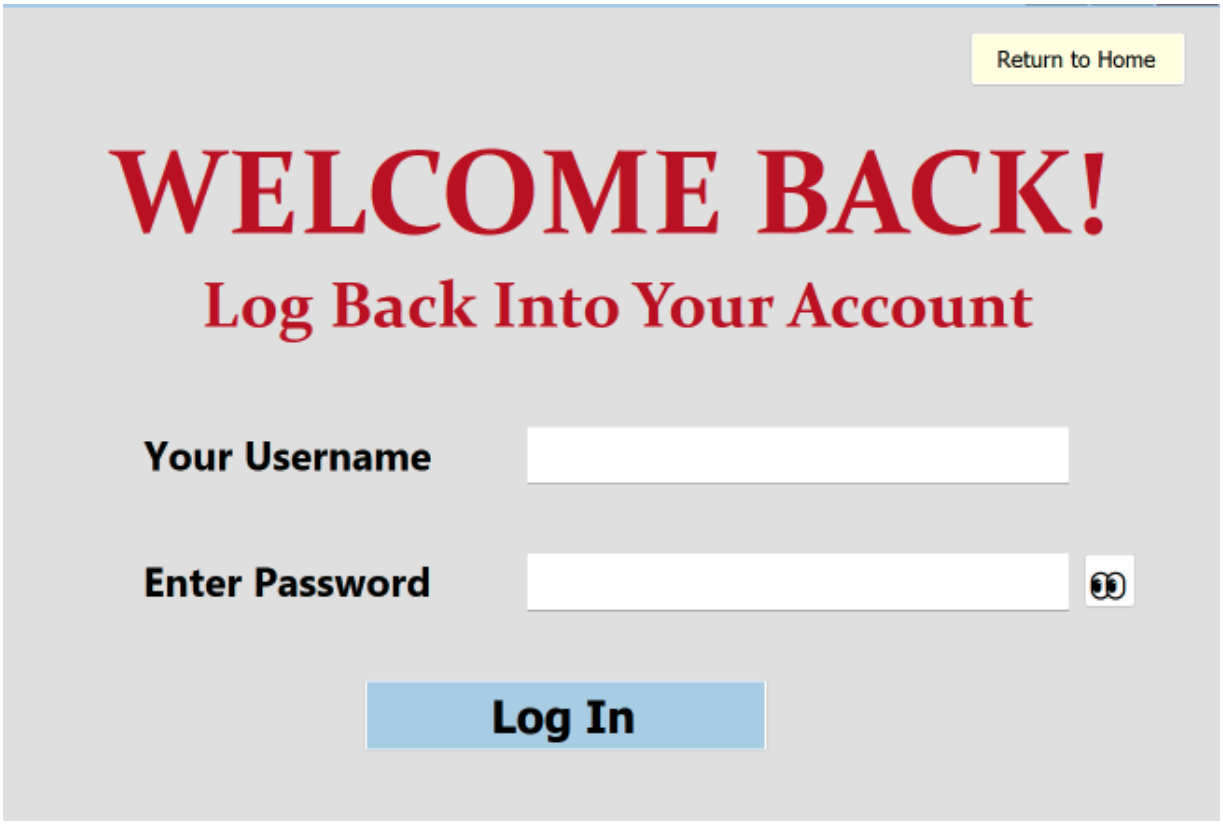
```
1 reference
public bool UsernameExists(string username)
{
    string sql = "SELECT COUNT(*) AS UserCount FROM Users WHERE Username = @username";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@username", username));
    int count = Convert.ToInt32(result.Rows[0]["UserCount"]); // takes the value from the first row, and the UserCount column
    if (count > 0)
    {
        return true;
    }
    else
    {
        return false;
    }

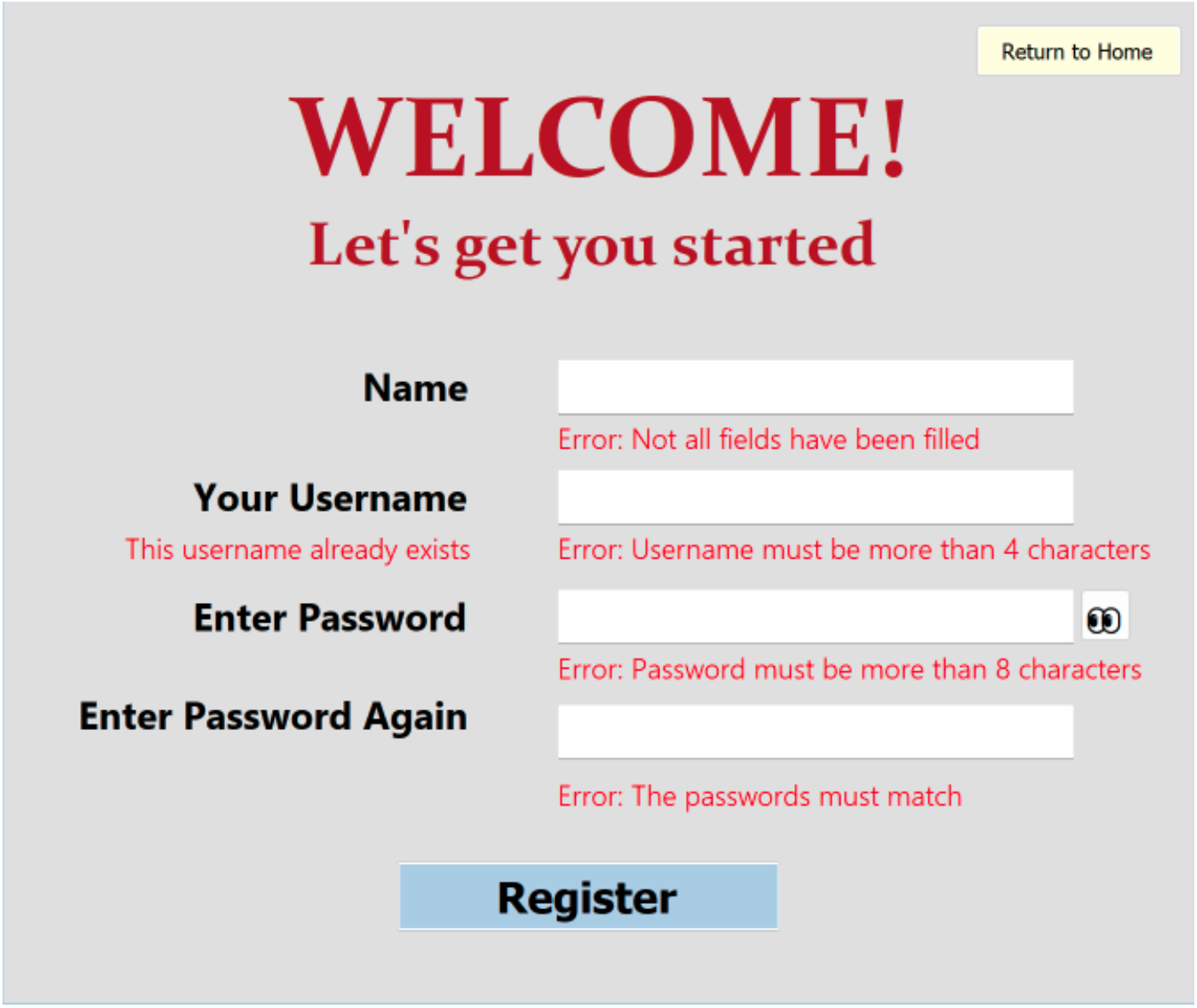
}

1 reference
public bool ValidateLogin(string username, string password)
{
    string sql = "SELECT Salt, HashedPassword FROM Users WHERE Username = @username";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@username", username));
    if (result.Rows.Count > 0)
    {
        string salt = result.Rows[0]["Salt"].ToString();
        string hash = result.Rows[0]["HashedPassword"].ToString();
        string pwd = HashPassword(password, salt);
        if (pwd == hash)
        {
            return true;
        }
        else
        {

            MessageBox.Show("Password Incorrect", "HTTP 401 UNAUTHORISED", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
    }
    else
    {
        MessageBox.Show("Username not Found","HTTP 401 UNAUTHORISED",MessageBoxButtons.OK,MessageBoxIcon.Error);
        return false;
    }
}
```

The database system I created is both effective and secure because it uses parameterised queries to prevent SQL injection and applies salting and hashing (via SHA-256) to protect user passwords, ensuring that plain-text passwords are never stored. I also chose to implement all of the database logic inside a dedicated class, which encapsulates the database operations into a single, reusable component. This follows OOP principles of encapsulation and abstraction, as the rest of the program can simply call methods like RegisterUser or ValidateLogin without needing to know how the database actually works. This makes the code cleaner, more modular, and easier to maintain. These design decisions provide a strong balance between security, scalability, and maintainability, ensuring the database system is robust.





My Registration and Log In pages use input validation techniques to ensure that all required fields are completed correctly and that the data entered is valid before it is processed. This includes checking for empty fields, enforcing the correct format for usernames and passwords, and preventing duplicate usernames by querying the database in advance. By validating the data at this stage, I reduce the risk of errors, improve the user experience, and strengthen security by ensuring only properly structured data is passed into the system.

```
public partial class LogIn : Form
{
    bool see = false;
    Database db = new Database();
    1 reference
    public LogIn()
    {
        InitializeComponent();
    }

    1 reference
    private void return_but_Click(object sender, EventArgs e)
    {
        HomePage form = new HomePage();
        this.Hide();
        form.Show();
    }

    1 reference
    private void log_in_Click(object sender, EventArgs e)
    {
        bool authenticate = db.ValidateLogin(username.Text, password.Text);
        if (authenticate)
        {
            Choose form = new Choose();
            this.Hide();
            form.Show();
        }
    }

    1 reference
    private void seePassword_Click(object sender, EventArgs e)
    {
        if (see == true)
        {
            seePassword.Text = "👀";
            password.PasswordChar = '*';

            see = false;
        }
        else
        {
            seePassword.Text = "🔓";
            password.PasswordChar = '\0';

            see = true;
        }
    }
}
```

By delegating authentication to the Database class, the code is more modular, reusable, and secure, since all credential handling is abstracted away from the UI. The login process uses the ValidateLogin() method, which provides reliable authentication, while the form simply responds to the feedback. I also added a password visibility toggle (👀 / 🔓), which enhances the user experience by allowing them to check for typing errors while keeping the password hidden by default to maintain security. This however was initially in the opposite order so that the button would show what mode the user was currently in rather than the mode that they could switch to. Following, surveying potential users, I found that they found it unintuitive and preferred it to be the other way around.

```
private void register_Click(object sender, EventArgs e)
{
    name_TextChanged(sender, e);
    username_TextChanged(sender, e);
    password_TextChanged(sender, e);
    password_repeat_TextChanged(sender, e);
    if (valid == true)
    {
        db.RegisterUser(name.Text, username.Text, password.Text);
        Choose form = new Choose();
        this.Hide();
        form.Show();
    }
}

2 references
private void name_TextChanged(object sender, EventArgs e)
{
    if (name.Text.Length == 0)
    {
        errormessage_missing.Visible = true;
        valid = false;
    }
    else
    {
        errormessage_missing.Visible = false;
        valid = true;
    }

}

2 references
private void username_TextChanged(object sender, EventArgs e)
{
    if (username.Text.Length <= 4)
    {
        errormessage_username.Visible = true;
        valid = false;
    }
    else
    {
        errormessage_username.Visible = false;

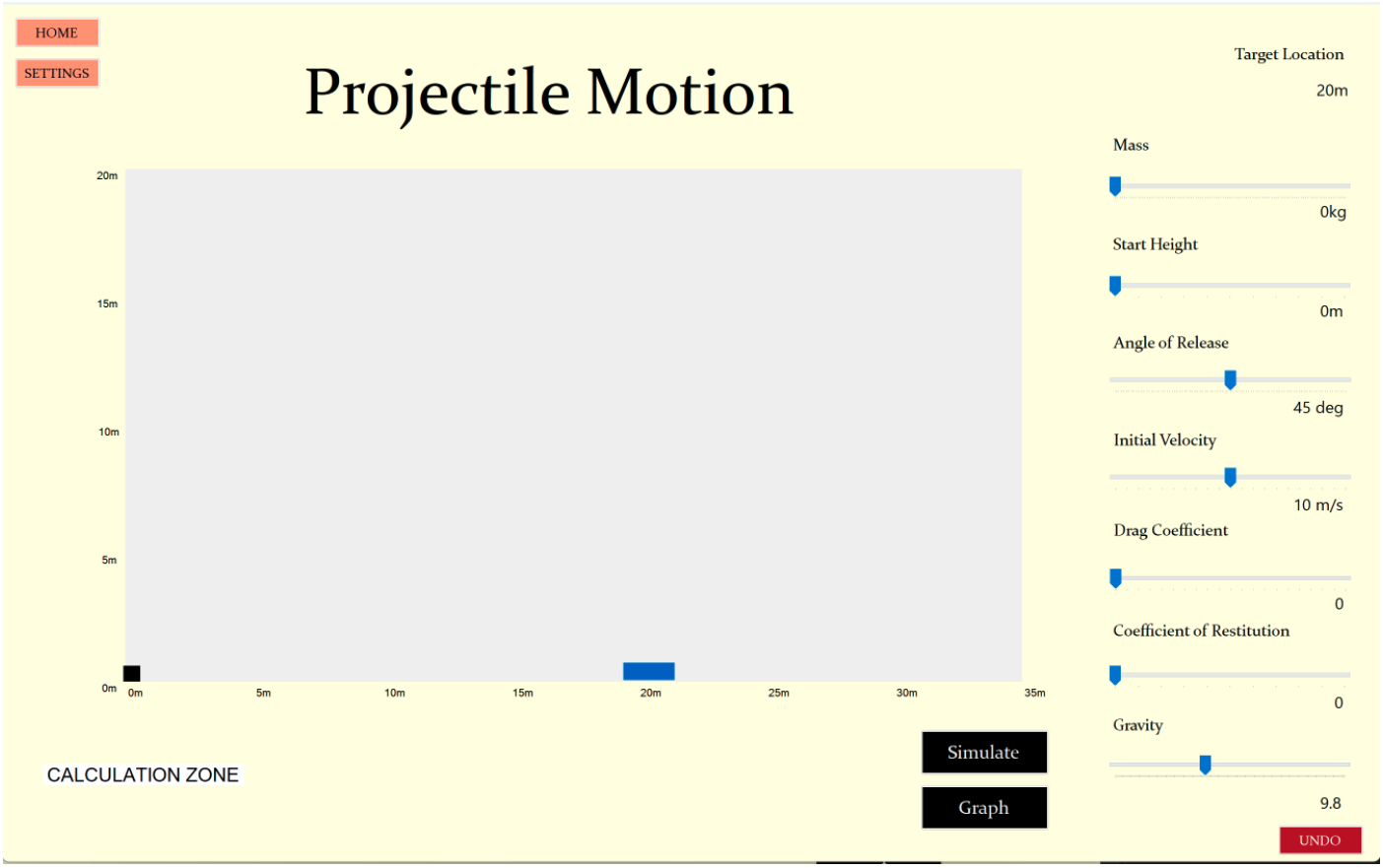
    }
    if (db.UsernameExists(username.Text))
    {
        errormessage_alreadyexists.Visible = true;
        valid = false;
    }
    else
    {
        errormessage_alreadyexists.Visible = false;
        if (username.Text.Length > 4) { valid = true; }

    }
}
```

The registration form uses input validation to ensure that the data meets the required criteria before it is sent to the database. This approach improves data integrity by preventing incomplete or invalid records from being created. I used a shared Boolean variable (valid) to track overall validity, as it provides a simple way to determine if all conditions are met.

My next part of developing this prototype was to make the projectile sandbox, and for that, I needed to decide the dimensions of the sandbox. I decided that my velocity would be from 0 to 20 ms^{-1} and that my angle of release would be from 0° to 90°. This would mean that the maximum vertical height would occur at 90° and the maximum horizontal distance would occur at 45° when the initial velocity is 20 ms^{-1}. This meant that when considering the motion to be in a particle in a vacuum, the maximum distance travelled would be ≈40m and the

maximum height would be ≈20m. However, since I would also like the user to experiment with bounce, I decided that the size of the sandbox would be 20m * 35m. This will allow the user to experiment with bounce whilst still allowing for more standard problems where the walls are not considered.



I initially began with the code below; however, this was flooring all my values instead of letting them remain as floating points and in order to understand what was going on, I used breakpoints to check how the values of COR and projectileCOR were changing. This is where I found that the divide by 10 was leading to the values being floored even though they were initialised as doubles. This was because the division taking place was of integers, so the calculation done is floor division rather than floating point division.

```
private void dragCoefficient_Scroll(object sender, EventArgs e)
{
    projectileDrag = dragCoefficient.Value / 10;
    lbl_drag.Text = projectileDrag.ToString();
}

1 reference
private void COR_Scroll(object sender, EventArgs e)
{
    projectileCOR = COR.Value / 10;
    lbl_cor.Text = projectileCOR.ToString();
}

1 reference
private void trackBar2_Scroll(object sender, EventArgs e)
{
    projectileGravity = gravity.Value / 10;
    lbl_gravity.Text = projectileGravity.ToString();
}
```

In my drag-and-drop algorithm for the target, I found that the target was dropping beneath the container, so in order to remedy this problem, I decided to move the clamping into the mouse move method

```
1 reference
private void target_MouseDown(object sender, MouseEventArgs e)
{
    mouseOffset = e.Location;
}
1 reference
private void target_MouseMove(object sender, MouseEventArgs e)
{
    Point newLocation = target.Parent.PointToClient(Control.MousePosition); //converts the current mouse screen coordinates into the coordinate system of the parent containe
    newLocation.Offset(-mouseOffset.X, -mouseOffset.Y); //adjust new location for offset
    target.Location = newLocation; //move target to this location
}
1 reference
private async void target_MouseUp(object sender, MouseEventArgs e)
{
    if (target.Location.X < 250)
    {
        target.Location = new Point(250, target.Location.Y);
    }
    if (target.Location.X > 1750)
    {
        target.Location = new Point(1700, target.Location.Y);
    }
    int finalY = 965;
    if (target.Location.Y > finalY)
    {
        target.Location = new Point(target.Location.X, 965);
    }
    while (target.Location.Y < finalY)
    {
        target.Location = new Point(target.Location.X, target.Location.Y + 5);
        await Task.Delay(10);
    }
    target.Location = new Point(target.Location.X, 965);
    lbl_target.Text = ((target.Location.X - 50) / 50).ToString() + "m";

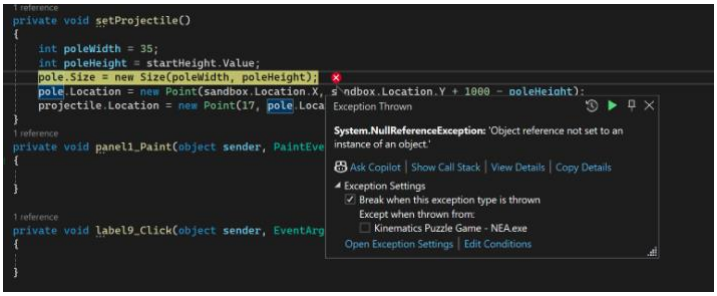

}
```

```
private void target_MouseMove(object sender, MouseEventArgs e)
{
    Point newLocation = target.Parent.PointToClient(Control.MousePosition); //converts the current mouse screen coordinates into the coordinate system of the parent co
    newLocation.Offset(-mouseOffset.X, -mouseOffset.Y); //adjust new location for offset
                                                        // Clamp X
    if (newLocation.X < 250) { newLocation.X = 250; }
    if (newLocation.X > 1700) {newLocation.X = 1700; }

    // Clamp Y
    if (newLocation.Y > 965) { newLocation.Y = 965; }
    target.Location = newLocation; //move target to this location
}
1 reference
private async void target_MouseUp(object sender, MouseEventArgs e)
{
    int finalY = 965;
    if (target.Location.Y > finalY)
    {
        target.Location = new Point(target.Location.X, 965);
    }
    while (target.Location.Y < finalY)
    {
        target.Location = new Point(target.Location.X, target.Location.Y + 5);
        await Task.Delay(10);
    }
    target.Location = new Point(target.Location.X, 965);
    lbl_target.Text = ((target.Location.X - 50) / 50).ToString() + "m";


}
```

While attempting to create a pole for the projectile to sit on, with a height that adjusts according to the height slider, I encountered an error. This was caused by the pole variable being shadowed within the method, meaning the class-level reference remained null and could not be accessed elsewhere in the code.



```
private void initialProjectile()
{
    GraphicsPath path = new GraphicsPath();
    path.AddEllipse(0, 0, 35, 35);
    projectile.Region = new Region(path);
    sandbox.Controls.Add(target);
    int poleWidth = 35;
    int poleHeight = startHeight.Value;
    pole = new Panel
    {
        Size = new Size(poleWidth, poleHeight),
        Location = new Point(sandbox.Location.X, sandbox.Location.Y + 1000 - poleHeight),
        BackColor = Color.BurlyWood,

    };
    sandbox.Controls.Add(pole);
}
```

```
1 reference
private void setProjectile()
{
    int poleWidth = 35;
    int poleHeight = startHeight.Value;
    pole.Size = new Size(poleWidth, poleHeight);

    // keep pole at bottom of sandbox
    pole.Location = new Point(0, sandbox.Height - poleHeight);

    // position projectile on top, centered
    projectile.Location = new Point(
        pole.Location.X + (pole.Width - projectile.Width) / 2,
        pole.Location.Y - projectile.Height
    );
}
```

This resolved the error; however, my code was still unable to create and modify the pole correctly. I was able to make the pole visible to the user by incorporating a conversion factor of 1 m = 50 px, but the projectile still did not appear in the correct position on top of the pole.

```
private void setProjectile()
{
    int poleWidth = 35;
    int poleHeight = startHeight.Value * 50;
    pole.Size = new Size(poleWidth, poleHeight);

    // keep pole at bottom of sandbox
    pole.Location = new Point(0, sandbox.Height - pole.Size.Height);

    // position projectile on top, centered
    projectile.Location = new Point(
        pole.Location.X + (pole.Width - projectile.Width) / 2,
        pole.Location.Y - projectile.Height
    );
    projectile.BringToFront();
}
```

The issue with the projectile not appearing correctly on top of the pole was caused by it not being added as a child of the sandbox panel, meaning its coordinates were being calculated relative to the wrong container. This misalignment caused both the X and Y positions to be incorrect. The problem was fixed by adding the projectile to the sandbox panel using sandbox.Controls.Add(projectile), which ensured that its position became relative to the pole. As a result, the projectile now appears correctly on top of the pole, and any changes to the pole's height are reflected accurately in the projectile's position.

The PhysicsEngine calculates the projectile's motion using time steps and incorporates gravity, drag, friction, and collisions to produce realistic behaviour. Trajectory tracking enables the path to be visualised and analysed, while coefficients of restitution and the epsilon threshold ensure accurate bouncing and prevent unnecessary motion once the projectile comes to a stop. Placing the calculations within a dedicated class separates the physics from the user interface, making the code easier to maintain and allowing dynamic

adjustments to projectile properties. This approach enhances both the realism and flexibility of the simulation, yielding a more accurate and engaging experience for the user.

```
internal class PhysicsEngine
{
    private double x, y;
    private double vx,vy;
    private double drag, eFloor, eWall, muFloor, gravity, epsilon;
    private int targetX;
    private int mass;
    private List<(double time, double x, double y)> trajectory;

    0 references
    public PhysicsEngine(int initialHeight, double dragCof, double CORfloor, double CORwall,double mu, double grav, int target, int projectileMass)
    {
        x = 0;
        y = initialHeight;
        vx = 0; vy = 0;
        drag = dragCof;
        eFloor = CORfloor;
        eWall = CORwall;
        muFloor = mu;
        gravity = grav;
        epsilon = 0.001;
        targetX = target;
        mass = projectileMass;
    }

    0 references
    public void SetProperties(int initialHeight, double dragCof, double CORfloor, double CORwall, double mu, double grav, int target, int projectileMass)
    {
        y = initialHeight;
        drag = dragCof;
        eFloor = CORfloor;
        eWall = CORwall;
        muFloor = mu;
        gravity = grav;
        targetX = target;
        mass = projectileMass;
    }

    0 references
    public void Reset()
    {
        x = 0;
        y = 0;
        vx = 0;
        vy = 0;
        drag = 0;
        eFloor = 0;
        eWall = 0;
        muFloor = 0;
        gravity = 0;
        targetX = 0;
        mass = 0;
        trajectory.Clear();
```

```
private string Update(double dt, int height, int width, double currentTime)
{
    double dragX = drag * vx;// drag forces
    double dragY = drag * vy;

    double ax = -dragX / mass; //acceleration
    double ay = -gravity - (dragY / mass);

    x += vx * dt; //update position
    y += vy * dt;

    if (y <= 0)
    {
        y = 0;
        if (CheckHit())
        {
            trajectory.Add((currentTime, x, y));
            return "hit";
        }
        vy = -vy * eFloor;
        vx = vx * (1 - muFloor);
    }
    if (y>= height) //reached ceiling
    {
        y = height;
        vy = -vy * eWall;
    }
    if (x <= 0)
    {
        x = 0;
        vx = -vx * eWall;
    }
    if (x >= width)
    {
        x = width;
        vx = -vx * eWall;
    }
    if (Math.Abs(vx) < epsilon && Math.Abs(vy) < epsilon && y == 0)
    {
        vx = 0;
        vy = 0;
        return "stopped";
    }
    trajectory.Add((currentTime, x, y));
    return "moving";
}
```

```
1 reference
private bool CheckHit()
{
    if (targetX <= x && x <= targetX + 2)
    {
        return true;
    }
    else
    {
        return false;
    }
}
```

```
1 reference
private async Task AnimateProjectile()
{
    List<(double time, double x, double y)> Trajectory = physics.GetAnimationTrajectory(sandbox.Height, sandbox.Width);
    foreach (var point in Trajectory)
    {
        projectile.Location = new Point((int)point.x, (int)point.y);
        projectile.BringToFront();
        int delay = (int)(100 * (1.0 / simSpeed));
        await Task.Delay(delay);
    }

}
1 reference
private async void Simulate_Click(object sender, EventArgs e)
{
    foreach (Control ctrl in this.Controls) //disable all the sliders so no furhter changes can be made
    {
        if (ctrl is TrackBar)
        {
            ctrl.Enabled = false;
        }
    }
    target.MouseDown -= target_MouseDown; //stop drag and drop functionality
    target.MouseMove -= target_MouseMove;
    target.MouseUp -= target_MouseUp;

    physics.Reset();
    physics.SetProperties(projectileStartHeight,projectileDrag,projectileCOR,0.5,projectileCOF,projectileGravity,target.Location.X,projectileMass);
    double time = 0.0;
    while (true)
    {
        string status = physics.Update(deltaTime, sandbox.Width / 50, sandbox.Height / 50, time);
        if (status == "hit" || status == "stopped")
        {
            if(status == "hit")
            {
                MessageBox.Show("Target Hit");
            }
            break;
        }
        time += deltaTime;
    }
    await AnimateProjectile();
}

1 reference
private void speed_Scroll(object sender, ScrollEventArgs e)
{
    simSpeed = speed.Value / 10.0;
    lbl_speed.Text = simSpeed.ToString() + "x";
}
```

When the simulation starts, all sliders are temporarily disabled and the target's drag-and-drop functionality is removed so the user can't make changes while the projectile is moving. The physics engine is then reset and set up with the current projectile parameters, including height, drag, friction, gravity, mass, and target position. The simulation calculates the projectile's motion in small time steps until it either hits the target or comes to a stop. Once the calculations are done, the AnimateProjectile method moves the projectile along its recorded trajectory on the screen, with the speed adjustable through the simSpeed setting. This approach keeps the simulation consistent, realistic, and easy for the user to follow, while still allowing them to experiment with different speeds.

```
1 reference
public string Update(double dt, int height, int width, double currentTime) //height and width in metres
{
    double dragX = drag * vx;// drag forces
    double dragY = drag * vy;

    double ax = -dragX / mass; //acceleration
    double ay = -gravity - (dragY / mass);

    vx += ax * dt;
    vy += ay * dt;


    x += vx * dt; //update position
    y += vy * dt;

    if (y <= 0)
    {
        y = 0;
        if (CheckHit())
        {
            trajectory.Add((currentTime, x, y));
            return "hit";
        }
        vy = -vy * eFloor;
        vx = vx * (1 - muFloor);
    }
    if (y >= height) //reached ceiling
    {
        y = height;
        vy = -vy * eWall;
    }
    if (x <= 0)
    {
        x = 0;
        vx = -vx * eWall;
    }
    if (x >= width)
    {
        x = width;
        vx = -vx * eWall;
    }
    if (((Math.Abs(vx) < epsilon || Math.Abs(vy) < epsilon) && y <= 0.1) || (y == 0  && eFloor == 0))
    {
        vx = 0;
        vy = 0;
        y = 0;
        return "stopped";
    }
    trajectory.Add((currentTime, x, y));
    return "moving";
}
```

The stop condition was loosened to ensure the simulation terminates even if only one of the velocities is effectively zero, preventing the projectile from continuing indefinitely due to tiny floating-point values and avoiding memory overflow.

```
    List<(double time, double x, double y)> trajectory = physics.GetTrajectory();
    List<double> Vx = new List<double>();
    List<double> Vy = new List<double>();
    List<double> V = new List<double>();
    List<double> Ax = new List<double>();
    List<double> Ay = new List<double>();
    List<double> A = new List<double>();
    List<double> timelist = new List<double>();
    chart1.Visible = true;
    for (int i = 0; i < trajectory.Count - 1; i++)
    {
        double Vxi = (trajectory[i + 1].x - trajectory[i].x) / deltaTime;
        double Vyi = (trajectory[i + 1].y - trajectory[i].y) / deltaTime;
        double Vi = Math.Sqrt(Vxi * Vxi + Vyi * Vyi);
        timelist.Add(trajectory[i].time);
        Vx.Add(Vxi);
        Vy.Add(Vyi);
        V.Add(Vi);

    }
    for (int i = 0; i < V.Count - 1; i++)
    {
        double Axi = (Vx[i + 1] - Vx[i]) / deltaTime;
        double Ayi = (Vy[i + 1] - Vy[i]) / deltaTime;
        double Ai = Math.Sqrt(Axi * Axi + Ayi * Ayi);
        Ax.Add(Axi);
        Ay.Add(Ayi);
        A.Add(Ai);
    }


    for (int i = 0; i < timelist.Count - 1; i++)
    {
        VxSeries.Points.AddXY(timelist[i], Vx[i]);
        VySeries.Points.AddXY(timelist[i], Vy[i]);
        VSeries.Points.AddXY(timelist[i], V[i]);
        AxSeries.Points.AddXY(timelist[i], Ax[i]);
        AySeries.Points.AddXY(timelist[i], Ay[i]);
        ASeries.Points.AddXY(timelist[i], A[i]);

    }

    foreach (Control ctrl in this.Controls)
    {
        if (ctrl is RadioButton)
        {
            ctrl.Visible = true;
        }
    }
}
else
{
```

The code derives the horizontal and vertical velocity at each step and by differentiating this, it can calculate the acceleration. All of these quantities are stored in lists and then plotted onto a chart allowing the user to select which lines they would like to see.

```
private void CalculationZone()
{
    if (projectileCOR == 0)
    {
        Calculation.Text = $"vx = {projectileU} * cos({projectileAngle})    vy = {projectileU} * sin({projectileAngle})\n";
        int verticalDisplacement = 0 - projectileStartHeight;
        double angleRadians = projectileAngle * Math.PI / 180.0;
        double vx = Math.Cos(angleRadians) * projectileU;
        double vy = Math.Sin(angleRadians) * projectileU;
        double t = (-vy + Math.Sqrt(vy * vy + 2 * projectileGravity * verticalDisplacement)) / projectileGravity;
        Calculation.Text += $"{verticalDisplacement} = {vy}t - 1/2 * {projectileGravity}t^2          t = {t}\n";
        Calculation.Text += $"horizontal distance = {vx} * {t} = {vx * t}";
        if (projectileDrag != 0)
        {
            Calculation.Text += "This calculation has considered no external factors";
        }
    }
    else
    {
        Calculation.Text = "Calculation Cannot be shown";
    }
```

I chose to provide a simpler version of the physics as it would not be possible to fit all the calculations on screen, and considering any external factors is not necessary for GCSE/A level maths and physics which is what this solution is targeted towards. In order to fix the quadratic, I decided to split it into separate steps.

```
private void CalculationZone()
{
    if (projectileCOR == 0)
    {
        Calculation.Text = $"vx = {projectileU} * cos({projectileAngle})    vy = {projectileU} * sin({projectileAngle})\n";
        int verticalDisplacement = 0 - projectileStartHeight;
        double angleRadians = projectileAngle * Math.PI / 180.0;
        double vx = Math.Cos(angleRadians) * projectileU;
        double vy = Math.Sin(angleRadians) * projectileU;
        double discriminant = vy * vy - 2 * projectileGravity * verticalDisplacement;
        double t = (-vy + Math.Sqrt(discriminant)) / (-projectileGravity);
        Calculation.Text += $"{verticalDisplacement} = {vy}t - 1/2 * {projectileGravity}t^2          t = {t}\n";
        Calculation.Text += $"horizontal distance = {vx} * {t} = {vx * t}";
        if (projectileDrag != 0)
        {
            Calculation.Text += "⚠ Note: This calculation ignores air resistance and other external factors.";
        }
    }
    else
    {
        Calculation.Text = "Calculation Cannot be shown";
    }
}
```

The final feature I need to implement for prototype 1 is the undo feature. I decided to use an object array as it allows me to store multiple data types.

Testing

Input	Testing	Justification	Expected Result	Result
Enter valid new username password	Log In / Registration	Tests account creation and storage which is essential for the rest of the solution	Account is created and saved, the user is let into the solution and can log in using this details next time	See Figure 5.1 **SUCCESS**
Enter Invalid Username/Password	Log In / Registration	Tests input validation	Error message shown; account not created	See Figure 5.2 **SUCCESS**

Launch Projectile from height 0m **Angle 45°** **Velocity 10ms^{-1}** **Drag = 0** **E = 0** **Gravity = 9.81ms^{-2}**	Projectile Motion	Test basic 2d projectile motion to ensure it follows physics accurately	Parabolic trajectory. Lands at distance of ≈ 10.2m	**SUCCESS**
Launch Projectile from height 5m **Angle 30°** **Velocity 15ms^{-1}** **Drag = 0.1** **E = 0** **Gravity = 9.81ms^{-2}**	Projectile Motion	Tests projectile motion with initial height and air resistance to show effect of air resistance	Projectile slows slightly and will land just short of ≈26m	See Figure 5.3 **SUCCESS**
Launch projectiles with varying coefficient of restitution (e)	Projectile Motion	To test bounce behaviour and simulate energy loss	Projectile bounces with height = previous height × coR; stops eventually if coR < 1	**PARTIAL SUCCESS** Energy loss and bounce occur however not as expected
Move Target	Projectile Motion	Test the system recognises where the target is	Visual UI change as well as the system recognising if the target has been hit	**SUCCESS**
Undo Last Action	Projectile Motion	Tests undo stack	Projectile state reverts to previous state with variables restored	**SUCCESS**
Test graphing capability with Vx, Vy and $A_{magnitude}$	Projectile Motion	Tests that position values are recorded and plotted over time	Multiple lines in different colours should appear on the graph	See Figure 5.4 **SUCCESS**

	UserID	Name	Username	Salt	HashedPassword	IsNewUser
1	3	Pranav	pran23	WPrNGbXWetxYKAzPwY19uQ==	EExZK/aFozexrCzwG0eM7pz8Fiupz0jBkXFMTGJ/t18=	1
2	5	testAccount	testing	ciGdO2yOOLV9FmV/AanU6w==	nPUhr0qna2XjG7RCRJLfeCRjgI6fi1QSldey5RmoveQ=	1

Figure 5. 1

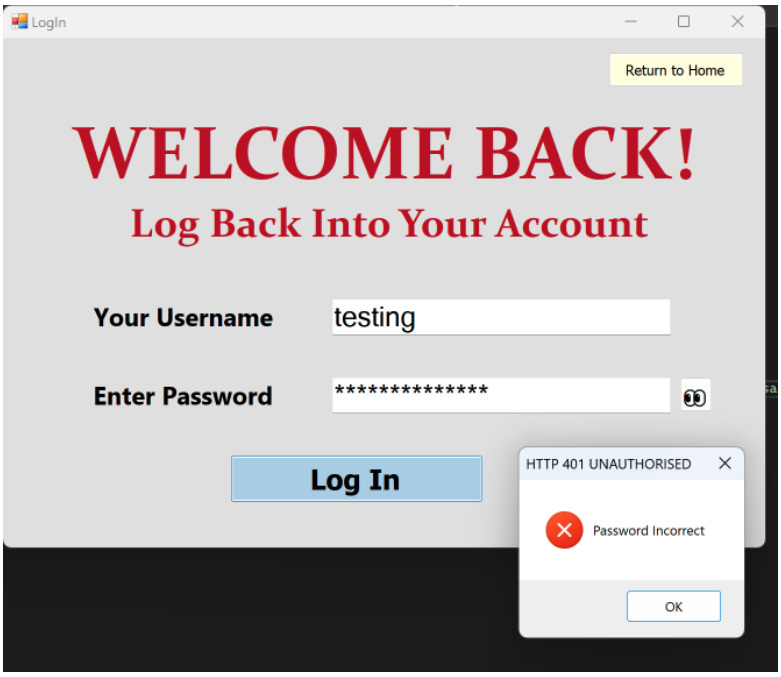


Figure 5. 2

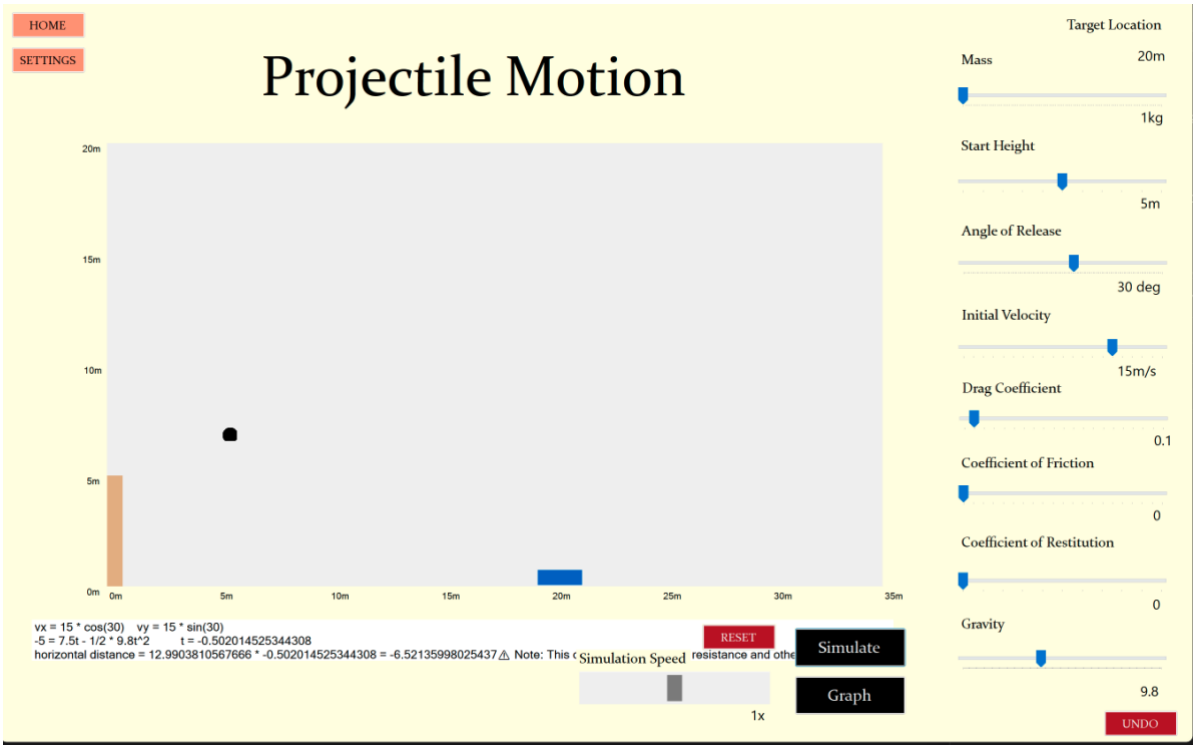


Figure 5. 3

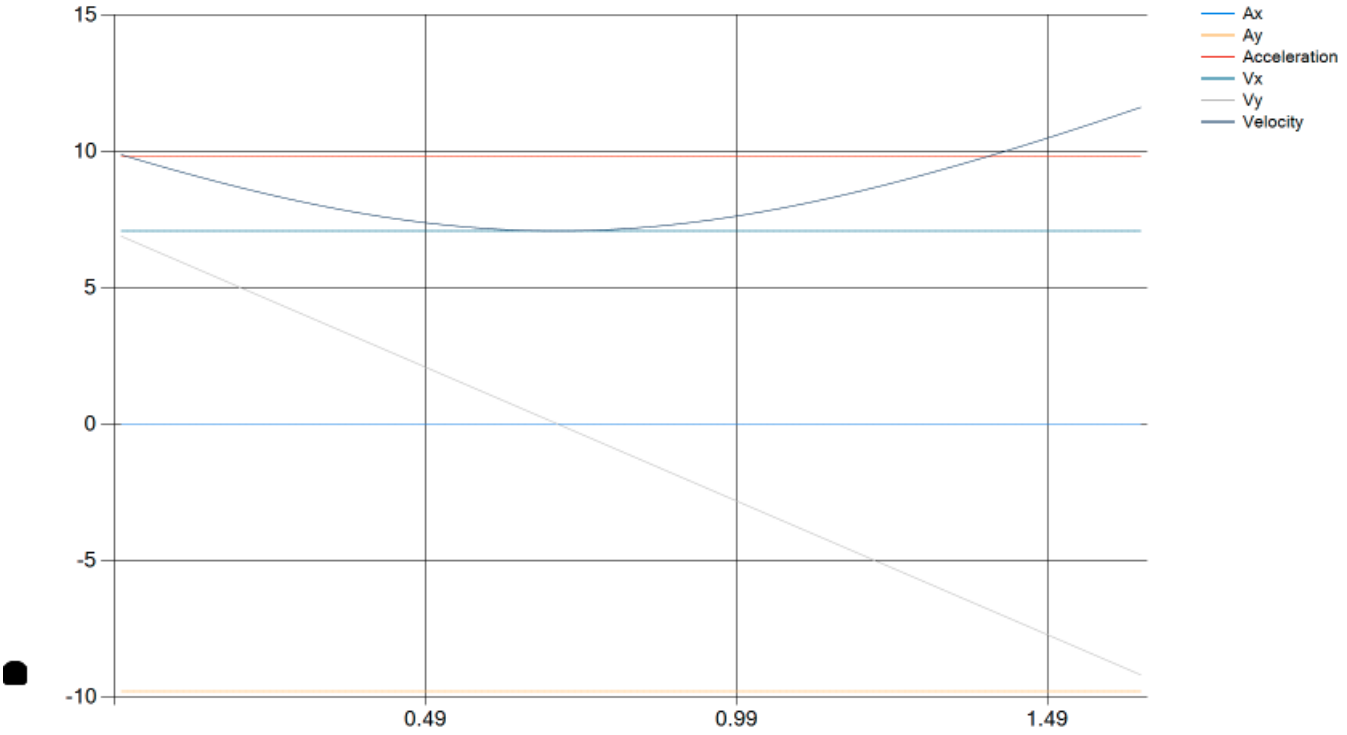


Figure 5. 4

Remedial Action

The partially failed test occurred due to the delta time being too large, which has since been fixed. All the other tests were successful, so no further action is required

Validation

As the UI mainly uses sliders, there is little extra validation necessary. The validation needed is in the login system and that has been handled through checking for duplicates and length checks.

Review

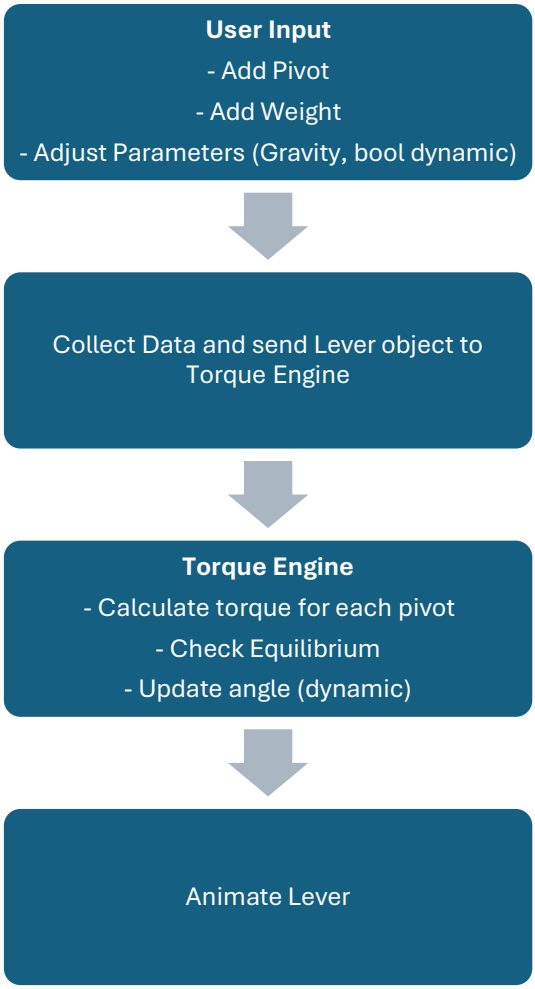
Following testing I can conclude that I was successfully able to implement

- Log In / registration system
- Projectile Motion Sandbox

In future prototypes I will ensure that the buttons to show/hide individual graphs works as expected rather than just showing all of the graphs and improving the calculation zone to correctly show results of calculation.

Prototype 2 – Torque Sandbox

The second prototype will focus on developing the torque sandbox to expand the simulation beyond projectile motion. Users will be able to experiment with parameters such as mass and pivot points. By first developing this in a sandbox, the complexity of the puzzle is avoided, keeping the scope manageable. It also allows me to focus on the core calculations and animation systems, ensuring they are thoroughly tested before being used in the puzzles. The Torque Sandbox Form will handle user interaction and animation, while a separate TorqueEngine class will perform all physics calculations. This separation ensures that the simulation is accurate and responsive, while keeping the form implementation simple and manageable.



Class Diagrams

I initially considered implementing this a child class to the Physics Engine Class from Prototype 1 however the calculations and values required were different with no overlap so it would not have any benefits which is why I ultimately decided to build it as a separate class as seen below

TourqueEngine
Key Attributes
`Double length`
`List<double> Pivots`
`List<(double mass, double position)> Weights`
`Double angle`
`Double gravity`
Int leverMass
`Double angularAcceleration`

Double angularVelocity = 0 Double angularPosition = 0 Bool tipping Double dt
Key Methods CalculateTorque() Update() CheckEquilibrium() AddWeight() RemoveWeight() AddPivot() RemovePivot() Reset() Getters

Algorithms

Add/Remove Weight

The weight is stored as a tuple in a list, and this algorithm will add/remove those tuples from the list that show what weights are acting on the lever. In this algorithm, the position will have to be given in metres to ensure that other calculations can work accordingly; however, in the UI, this will be abstracted to Positions 1-8 with the user being able to choose the weight through sliders, as it will allow for more customisation and the user to build problems that better suit their needs. A similar algorithm will also be used for adding and remove pivots.

```
AddWeight(mass, position)
      Weights.add((mass,position))

RemoveWeight(position)
      Count = 0
      FOREACH weight in Weights
            IF weight.position == position
                  Break
            Count += 1
      Weights.RemoveAt(count)
```

Reset Lever

Clears all weights and pivots and resets the angle and angular velocity to 0. This allows for repeated experimentation without having to call multiple instances of the same class, which would increase memory usage.

Calculate Torque

This algorithm calculates the total torque that has been applied to the lever. I have chosen to use recursion in this algorithm as it mirrors the definition of torque, which is “total moment = moment of first force + moment of second force …”.

```
CalculateTorque(PivotPosition, index)
      IF index > Weights.Count
            Return 0
      Distance = Weights[index].position – PivotPosition
      Force = Weights[index].mass * gravity
      Moment = Distance * Force
      Return Moment + CalculateTorque(PivotPosition, index + 1)
```

Update()

The update function is what the user would call to allow the simulation to begin. The user will then repeatedly call the update tipping until it returns false and redraw the lever so that it represents the values returned

```
Update(angularVelocity,anglularPosition, dt)
      Weights.Add((leverMass, length/2))
      Torque = CalculateTorque(Pivots[0],0)
      ReactionPivot2 = Torque/Math.Abs(Pivots[1] – Pivot[0])
      DownwardForce = 0
      FOREACH weight in Weights
            DownwardForce += weight.mass * 9.8
      ReactionPivot1 = DownwardForce – ReactionPivot2
      IF ReactionPivot2 < 0
            TippingMovement(1)
      ELSE IF ReactionPivot1 < 0
            TippingMovement(0)
```

```
TippingMovement(index)
      Torque = CalculateTorque(Pivots[index],0)
      MomentofInertia = 0.083 * length * Math.Pow(0.2,3)
      angularAcceleration = Torque/MomentofInertia
      dt = 0
      tipping = true

UpdateTipping()
      IF tipping
            while Math.Abs(angularPosition * 180/Math.Pi) < 45
                  angularVelocity += angularAcceleration * dt
                  anglularPosition += angularVelocityNew * dt
                  dt += 0.01
                  return true
      return false
```

Animation

The purpose of RedrawLever is to visually represent the current state of the lever and weights on the WinForms form at each frame. It takes the lever angle and the list of weights as inputs and draws them accordingly. I chose to use a redraw function with a timer to ensure smooth animation without freezing the UI.

```
RedrawLever(active, pivotPoints, leverLength, angle, weights,pivotOffset)
      activePivotIndex = pivotPoints[active]
      FOREACH pivot IN pivots
            Graphics.FillEclipse(color = Blue, centre(pivot.x,pivot.y), radius=5)
      leftEndX  = pivot.X - pivotOffset * COS(angle)
      leftEndY  = pivot.Y + pivotOffset * SIN(angle)
      rightEndX = pivot.X + (leverLength - pivotOffset) * COS(angle)
      rightEndY = pivot.Y - (leverLength - pivotOffset) * SIN(angle)
      FOR EACH weight IN weights
          distanceFromPivot = weight.position - pivotOffset
          weightX = pivot.X + distanceFromPivot * COS(angle)
          weightY = pivot.Y - distanceFromPivot * SIN(angle)
```

Usability Features

This prototype will use sliders and buttons for the user to input data, as it allows for easier validation and a lower chance of error. The user will be given the choice of 10 different points along the 20m lever where they can use a slider to place the amount of mass they would like or a button to add a pivot there of which they can only have a maximum of 2.

Key Variables

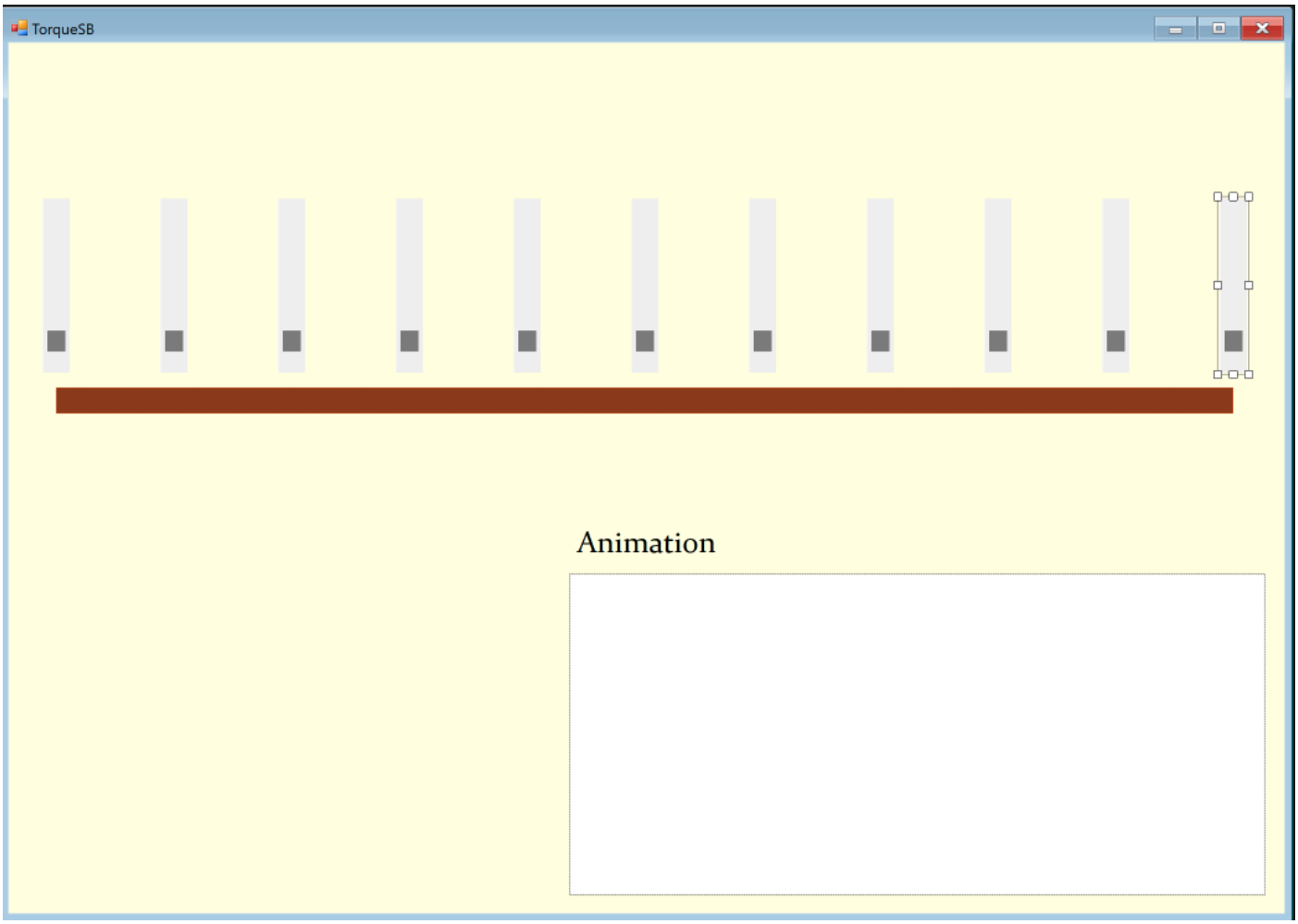
Variable Name	Data Type	Description	Justification
length	double	Length of the lever which will be 20m in this prototype	Required for rendering the simulation.
Pivots	List<double>	A list of the positions of the pivots for calculations	Needed to calculate reaction forces, torque, and tipping behaviour.
Weights	List<(double mass, double position)>	List of all weights with mass and position along the lever	Required to calculate torque, downward force and tipping effects
gravity	Double	Acceleration due to gravity Initially set to 9.8 but the user will be able to change this	Used to calculate the forces from the weights
LeverMass	Integer	Mass of the lever	Needed to include lever weight in torque and downward force calculations
angularAcceleration	Double	Angular acceleration of the lever	Used to update angular velocity and angle over time
angularVelocity	Double	Current angular velocity of the lever	Updated each frame to calculate new angle
angularPosition	Double	Current angular position of the lever	Tracks rotation for physics and rendering
tipping	Boolean	Indicates if the lever would tip with the masses placed on it	Controls whether tipping calculations and animation updates should run
dt	Double	Delta time – time elapsed in calculation	Needed for numerical integration of motion
DownwardForce	Integer	Total downward force from all weights and the lever	Required to calculate reaction forces at pivots
pivotOffset	Integer	Distance from the lever end to pivot 1	Needed to correctly calculate lever end positions for rendering the simulation

Testing

Input	Testing	Justification	Expected Result
Two equal weights on opposite sides of a single pivot	Equilibrium	Checks that the torque and reaction forces are measured correctly	Lever stays balanced
Four unequal weights on opposite sides of a single pivot	Single Pivot	Tests basic torque calculation and tipping with a single pivot	Lever Rotates
Place one weight on a double pivot system	Double Pivot	Checks that tipping occurs correctly around the pivot experiencing negative reaction force	Lever tips around pivot1 if ReactionPivot2 < 0
Place three weights on a double pivot system	Double Pivot	Checks that the calculations are	Lever tips up to ±45° depending on weight location
Place 10 weights and 2 pivots	Boundary Test	Check that the system can handle the maximum capacity of weight positions and pivots	Lever tips up to ±45° depending on weight location
Gravity = 0	Zero Gravity	Tests effects of zero gravity.	Lever remains static, no tipping or rotation
Place more than 2 pivots	Validation	The calculations only allow for a maximum of 2 pivots so the front end should ensure that no more than 2 pivots are selected	Error Message

Development

I initially planned to have 10 discrete positions for the masses and pivots to act on a 20m lever; however, when setting up the environment, I realised that using 10 whilst ensuring that the first acts at 0m and the last acts at 20m would result in non-integer values for distance and would also result in the lack of a middle point where a pivot or mass could act. I have chosen to use vertical scroll bars to allow the user to input mass; however, since they have the maximum value at the bottom, I will have to flip the value before it is used or displayed.



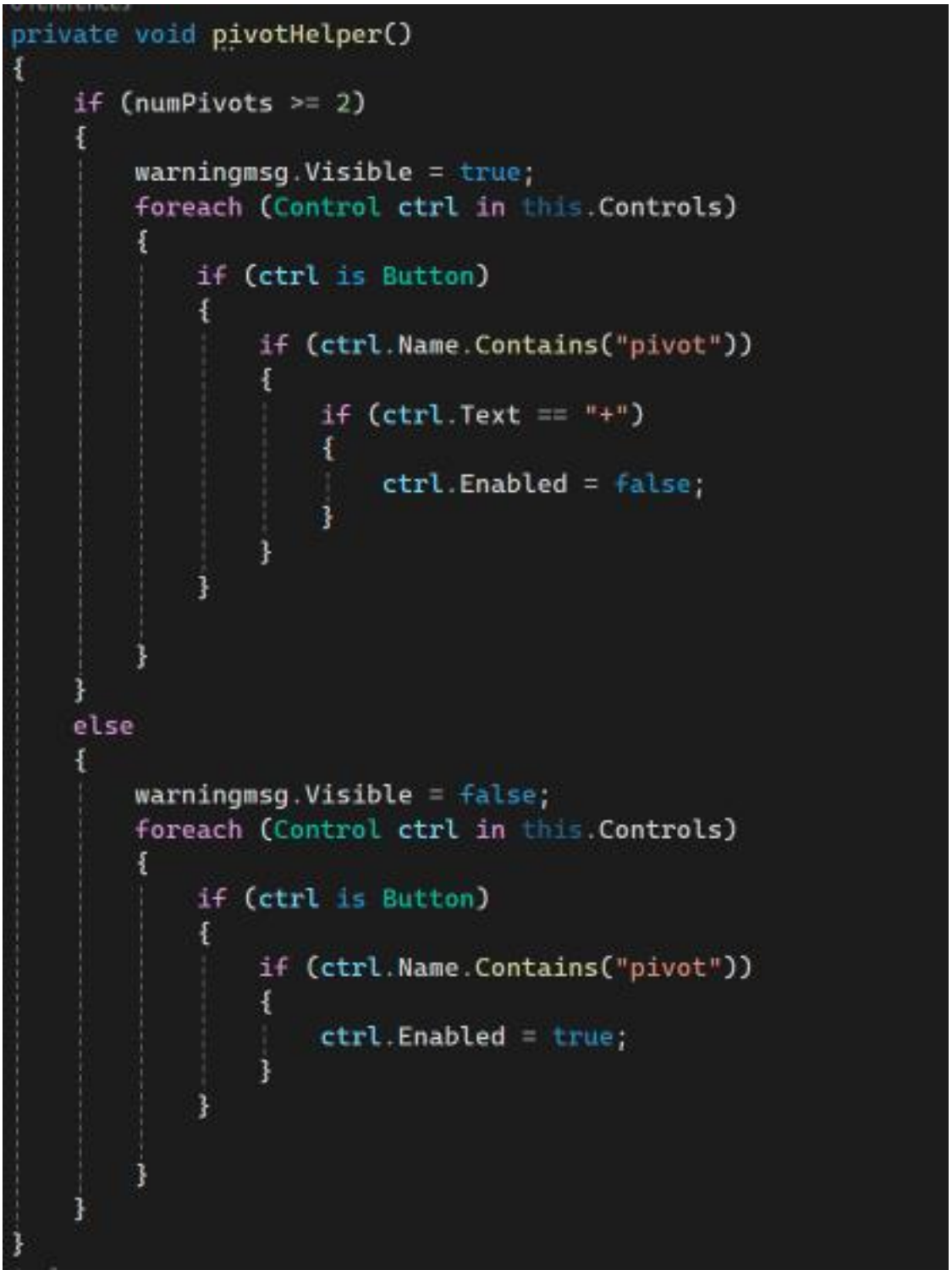
The following algorithm draws a pivot at the point the user selects. It keeps track of the pivot points through a list and draws all the pivots in that list. By using OnPaint, the application ensures that pivots persist even when the form is resized, minimised, or covered by another window. This approach is more reliable than drawing directly in the button click event using CreateGraphics(), which would result in pivots disappearing if the form were refreshed.

```
private void DrawPivot(Graphics g, Point top)
{
    int halfWidth = 25;
    Point leftBase = new Point(top.X - halfWidth, top.Y + 70); //70 is height of pivot
    Point rightBase = new Point(top.X + halfWidth, top.Y + 70);
    Point[] trianglePoints = { top, leftBase, rightBase };
    g.DrawPolygon(Pens.Black, trianglePoints);
    g.FillPolygon(Brushes.LightBlue, trianglePoints);
}

0 references
protected override void OnPaint (PaintEventArgs e)
{
    base.OnPaint(e);
    Graphics g = e.Graphics;
    foreach (Point top in pivotTops)
    {
        DrawPivot(g, top);
    }
}
```

```
private void pivot2_Click(object sender, EventArgs e)
{
    if (!p2)
    {
        Point top = new Point(pivot2.Location.X + 22, 599);
        pivotTops.Add(top);
        pivot2.BackColor = Color.LightSalmon;
        pivot2.Text = "-";
        this.Invalidate();
        p2 = true;
    }
    else
    {
        Point top = new Point(pivot11.Location.X + 22, 599);
        pivotTops.Remove(top);
        pivot2.BackColor = SystemColors.GradientInactiveCaption;
        pivot2.Text = "+";
        this.Invalidate();
        p2 = false;
    }
}
```

To ensure that only a maximum of two pivots are operating at once, I created a helper method to check and disable the button to add more pivots until there are fewer than two. A warning message will also show so that the user knows what has happened.



```
private void pivotHelper()
{
    if (numPivots >= 2)
    {
        warningmsg.Visible = true;
        foreach (Control ctrl in this.Controls)
        {
            if (ctrl is Button)
            {
                if (ctrl.Name.Contains("pivot"))
                {
                    if (ctrl.Text == "+")
                    {
                        ctrl.Enabled = false;
                    }
                }
            }

        }
    }
    else
    {
        warningmsg.Visible = false;
        foreach (Control ctrl in this.Controls)
        {
            if (ctrl is Button)
            {
                if (ctrl.Name.Contains("pivot"))
                {
                    ctrl.Enabled = true;
                }
            }

        }
    }
}
```

When trying to build the undo stack, I found that refactoring the pivot code significantly simplifies implementing an effective Undo system. Originally, each pivot had its own Boolean flag and individual click handler, meaning that reversing a user action would require writing separate logic for each of the eleven pivots, which is repetitive and error-prone. By abstracting the pivot state into a single HashSet, which I chose to use to avoid duplicate entries and using one click handler, it allows each pivot placement or removal to be recorded consistently on an Undo stack, making it possible to reverse the last action with a single, uniform block of code. This approach reduces code duplication and increases maintainability.

```
public TorqueSB()
{
    InitializeComponent();
    pivot1.Click += PivotClick;
    pivot2.Click += PivotClick;
    pivot3.Click += PivotClick;
    pivot4.Click += PivotClick;
    pivot5.Click += PivotClick;
    pivot6.Click += PivotClick;
    pivot7.Click += PivotClick;
    pivot8.Click += PivotClick;
    pivot9.Click += PivotClick;
    pivot10.Click += PivotClick;
    pivot11.Click += PivotClick;
}


11 references
private void PivotClick(object sender, EventArgs e)
{
    Button pivotButton = sender as Button;
    int index = int.Parse(pivotButton.Name.Replace("pivot", ""));
    Point top = new Point(pivotButton.Location.X + 22, 599);
    bool exists = PlacedPivots.Contains(index); //check if a pivot already exists at that location

    if (!exists)
    {
        pivotTops.Add(top);
        pivotButton.BackColor = Color.LightSalmon;
        pivotButton.Text = "-";

    }
    else
    {
        pivotTops.Remove(top);
        pivotButton.BackColor = SystemColors.GradientInactiveCaption;
        pivotButton.Text = "+";
    }
    pivotHelper();
    this.Invalidate();
}
```

In order to improve the code from prototype 1, I decided to build a class where each object will hold a change in state.

```
1 reference
internal class State
{
    1 reference
    public string Type { get; set; }
    1 reference
    public int Index { get; set; }
    1 reference
    public object OldValue { get; set; }

    0 references
    public State(string type, int index, object oldValue)
    {
        Type = type;
        Index = index;
        OldValue = oldValue;
    }
}
```

```
private void MassScroll(object sender, EventArgs e)
{
    VScrollBar slider = sender as VScrollBar;
    int index = int.Parse(slider.Name.Replace("mass", ""));
    Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
    int oldValue = int.Parse(lbl.Text);
    lbl.Text = (slider.Maximum - slider.Value).ToString() + "kg"; // scroll bar has maximum at bottom so to make it more intuitive this method is being used to flip the values
    State state = new State("mass", index, oldValue);
}
```

```
1 reference
private void undo_Click(object sender, EventArgs e)
{
    State top = StateStack.Pop();
    string type = top.Type;
    if (type == "gravity")
    {
        double oldValue = Convert.ToDouble(top.OldValue);
        gravity.Value = Convert.ToInt32(oldValue * 10);
        lbl_gravity.Text = oldValue.ToString();
    }
    if (type == "mass")
    {
        int oldValue = Convert.ToInt32(top.OldValue);
        int index = top.Index;
        Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
        VScrollBar slider = this.Controls.Find("mass" + index, true).FirstOrDefault() as VScrollBar;
        slider.Value = oldValue;
        lbl.Text = oldValue.ToString() + "kg";
    }
    if (type == "pivot")
    {
        bool oldValue = Convert.ToBoolean(top.OldValue);
        int index = top.Index;
        Button pivotButton = this.Controls.Find("pivot" + index, true).FirstOrDefault() as Button;
        Point topPoint = new Point(pivotButton.Location.X + 22, 599);
        if (oldValue)
        {
            pivotTops.Add((topPoint, index));
            pivotButton.BackColor = Color.LightSalmon;
            pivotButton.Text = "-";

        }
        else
        {
            pivotTops.Remove((topPoint, index));
            pivotButton.BackColor = SystemColors.GradientInactiveCaption;
            pivotButton.Text = "+";
        }
        pivotHelper();
        this.Invalidate();
    }
```

```
private void clear_Click(object sender, EventArgs e)
{
    animationPanel.Visible = false;
    pivotTops.Clear();
    PlacedPivots.Clear();
    foreach (Control ctr in this.Controls)
    {
        if (ctr is Button && ctr.Name.Contains("pivot"))
        {
            ctr.BackColor = SystemColors.GradientInactiveCaption;
            ctr.Text = "+";
        }
        if (ctr is VScrollBar slider)
        {
            int index = int.Parse(slider.Name.Replace("mass", ""));
            Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
            slider.Value = slider.Maximum; //maximum will take the slider to the bottom
            lbl.Text =  " 0kg";
        }
    }
    this.Invalidate();
    warningmsg.Visible = false;
}
```

I then built the TorqueEngine() class and the animation. For the CalculateTorque method, I chose to use recursion as it mimics how torque is defined.

```
1 reference
public double CalculateTorque(double PivotPoistion, int index)
{
    if (index > weights.Count)
    {
        return 0;
    }
    double distance = weights[index].position - PivotPoistion; // if distance is positive then it will be a clockwise moment and if it is negative then it will be an anticlo
    double force = weights[index].mass * gravity;
    double moment = distance * force;
    return moment + CalculateTorque(PivotPoistion, index + 1);
}
```

```
public bool UpdateTipping()
{
    if (Math.Abs(angularAcceleration * 180 / Math.PI) < 45)
    {

        angularVelocity += angularAcceleration * dt;
        angularPosition += angularVelocity * dt;
        dt += 0.01;
        return true;

    }
    return false;
}
```

When testing my code I found that the stopping function did not work as expected so I updated the UpdateTipping() method.

```
1 reference
public bool UpdateTipping()
{
    angularVelocity += angularAcceleration * dt;
    angularPosition += angularVelocity * dt;
    dt += 0.01;
    double angleDeg = angularPosition * 180 / Math.PI;
    if (Math.Abs(angleDeg) >= 45)
    {
        tipping = false;
        angularPosition = Math.Sign(angleDeg) * 45 * Math.PI / 180;
        angularVelocity = 0;
        angularAcceleration = 0;
        return false;

    }
    return true;
}
```

```
3 references
internal class TorqueEngine
{
    4 references
    public double length { get; set; }
    12 references
    public List<double> pivots { get; set; }
    11 references
    public List<(double mass, double position)> weights { get; set; }
    2 references
    public double angle { get; set; }
    4 references
    public double gravity { get; set; }
    3 references
    public int leverMass { get; set; }
    5 references
    public double angularAcceleration { get; set; }
    5 references
    public double angularVelocity { get; set; }
    6 references
    public double angularPosition { get; set; }
    public bool tipping;
    public double dt;
    9 references
    public int active { get; set; }
```

```
public TorqueEngine(double leverLength, double Gravity, int mass)
{
    length = leverLength;
    pivots = new List<double>();
    weights = new List<(double mass, double position)>();
    gravity = Gravity;
    leverMass = mass;
    angle = 0;
    angularAcceleration = 0;
    angularVelocity = 0;
    angularPosition = 0;
    tipping = false;
    dt = 0;
    active = -2;//no pivot
    weights.Add((leverMass, length / 2));

}

3 references
public double CalculateTorque(double PivotPosition, int index)
{
    if (index >= weights.Count)
    {
        return 0;
    }

    double distance = weights[index].position - PivotPosition; // if distance is positive then it will be a clockwise moment and if it is negative then it will be an anticlockwise moment
    double force = weights[index].mass * gravity;
    double moment = distance * force;
    return moment + CalculateTorque(PivotPosition, index + 1);
}

2 references
public void TippingMovement(int index)
{
    double Torque = CalculateTorque(pivots[index], 0);
    double MomentofInertia = (leverMass * Math.Pow(length, 2)) / 12.0;
    angularAcceleration = Torque / MomentofInertia;
    tipping = true;
}

1 reference
public bool UpdateTipping()
{
    angularVelocity += angularAcceleration * dt;
    angularPosition += angularVelocity * dt;
    dt += 0.01;
    double angleDeg = angularPosition * 180 / Math.PI;
    if (Math.Abs(angleDeg) >= 45)
    {
        tipping = false;
        angularPosition = Math.Sign(angleDeg) * 45 * Math.PI / 180;
        angularVelocity = 0;
        angularAcceleration = 0;
        return false;

    }
    return true;
}
```

```
1 reference
public void Update()
{

    double Torque = CalculateTorque(pivots[0], 0);
    double ReactionPivot2 = 0;
    double ReactionPivot1 = 0;
    if (pivots.Count == 2)
    {
        ReactionPivot2 = Torque / Math.Abs(pivots[1] - pivots[0]);
        double DownwardForce = 0;
        foreach (var weight in weights)
        {
            DownwardForce += weight.mass * 9.8;
        }
        ReactionPivot1 = DownwardForce - ReactionPivot2;
    }
    if (pivots.Count == 1 && Torque != 0)
    {
        ReactionPivot1 = -1;
    }
    else
    {
        active = -2; //error;
    }
    if (ReactionPivot2 < 0)
    {
        TippingMovement(1);
        active = 1;
    }
    else if (ReactionPivot1 < 0)
    {
        TippingMovement(0);
        active = 0;
    }
    else
    {
        active = -1; //balanced
    }

}
```

```
1 reference
public void AddWeight(int mass, double position)
{
    weights.Add((mass, position));
}
1 reference
public void RemoveWeight(double position)
{
    int count = 0;
    foreach (var weight in weights)
    {
        if (weight.position == position)
        {
            break;
        }
        count += 1;
    }
    weights.RemoveAt(count);
}

1 reference
public void AddPivots(double position)
{
    pivots.Add(position);
}

1 reference
public void RemovePivots(double position)
{
    int count = 0;
    foreach (double pos in pivots)
    {
        if (pos == position)
        {
            break;
        }
        count += 1;
    }
    pivots.RemoveAt(count);
}

1 reference
public void Reset()
{
    angle = 0;
    gravity = 9.8;
    angularAcceleration = 0;
    angularPosition = 0;
    angularVelocity = 0;
    dt = 0;
    active = 0;
    tipping = false;
    pivots.Clear();
    weights.Clear();
}
```

```
4 references
public partial class TorqueSB : Form
{

    Stack<State> StateStack = new Stack<State>();
    HashSet<int> PlacedPivots = new HashSet<int>();
    List<(Point coordinate, int index)> pivotTops = new List<(Point coordinate, int index)>(); //stores the top vertex of the pivots so that they can be drawn and deleted
    List<(Point leftCoordinate, Point rightCoordinate)> beams = new List<(Point leftCoordinate, Point rightCoordinate)>();
    List<Point> weights = new List<Point>();
    TorqueEngine engine = new TorqueEngine(20.0, 9.8, 5);
    1 reference
    public TorqueSB()
    {
        InitializeComponent();
        pivot1.Click += PivotClick;
        pivot2.Click += PivotClick;
        pivot3.Click += PivotClick;
        pivot4.Click += PivotClick;
        pivot5.Click += PivotClick;
        pivot6.Click += PivotClick;
        pivot7.Click += PivotClick;
        pivot8.Click += PivotClick;
        pivot9.Click += PivotClick;
        pivot10.Click += PivotClick;
        pivot11.Click += PivotClick;
        mass1.Scroll += MassScroll;
        mass2.Scroll += MassScroll;
        mass3.Scroll += MassScroll;
        mass4.Scroll += MassScroll;
        mass5.Scroll += MassScroll;
        mass6.Scroll += MassScroll;
        mass7.Scroll += MassScroll;
        mass8.Scroll += MassScroll;
        mass9.Scroll += MassScroll;
        mass10.Scroll += MassScroll;
        mass11.Scroll += MassScroll;
    }

    3 references
    private void stackHelper()
    {
        if (StateStack.Count >= 5)
        {
            State topElement = StateStack.Pop();
            StateStack.Clear();
            StateStack.Push(topElement);
        }

    }
```

```
11 references
private void PivotClick(object sender, EventArgs e)
{
    Button pivotButton = sender as Button;
    int index = int.Parse(pivotButton.Name.Replace("pivot", ""));
    Point top = new Point(pivotButton.Location.X + 22, 599);
    bool exists = PlacedPivots.Contains(index); //check if a pivot already exists at that location
    State state = new State("pivot", index, exists);
    stackHelper();
    StateStack.Push(state);
    if (!exists)
    {
        pivotTops.Add((top, index));
        pivotButton.BackColor = Color.LightSalmon;
        pivotButton.Text = "-";
        PlacedPivots.Add(index);
        engine.AddPivots(index - 1); //pivot distance added in metres
    }
    else
    {
        pivotTops.Remove((top, index));
        pivotButton.BackColor = SystemColors.GradientInactiveCaption;
        pivotButton.Text = "+";
        PlacedPivots.Remove(index);
        engine.RemovePivots(index - 1);
    }
    pivotHelper();
    this.Invalidate();
}


11 references
private void MassScroll(object sender, EventArgs e)
{
    VScrollBar slider = sender as VScrollBar;
    int index = int.Parse(slider.Name.Replace("mass", ""));
    Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
    int oldValue = int.Parse(lbl.Text.Replace("kg", ""));
    lbl.Text = (slider.Maximum - slider.Value).ToString() + "kg"; // scroll bar has maximum at bottom so to make it more intuitive this method is being used to flip the values
    State state = new State("mass", index, oldValue);
    stackHelper();
    StateStack.Push(state);
    if (slider.Value == 0)
    {
        engine.RemoveWeight(index - 1);
    }
    else
    {
        engine.AddWeight(slider.Value, index - 1);
    }
}
```

```
private void DrawPivot(Graphics g, Point top)
{
    int halfWidth = 25;
    Point leftBase = new Point(top.X - halfWidth, top.Y + 70); //70 is height of pivot
    Point rightBase = new Point(top.X + halfWidth, top.Y + 70);
    Point[] trianglePoints = { top, leftBase, rightBase };
    g.DrawPolygon(Pens.Black, trianglePoints);
    g.FillPolygon(Brushes.LightBlue, trianglePoints);
}

1 reference
private void DrawWeight(Graphics g, Point centre)
{
    int radius = 10;
    g.DrawEllipse(Pens.Black, centre.X - radius, centre.Y - radius, radius * 2, radius * 2);
    g.FillEllipse(Brushes.Black, centre.X - radius, centre.Y - radius, radius * 2, radius * 2);
}

1 reference
private void DrawBeam(Graphics g, Point left, Point right)
{
    using (Pen thickPen = new Pen(Color.SaddleBrown, 10))
    {
        g.DrawLine(thickPen, left.X, left.Y, right.X, right.Y);
    }
}

0 references
protected override void OnPaint(PaintEventArgs e)
{
    base.OnPaint(e);
    Graphics g = e.Graphics;
    foreach (var top in pivotTops)
    {
        DrawPivot(g, top.coordinate);
    }


}
```

```
private void pivotHelper()
{
    if (PlacedPivots.Count >= 2)
    {
        warningmsg.Visible = true;
        foreach (Control ctrl in this.Controls)
        {
            if (ctrl is Button)
            {
                if (ctrl.Name.Contains("pivot"))
                {
                    if (ctrl.Text == "+")
                    {
                        ctrl.Enabled = false;
                    }
                }
            }

        }
    }
    else
    {
        warningmsg.Visible = false;
        foreach (Control ctrl in this.Controls)
        {
            if (ctrl is Button)
            {
                if (ctrl.Name.Contains("pivot"))
                {
                    ctrl.Enabled = true;
                }
            }

        }
    }
}
```

```
private void undo_Click(object sender, EventArgs e)
{
    try
    {
        State top = StateStack.Pop();
        string type = top.Type;
        if (type == "gravity")
        {
            double oldValue = Convert.ToDouble(top.OldValue);
            gravity.Value = Convert.ToInt32(oldValue * 10);
            lbl_gravity.Text = oldValue.ToString();
        }
        if (type == "mass")
        {
            int oldValue = Convert.ToInt32(top.OldValue);
            int index = top.Index;
            Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
            VScrollBar slider = this.Controls.Find("mass" + index, true).FirstOrDefault() as VScrollBar;
            slider.Value = oldValue;
            lbl.Text = oldValue.ToString() + "kg";
        }
        if (type == "pivot")
        {
            bool oldValue = Convert.ToBoolean(top.OldValue);
            int index = top.Index;
            Button pivotButton = this.Controls.Find("pivot" + index, true).FirstOrDefault() as Button;
            Point topPoint = new Point(pivotButton.Location.X + 22, 599);
            if (oldValue)
            {
                pivotTops.Add((topPoint, index));
                pivotButton.BackColor = Color.LightSalmon;
                pivotButton.Text = "-";

            }
            else
            {
                pivotTops.Remove((topPoint, index));
                pivotButton.BackColor = SystemColors.GradientInactiveCaption;
                pivotButton.Text = "+";
            }
            pivotHelper();
            this.Invalidate();
        }
    }
    catch
    {
        MessageBox.Show("UNDO LIMIT REACHED", "HTTP ERROR 409", MessageBoxButtons.OK, MessageBoxIcon.Stop);
    }


}
```

```
1 reference
private void RedrawLever()
{
    int PivotLeftMargin = 77;
    double Scale = 95;
    beams.Clear();
    weights.Clear();
    int active = engine.active;
    double pivotPosMetres = engine.pivots[active];
    double angle = engine.angularPosition;
    int pivotX = PivotLeftMargin + (int)(pivotPosMetres * Scale);
    int pivotY = 599;
    // Left and right ends of the lever in metres
    double leftDistance = pivotPosMetres;
    double rightDistance = engine.length - pivotPosMetres;

    Point left = new Point(
        pivotX - (int)(leftDistance * Math.Cos(angle) * Scale),
        pivotY + (int)(leftDistance * Math.Sin(angle) * Scale)
    );

    Point right = new Point(
        pivotX + (int)(rightDistance * Math.Cos(angle) * Scale),
        pivotY - (int)(rightDistance * Math.Sin(angle) * Scale)
    );
    beams.Add((left, right));
    foreach (var weight in engine.weights)
    {
        double distancefromPivot = weight.position - pivotPosMetres;
        Point weightXY = new Point(
        pivotX + (int)(distancefromPivot * Math.Cos(angle) * Scale),
        pivotY - (int)(distancefromPivot * Math.Sin(angle) * Scale)
        );
        weights.Add(weightXY);
    }
    animationPanel.Invalidate();

}

1 reference
private async void Animate()
{
    while (engine.UpdateTipping())
    {
        // Redraw lever / animation
        RedrawLever();
        animationPanel.Invalidate();


        await Task.Delay(16);
    }
}
```

```
private void Simulate_Click(object sender, EventArgs e)
{

    engine.Update();
    bool animation = true;
    if (engine.active == -1)
    {
        MessageBox.Show("Balanced", "SIMULATION RESULT", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    else if (engine.active == -2)
    { MessageBox.Show("Cannot Simulate without pivot", "SIMULATION EROR", MessageBoxButtons.OK, MessageBoxIcon.Error); }
    else
    {
        pivotTops.Clear();
        animationPanel.Enabled = true;
        animationPanel.Visible = true;
        gravity.Visible = false;
        gravity_text.Visible = false;
        lbl_gravity.Visible = false;
        Animate();
    }
}

1 reference
private void animationPanel_Paint(object sender, PaintEventArgs e)
{
    Graphics g = e.Graphics;
    g.SmoothingMode = System.Drawing.Drawing2D.SmoothingMode.AntiAlias;

    foreach (var beam in beams)
    {
        DrawBeam(g, beam.leftCoordinate, beam.rightCoordinate);
    }
    foreach (Point weight in weights)
    {
        DrawWeight(g, weight);
    }
}
```

I chose to perform all physics calculations in metres, which allows forces, torques, and angular motion to remain physically realistic, while converting these values to pixels for drawing ensures that the lever, pivots, and weights are correctly displayed on the screen.

```
public (double torque, List<string> log) CalculateTorqueWithLog(double pivotPosition, int? otherPivotIndex = null)
{
    List<string> log = new List<string>();
    double torque = 0;

    for (int i = 0; i < weights.Count; i++)
    {
        double pos = weights[i].position;
        double dist = pos - pivotPosition;
        double m = weights[i].mass;

        // skip if it's between two pivots
        if (otherPivotIndex.HasValue)
        {
            double otherPivot = pivots[otherPivotIndex.Value];
            double minP = Math.Min(pivotPosition, otherPivot);
            double maxP = Math.Max(pivotPosition, otherPivot);
            if (pos > minP && pos < maxP)
                continue;
        }

        double contrib = m * dist;
        log.Add($"{m}kg at {pos}m which is a distance {dist}m from the pivot → torque {contrib}");
        torque += contrib;
    }

    return (torque, log);
}
```

```
private void UpdateCalculationZone()
{
    if (CalculationZone == null) return;
    var sb = new System.Text.StringBuilder();

    if (engine.pivots.Count == 0)
    {
        sb.AppendLine("No pivots placed.");
    }
    else
    {
        for (int i = 0; i < engine.pivots.Count; i++)
        {
            int? other = null;
            if (engine.pivots.Count == 2)
                other = (i == 0 ? 1 : 0);

            var (torque, log) = engine.CalculateTorqueWithLog(engine.pivots[i], other);

            sb.AppendLine($"Pivot {i + 1} at {engine.pivots[i]}m");
            foreach (var line in log)
                sb.AppendLine("  " + line);
            sb.AppendLine($"Total torque: {torque}");
            sb.AppendLine();
        }
    }

    CalculationZone.Text = sb.ToString();
}
```

This feature displays a clear breakdown of the torque calculation used in the simulation. When the user runs the model, the program lists each weight, its position, and its contribution to the total torque about each pivot. The method CalculateTorqueWithLog in the engine performs the same calculation logic asCalculateTorque: for every weight, it multiplies mass by its distance from the pivot. It then records each step in a log. The simulation's UpdateCalculationZone method formats this log and outputs it to the on-screen calculation area, helping users understand how the final torque is derived and ensuring the engine's results match the puzzle's logic.

```
private void undo_Click(object sender, EventArgs e)
{
    try
    {
        State top = StateStack.Pop();
        string type = top.Type;
        if (type == "gravity")
        {
            double oldValue = Convert.ToDouble(top.OldValue);
            gravity.Value = Convert.ToInt32(oldValue * 10);
            lbl_gravity.Text = oldValue.ToString();
        }
        if (type == "mass")
        {
            int oldValue = Convert.ToInt32(top.OldValue);
            int index = top.Index;
            Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
            VScrollBar slider = this.Controls.Find("mass" + index, true).FirstOrDefault() as VScrollBar;
            slider.Value = oldValue;
            lbl.Text = oldValue.ToString() + "kg";
        }
        if (type == "pivot")
        {
            bool oldValue = Convert.ToBoolean(top.OldValue);
            int index = top.Index;
            Button pivotButton = this.Controls.Find("pivot" + index, true).FirstOrDefault() as Button;
            Point topPoint = new Point(pivotButton.Location.X + 22, 599);
            if (oldValue)
            {
                pivotTops.Add((topPoint, index));
                pivotButton.BackColor = Color.LightSalmon;
                pivotButton.Text = "-";

            }
            else
            {
                pivotTops.Remove((topPoint, index));
                pivotButton.BackColor = SystemColors.GradientInactiveCaption;
                pivotButton.Text = "+";
            }
            pivotHelper();
            this.Invalidate();
        }
    }
    catch
    {
        MessageBox.Show("UNDO LIMIT REACHED", "HTTP ERROR 409", MessageBoxButtons.OK, MessageBoxIcon.Stop);
    }


}
```

I found that there was an error in the way that the mass and position were passed to the TorqueEngine

```
11 references
private void MassScroll(object sender, EventArgs e)
{
    VScrollBar slider = sender as VScrollBar;
    int index = int.Parse(slider.Name.Replace("mass", ""));
    Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
    int oldValue = int.Parse(lbl.Text.Replace("kg", ""));
    lbl.Text = (slider.Maximum - slider.Value).ToString() + "kg"; // scroll bar has maximum at bottom so to make it more intuitive this method is being used to flip the valu
    State state = new State("mass", index, oldValue);
    stackHelper();
    StateStack.Push(state);
    int listIndex = placedMasses.FindIndex(p => p.index == index);
    int mass = slider.Maximum - slider.Value;
    if (mass == 0)
    {
        try
        {
            placedMasses.RemoveAt(listIndex);
        }
        catch { }
    }
    else if (listIndex < 0)
    {
        placedMasses.Add((mass, index));

    }
    else
    {
        placedMasses[listIndex] = (mass,index);
    }
```

```
public void CleanList()
{
    placedMasses = placedMasses
        .GroupBy(p => p.index)
        .Select(g => g.First())
        .ToList();
}
1 reference
private void Simulate_Click(object sender, EventArgs e)
{
    engine.weights.Clear();
    CleanList();
    foreach (var weight in placedMasses)
    {
        engine.AddWeight(weight.mass, 2*(weight.index - 1));
        MessageBox.Show(weight.mass + " at position " + 2 * (weight.index - 1));
    }
    /*foreach(Control ctrl in this.Controls)
    {
        if (ctrl is VScrollBar slider && )
    }*/
    engine.Update();
    bool animation = true;
    if (engine.active == -1)
    {
        MessageBox.Show("Balanced", "SIMULATION RESULT", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    else if (engine.active == -2)
    { MessageBox.Show("Cannot Simulate without pivot", "SIMULATION EROR", MessageBoxButtons.OK, MessageBoxIcon.Error); }
    else
    {
        pivotTops.Clear();
        animationPanel.Enabled = true;
        animationPanel.Visible = true;
        gravity.Visible = false;
        gravity_text.Visible = false;
        lbl_gravity.Visible = false;
        Animate();
    }
}
```

Testing

Input	Testing	Justification	Expected Result	Outcome
Two equal weights on opposite sides of a single pivot	Equilibrium	Checks that the torque and reaction forces are measured correctly	Lever stays balanced	**SUCCESS**
Four unequal weights on opposite sides of a single pivot	Single Pivot	Tests basic torque calculation and tipping with a single pivot	Lever Rotates	**SUCCESS** See Figure 6.1
Place one weight on a double pivot system	Double Pivot	Checks that tipping occurs correctly around the pivot experiencing negative reaction force	Lever tips around pivot1 if ReactionPivot2 < 0	**SUCCESS**
Place three weights on a double pivot system	Double Pivot	Checks that the calculations are	Lever tips up to ±45° depending on weight location	**SUCCESS**
Place 10 weights and 2 pivots	Boundary Test	Check that the system can handle the maximum capacity of	Lever tips up to ±45° depending on weight location	**SUCCESS**

		weight positions and pivots		
Gravity = 0	Zero Gravity	Tests effects of zero gravity.	Lever remains static, no tipping or rotation	**SUCCESS**
Place more than 2 pivots	Validation	The calculations only allow for a maximum of 2 pivots so the front end should ensure that no more than 2 pivots are selected	Error Message	**SUCCESS** See Figure 6.2

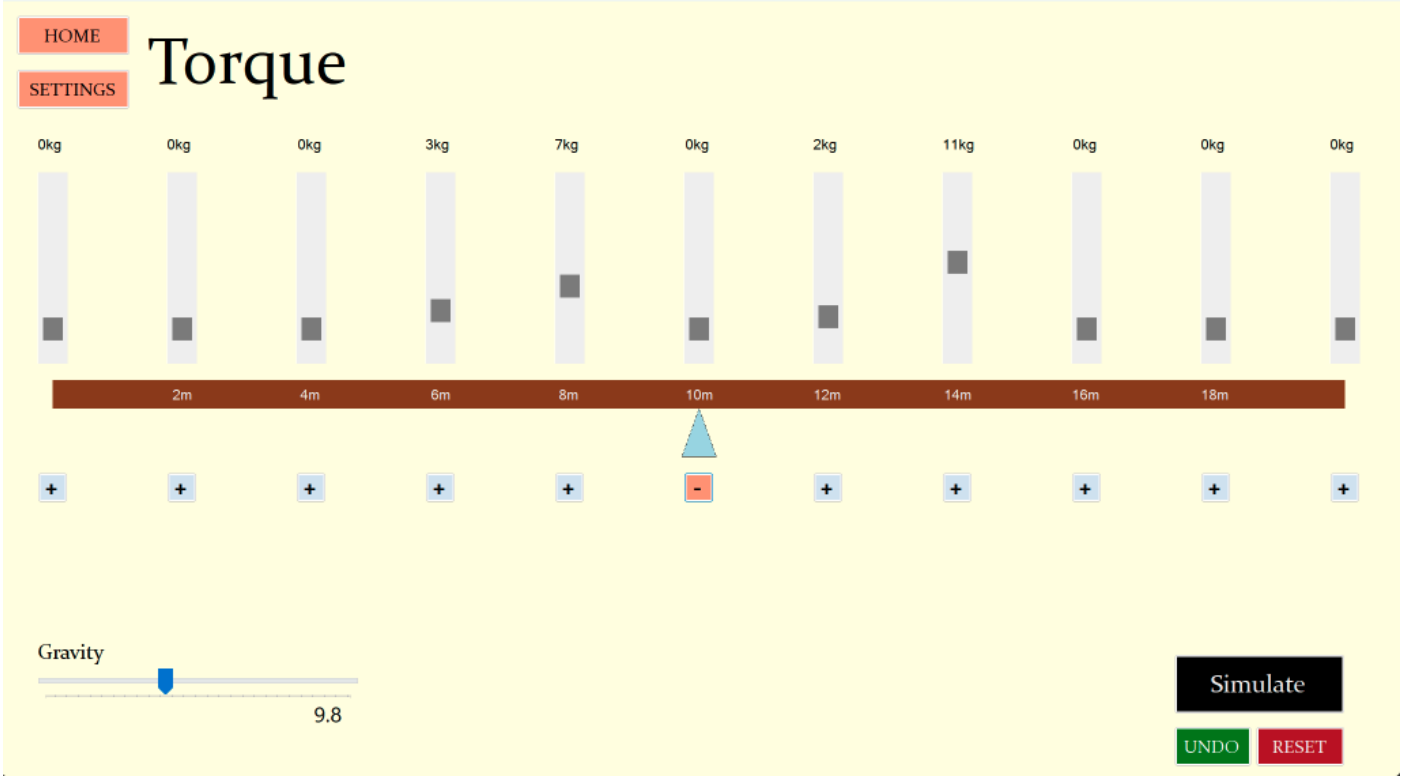


Figure 6. 1

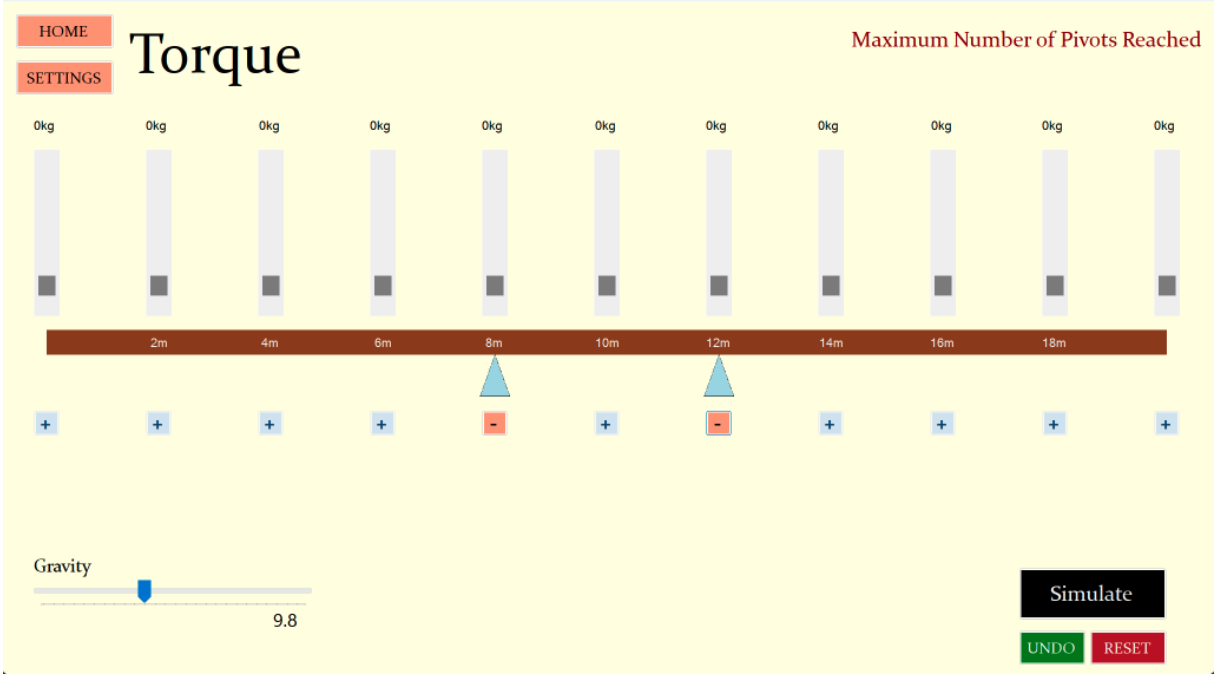


Figure 6. 2

Remedial Action

All the tests were successful, so no further action is required

Validation

As the UI mainly only sliders, there is little extra validation necessary. All user interactions behaved as normal which showed that the prototype was robust.

Review

Following testing I can conclude that I was successfully able to implement

- Torque Sandbox

Prototype 3 –Puzzles

Prototype 3 is the projectile motion puzzle and torque puzzle. Unlike the previous sandbox prototypes, which allowed users to freely adjust variables and observe the resulting motion, this prototype frames the simulation as a structured challenge. For each projectile motion puzzle, the user is presented with a target and environmental parameters such as projectile mass, gravity, and drag, which are set randomly. The player must adjust the projectile's launch height and angle to hit the target; in some puzzles, the initial velocity can also be changed, while in others it is fixed, introducing a layer of selective challenge. For each torque puzzle, masses are randomly placed, and the user has to add masses at different nodes to balance the beam. No additional obstacles or levels are included at this stage; the focus is on providing a playable puzzle that demonstrates both the physics simulation and user interaction, while laying the groundwork for more complex features in future prototypes.

Class Diagrams:

ProjectilePuzzleEngine

ProjectilePuzzleEngine is a child of PhysicsEngine (Prototype 1). It generates a puzzle by randomly setting environmental and projectile variables, ensures the puzzle is valid using a helper function, stores the correct answer values, and calculates the user's score based on accuracy, timing, and analytical input.

ProjectilePuzzleEngine : Physics Engine

Key Attributes
`List allowedVariables`
`List fixedVariables`
`List finalValues`
`List startValues`

Key Methods
`GeneratePuzzle()`
`ValidatePuzzle()`
`CalcualateScore()`

TorquePuzzleEngine

TorquePuzzleEngine is a child of TorqueEngine (Prototype 2). It generates a puzzle by randomly setting pivots and masses, ensures the puzzle is valid using a helper function, stores the correct answer values, and calculates the user's score based on accuracy, timing, and analytical input.

TorquePuzzleEngine : TorqueEngine

Key Attributes
`List allowedVariables`
`List fixedVariables`
`List finalValues`
`List startValues`

Key Methods
`GeneratePuzzle()`
`ValidatePuzzle()`
`CalcualateScore()`

Database

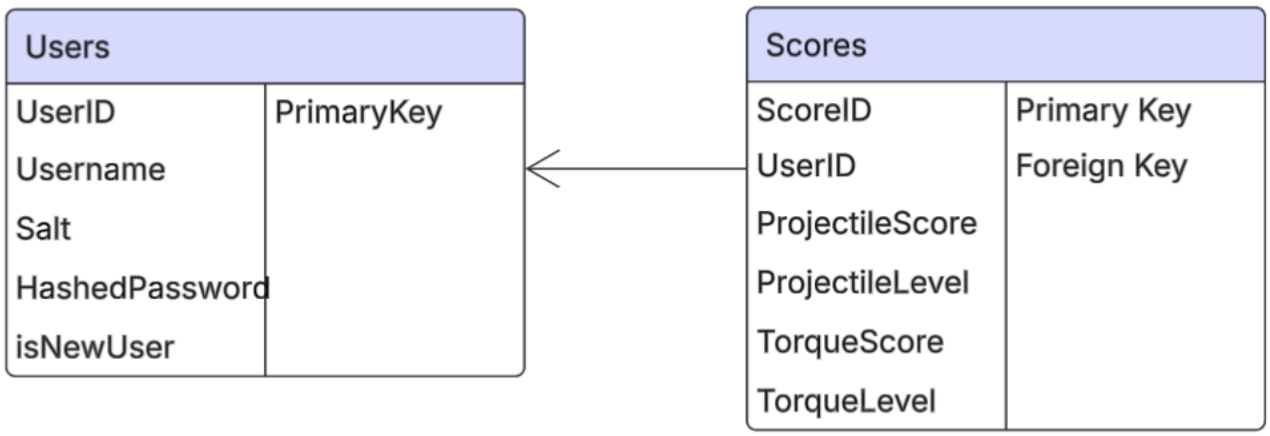
In addition, I am planning to enhance the existing database class from Prototype 1 by adding an UpdateScore function. This function will allow the application to record and update the user's scores for each puzzle or level directly in the database, linking them to the logged-in user via a foreign key. By doing this, both projectile and torque puzzle scores can be persistently stored and retrieved across sessions. I will also be adding a leaderboard sort function to retrieve the data for the leaderboard.

Database
Key Attributes
`ConnectionString`
Key Methods
`RegisterUser()`
`ValidateLogin()`
`UsernameExists()`
`ExecuteQuery()`
`ExecuteNonQuery()`
`UpdateScores()`
`SortScores()`

I will also be using a static class to pass the UserID to different forms

Session
Key Attributes
`CurrentUserID`
`Username`

Database



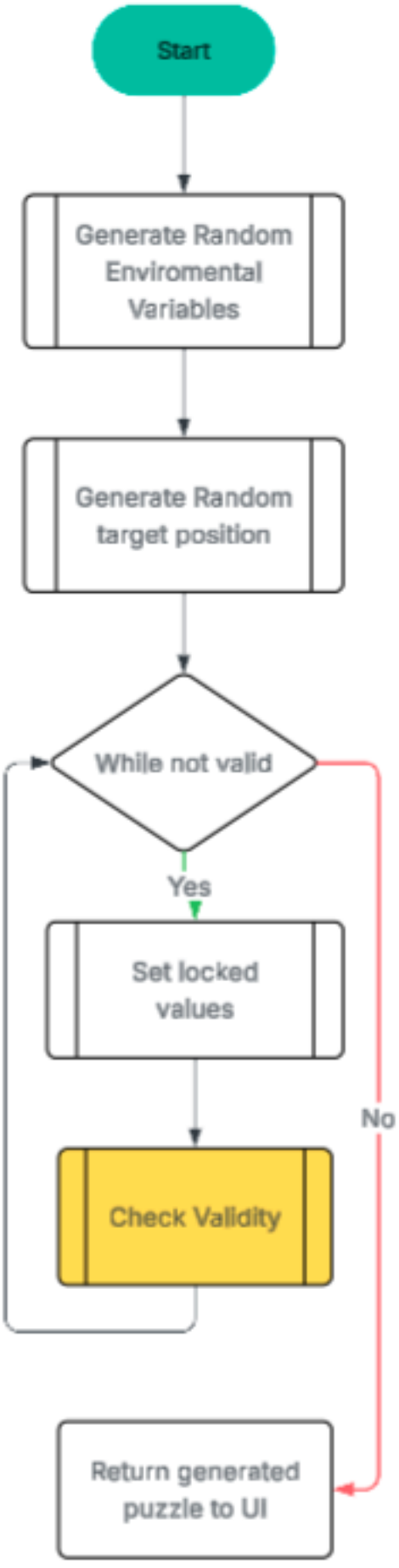
The above ER diagram shows a one-to-one relationship between the Users and Scores tables. Each user has a single set of scores, with the Scores table storing projectile and torque scores, corresponding levels, all linked to the user via the UserID foreign key. This ensures every user has exactly one score record.

```
CREATE TABLE Scores(
ScoreID INT PRIMARY KEY AUTOINCREMENT,
UserID INT,
ProjectileScore INT DEFAULT 0,
ProjectileLevel INT DEFAULT 1,
TorqueScore INT DEFAULT 0,
TorqueLevel INT DEFAULT 1,
LeaderboardPosition INT,
FOREIGN KEY(USERID) REFERENCES Users(USERID) ON DELETE CASCADE
)
```

Algorithms

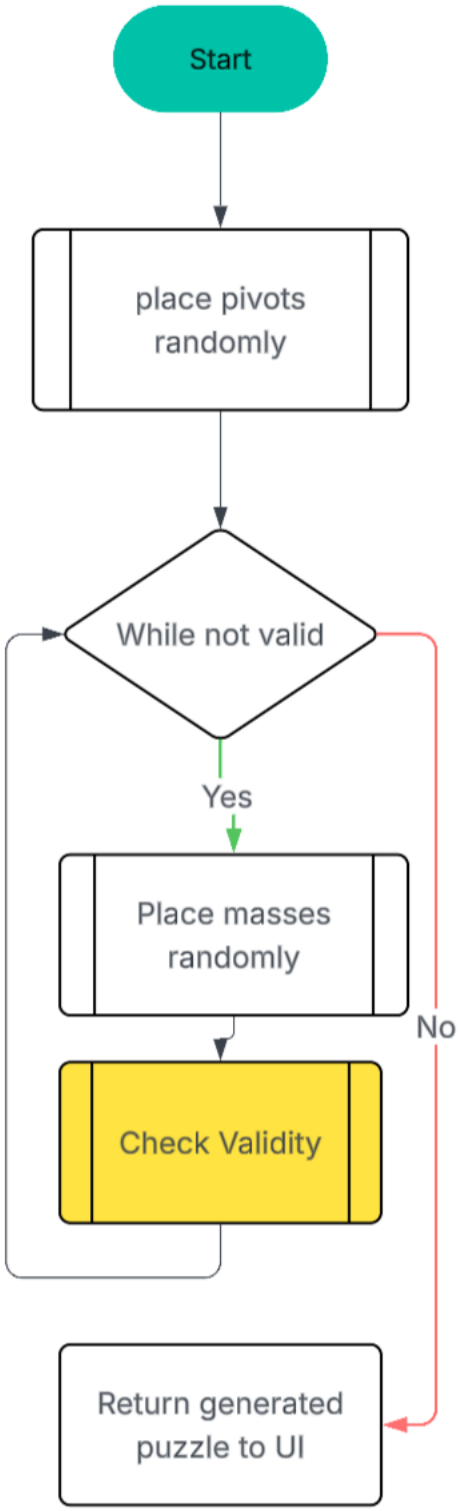
Projectile Puzzle Generation

This algorithm begins by applying randomised environmental variables and choosing a random target position. It will then set the projectile variables ensuring that some are fixed and some are user controllable. It then validates the puzzle using a helper function to ensure that a solution exists. If the puzzle is invalid, the algorithm discards the current values and repeats the random generation until a valid puzzle is produced. Finally, the algorithm stores the answer variables and the fixed/random configuration in the PuzzleEngine and returns the generated puzzle to the UI, ready for the user to attempt and for scoring.



Torque Puzzle Generation

This algorithm begins by randomly placing a maximum of two pivots on the beam. It then places the masses randomly at different positions along the beam. After placement, it checks whether the system is already balanced or whether it can be balanced within the allowed restrictions of the solution. If the configuration is invalid, the algorithm discards the current setup and restarts the random generation process. Finally, it stores the correct solution and fixed/random variable configuration in the PuzzleEngine and returns the puzzle to the UI, ready for the user to attempt and for scoring.



Scoring

The scoring algorithm calculates a user's points based on accuracy and efficiency. When the user submits their solution first compares the user's input values against the pre-calculated correct values stored in the PuzzleEngine. Finally the algorithm updates the score in the database.

```
CalculateScore(userInput, timeTaken, usedSimulation,numberofAttempts)
      Set score = 0
      If userInput is exactly answer
            Then score += fullPoints
      Else if the input is within allowed error margin
            Then score += partialPoints

      If timeTaken is greater than fast threshold
            Then score += speedBonus

      If numberofAttempts is 1 and usedSimulation is false
            Then score += analyticalBonus
      Else if numberofAttempts is 1
            Then score += firstattemptBonus
```

Usability Features

The form provides a button to switch between Sandbox and Puzzle modes, allowing the user to experiment freely or attempt structured puzzles. When in Puzzle mode, specific sliders or input controls are disabled according to the puzzle's rules, preventing the user from changing fixed variables. This ensures that the user can focus only on the adjustable parameters required to solve the puzzle, enhancing clarity and reducing accidental errors. Additionally, visual cues such as labels or highlights indicate which controls are active and which are restricted, improving overall usability and guiding the user through the puzzle-solving process.

Key Variables

Variable Name	Data Type	Description	Justification
allowedVariables	list	Variables that the user is allowed to manipulate	Clearly defines which inputs are user controlled to enforce puzzle rules.
fixedVariables	List	Variables that are pre-set and cannot be changed by the user	Ensures that the puzzle cannot be solved by changing factors that weren't meant to be changed
startValues	Dictionary	Initial values for all puzzle parameters	Records the starting state to display to the user and support reset functionality
finalValues	Dictionary	Correct values for the solution	Used to calculate scores based on accuracy and

			verify if puzzle is solved correctly
ProjectileScore	Int	Stores the user's score for the projectile puzzle	Tracks progress and allows for comparison between attempts
PuzzleValid	Bool	Indicates whether the randomly generated puzzle is solvable	Prevents the user from being presented impossible puzzles
CurrentUserID	Int	Stores the logged-in user's ID	Links scores and progress to the correct user in the database

Testing

Input	Testing	Justification	Expected Result
Correctly Solve projectile motion puzzle - **Ensuring the speed threshold is met**	Projectile Motion Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points
Correctly Solve projectile motion puzzle - **Ensuring the first time threshold is met**	Projectile Motion Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points
Correctly Solve projectile motion puzzle - **Without using the simulation to check**	Projectile Motion Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points
Correctly Solve torque puzzle Ensuring the speed threshold is met	Torque Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points
Correctly Solve torque puzzle - **Ensuring the first time threshold is met**	Torque Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points

Correctly Solve torque puzzle - **Without using the simulation to check**	Torque Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points

Development

I began by setting up the session static class that would allow me to use variables such as the UserID across multiple forms. To do this, I first had to write a method that would retrieve the UserID for a given username.

```
public int GetUserID(string username)
{
    int UserID = -1;
    string sql = "SELECT UserID From Users WHERE Username = @username";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@username", username));
    if (result.Rows.Count > 0)
    { UserID = Convert.ToInt32(result.Rows[0]["UserID"]); }
    return UserID;
}
```

Then, when the user is logging in or registering this data is stored to session as seen in the code snippet below.

```
private void log_in_Click(object sender, EventArgs e)
{
    bool authenticate = db.ValidateLogin(username.Text, password.Text);
    if (authenticate)
    {
        int UserID = db.GetUserID(username.Text);
        Session.UserID = UserID;
        Session.Username = username.Text;
        Choose form = new Choose();
        this.Hide();
        form.Show();
    }
}
```

```
7 references
internal class Session
{
    4 references
    public static string Username { get; set; }
    3 references
    public static int UserID { get; set; }
}
```

Following this I added a button to allow the user to toggle between sandbox and puzzle mode.

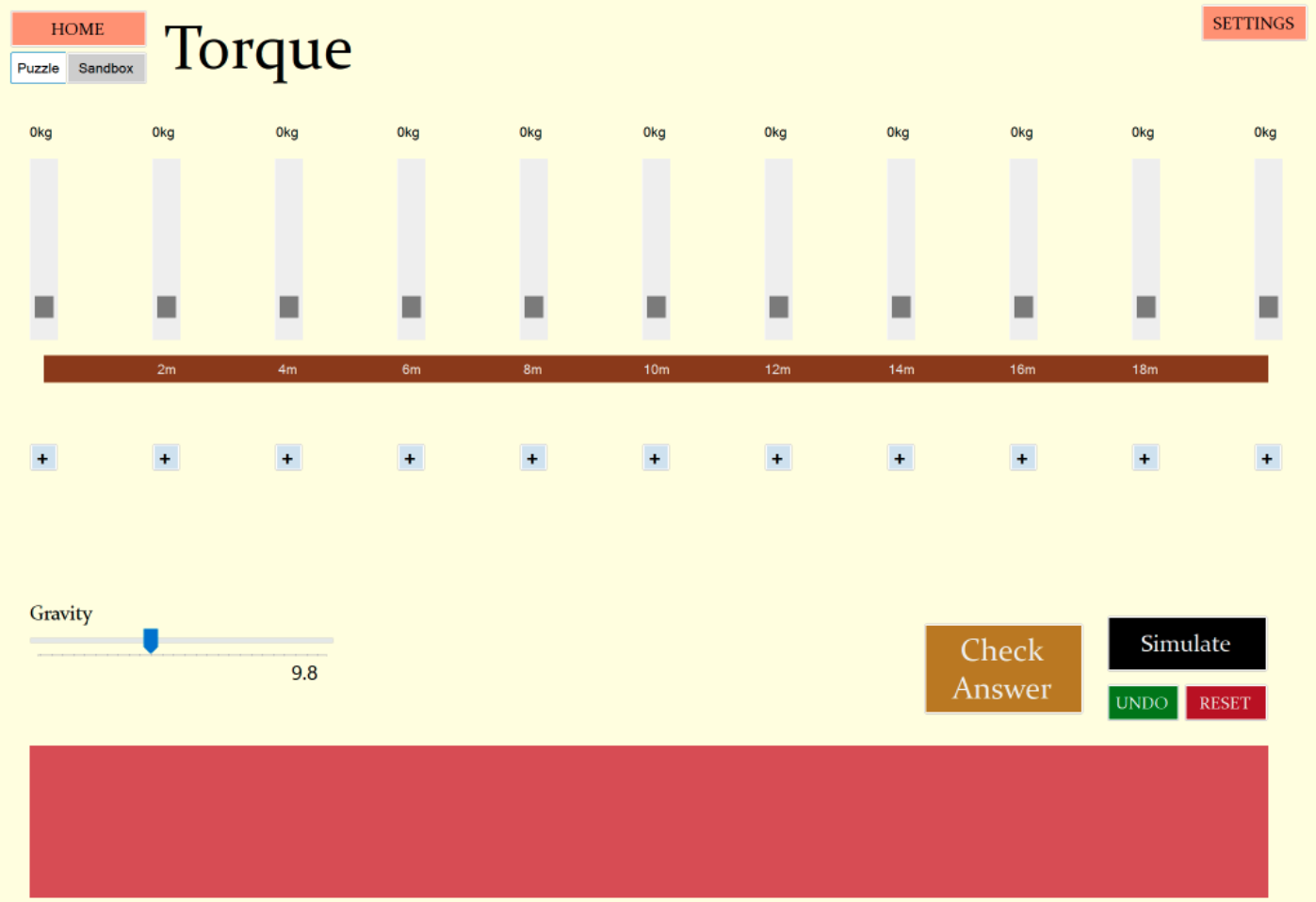


In order to display the question: *A uniform beam of length 20.0m and mass 5.0 kg is pivoted as shown. Additional point masses may be placed at distinct positions on the beam. Using the diagrammed pivot and initial masses, place one or more extra masses so that the beam is in static equilibrium.* I initially wrote the following function:

```
private void puzzle_Click(object sender, EventArgs e)
{
    mode = "puzzle";
    sandbox.BackColor = Color.LightGray;
    puzzle.BackColor = Color.White;
    foreach (Control ctrl in this.Controls)
    {
        if (ctrl.Name.StartsWith("Question"))
        {
            ctrl.Visible = true;
        }
    }

}
```

However since the text was a child to the parent panel. Only the parent panel would show and the solution would output the following.



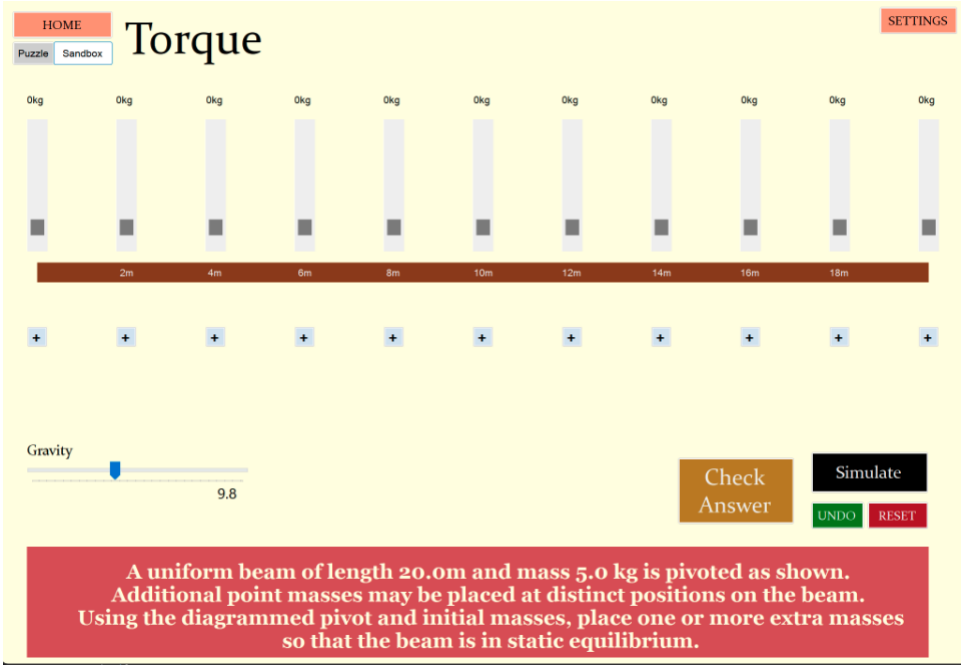
In order to remedy this, I had to write a recursive function to search the children of each parent node as well

```
1 reference
private void puzzle_Click(object sender, EventArgs e)
{
    mode = "puzzle";
    sandbox.BackColor = Color.LightGray;
    puzzle.BackColor = Color.White;
    Question(this);

}

3 references
private void Question(Control parent)
{
    foreach (Control ctrl in parent.Controls)
    {
        if (ctrl.Name.StartsWith("Question"))
        {
            ctrl.Visible = !ctrl.Visible; // flip visibility
        }

        // recursively search children
        if (ctrl.HasChildren)
        {
            Question(ctrl);
        }
    }
}
```



However when repeatedly switching between the toggles, I found that the text was not always displaying. See *Prototype 3 – Video 1*. The issue was that child controls were being toggled twice, once indirectly when their parents' visibility changed, and again directly in the recursion, which caused them to remain hidden. I

overcame this by making all children explicitly follow their parents' new visibility state so they appear/disappear together.

```
private void Question(Control parent)
{
    foreach (Control ctrl in parent.Controls)
    {
        if (ctrl.Name.StartsWith("Question"))
        {
            bool newState = !ctrl.Visible;
            ctrl.Visible = newState;

            // ensure all children match parent
            SetChildrenVisibility(ctrl, newState);
        }

        // recursively search children
        else if (ctrl.HasChildren)
        {
            Question(ctrl);
        }
    }
}

2 references
private void SetChildrenVisibility(Control parent, bool visible)
{
    foreach (Control child in parent.Controls)
    {
        child.Visible = visible;
        if (child.HasChildren)
        {
            SetChildrenVisibility(child, visible);
        }
    }
}
```

However, this led to problems with double-clicking and content being out of sync, so I decided to send the subroutine what the visibility should be.

```
private void puzzle_Click(object sender, EventArgs e)
{
    mode = "puzzle";
    sandbox.BackColor = Color.LightGray;
    puzzle.BackColor = Color.White;
    Toggle(this,true,"Question");
    CheckAnswer.Visible = true;
    nextPuzzle.Visible = true;
    puzzle.Enabled = false;
    sandbox.Enabled = true;
}

3 references
private void Toggle(Control parent,bool newState, string name)
{
    foreach (Control ctrl in parent.Controls)
    {
        if (ctrl.Name.StartsWith(name))
        {
            ctrl.Visible = newState;

        }

        // recursively search children
        if (ctrl.HasChildren)
        {
            Toggle(ctrl,newState,name);
        }
    }
}
```

I then created the scores table in the database to persistently store the user data

```
CREATE TABLE Scores (
    ScoreID             INTEGER PRIMARY KEY AUTOINCREMENT,
    UserID              INT,
    ProjectileScore     INT     DEFAULT 0,
    ProjectileLevel     INT     DEFAULT 1,
    TorqueScore         INT     DEFAULT 0,
    TorqueLevel         INT     DEFAULT 1,
    LeaderboardPosition INT,
    FOREIGN KEY (
        UserID
    )
    REFERENCES Users (UserID) ON DELETE CASCADE
);
```

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	Default value
1	ScoreID	INTEGER	🔑							*NULL*
2	UserID	INT		🔗						*NULL*
3	ProjectileScore	INT								0
4	ProjectileLevel	INT								1
5	TorqueScore	INT								0
6	TorqueLevel	INT								1
7	LeaderboardPosition	INT								*NULL*

Following this I began to implement my child class that will handle the puzzle logic for the torque puzzles. I chose to implement this as a child class rather than extending the parent class, as it would allow me to encapsulate this puzzle logic from the simulation logic making the code easier to follow.

```
1 reference
internal class TorquePuzzle : TorqueEngine
{
    0 references
    public TorquePuzzle(double leverLength, double gravity, int mass)
    : base(leverLength, gravity, mass)
    {

    }

    0 references
    public void SetRandomPivots()
    {
        Random rnd = new Random();
        int numPivots = rnd.Next(1, 3);
        for (int i = 0; i < numPivots; i++)
        {

            int random = rnd.Next(0, 11) * 2;
            if (i > 0)
            {
                while (pivots[i - 1] == random)
                {
                    random = rnd.Next(0, 11) * 2;
                }
            }
            pivots[i] = random;
        }
    }
}
```

```
internal class TorquePuzzle : TorqueEngine
{
    0 references
    public TorquePuzzle(double leverLength, double gravity, int mass)
    : base(leverLength, gravity, mass)
    {

    }

    0 references
    public void SetRandomPivots()
    {
        Random rnd = new Random();
        int numPivots = rnd.Next(1, 3);
        for (int i = 0; i < numPivots; i++)
        {

            int random = rnd.Next(0, 11) * 2;
            if (i > 0)
            {
                while (pivots[i - 1] == random) //ensures the pivot is not placed in the same position twice
                {
                    random = rnd.Next(0, 11) * 2;
                }
            }
            pivots[i] = random;
        }
    }

    0 references
    public void SetMasses()
    {
        Random rnd = new Random();
        int numMasses = rnd.Next(0,3);
        for(int i = 0; i < numMasses; i++)
        {
            int mass = rnd.Next(0, 21);
            int position = rnd.Next(0, 11) * 2;
            weights[i] = (mass, position);
        }
    }
}
```

```
public void SetPuzzle()
{
    double torque = 0;
    while (torque == 0) //ensures that the puzzle isnt already solved
    {
        SetMasses();
        SetRandomPivots();
        torque = CalculateTorque(pivots[0],0);
    }
}
```

The above code will create a puzzle that is not already balanced; however, it does not account for whether the puzzle is solvable yet with the environmental constraints. To ensure this, an algorithm will be needed to find at least one solution to the problem within the constraints.

```
Valid(torque,pivotPosition)
Valid = false
IF torque is negative //ie anticlockwise moment
      Find positions of all the available mass positions to the right of the active
      Pivot
      Find mass values for those selected positions that would give a moment equal
      to the absolute value of the torque
      If found, valid = true
IF torque is positive //ie clockwise moment
      Find positions of all the available mass positions to the left of the active
      Pivot
      Find mass values for those selected positions that would give a moment equal
      to the absolute value of the torque
      If found, valid = true
Return valid
```

In order to find the mass values, I needed another algorithm which could solve the following problem

There is:
n masses *(where n < 10 and is known)*

Each mass has a known distance, distance[i] and an unknown value weight[i] *(where weight[i] < 20)*

The total moment, S is known

S = $\sum_{k=0}^{n-1} \text{weight}[i] * distance[i]$

To find whether there is a solution to this Diophantine problem, I am going to use a backtracking algorithm. By considering this problem as a tree structure, where each level represents a mass, and each possible weight for that mass forms a branch and by systematically exploring this tree whilst pruning impossible branches I can efficiently determine whether a valid set of masses exist in the constraints.

```
FindValues(currentItem,runningSum,distances[],totalMoment)
      IF runningSum == totalMoment //base case
            RETURN true
      IF currentItem == n //base case where n is total number of masses
            RETURN false
      For w from 0 to 20
            sum = runningSum + w * distances[currentItem]
            if FindValue(currentItem + 1, sum, distances[], totalMoment)
                  return true
      return false
```

Each mass is considered in turn, and every possible weight (0–20) is tried, with the running total moment updated by adding the item's contribution (mass × distance). If the running total exceeds the target, that branch is skipped, as higher weights would only make the total larger. There are two base cases: 1) if the running total matches the target, the algorithm stops immediately; 2) if all items have been assigned but the total has not been reached, the current path fails.

```
public bool Validate(double torque, int pivotIndex)
{
    bool valid = false;
    List<int> Distances = new List<int>();
    List<int> massPositions = new List<int>();
    int pivotPosition = Convert.ToInt32(pivots[pivotIndex]);
    int numPivots = 0;
    if (torque < 0)
    {
        numPivots = 20 - pivotPosition / 2;
        for (int i = 0; i < numPivots; i++)
        {
            massPositions.Add(pivotPosition + (i + 1) * 2); //all the positions to the right of the pivot
        }
    }
    else
    {
        numPivots =  pivotPosition / 2;
        for (int i = 0; i < numPivots; i++)
        {
            massPositions.Add(pivotPosition - (i + 1) * 2); //all the positions to the right of the pivot
        }
        massPositions.Reverse();
    }
    foreach ((double mass, double position) weight in weights)
    {
        if (massPositions.Contains(Convert.ToInt32(weight.position)))
        {
            massPositions.Remove(Convert.ToInt32(weight.position)); //remove points where mass is already acting
        }
    }
    foreach (int position in massPositions)
    {
        Distances.Add(Math.Abs(pivotPosition - position));
    }
    valid = FindValues(0, 0, Distances, torque,numPivots);
    return valid;
}
```

```
public bool FindValues(int currentitem, double runningSum, List<int> distances, double totalMoment, int numPivots)
{
    if (runningSum == totalMoment)
    {
        return true;
    }
    if (currentitem == numPivots -1)
    {
        return false;
    }
    for (int w = 0; w <= 20; w++)
    {
        double sum = runningSum + w * distances[currentitem];
        if (FindValues(currentitem + 1, sum, distances, totalMoment, numPivots))
        {
            return true;
        }
    }
    return false;
}
```

```
public void SetPuzzle()
{
    double torque = 0;
    double ReactionPivot2 = 0;
    double ReactionPivot1 = 0;
    while (torque == 0) //ensures that the puzzle isnt already solved
    {
        SetMasses();
        SetRandomPivots();
        torque = CalculateTorque(pivots[0], 0);
    }
    if (pivots.Count > 1) //if there are more than two pivots this algorithm will check which pivot is causing the moment
    {
        ReactionPivot2 = torque / Math.Abs(pivots[1] - pivots[0]);
        double DownwardForce = 0;
        foreach (var weight in weights)
        {
            DownwardForce += weight.mass * gravity;
        }
        ReactionPivot1 = DownwardForce - ReactionPivot2;
    }
    bool valid = false;
    while (valid == false)
    {
        if (ReactionPivot1 < 0 || pivots.Count == 1)
        {
            valid = Validate(torque, 0);
        }
        else
        {
            valid = Validate(torque, 1);
        }
    }
    foreach (var weight in weights)
    {
        startMass[Convert.ToInt32(weight.position/2)] = Convert.ToInt32(weight.mass);
    }
}
```

When testing the program, I found that masses were being placed on the pivot which is something that I didn't want occurring, so I added another constraint to the mass generation

Pivots are located at 16m,
16kg mass is located at 16m
1kg mass is located at 1m

```
public void SetMasses()
{
    weights.Clear();
    Random rnd = new Random();
    int numMasses = rnd.Next(1, 4);
    for (int i = 0; i < numMasses; i++)
    {
        int mass = rnd.Next(0, 21);
        int position = rnd.Next(0, 11) * 2;
        while (position == pivots[0]) { position = rnd.Next(0, 11) * 2; }
        weights.Add((mass, position));
    }
}
```

As the code seemed to be stuck in an infinite loop, I decided to remove all conditional loops

```
int random = rnd.Next(0, 11) * 2;
if (i > 0)
{
    while (pivots[i - 1] == random) //ensures the pivot is not placed in the same position twice
    {
        random = rnd.Next(0, 11) * 2;
    }
}
pivots.Add(random);
```

```
public void SetRandomPivots()
{
    pivots.Clear();
    Random rnd = new Random();
    int numPivots = rnd.Next(1, 3);
    for (int i = 0; i < numPivots; i++)
    {

        int random = rnd.Next(0, 11) * 2;
        if (i > 0)
        {
            if (pivots[i - 1] == random) //ensures the pivot is not placed in the same position twice
            {
                if (random != 20) { random += 2; }
                else { random -= 2; }
            }
        }
        pivots.Add(random);
    }
}
```

When this didn't fix the problem, I placed breakpoints and found that the program was getting stuck in the FindValues function. On further inspection, I found that my pruning was not effective enough as the number of calls was still too large.

```
public bool FindValues(int currentitem, double runningSum, List<int> distances, double totalMoment, int numPivots)
{
    if (Math.Abs(runningSum - totalMoment) < 1e-6)
    {
        return true;
    }
    if (currentitem >= distances.Count)
    {
        return false;
    }
    if (runningSum > totalMoment) { return false; }
    for (int w = 0; w <= 20; w++)
    {
        double sum = runningSum + w * distances[currentitem];
        if (FindValues(currentitem + 1, sum, distances, totalMoment, numPivots))
        {
            return true;
        }
    }
    return false;
}
```

However, this still did not solve the problem as whilst Find Values was working as expected, Validate was constantly returning false leading to a infinite loop. Whilst further investigating, I found that the reason for this was that the torque value calculated in set puzzle was very large which meant that the puzzle could never be valid. I found that the issue was the calculate torque function was considering gravity whilst the validating functions where not. To remedy this, I have set gravity to 1 as in this context is it just an arbitrary scalar to the entire equation so has no overall impact.

```
public void SetMasses()
{
    weights.Clear();
    Random rnd = new Random();
    int numMasses = rnd.Next(1, 4); // 1-3 masses

    var usedPositions = new HashSet<int>();
    usedPositions.Add(Convert.ToInt32(pivots[0]));
    if (pivots.Count > 1)
    {
        usedPositions.Add(Convert.ToInt32(pivots[1]));
    }
    for (int i = 0; i < numMasses; i++)
    {
        int position;
        // pick a free position that is not a pivot
        do
        {
            position = rnd.Next(0, 11) * 2;
        } while (usedPositions.Contains(position));

        int mass;
        // pick a mass that is not numerically equal to its position
        do
        {
            mass = rnd.Next(1, 21); // 1-20 kg
        } while (mass == position);

        weights.Add((mass, position));
        usedPositions.Add(position); // mark this position as taken
    }

}
```

As the logic now worked as expected, I began coding the UI for displaying the puzzle. I wanted the scroll bars that had been set to not be visible so that the user does not try to change them so to make it obvious that there is a mass there I decided to place a rectangle in that location with its height proportional to its mass.

```
private void displayPuzzle()
{
    for (int index = 1; index <= 11; index++)
    {
        Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
        VScrollBar mass = this.Controls.Find("mass" + index, true).FirstOrDefault() as VScrollBar;
        lbl.Text = puzzleEngine.startMass[index-1] + "kg";
        mass.Value = mass.Maximum - puzzleEngine.startMass[index-1];
        if (puzzleEngine.startMass[index - 1] > 0 )
        {
            mass.Visible = false;
            Panel displayMass = new Panel();
            displayMass.BackColor = Color.Black;
            displayMass.Width = mass.Width;
            displayMass.Height = mass.Height * (mass.Value/20);
            displayMass.Location = new Point(mass.Location.X, mass.Location.Y + mass.Height);
            this.Controls.Add(displayMass);
            displayMass.BringToFront();


        }
    }
}
```

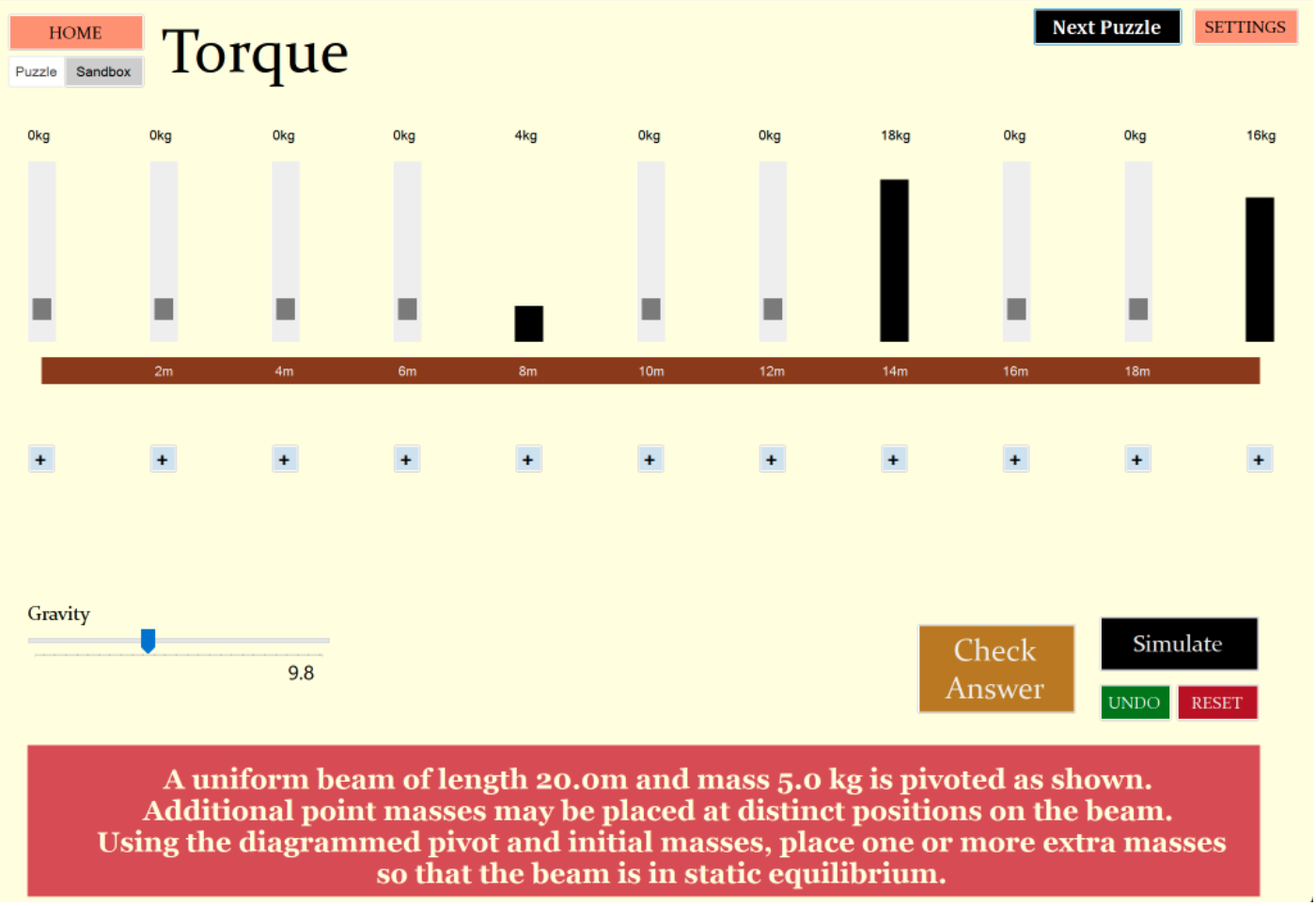
However since this line: displayMass.Height = mass.Height * (mass.Value/20); was integer division with the fraction always being ≤1 the height would always be 0 unless the mass.Value was 20 (Maximum value). To remedy this, I performed the calculation as a floating point and then converted to integer at the end as so:

```
private void displayPuzzle()
{
    for (int index = 1; index <= 11; index++)
    {
        Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
        VScrollBar mass = this.Controls.Find("mass" + index, true).FirstOrDefault() as VScrollBar;
        lbl.Text = puzzleEngine.startMass[index-1] + "kg";
        mass.Value = mass.Maximum - puzzleEngine.startMass[index-1];
        if (puzzleEngine.startMass[index - 1] > 0 )
        {
            mass.Visible = false;
            Panel displayMass = new Panel();
            displayMass.BackColor = Color.Black;
            displayMass.Width = mass.Width;
            double height = mass.Height * (mass.Value / 20.0);
            displayMass.Height = Convert.ToInt32(height);
            displayMass.Location = new Point(mass.Location.X, mass.Location.Y + mass.Height);
            this.Controls.Add(displayMass);
            displayMass.BringToFront();


        }
    }
}
```

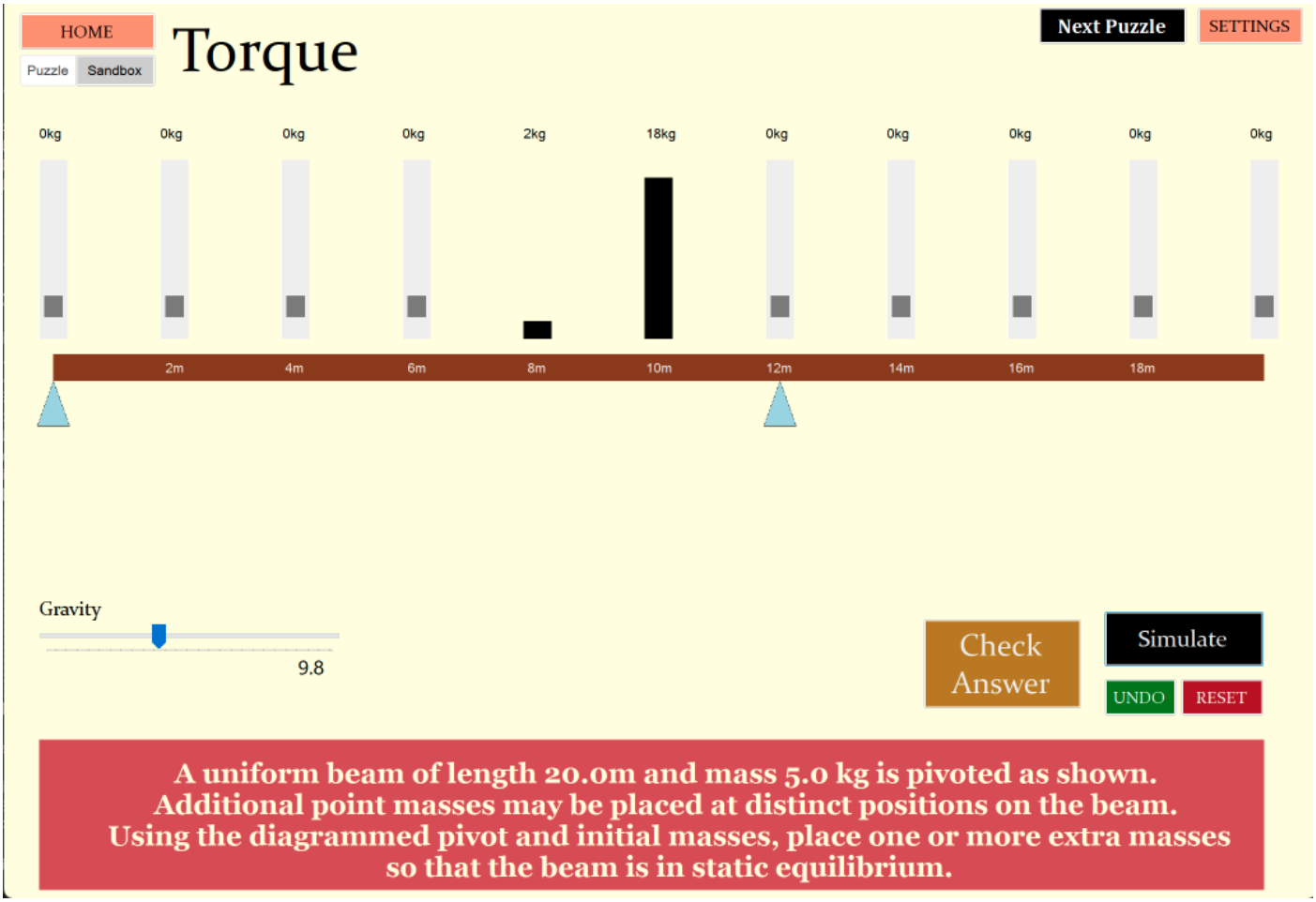
Now the mass was showing however it was still not formatting properly.

```
private void displayPuzzle()
{
    for (int index = 1; index <= 11; index++)
    {
        int massValue = puzzleEngine.startMass[index - 1];
        Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
        VScrollBar mass = this.Controls.Find("mass" + index, true).FirstOrDefault() as VScrollBar;
        Button pivotBut = this.Controls.Find("pivot" + index, true).FirstOrDefault() as Button;
        pivotBut.Visible = false;
        lbl.Text = massValue + "kg";
        mass.Value = mass.Maximum - massValue;
        if (massValue > 0)
        {
            mass.Visible = false;
            Panel displayMass = new Panel();
            displayMass.Name = "displayMass" + index;
            displayMass.BackColor = Color.Black;
            displayMass.Width = mass.Width;
            double height = mass.Height * (massValue / 20.0);
            displayMass.Height = Convert.ToInt32(height);
            displayMass.Location = new Point(mass.Location.X, mass.Location.Y + mass.Height - displayMass.Height);
            this.Controls.Add(displayMass);
            displayMass.BringToFront();
        }
    }
}
```



I know needed to display the pivot(s) that where acting on the system.

```
private void displayPuzzle()
{
    for (int index = 1; index <= 11; index++)
    {
        int massValue = puzzleEngine.startMass[index - 1];
        Label lbl = this.Controls.Find("lbl_m" + index, true).FirstOrDefault() as Label;
        VScrollBar mass = this.Controls.Find("mass" + index, true).FirstOrDefault() as VScrollBar;
        Button pivotBut = this.Controls.Find("pivot" + index, true).FirstOrDefault() as Button;
        pivotBut.Visible = false;
        lbl.Text = massValue + "kg";
        mass.Value = mass.Maximum - massValue;
        if (massValue > 0)
        {
            mass.Visible = false;
            Panel displayMass = new Panel();
            displayMass.Name = "displayMass" + index;
            displayMass.BackColor = Color.Black;
            displayMass.Width = mass.Width;
            double height = mass.Height * (massValue / 20.0);
            displayMass.Height = Convert.ToInt32(height);
            displayMass.Location = new Point(mass.Location.X, mass.Location.Y + mass.Height - displayMass.Height);
            this.Controls.Add(displayMass);
            displayMass.BringToFront();
        }
    }
    foreach (int pivot in puzzleEngine.pivots)
    {
        int index = pivot/2 + 1;
        Button pivotBut = this.Controls.Find("pivot" + index, true).FirstOrDefault() as Button;
        PivotClick(pivotBut, EventArgs.Empty);
    }
    warningmsg.Visible = false;
}
```



I wanted to allow the user to simulate and then go back to where they left off rather than resetting the sandbox/puzzle to the starting state. To do this, I decided to use another button. In order to ensure that I was able to call simulate again after that, I had to ensure that variables in the engine such as angle, and tipping were reset but that the pivots and masses list were not cleared.

```
private void closeSim_Click(object sender, EventArgs e)
{
    animationPanel.Visible = false;
    engine.PartialReset();
    this.Invalidate();
    closeSim.Visible = false;

}
```

```
public void Reset()
{
    angle = 0;
    gravity = 9.8;
    angularAcceleration = 0;
    angularPosition = 0;
    angularVelocity = 0;
    dt = 0;
    active = 0;
    tipping = false;
    pivots.Clear();
    weights.Clear();
}

1 reference
public void PartialReset()
{
    angle = 0;
    gravity = 9.8;
    angularAcceleration = 0;
    angularPosition = 0;
    angularVelocity = 0;
    dt = 0;
    active = 0;
    tipping = false;
}
```

I then started building the logic behind solving the puzzle. To do this, I checked whether what the user placed was balanced, as there could be a large number of ways that this problem could be solved so finding all the solutions and then comparing that to the user's answer would be inefficient.

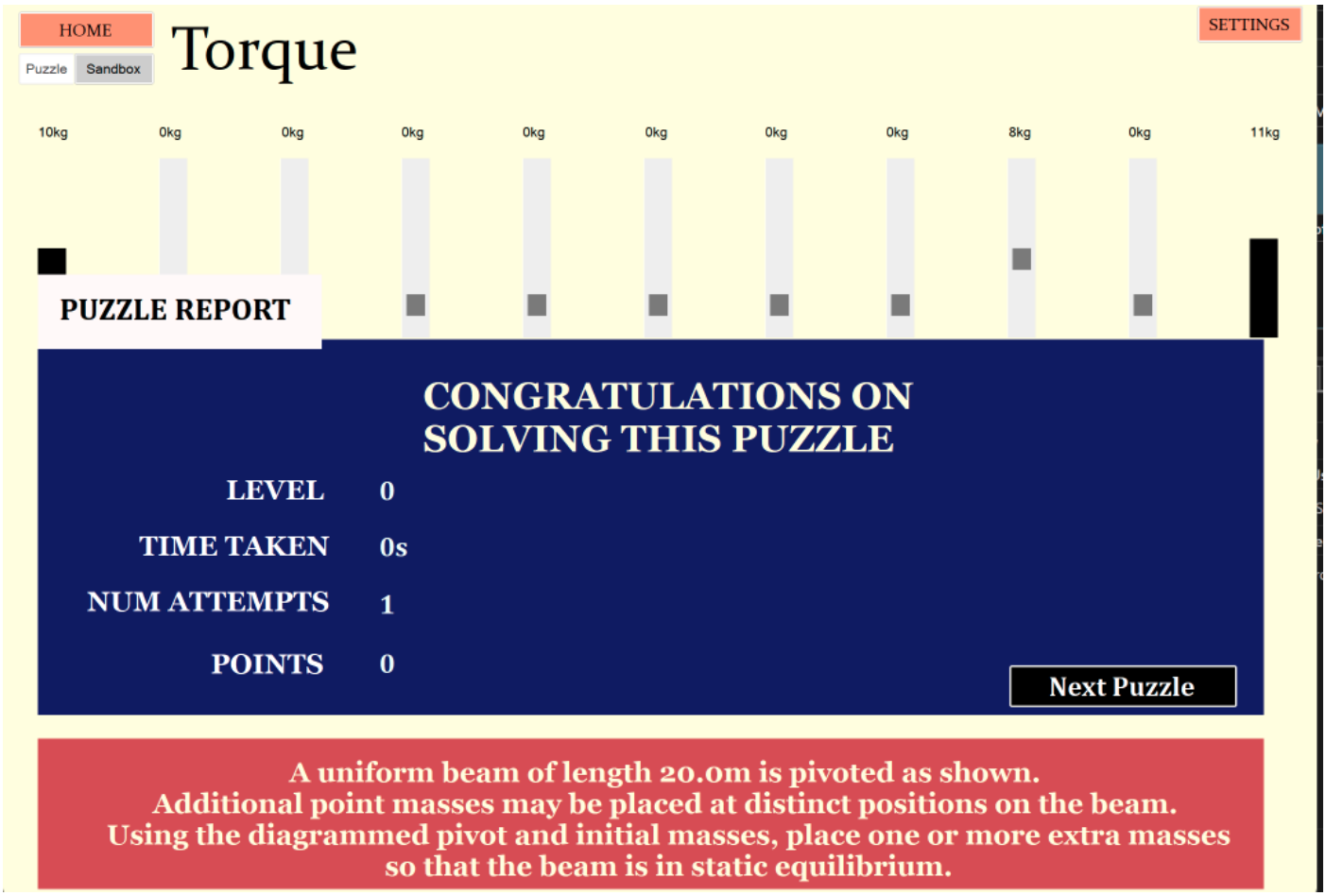
```
private void CheckAnswer_Click(object sender, EventArgs e)
{
    puzzleAttempts++;
    engine.weights.Clear();
    CleanList();
    foreach (var weight in placedMasses)
    {
        engine.AddWeight(weight.mass, 2 * (weight.index - 1));

    }
    engine.Update();
    if(engine.active == -1)
    {
        //puzzle solved
        Toggle(this, true, "Solved"); //displays the solved banner
        nextPuzzle.Visible = false; //ensures users do not click the next puzzle button that doesnt remvoe the banner
        SolvedAttempts.Text = puzzleAttempts.ToString();
        int points = 5; //points awarded for solving a problem
    }
    else
    {
        // incorrect

        MessageBox.Show("Uh Oh! Looks like that isn't quite right\nWhy don't you give it another try", $"Puzzle #{puzzleCode}", MessageBoxButtons.OK, Mess
    }


}
```

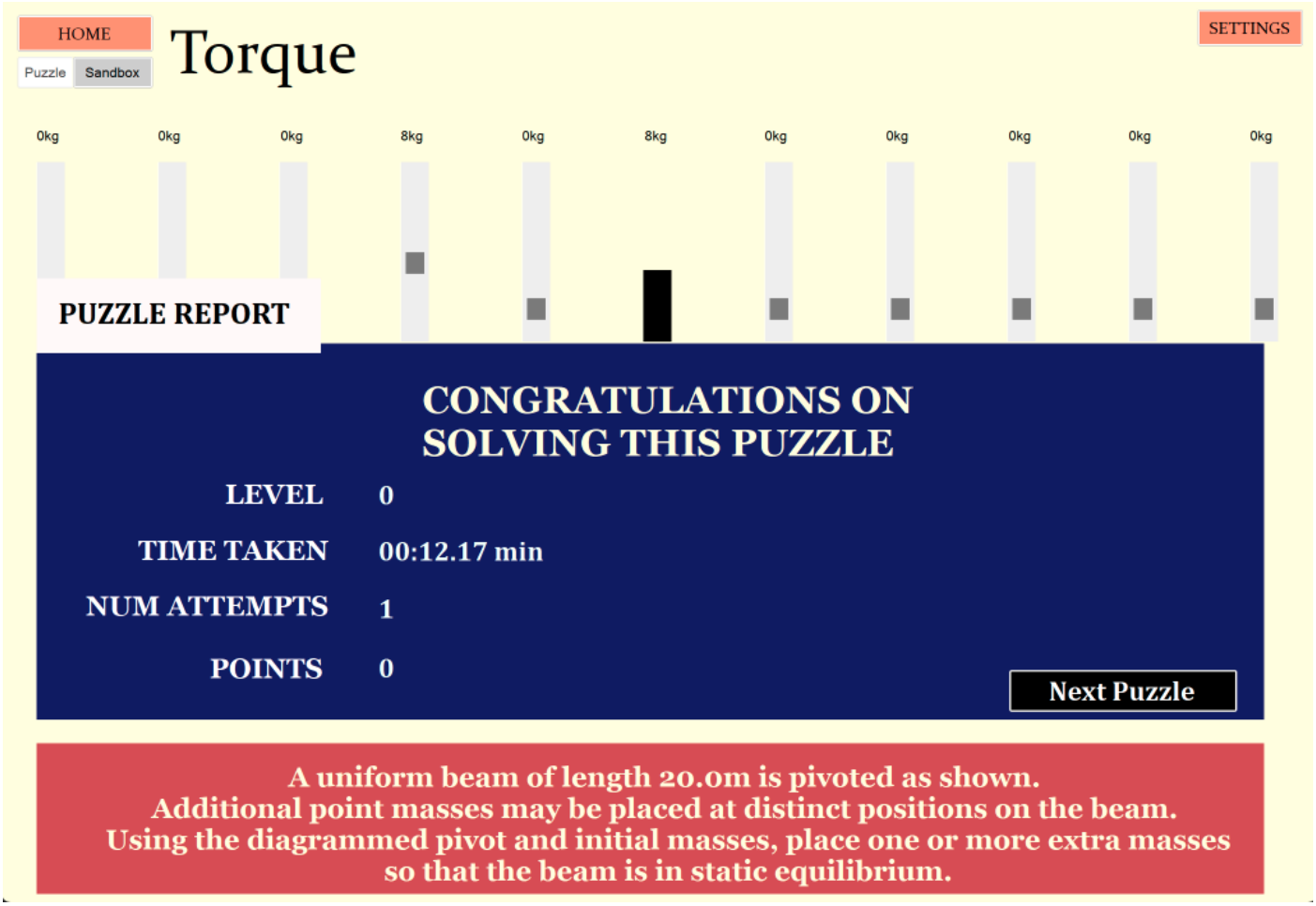
From there, I needed to find the time taken by the user and to do this I used a stopwatch that is built into windows forms.




```
Stopwatch stopwatch = new Stopwatch();
```

```
private void puzzle_Click(object sender, EventArgs e)
{
    mode = "puzzle";
    sandbox.BackColor = Color.LightGray;
    puzzle.BackColor = Color.White;
    Toggle(this, true, "Question");
    CheckAnswer.Visible = true;
    nextPuzzle.Visible = true;
    puzzle.Enabled = false; //no longer able to switch to puzzle mode as user is already in puzzle mode
    sandbox.Enabled = true;
    puzzleEngine.SetPuzzle();
    displayPuzzle();
    usedSimulation = false;
    Random rnd = new Random();
    puzzleCode = rnd.Next(1000, 9999);
    stopwatch.Reset();
    stopwatch.Start();
}
```

```
stopwatch.Stop();
TimeSpan elapsed = stopwatch.Elapsed;
string timeElapsed = $"{elapsed.Minutes:D2}:{elapsed.Seconds:D2}.{elapsed.Milliseconds / 10:D2} min";
```

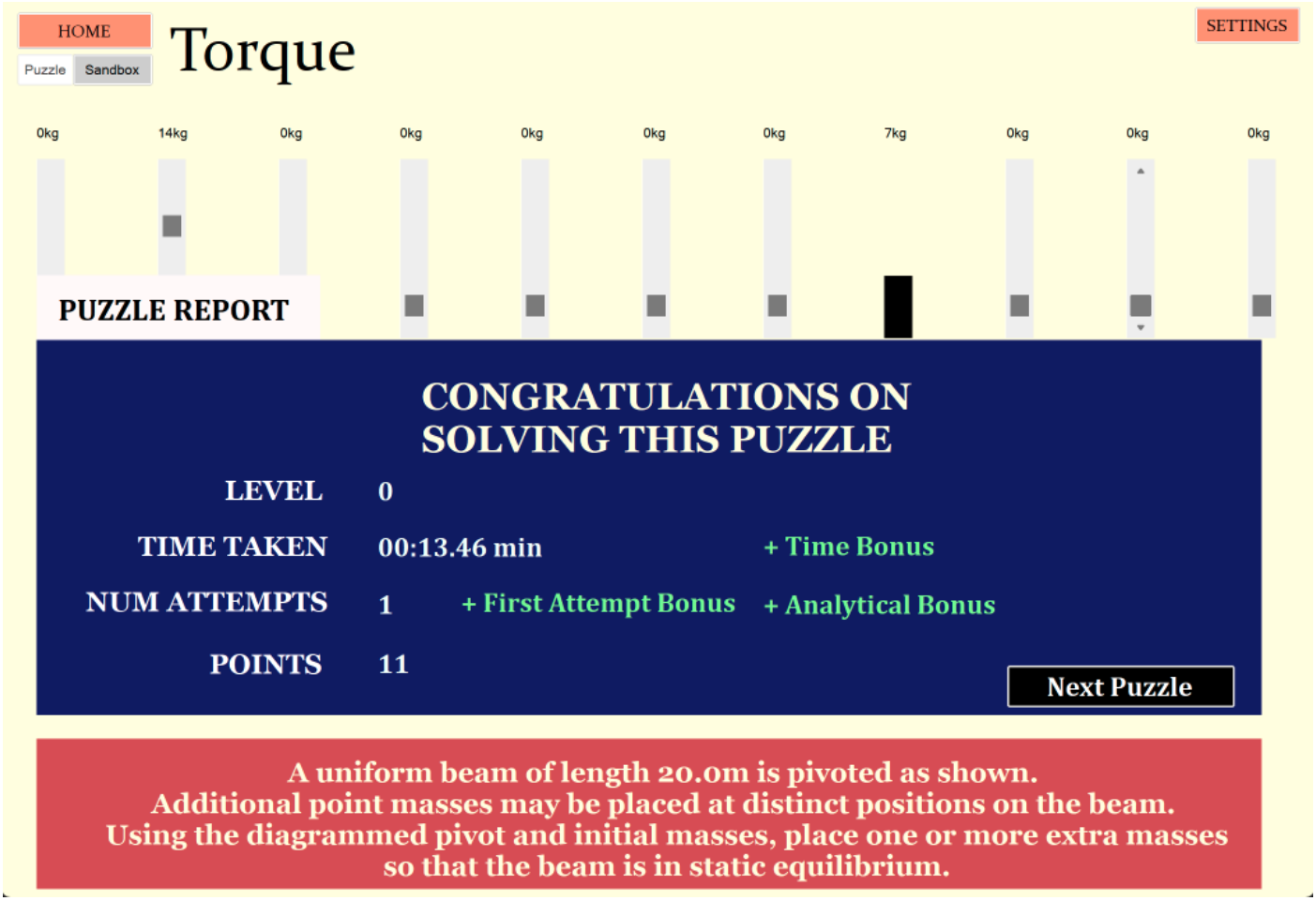


Once this worked, I then moved on to calculating the points for the puzzle. For scoring this is the values that I have used.

	Points
Solving Correctly	+5
Per Bonus	+2

```
if(engine.active == -1)
{
    //puzzle solved
    stopwatch.Stop();
    TimeSpan elapsed = stopwatch.Elapsed;
    string timeElapsed = $"{elapsed.Minutes:D2}:{elapsed.Seconds:D2}.{elapsed.Milliseconds / 10:D2} min";
    int seconds = (int)stopwatch.Elapsed.TotalSeconds;
    Toggle(this, true, "Solved"); //displays the solved banner
    nextPuzzle.Visible = false; //ensures users do not click the next puzzle button that doesnt remvoe the banner
    SolvedAttempts.Text = puzzleAttempts.ToString();
    SolvedTime.Text = timeElapsed;
    int points = 5; //points awarded for solving a problem


    if (seconds < 30)
    {
        TimeBonus.Visible = true;
        points += 2;
    }
    if (puzzleAttempts == 1)
    {
        FirstAttemptBonus.Visible = true;
        points += 2;
        if (!usedSimulation)
        {
            AnalyticalBonus.Visible = true;
            points += 2;
        }
    }
    SolvedPoints.Text = points.ToString();
```

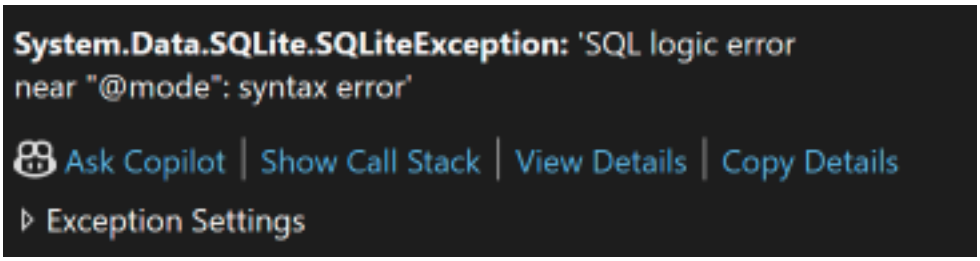


My next step was to add this data to the database so that it could be used in leaderboards.

```
public void AddScore(int UserID,  int Score, string mode)
{
    mode += "Score";
    string sql = "UPDATE Scores SET @mode = @mode + @Score, TotalScore = TotalScore + @Score WHERE UserID = @UserID";
    ExecuteNonQuery(sql,
        new SQLiteParameter("@mode", mode),
        new SQLiteParameter("@Score", Score),
        new SQLiteParameter("@UserID", UserID)
        );
}
```

```
db.AddScore(Session.UserID, points, "Torque");
```

However when trying to run this code, I encountered this error



I found that this error occurred because I attempted to parameterise the column headings of the table rather than passing them directly into the sql statement. The fixed version is below.

```
public void AddScore(int UserID,  int Score, string mode)
{
    mode += "Score";
    string sql = $"UPDATE Scores SET {mode} = {mode} + @Score, TotalScore = TotalScore + @Score WHERE UserID = @UserID";
    ExecuteNonQuery(sql,
        new SQLiteParameter("@Score", Score),
        new SQLiteParameter("@UserID", UserID)
        );
}
```

However, when I tried to run this code, I found that nothing was being added to the table and the reason was that, I was trying to update the table where UserID == @UserID however, the scores table was empty. So I've added the default values to the scores table when initialising a new table

```
int userID = GetUserID(username);
string sql2 = @"
    INSERT INTO Scores(UserID, ProjectileScore, ProjectileLevel, TorqueScore, TorqueLevel, LeaderboardPosition, TotalScore)
    VALUES(@UserID,0,1,0,1,0,0);
    ";
ExecuteNonQuery(sql2,
    new SQLiteParameter("@UserID",userID)
    );
```

	ScoreID	UserID	Projectile$	ProjectileL	TorqueSc	TorqueLe	Leaderb	TotalScore
1	1	3	0	0	11	0	0	11
2	2	11	0	1	11	1	0	11

As my torque puzzles were now working as expected, I moved on to making my projectile puzzles. For this the first thing that I did was ensure that I built a child class to handle all the logic behind making a puzzle.

```
internal class ProjectilePuzzle : PhysicsEngine
{
    int level;
    int pivotPosition = 20;
    public Dictionary<string, double> envFactors = new Dictionary<string, double>();
    int startHeight = 0;
    int angle = 0;
    int initVel = 0;
    double gravity = 0;
    0 references
    public ProjectilePuzzle(int userLevel) : base(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
    {
        level = userLevel;
        envFactors["mass"] = 1;
        envFactors["drag"] = 0;
        envFactors["COF"] = 0;
        envFactors["COR"] = 0;
    }
    0 references
    public void setPuzzle()
    {

    }
```

```
public void setEnvironmentalFactors()
{
    Random rnd = new Random();
    int mass = rnd.Next(1, 101);
    double drag = rnd.Next(0, 20) / 10.0;
    double cof = rnd.NextDouble();
    double cor = rnd.NextDouble();
    envFactors["mass"] = mass;
    envFactors["drag"] = drag;
    envFactors["COF"] = cof;
    envFactors["COR"] = cor;
}

0 references
public void setPivotPosition()
{
    Random rnd = new Random();
    pivotPosition = rnd.Next(5, 35);
}

0 references
public void setStartHeight()
{
    Random rnd = new Random();
    startHeight = rnd.Next(0, 10);
}

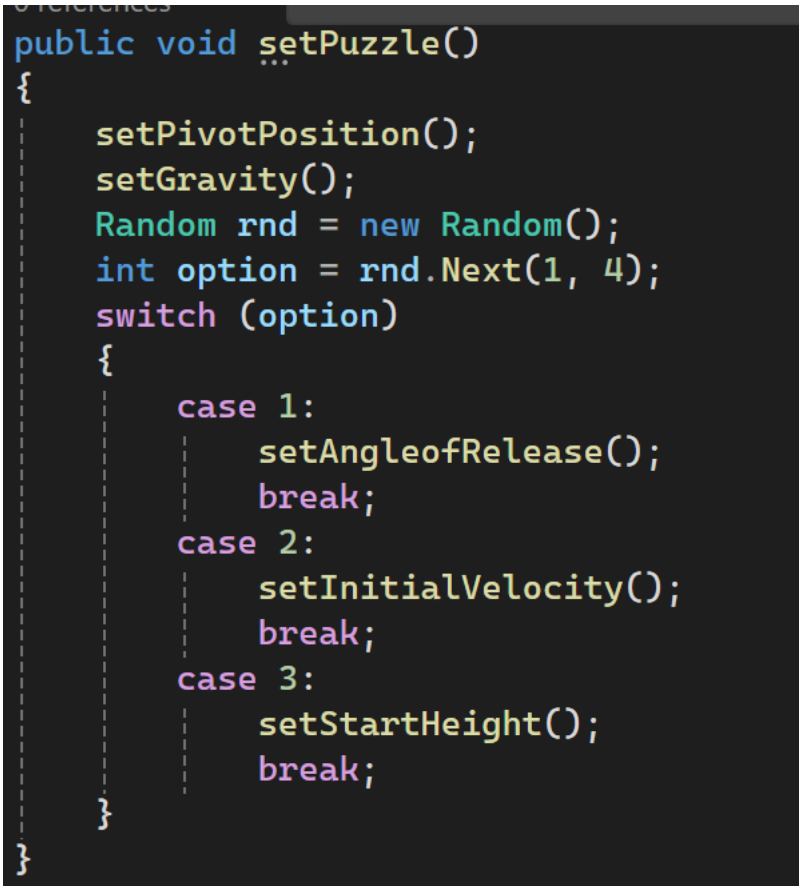
0 references
public void setAngleofRelease()
{
    Random rnd = new Random();
    angle = rnd.Next(-45, 90);
}
```

```
public void setInitialVelocity()
{
    Random rnd = new Random();
    rnd.Next(1, 20);
}

0 references
public void setGravity()
{
    Random rnd = new Random();
    gravity = rnd.Next(50, 250) / 10.0;
}
```

I chose to separate all the random assignments into separate functions as in this puzzle it is randomly decided what is pre set and what the user has to change in order to solve the puzzle. If the ending position and gravity

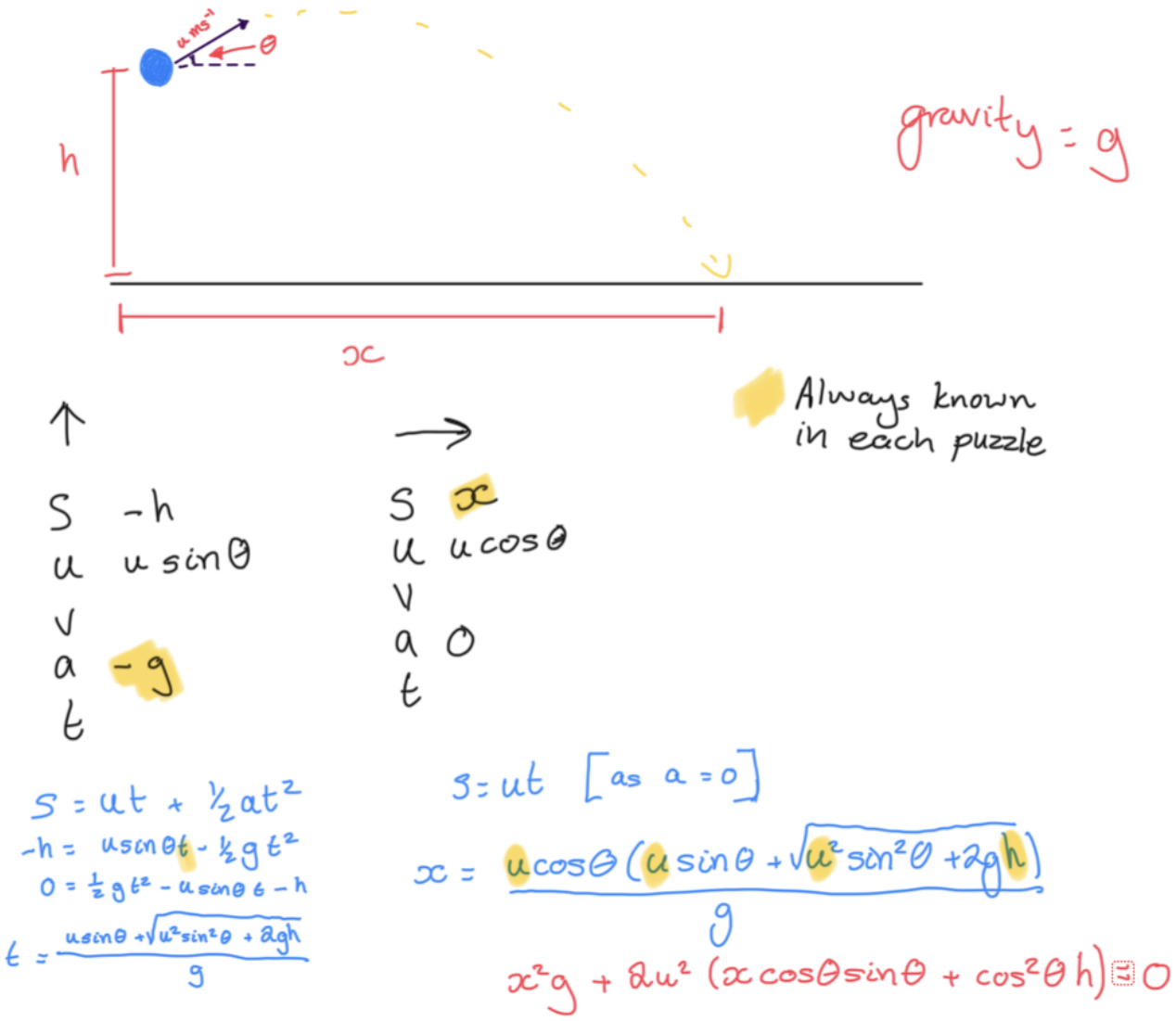
are always pre set and one of angle, initial velocity and height is set then the puzzle will always have only one possible solution.



```
public void setPuzzle()
{
    setPivotPosition();
    setGravity();
    Random rnd = new Random();
    int option = rnd.Next(1, 4);
    switch (option)
    {
        case 1:
            setAngleofRelease();
            break;
        case 2:
            setInitialVelocity();
            break;
        case 3:
            setStartHeight();
            break;
    }
}
```

I chose to use a switch-case statement rather than if-else as it allows for better readability and performance in my situation.

When formulating out my problem



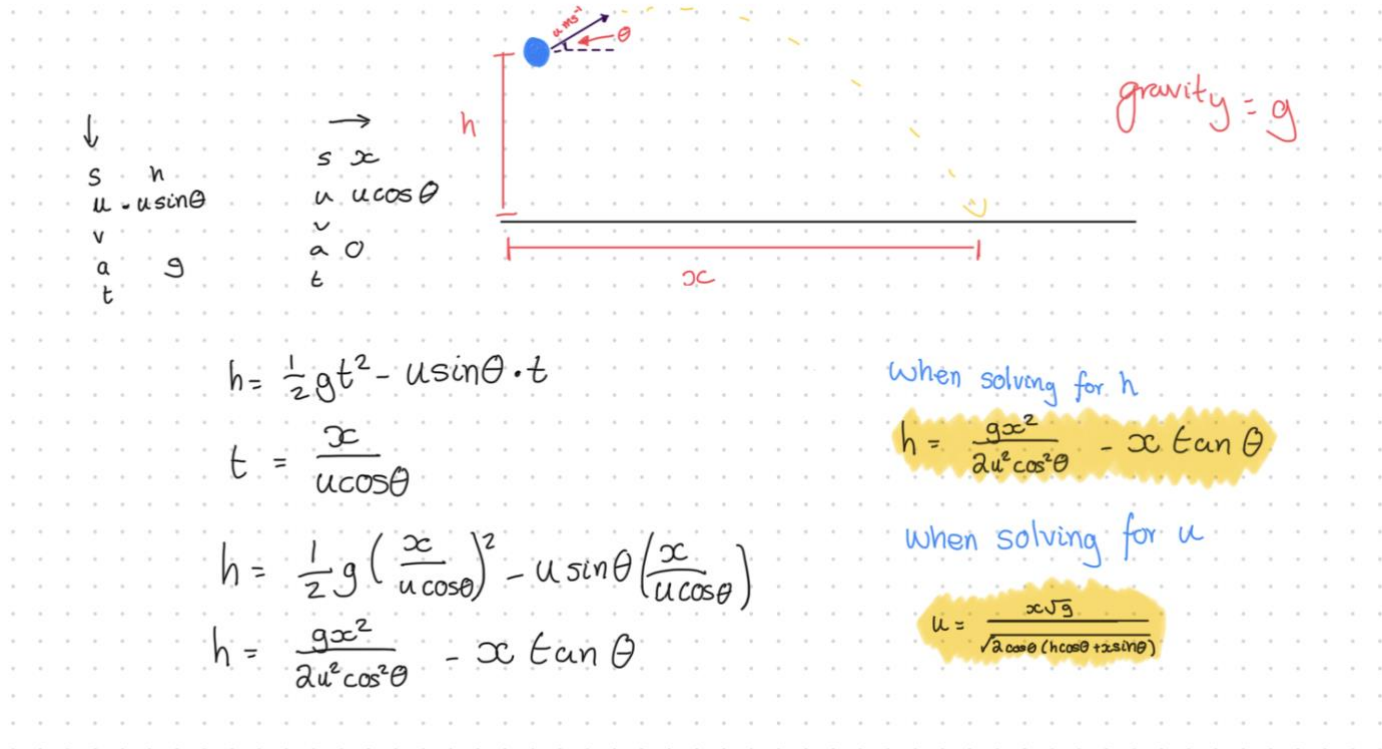
From these equations I found that, I could only have a maximum of one unknown to be able to definitively solve the problem so due to this, I decided to set up my levels as follows

Level	Type of Problem
Level 1	One Unknown **without** environmental factors
Level 2	Two Unknowns **without** environmental factors
Level 3	One Unknown **with** environmental factors
Level 4	Two Unknowns **with** environmental factors

In all cases, x (horizontal distance) and g (gravitational acceleration) are set. The unknowns are chosen from {θ, v_0, h}, with the other(s) provided. Environmental factors such as mass, drag, coefficient of friction (COF), or coefficient of restitution (COR) are included only in Levels 3 and 4. Known parameters are randomly selected within their allowed ranges, and the unknown(s) are computed using the appropriate equations of motion, including adjustments for environmental factors when relevant. The program then checks whether the solution is physically valid, such as ensuring integer values, realistic launch angles, and heights within the permitted range. If the solution is invalid, the process repeats with a new random selection until a valid puzzle is found. This method guarantees that each generated puzzle is solvable, preserves randomness by producing a fresh challenge every time, and remains efficient without the need to precompute or store large sets of puzzles, while also being flexible enough to accommodate additional constraints or environmental factors in higher-level problems. To improve computational efficiency, the program does not rely on pure random guessing but instead implements pruning and constraints during puzzle generation. First, candidate values for parameters

are randomly selected only within feasible ranges: for example, θ avoids angles where cos θ is near zero (which would make computations invalid or produce extreme results), and v_0 is constrained so that the resulting height h can plausibly fall within the allowed range. Second, by including the unknown(s) in the random selection and immediately computing the other parameter(s), the program significantly increases the probability of generating a valid solution on each iteration, thereby reducing the number of wasted trials. Invalid solutions are discarded early, avoiding unnecessary calculations with unrealistic values. These strategies—constrained sampling, early validation, and targeted computation—prune the search space, ensuring that each generated puzzle is both solvable and produced efficiently, even for higher-level problems with environmental factors.

```
public void setAngleofRelease()
{
    Random rnd = new Random();
    bool valid = false;
    while (!valid)
    {
        angle = rnd.Next(-45, 90);
        double thetaRad = angle * Math.PI / 180;
        // Prune impossible angles where cosθ too small
        if (Math.Abs(Math.Cos(thetaRad)) > 0.01)
        {
            valid = true;
        }
    }
}
```



On further inspection, I found that it was impossible to algebraically solve for theta and instead would have to be solved with numerical methods. So, due to this, I changed my levels slightly to reflect this

Level	Type of Problem	Always Known
Level 1	One Unknown **without** environmental factors	Target Position, Gravity and Angle of Release
Level 2	Two Unknowns **without** environmental factors	Target Position, Gravity and Angle of Release
Level 3	Three Unknowns **without** environmental factors	Target Position and Gravity

```
public bool calculateVelocity()
{
    double thetaRad = angle * Math.PI / 180.0;
    double cosTheta = Math.Cos(thetaRad);
    double sinTheta = Math.Sin(thetaRad);
    double tanTheta = Math.Tan(thetaRad);
    double rootedTerm = 2 * cosTheta * (startHeight * cosTheta + targetPosition * sinTheta);
    double denominator = Math.Pow(rootedTerm, 0.5);
    double numerator = targetPosition * Math.Pow(gravity, 0.5);
    double velocity = numerator/denominator;
    if (velocity % 1 == 0) //check if velocity is an integer
    {
        initVel = Convert.ToInt32(velocity);
        return true;
    }
    else
    {
        return false;
    }

}
```

```
public bool calculateStartHeight()
{
    double thetaRad = angle * Math.PI / 180.0;
    double cosTheta = Math.Cos(thetaRad);
    double sinTheta = Math.Sin(thetaRad);
    double tanTheta = Math.Tan(thetaRad);
    double fracNum = gravity * Math.Pow(targetPosition, 2);
    double fracDenom = 2 * initVel * initVel * cosTheta * cosTheta;
    double frac = fracNum / fracDenom;
    double height = frac - (targetPosition * tanTheta);
    if (height % 1 == 0) //check if height is an integer
    {
        startHeight = Convert.ToInt32(height);
        return true; //valid result found
    }
    else
    {
        return false;
    }
}
```

The following pseudocode shows how I plan on setting up a puzzle

```
Function SetLevel1Puzzle()
      Valid = false;
      Select which of startHeight and velocity are unknown
      While valid is false
            set angle
            Set target
            Set gravity
            Set one of startHeight and velocity
            If startheight was set then  valid = calculateVelocity()
            If velocity was set then valid = calcualateStartHeight()
```

```
public void setLevel1Puzzle()
{
    bool valid = false;
    Random rnd = new Random();
    int option = rnd.Next(1, 3);
    while (!valid)
    {
        setAngleofRelease();
        setTargetPosition();
        setGravity();
        switch (option)
        {
            case 1:
                setStartHeight();
                valid = calculateVelocity();
                break;
            default:
                setInitialVelocity();
                valid = calculateStartHeight();
                break;
        }
    }
}
```

This I however found to be rather inefficient as it was taking around 1000 attempts to find a puzzle each time. For this reason, I decided to go for a more deterministic approach. First, the function randomly selects whether the unknown variable will be the initial velocity or the start height. It then sets a launch angle, gravity, and chooses a known integer value for the other variable using the existing functions. To guarantee a valid solution, the target position is iterated over a small, predefined range until the calculation functions (calculateVelocity() or calculateStartHeight()) produce an integer for the unknown. This approach reuses the existing functions while pruning impossible angles and restricting values to realistic ranges, ensuring the puzzle is physically

correct, efficiently generated, and always solvable with integers. By generating the puzzle in this way, the system avoids repeated random attempts or rounding, producing reliable and consistent puzzles for the player.

```
public void setLevel1Puzzle()
{
    Random rnd = new Random();
    bool unknownIsVelocity = rnd.Next(0, 2) == 0; // true = velocity unknown, false = startHeight unknown
    setAngleofRelease();
    setGravity();
    if (unknownIsVelocity)
    {

        setStartHeight();
    }
    else
    {

        setInitialVelocity();
    }
    for (int x = 5; x <= 35; x++)
    {
        targetPosition = x;

        if (unknownIsVelocity)
        {
            if (calculateVelocity())
            {

                break;
            }
        }
        else
        {
            if (calculateStartHeight())
            {

                break;
            }
        }
    }
}
```

However, this approach also proved unsuccessful. Because of the limited number of viable options, I instead decided to generate a dictionary containing all possible combinations and then selectively hide and reveal elements from it to create each puzzle.

```
public void generatePuzzleBank()
{
    int id = 0;
    Random rnd = new Random();

    for (int g = 50; g <= 250; g++)
    {
        double gravity = g / 10.0;

        for (int angle = 5; angle < 85; angle += 5) // avoid flat or vertical
        {
            for (int h = 0; h <= 10; h++) // startHeight
            {
                for (int v = 1; v <= 25; v++) // velocity
                {
                    // Try to compute target position using projectile formula
                    double thetaRad = angle * Math.PI / 180.0;
                    double cosTheta = Math.Cos(thetaRad);
                    double sinTheta = Math.Sin(thetaRad);
                    double tanTheta = Math.Tan(thetaRad);

                    double xSquared = (2 * v * v * cosTheta * cosTheta * (h + v * v * sinTheta * sinTheta / (2 * gravity))) / gravity;

                    if (xSquared > 0)
                    {
                        double x = Math.Sqrt(xSquared);

                        // keep only integer target positions
                        if (Math.Abs(x - Math.Round(x)) < 0.0001)
                        {
                            int target = (int)Math.Round(x);
                            if (target >= 5 && target <= 35) // target must be within bounds
                            {
                                puzzleBank[id++] = (h, angle, v, target, gravity);
                            }
                        }
                    }
                }
            }
        }
    }
}
```

This dictionary is built by systematically iterating through reasonable ranges for start height, angle of release, initial velocity, gravity, and target position. For each combination, I apply the projectile motion equations and only store it in the dictionary if the result produces integer values for all parameters and falls within realistic game bounds. Each entry in the dictionary is stored with a unique ID, making it easy to retrieve and manage puzzles.

This approach guarantees that every puzzle presented to the user is valid, avoids unnecessary computation at runtime, and ensures consistency across different difficulty levels. It also allows me to simply “hide” one of the parameters (e.g., start height or velocity) when presenting the puzzle, turning the complete solution into a challenge for the player.

This successfully creates 315 unique setups that can be used to form puzzles as seen below.

ID: 0 | StartHeight: 0, Angle: 15, Velocity: 10, Target: 5, Gravity: 5
ID: 1 | StartHeight: 0, Angle: 15, Velocity: 20, Target: 20, Gravity: 5
ID: 2 | StartHeight: 5, Angle: 30, Velocity: 10, Target: 15, Gravity: 5
ID: 3 | StartHeight: 0, Angle: 45, Velocity: 10, Target: 10, Gravity: 5
ID: 4 | StartHeight: 0, Angle: 75, Velocity: 10, Target: 5, Gravity: 5
ID: 5 | StartHeight: 0, Angle: 75, Velocity: 20, Target: 20, Gravity: 5
ID: 6 | StartHeight: 9, Angle: 70, Velocity: 10, Target: 9, Gravity: 5.1
ID: 7 | StartHeight: 10, Angle: 70, Velocity: 12, Target: 12, Gravity: 5.2
ID: 8 | StartHeight: 3, Angle: 20, Velocity: 21, Target: 34, Gravity: 5.3
ID: 9 | StartHeight: 0, Angle: 15, Velocity: 18, Target: 15, Gravity: 5.4
ID: 10 | StartHeight: 0, Angle: 45, Velocity: 18, Target: 30, Gravity: 5.4
ID: 11 | StartHeight: 5, Angle: 60, Velocity: 6, Target: 5, Gravity: 5.4
ID: 12 | StartHeight: 0, Angle: 75, Velocity: 18, Target: 15, Gravity: 5.4
ID: 13 | StartHeight: 0, Angle: 15, Velocity: 22, Target: 22, Gravity: 5.5
ID: 14 | StartHeight: 0, Angle: 45, Velocity: 11, Target: 11, Gravity: 5.5
ID: 15 | StartHeight: 0, Angle: 75, Velocity: 22, Target: 22, Gravity: 5.5
ID: 16 | StartHeight: 1, Angle: 80, Velocity: 17, Target: 9, Gravity: 5.6
ID: 17 | StartHeight: 4, Angle: 30, Velocity: 15, Target: 23, Gravity: 5.7
ID: 18 | StartHeight: 6, Angle: 10, Velocity: 5, Target: 7, Gravity: 6
ID: 19 | StartHeight: 0, Angle: 15, Velocity: 12, Target: 6, Gravity: 6
ID: 20 | StartHeight: 0, Angle: 15, Velocity: 24, Target: 24, Gravity: 6
ID: 21 | StartHeight: 1, Angle: 30, Velocity: 12, Target: 12, Gravity: 6
ID: 22 | StartHeight: 6, Angle: 30, Velocity: 12, Target: 18, Gravity: 6
ID: 23 | StartHeight: 6, Angle: 35, Velocity: 17, Target: 30, Gravity: 6
ID: 24 | StartHeight: 0, Angle: 45, Velocity: 12, Target: 12, Gravity: 6
ID: 25 | StartHeight: 0, Angle: 45, Velocity: 18, Target: 27, Gravity: 6
ID: 26 | StartHeight: 2, Angle: 60, Velocity: 20, Target: 30, Gravity: 6
ID: 27 | StartHeight: 3, Angle: 60, Velocity: 12, Target: 12, Gravity: 6
ID: 28 | StartHeight: 8, Angle: 60, Velocity: 8, Target: 8, Gravity: 6
ID: 29 | StartHeight: 0, Angle: 75, Velocity: 12, Target: 6, Gravity: 6
ID: 30 | StartHeight: 0, Angle: 75, Velocity: 24, Target: 24, Gravity: 6
ID: 31 | StartHeight: 3, Angle: 15, Velocity: 7, Target: 7, Gravity: 6.1
ID: 32 | StartHeight: 0, Angle: 45, Velocity: 21, Target: 35, Gravity: 6.3
ID: 33 | StartHeight: 0, Angle: 15, Velocity: 16, Target: 10, Gravity: 6.4
ID: 34 | StartHeight: 10, Angle: 30, Velocity: 16, Target: 30, Gravity: 6.4
ID: 35 | StartHeight: 0, Angle: 45, Velocity: 8, Target: 5, Gravity: 6.4
ID: 36 | StartHeight: 0, Angle: 45, Velocity: 16, Target: 20, Gravity: 6.4
ID: 37 | StartHeight: 5, Angle: 60, Velocity: 16, Target: 20, Gravity: 6.4
ID: 38 | StartHeight: 0, Angle: 75, Velocity: 16, Target: 10, Gravity: 6.4
ID: 39 | StartHeight: 10, Angle: 20, Velocity: 13, Target: 23, Gravity: 6.5
ID: 40 | StartHeight: 0, Angle: 45, Velocity: 13, Target: 13, Gravity: 6.5
ID: 41 | StartHeight: 2, Angle: 70, Velocity: 17, Target: 15, Gravity: 6.5
ID: 42 | StartHeight: 7, Angle: 5, Velocity: 12, Target: 17, Gravity: 7
ID: 43 | StartHeight: 0, Angle: 15, Velocity: 14, Target: 7, Gravity: 7
ID: 44 | StartHeight: 9, Angle: 25, Velocity: 14, Target: 23, Gravity: 7
ID: 45 | StartHeight: 7, Angle: 30, Velocity: 14, Target: 21, Gravity: 7
ID: 46 | StartHeight: 0, Angle: 45, Velocity: 14, Target: 14, Gravity: 7
ID: 47 | StartHeight: 0, Angle: 75, Velocity: 14, Target: 7, Gravity: 7
ID: 48 | StartHeight: 7, Angle: 80, Velocity: 22, Target: 13, Gravity: 7
ID: 49 | StartHeight: 7, Angle: 50, Velocity: 11, Target: 13, Gravity: 7.1
ID: 50 | StartHeight: 0, Angle: 15, Velocity: 12, Target: 5, Gravity: 7.2
ID: 51 | StartHeight: 0, Angle: 15, Velocity: 24, Target: 20, Gravity: 7.2
ID: 52 | StartHeight: 7, Angle: 15, Velocity: 17, Target: 25, Gravity: 7.2
ID: 53 | StartHeight: 5, Angle: 30, Velocity: 12, Target: 15, Gravity: 7.2
ID: 54 | StartHeight: 5, Angle: 35, Velocity: 17, Target: 25, Gravity: 7.2
ID: 55 | StartHeight: 0, Angle: 45, Velocity: 12, Target: 10, Gravity: 7.2
ID: 56 | StartHeight: 0, Angle: 75, Velocity: 12, Target: 5, Gravity: 7.2
ID: 57 | StartHeight: 0, Angle: 75, Velocity: 24, Target: 20, Gravity: 7.2
ID: 58 | StartHeight: 4, Angle: 30, Velocity: 16, Target: 21, Gravity: 7.3
ID: 59 | StartHeight: 0, Angle: 45, Velocity: 15, Target: 15, Gravity: 7.5
ID: 60 | StartHeight: 10, Angle: 60, Velocity: 10, Target: 10, Gravity: 7.5
ID: 61 | StartHeight: 9, Angle: 15, Velocity: 17, Target: 27, Gravity: 7.6
ID: 62 | StartHeight: 6, Angle: 40, Velocity: 21, Target: 35, Gravity: 7.6

```
public void setLevel1Puzzle()
{

    puzzlekey = rnd.Next(puzzleBank.Count); // pick random puzzle

    var puzzle = puzzleBank[puzzlekey];
    int choice = rnd.Next(2);
    startingValues.angle = puzzle.angle;
    startingValues.target = puzzle.target;
    startingValues.gravity = puzzle.gravity;

    switch (choice)
    {
        case 0:
            startingValues.startHeight = -1; //what will be hidden from user
            startingValues.velocity = puzzle.velocity;
            startHeight = puzzle.startHeight;

            break;
        case 1:
            startingValues.velocity = -1; //what will be hidden from user
            startingValues.startHeight = puzzle.startHeight;
            initVel = puzzle.velocity;
            break;
    }

}
0 references
public void setLevel2Puzzle()
{

    puzzlekey = rnd.Next(puzzleBank.Count); // pick random puzzle

    var puzzle = puzzleBank[puzzlekey];
    int choice = rnd.Next(2);
    startingValues.angle = puzzle.angle;
    startingValues.target = puzzle.target;
    startingValues.gravity = puzzle.gravity;
    startingValues.startHeight = -1; //what will be hidden from user
    startingValues.velocity = -1; //what will be hidden from user
    startHeight = puzzle.startHeight;
    initVel = puzzle.velocity;
}
```

```
public void setLevel3Puzzle()
{

    puzzlekey = rnd.Next(puzzleBank.Count); // pick random puzzle

    var puzzle = puzzleBank[puzzlekey];
    int choice = rnd.Next(3);
    startingValues.target = puzzle.target;
    startingValues.gravity = puzzle.gravity;
    switch (choice)
    {
        case 0:
            startingValues.startHeight = -1; //what will be hidden from user
            startingValues.velocity = puzzle.velocity;
            startingValues.angle = puzzle.angle;
            startHeight = puzzle.startHeight;
            break;
        case 1:
            startingValues.velocity = -1; //what will be hidden from user
            startingValues.startHeight = puzzle.startHeight;
            startingValues.angle = puzzle.angle;
            initVel = puzzle.velocity;
            break;
        case 2:
            startingValues.angle = -1; //what will be hidden from user
            startingValues.startHeight = puzzle.startHeight;
            startingValues.velocity = puzzle.velocity;
            angle = puzzle.angle;
            break;
    }
}
```

This same method however, could not be used for Levels 4-6 as they also included the environmental factors. Thus, I decided to stop at level 3. The reason for this is that once factors such as drag or friction are introduced, the equations of motion can no longer be solved neatly using the standard kinematic formulae. Instead, they would require iterative or numerical solutions, which would make it much harder to guarantee that the puzzles always have an integer solution for the missing variable.

```
public void setPuzzle()
{
    int puzzleDifficulty = rnd.Next(1, level + 1);
    switch (puzzleDifficulty)
    {
        case 1:
            setLevel1Puzzle();
            break;
        case 2:
            setLevel2Puzzle();
            break;
        case 3:
            setLevel3Puzzle();
            break;
        default:
            setLevel1Puzzle();
            break;
    }
}
```

I now needed to build the UI and display the result of the user's input to the puzzle



```
private void setPuzzle()
{
    puzzle.level = db.GetLevel(Session.UserID,"Projectile");
    puzzle.setPuzzle();
    lbl_target.Text = puzzle.startingValues.target.ToString() + "m";
    gravity.Value = Convert.ToInt32(puzzle.startingValues.gravity * 10);
    gravity.Enabled = false;
    lbl_gravity.Text = puzzle.startingValues.gravity.ToString();
    if (puzzle.startingValues.startHeight == -1)
    {
        unknowns.Add("startHeight");
        SetLabelBold(HeightTitle, true);
    }
    else
    {
        lbl_startHeight.Text = puzzle.startingValues.startHeight.ToString() + "m";
        startHeight.Value = puzzle.startingValues.startHeight;
        startHeight.Enabled = false;
    }
    if (puzzle.startingValues.velocity == -1)
    {
        unknowns.Add("initVel");
        SetLabelBold(VelocityTitle, true);
    }
    else
    {
        lbl_initialVelocity.Text = puzzle.startingValues.velocity.ToString() + "m/s";
        initialVelocity.Value = puzzle.startingValues.velocity;
        initialVelocity.Enabled = false;
    }
    if (puzzle.startingValues.angle == -1)
    {
        unknowns.Add("angle");
        SetLabelBold(AngleTitle, true);
    }
    else
    {
        lbl_degree.Text = puzzle.startingValues.angle.ToString() + " deg";
        angle.Value = puzzle.startingValues.angle;
        angle.Enabled = false;
    }
    mass.Enabled = false;
    dragCoefficient.Enabled = false;
    COF.Enabled = false;
    COR.Enabled = false;

}
```

```
private void CheckAnswer_Click(object sender, EventArgs e)
{
    puzzleAttempts++;
    bool solved = false;
    foreach(string unknown in unknowns)
    {
        solved = false;
        if (unknown == "initVel")
        {
            if (initialVelocity.Value == puzzle.initVel)
            {
                solved = true;
            }
        }
        else if (unknown == "angle")
        {
            if (angle.Value == puzzle.angle)
            {
                solved = true;
            }
        }
        else if (unknown == "startHeight")
        {
            if (startHeight.Value == puzzle.startHeight)
            {
                solved = true;
            }
        }
    }
```

```
private double GetHorizontalDistance()
{
    double theta = angle.Value * Math.PI / 180.0;
    double velocity = initialVelocity.Value;
    double vX = velocity * Math.Cos(theta);
    double vY = velocity * Math.Sin(theta);
    double grav = gravity.Value;
    double height = startHeight.Value;
    double t = (vY + Math.Sqrt(vY * vY + 2 * grav * height)) / grav;
    double x = vX * t;
    return x;
}
```

```
private void puzzleSolved()
{
    Toggle(this, true, "Solved");
    SolvedAttempts.Text = puzzleAttempts.ToString();
    stopwatch.Stop();
    TimeSpan elapsed = stopwatch.Elapsed;
    string timeElapsed = $"{elapsed.Minutes:D2}:{elapsed.Seconds:D2}.{elapsed.Milliseconds / 10:D2} min";
    int seconds = (int)stopwatch.Elapsed.TotalSeconds;
    nextPuzzleBUT.Visible = false;
    SolvedLevel.Text = puzzle.level.ToString();
    SolvedTime.Text = timeElapsed;
    if (seconds < 60)
    {
        TimeBonus.Visible = true;
        points += 2;
    }
    if (puzzleAttempts == 1)
    {
        FirstAttemptBonus.Visible = true;
        points += 2;
        if (!usedSimulation)
        {
            AnalyticalBonus.Visible = true;
            points += 2;
        }
    }
    SolvedPoints.Text = points.ToString();
    db.AddScore(Session.UserID, points, "Projectile");
}
```

```
if (solved)
{

    points = 5;
    puzzleSolved();
    ExactPoints.Visible = true;

}
else if (Math.Abs(puzzle.startingValues.target - GetHorizontalDistance()) < 2.0)
{
    points = 3;
    partialPoints.Visible = true;
    puzzleSolved();
}
else
{
    MessageBox.Show("Uh Oh! Looks like that isn't quite right\nWhy don't you give it another try", $"Puzzle #{puzzleCode}", MessageBoxButtons.OK, Mess

}
```

In developing the projectile puzzle, I reused a significant amount of code from my previous Torque Puzzle project, which allowed me to efficiently implement core functionality such as variable storage and answer checking. To validate the player's input, I compared their answer against the correct value stored within the class. Recognising that calculations can involve rounding or approximations, I implemented a system where answers that were close to the correct value received partial credit. This approach ensured that the puzzle was both fair and robust, while leveraging previously tested code to reduce errors and development time.

Testing

Input	Testing	Justification	Expected Result	Result
Correctly Solve projectile motion puzzle - **Ensuring the speed threshold is met**	Projectile Motion Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points	**SUCCESSFUL** *See Figure 7.1*

Correctly Solve projectile motion puzzle - **Ensuring the first time threshold is met**	Projectile Motion Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points	**SUCCESSFUL** *See Prototype 3 Video 2*
Correctly Solve projectile motion puzzle - **Without using the simulation to check**	Projectile Motion Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points	**SUCCESSFUL** *See Figure 7.2*
Projectile Puzzle Generates a solvable random puzzle	Projectile Puzzle	Ensures that the puzzle can be used by the user	Solution only produces solvable puzzles	**SUCCESSFUL** *See Prototype 3 Video 3*
Projectile Puzzle allows for partial credit	Projectile Puzzle	Accounts for rounding errors and small discrepancies Ensures that puzzle doesn't unfairly punish users.	Solution awards partial credit	**UNSUCCESSFUL** *Due to the nature of the puzzle only allowing integer inputs the partial credit threshold will never be met.*
Torque Puzzle Generates a solvable random puzzle	Torque Puzzle	Ensures that the puzzle can be used by the user	Solution only produces solvable puzzles	**SUCCESSFUL** *See Prototype 3 Video 4*
Correctly Solve torque puzzle - **Ensuring the speed threshold is met**	Torque Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points	**SUCCESSFUL** *See Figure 7.3*
Correctly Solve torque puzzle - **Ensuring the first time threshold is met**	Torque Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points	**SUCCESSFUL** *See Figure 7.4*

Correctly Solve torque puzzle - **Without using the simulation to check**	Torque Puzzle and scoring	Ensures that puzzle works as expected and extra points are awarded when necessary	Solution recognises the correct solution and awards the bonus points	**SUCCESSFUL** *See Prototype 3 Video 5*
Adds scores to database	Scoring	Ensures that scores are stored persistently for leaderboards and user classification in future prototypes	Scores are updated in database	**SUCCESSFUL** *See Figure 7.5*

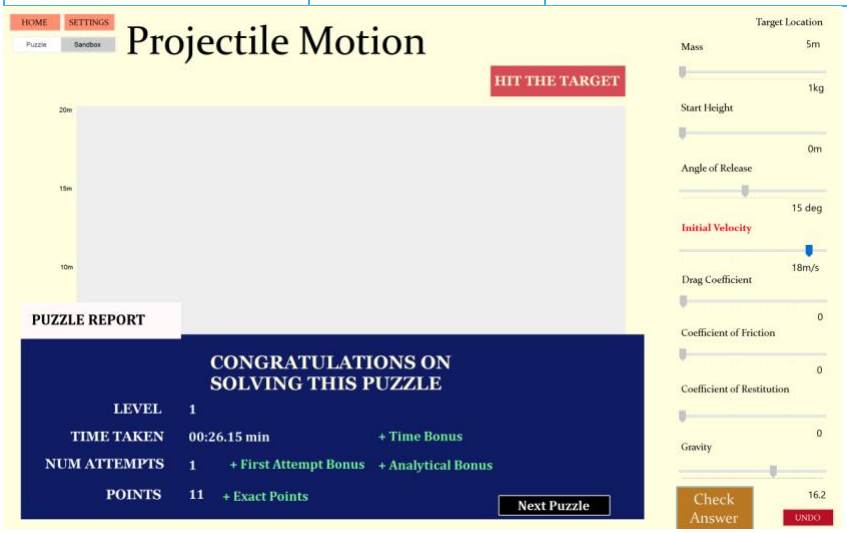


Figure 7. 1

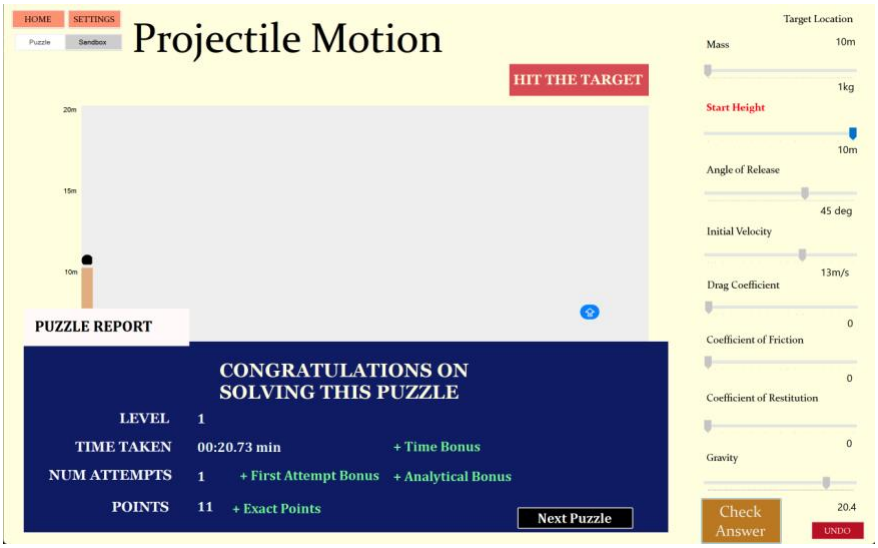


Figure 7. 2

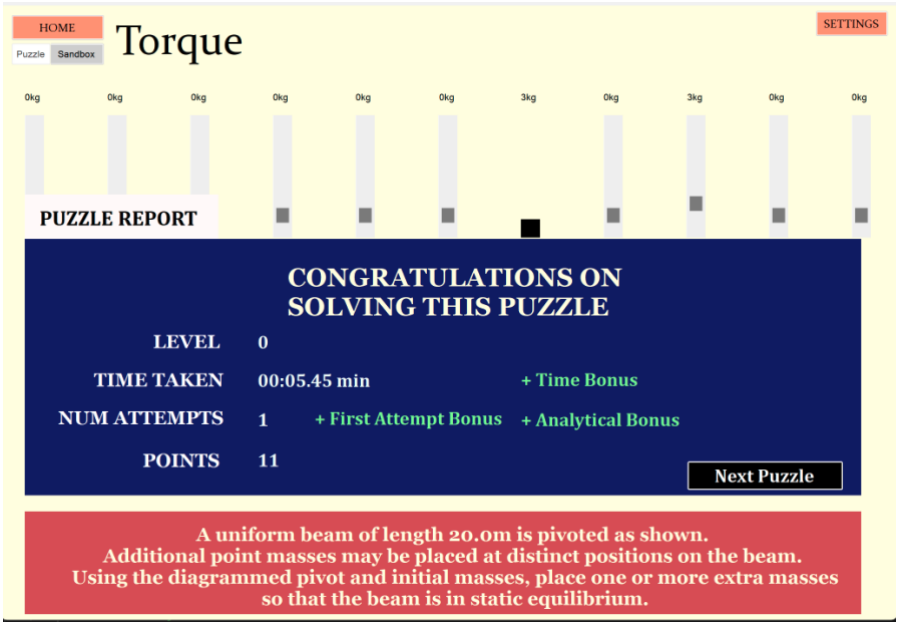


Figure 7. 3

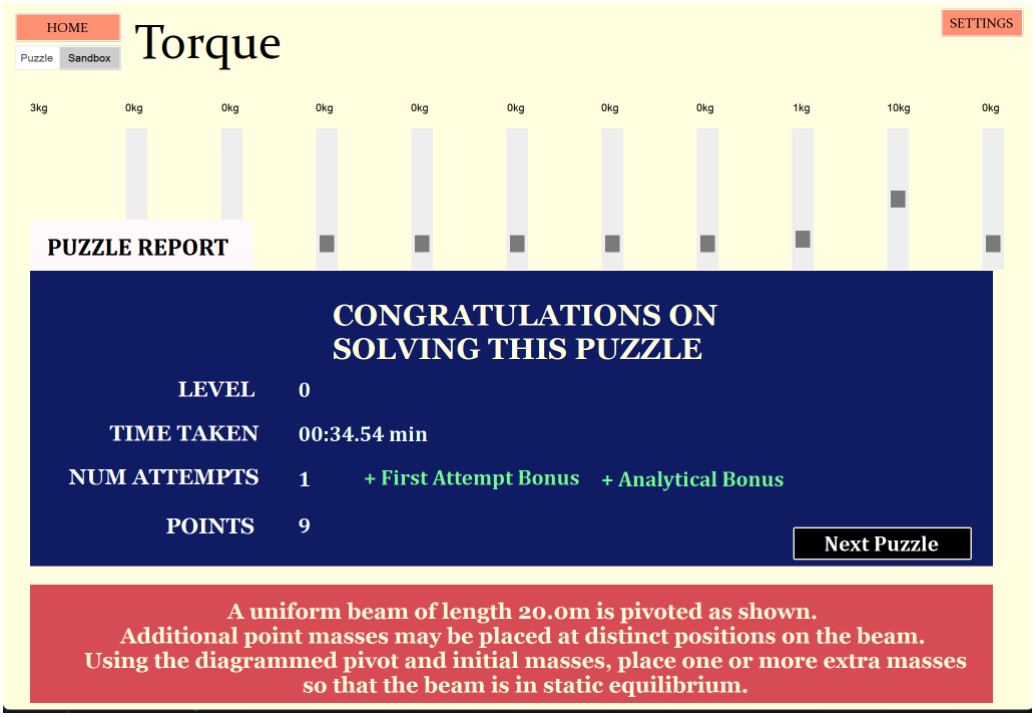


Figure 7. 4

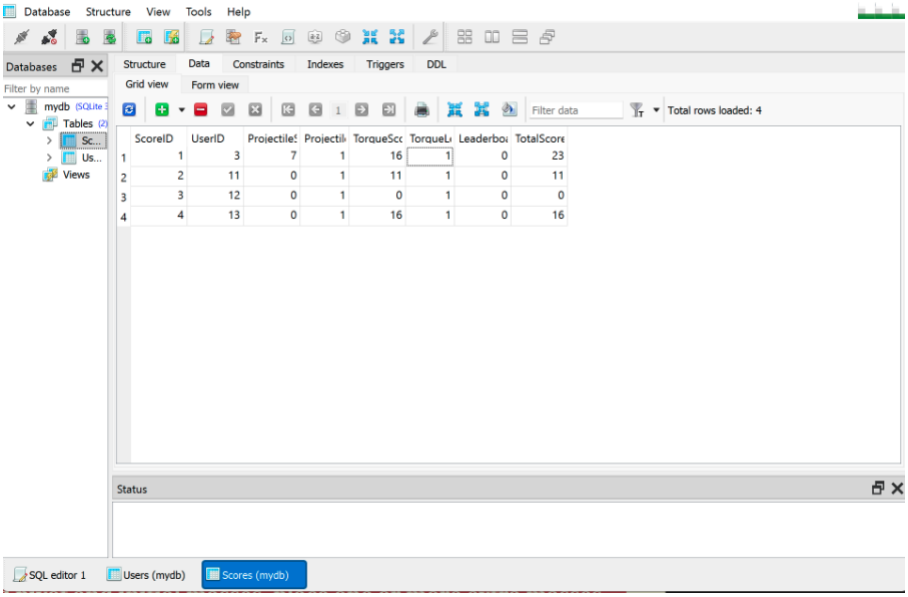


Figure 7. 5

Remedial Action

My unsuccessful test was due to the fact that since all inputs (apart from gravity and other environmental factors) are integers, the condition to allow for partial credit will never be met.

Validation

In this prototype, I developed a projectile and torque puzzle system that generates valid puzzles using integer-based physics calculations. To ensure puzzles are always solvable, I precomputed a dictionary of valid combinations of start height, launch angle, initial velocity, target position, and gravity for the projectile motion. During puzzle generation, the program selects a random entry from this dictionary and hides one variable for the player to solve, while the remaining values and the correct answer are stored internally.

Review

Following testing I can conclude that I was successfully able to implement

- Log In /Registration System
- Torque Sandbox
- Projectile Motion Sandbox
- Torque Puzzles
- Projectile Motion Puzzles
- Scoring

Prototype 4 – Leaderboard

My fourth prototype will be to create a leaderboard that displays the top ten users in each projectile and torque puzzle, as well as an overall leaderboard. Below is my mockup of how I intend the UI to look.



Class Diagrams

This prototype will extend the functionality of the existing database class. By reusing its methods for data retrieval and storage, the leaderboard system will efficiently access player scores and process them to determine rankings.

Database

To enable the leaderboard to display the top ten users, data must be retrieved from the database in descending order of score. The following SQL query accomplishes this:

```
SELECT User.Username, Scores.Score, Scores.Level
FROM Scores
INNER JOIN Users ON Scores.UserID = Users.UserID
ORDER BY Scores.Score DESC
```

The leaderboard prototype builds on the existing database class, introducing functionality to retrieve, rank, and display player scores. Unlike previous prototypes, it incorporates logic to select the top ten scores and link them to usernames stored in a separate Users table via a UserID foreign key. This is achieved using an SQL join

query, ensuring that each score is correctly associated with the corresponding player. Scores are sorted in descending order so that the highest-scoring players appear at the top, providing a clear and concise leaderboard for users. The design also allows for future extensions, such as filtering by difficulty level, tracking completion times, or displaying historical scores, without requiring changes to the underlying database structure.

Key Algorithms

Ranking Algorithm

The ranking algorithm assigns a position to each player based on their score. If two or more players have the same score, they are assigned the same rank to account for ties, and the next rank continues sequentially after the group of tied players. For example, if three players are tied for second place, then the next player is ranked fifth, not third.

```
rank = 1;
loop through each score
      if tie
            assign the rank of the previous item
      else;
            assign current rank
      add to tuple list
      rank ++;
```



Usability Features

The leaderboard has been designed for clarity and ease of use. It displays only the top 10 players allowing users to quickly identify the highest scores, while the current player's rank is always shown, even if they are outside the top 10. The leaderboard updates dynamically whenever new scores are added as well as visual highlights such as emphasising the current player's row, making personal performance easy to track.

Key Variables

Variable Name	Data Type	Description	Justification
LeaderboardData	List<(int, string, string, int)>	List of tuples used to populate the UI	Provides a structured, easy-to-iterate format for displaying leaderboard data in the form.
Rank	Int	Temporary counter for assigning player ranks during iteration	Necessary for the ranking algorithm to generate accurate ranks for each player
currentUser	String	Holds the current users username	Sllows the game to display the player's rank even if they are not in the top 10.

Testing

Input	Testing	Justification	Expected Result
15 players with unique scores	Sorting and top 10 selection	Ensures leaderboard correctly displays highest scores	Top 10 players displayed in descending order of score
Two players with identical scores	Tie handling in ranking	Ensure fairness when multiple players have the same score	Both players have the same rank and the next rank is adjusted appropriately
Current player outside top 10	Current player rank display	Ensures players see their own rank even if not in top 10.	Current player rank shown separately.
New high score added	Dynamic update of leaderboard	Verify leaderboard updates when new high scores are added	Leaderboard updates to show new top 10 with updated ranks

Development

My first step in developing this prototype was to collect the data from the database

```
public List<(string Username, int Score, int Level, int Rank)> GetTorqueLeaderboardScores()
{
    var leaderboard = new List<(string, int, int, int)>();
    string sql = @"SELECT Users.Username, Scores.TorqueScore, Scores.TorqueLevel FROM Scores INNER JOIN Users ON Scores.UserID = Users.UserID ORDER BY Scores.TorqueScore DES
    DataTable result = ExecuteQuery(sql);
    foreach (DataRow row in result.Rows)
    {
        leaderboard.Add((
            Username: row["Username"].ToString(),
            Score: Convert.ToInt32(row["TorqueScore"]),
            Level: Convert.ToInt32(row["TorqueLevel"]),
            Rank: 0
            ));
    }
    return leaderboard;
}
0 references
public List<(string Username, int Score, int Level, int Rank)> GetProjectileLeaderboardScores()
{
    var leaderboard = new List<(string, int, int, int)>();
    string sql = @"SELECT Users.Username, Scores.ProjectileScore, Scores.ProjectileLevel FROM Scores INNER JOIN Users ON Scores.UserID = Users.UserID ORDER BY Scores.Project
    DataTable result = ExecuteQuery(sql);
    foreach (DataRow row in result.Rows)
    {
        leaderboard.Add((
            Username: row["Username"].ToString(),
            Score: Convert.ToInt32(row["ProjectileScore"]),
            Level: Convert.ToInt32(row["ProjectileLevel"]),
            Rank: 0
            ));
    }
    return leaderboard;

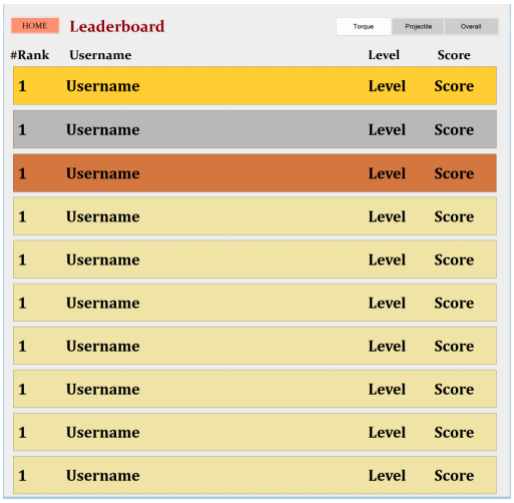
}
0 references
public List<(string Username, int Score, int Level, int Rank)> GetTotalLeaderboardScores()
{
    var leaderboard = new List<(string, int, int, int)>();
    string sql = @"SELECT Users.Username, Scores.TotalScore FROM Scores INNER JOIN Users ON Scores.UserID = Users.UserID ORDER BY Scores.TotalScore DESC";
    DataTable result = ExecuteQuery(sql);
    foreach (DataRow row in result.Rows)
    {
        leaderboard.Add((
            Username: row["Username"].ToString(),
            Score: Convert.ToInt32(row["TotalScore"]),
            Level: 0,
            Rank: 0
            ));
    }
    return leaderboard;

}
```

The game features three types of leaderboards: Torque, Projectile, and Total. Each leaderboard retrieves scores from the database using a dedicated method. The Scores and Users tables are linked by UserID, and an SQL INNER JOIN is used to combine the data. The queries select the relevant scores and levels (for Torque and Projectile) or just the total score, and order the results in descending order so that higher scores appear first.

The retrieved data is stored in a list of tuples (Username, Score, Level, Rank). Using tuples allows multiple pieces of information to be grouped together per player without creating separate classes, keeping the design simple and easy to iterate over for UI display. The Rank field is initially set to zero and is later assigned using the ranking algorithm, which calculates player positions and handles ties.

Following this I then designed the UI for the leaderboard as shown below



```
private void rankingAlgorithm()
{
    switch (currentLeaderboard)
    {
        case "torque":
            leaderboardData = db.GetTorqueLeaderboardScores();
            break;
        case "projectile":
            leaderboardData = db.GetProjectileLeaderboardScores();
            break;
        case "overall":
            leaderboardData = db.GetTotalLeaderboardScores();
            break;

    }

    int rank = 1;
    var item = leaderboardData[0];
    leaderboardData[0] = (item.Username, item.Score, item.Level, 1);
    for (int i = 1; i < leaderboardData.Count; i++)
    {
        item = leaderboardData[i];
        if (item.Score == leaderboardData[i - 1].Score)
        {
            item.Rank = leaderboardData[i - 1].Rank;
        }
        else
        {
            item.Rank = rank;
        }
        leaderboardData[i] = item;
        rank++;
    }
}
```

The ranking algorithm method assigns player ranks based on their scores. It first retrieves leaderboard data from the database depending on the selected mode - Torque, Projectile, or Overall. A rank counter begins at 1, with the first player automatically assigned rank 1. The algorithm then loops through the remaining players, comparing each score with the previous one. If two players have identical scores, they receive the same rank; otherwise, the rank increases by one. Because tuples are immutable, each updated entry is reassigned to the list. This approach ensures the leaderboard displays ranks fairly and accurately while maintaining simplicity and efficiency across all game modes.

```
private void populate()
{
    rankingAlgorithm();
    ClearLeaderboard();
    int index = 1;
    bool userPlaced = false;
    foreach(var data in leaderboardData)
    {
        if (data.Username == Session.Username)
        {
            userPlaced = true;
        }
        Label rankLabel = this.Controls.Find($"rank{index}", true).FirstOrDefault() as Label;
        Label nameLabel = this.Controls.Find($"name{index}", true).FirstOrDefault() as Label;
        Label levelLabel = this.Controls.Find($"Level{index}", true).FirstOrDefault() as Label;
        Label scoreLabel = this.Controls.Find($"Score{index}", true).FirstOrDefault() as Label;

        if (index == 10 && !userPlaced)
        {
            var player = leaderboardData.FirstOrDefault(p => p.Username == Session.Username);
            rankLabel.Text = player.Rank.ToString();
            nameLabel.Text = player.Username;
            if (currentLeaderboard != "overall")
            {
                levelLabel.Text = player.Level.ToString();
            }
            scoreLabel.Text = player.Score.ToString();
        }
        else
        {
            rankLabel.Text = data.Rank.ToString();
            nameLabel.Text = data.Username;
            if (currentLeaderboard != "overall")
            {
                levelLabel.Text = data.Level.ToString();
            }
            scoreLabel.Text = data.Score.ToString();
        }

        if (data.Username == Session.Username && data.Rank > 3)
        {
            Panel panel = this.Controls.Find($"panel{index}", true).FirstOrDefault() as Panel;
            panel.BackColor = Color.LightBlue;
        }
        index++;
    }
```

The populate() method updates the leaderboard by first ranking players and clearing old data. It loops through the ranked list, dynamically assigning usernames, scores, levels, and ranks to labels on the form. If the current user is not in the top ten, their details are still shown in the final position. The user's panel is highlighted for visibility. The Clear Leaderboard() method resets all labels and panel colours before new data is displayed, ensuring the leaderboard remains accurate, clean, and easy to read.

Testing

Input	Testing	Justification	Expected Result	Result
15 players with unique scores	Sorting and top 10 selection	Ensures leaderboard correctly displays highest scores	Top 10 players displayed in descending order of score	**SUCCESS** *See Figure 8.1*
Two players with identical scores	Tie handling in ranking	Ensure fairness when multiple players have the same score	Both players have the same rank and the next rank is adjusted appropriately	**SUCCESS** *See Figure 8.2*
Current player outside top 10	Current player rank display	Ensures players see their own rank even if not in top 10.	Current player rank shown separately.	**SUCCESS** *See Figure 8.2*

New high score added	Dynamic update of leaderboard	Verify leaderboard updates when new high scores are added	Leaderboard updates to show new top 10 with updated ranks	**SUCCESS** *See Video Figure 8.3*

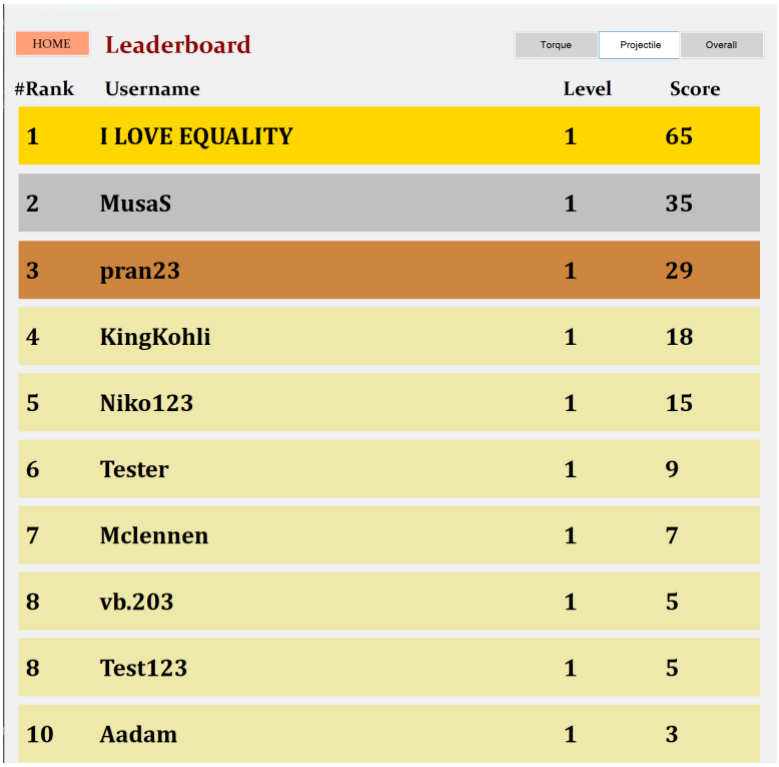


Figure 8. 1

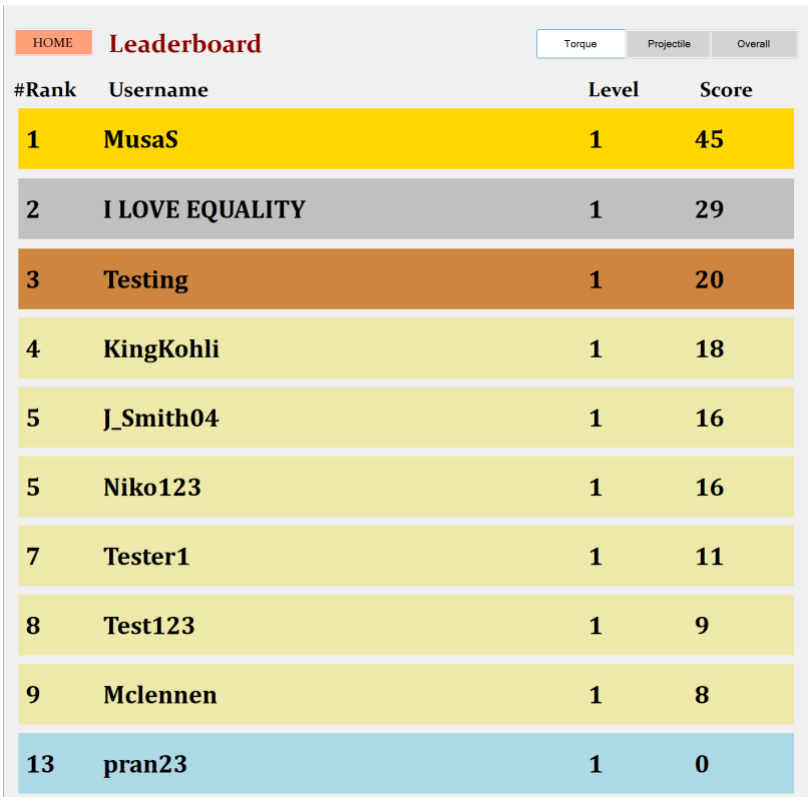


Figure 8. 2

Remedial Action

All tests were successful so no further remedial action is needed.

Validation

In this prototype, the data comes from a database where the data is what is expected so no further validation is required.

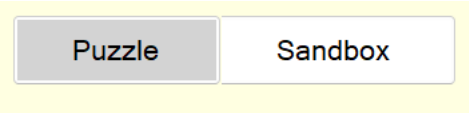
Review

Following testing I can conclude that I was successfully able to implement

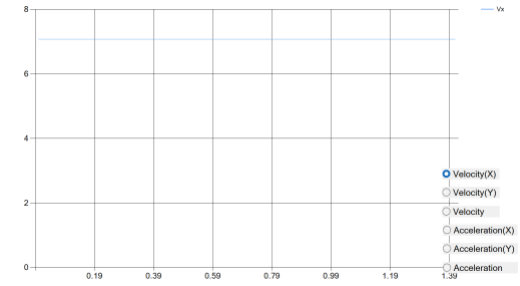
- Log In /Registration System
- Torque Sandbox
- Projectile Motion Sandbox
- Torque Puzzles
- Projectile Motion Puzzles
- Scoring
- Leaderboard

Stakeholder Analysis

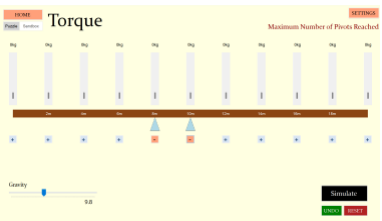
My stakeholder, Nikodem, tested the program so far, and he found that the UI was easy to navigate and was easy to understand, apart from the puzzle, sandbox switch button, as he found it difficult to understand which mode it was currently in. In order to counter this, he suggested that the name of the mode to be displayed to the user to make it clearer which mode the user was currently in.



He also found that the graph mode was currently too thin and he would prefer to be able to customise the graph and be able to zoom in on certain sections to help improve his learning. He also suggested that the user should be able to overlay multiple graphs over each other. Additionally, he found that the graphs for Acceleration were not always accurate which would cause issues for user learning.

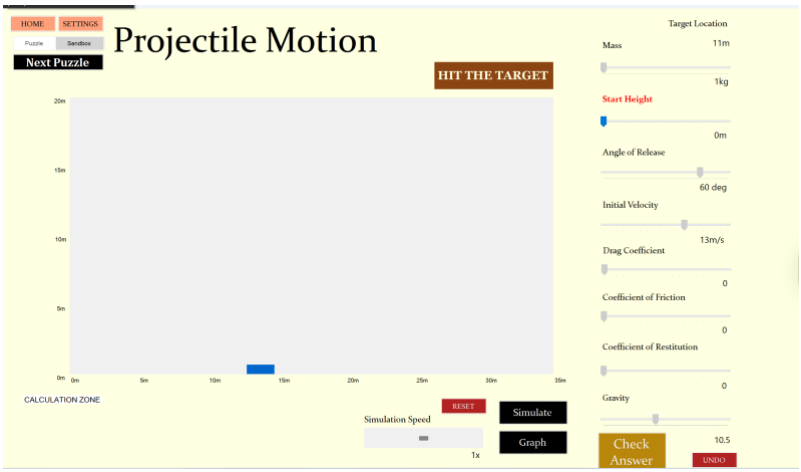


With the torque sandbox mode he found the message displayed confusing as he felt it meant he could only allow one pivot even though the message was supposed to display that the user could place up to two pivots.

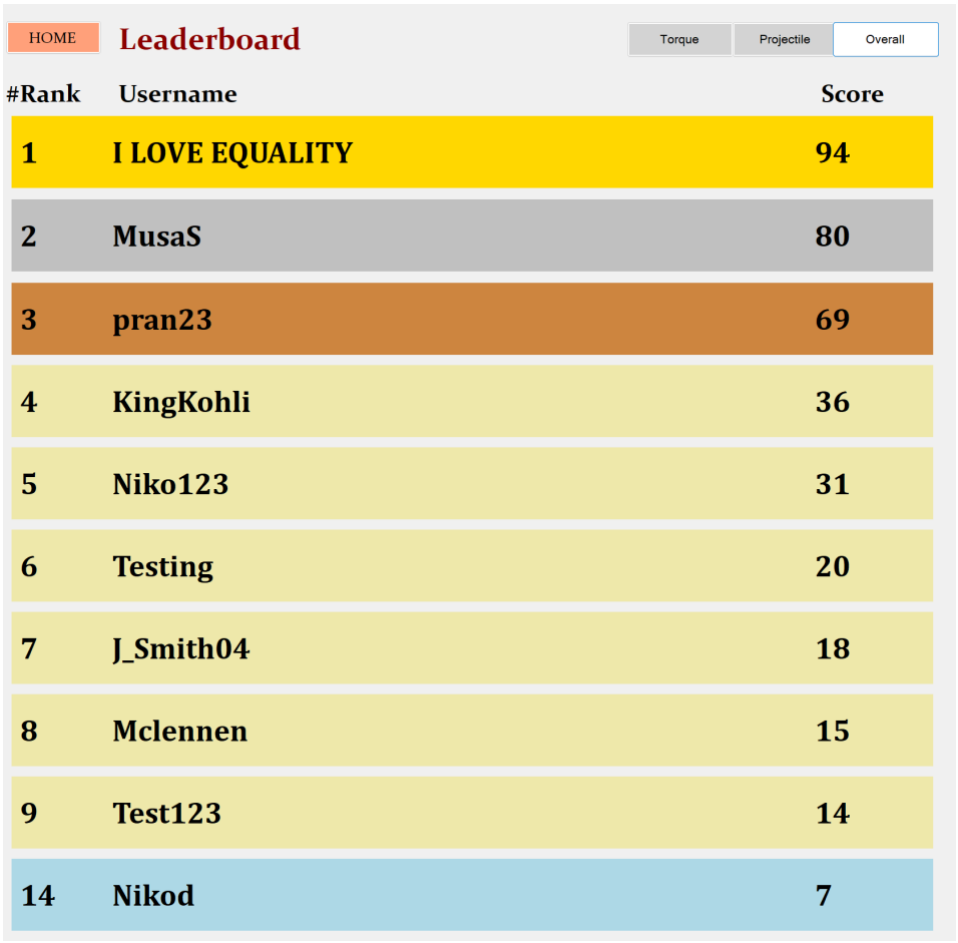


When considering the puzzles he found that for projectile motion it was difficult to tell what the initial variables were as there were many redundant sliders such as drag, coefficient of friction and coefficient of restitution

that the user does not need to consider. Nikodem suggested that to make this more user friendly, I should hide those sliders and also in the question rather than just hit the target I should clearly show what the initial values are.



With the leaderboard, he found it very easy to use and appreciated the use of colours for 1st 2nd and 3rd as well as the users current rank which he said 'makes it very easy to find your rank on the leaderboard'. With the accounts he felt that the user should be given the chance to change their username and password.



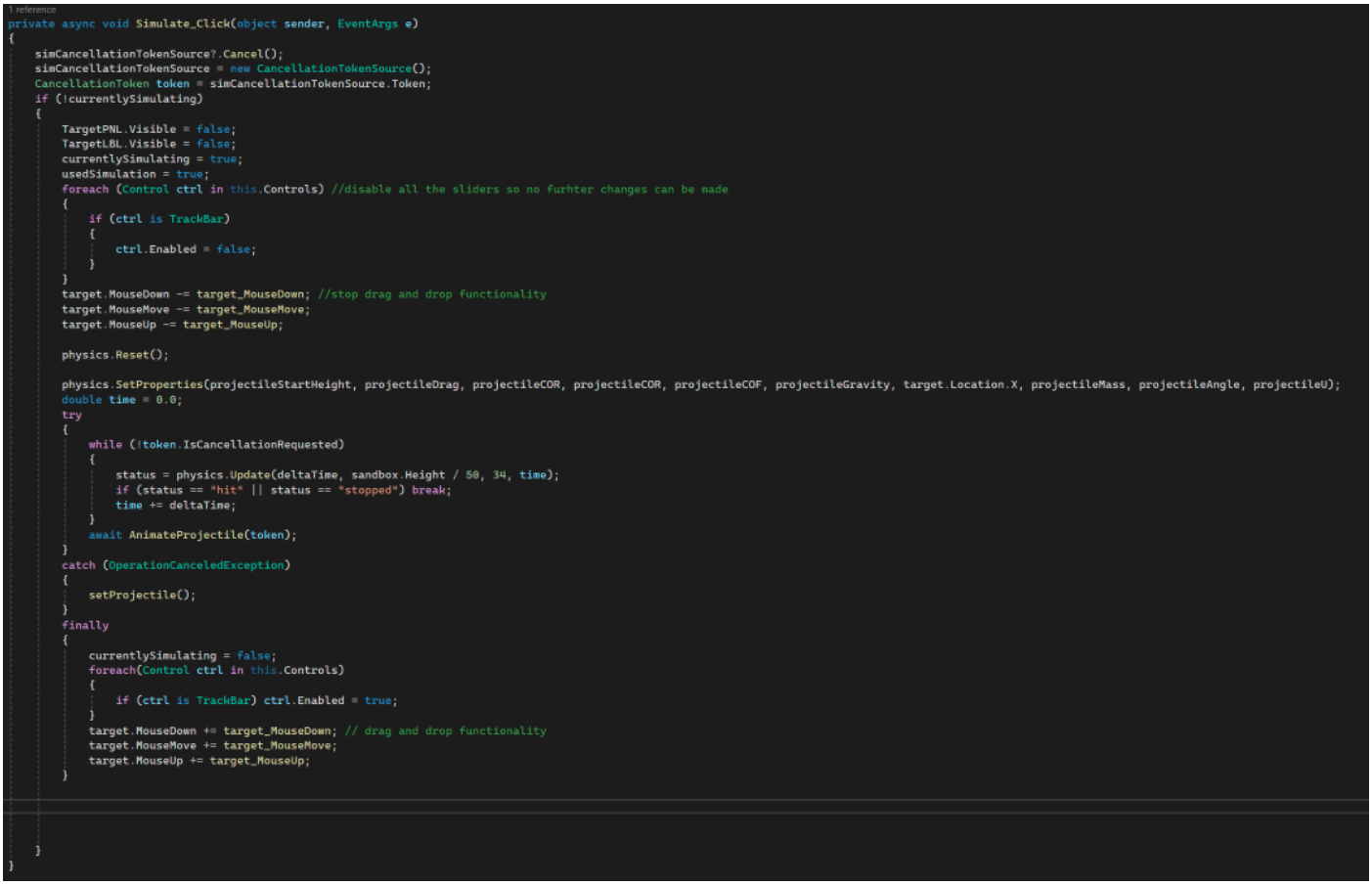
Within the solution, Nikodem also found a few bugs. The first bug was that the program was applying Coefficient of Restitution at the wrong spot even when it shouldn't be, see video file *Stakeholder Analysis – Video 1*. This, however, was a simple fix due to the height and width being passed to the class the opposite way around. However, once this was fixed, another error became apparent, see *Stakeholder Analysis – Video 2*. This error was that when a simulation was infinitely long, there was no way to stop the current simulation. In

addition, the collision detection with the target was not working properly. Additionally, Nikodem also found that the puzzle for torque also had a few physics errors that needed correcting.

Remedial Action

To fix the error found in *Stakeholder Analysis – Video 2,* I researched how to cancel an operation using the CancellationTokenSource Class (Microsoft Learn, n.d.) . This prevents multiple overlapping simulations and allows the user to quickly adjust parameters and rerun the simulation. To fix this, I implemented the following solution.

A CancellationTokenSource was added to allow running simulation tasks to be cancelled immediately. The Simulate method checks if a simulation is already running. If it is, it cancels the existing simulation before starting a new one. The reset method also cancels any ongoing simulation, ensuring the projectile resets instantly.



```
1 reference
private async void Simulate_Click(object sender, EventArgs e)
{
    simCancellationTokenSource?.Cancel();
    simCancellationTokenSource = new CancellationTokenSource();
    CancellationToken token = simCancellationTokenSource.Token;
    if (!currentlySimulating)
    {
        TargetPNL.Visible = false;
        TargetLBL.Visible = false;
        currentlySimulating = true;
        usedSimulation = true;
        foreach (Control ctrl in this.Controls) //disable all the sliders so no furhter changes can be made
        {
            if (ctrl is TrackBar)
            {
                ctrl.Enabled = false;
            }
        }
        target.MouseDown -= target_MouseDown; //stop drag and drop functionality
        target.MouseMove -= target_MouseMove;
        target.MouseUp -= target_MouseUp;

        physics.Reset();

        physics.SetProperties(projectileStartHeight, projectileDrag, projectileCOR, projectileCOR, projectileCOF, projectileGravity, target.Location.X, projectileMass, projectileAngle, projectileU);
        double time = 0.0;
        try
        {
            while (!token.IsCancellationRequested)
            {
                status = physics.Update(deltaTime, sandbox.Height / 50, 34, time);
                if (status == "hit" || status == "stopped") break;
                time += deltaTime;
            }
            await AnimateProjectile(token);
        }
        catch (OperationCanceledException)
        {
            setProjectile();
        }
        finally
        {
            currentlySimulating = false;
            foreach(Control ctrl in this.Controls)
            {
                if (ctrl is TrackBar) ctrl.Enabled = true;
            }
            target.MouseDown += target_MouseDown; // drag and drop functionality
            target.MouseMove += target_MouseMove;
            target.MouseUp += target_MouseUp;
        }



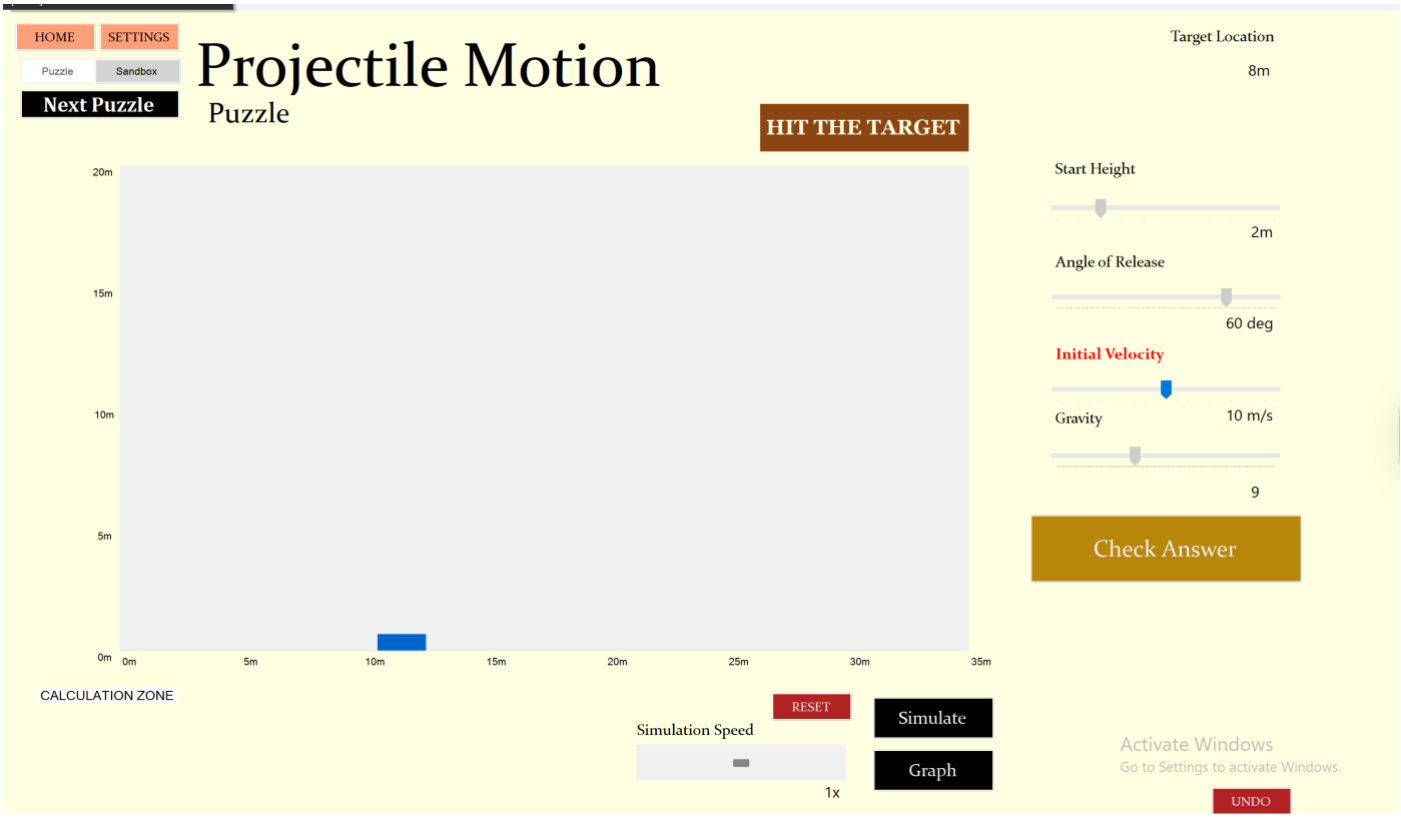
    }
}
```

To resolve the issue with collision detection, I reordered the program logic so that the collision check is performed first, before evaluating any other conditions. This adjustment has ensured that the collision

detection now functions correctly, as demonstrated in *Stakeholder Analysis – Video 3*.

```
if (CheckHit())
{
    trajectory.Add((currentTime, x, y));
    return "hit";
}

if (y <= 0)
{
    y = 0; // dont go below to floor
    if (Math.Abs(vy) > epsilon) // only bounce if vertical speed is high enough
    {
        vy = -vy * eFloor;
        vx *= (1 - muFloor); // apply floor friction
    }
    else
    {
        vy = 0;
        vx = 0;
        return "stopped";
    }
```



I will continue to build the following features in future prototypes

- Graph Customisability
- Username/Password Editing

Prototype 5 – Usability Features

In this prototype, I will build usability features such as providing correct answers for puzzles to help guide learning. Additionally, I will include user interface customisation options, allowing users to personalise aspects of the interface to suit their preferences, thereby improving accessibility and overall user experience.

Show Answers

For the projectile motion puzzles, implementing the "Show Answer" feature will be straightforward since the correct answer is already stored within the program. However, for the torque puzzle, this feature will be more complex to develop, as multiple possible combinations can correctly solve the puzzle.

```
Calculate torque about first pivot
Calculate torque about second pivot
Pick pivot with larger |torque|
If torque < 0
      Candidate positions are to the right
Else
      Candidate position to the left
Remove the position already occupied
Convert positions into distance from pivot
Recursively search for a solution
```

Font and Sizing

I plan to allow the user to change the font and font size to their preferences. I plan to allow the user to change the font style and font size according to their personal preferences. This will be done through a simple settings menu where users can select from a range of clear, readable fonts and adjust the size of the text to suit their needs. Giving users control over the font and size improves accessibility and usability, as it allows them to adapt the interface to their individual visual requirements. This feature helps users with visual impairments or reading difficulties engage with the program more comfortably, ensuring the interface is inclusive and user-friendly.

Colours

I plan to allow the user to choose between a light and dark theme for the program. Both themes will have a consistent combination of background, text, and highlight colours, which users can select through a simple settings menu to personalise their interface. Providing multiple colour themes improves accessibility and usability by allowing users to select colours that suit their visual preferences or needs. It also enhances the overall user experience by giving users more control over the program's look and feel, making the interface more engaging and comfortable to use.

Graph Customisability

I plan to allow the user to create multiple lines on the graph and change the colour of each line individually. Instead of using radio buttons to select lines, users will be able to click directly on a line or a corresponding label to modify its properties, making the interface more intuitive and flexible. This feature improves usability by giving users more control over how data is displayed, making it easier to compare multiple datasets visually. Allowing line selection without radio buttons allows for multiple lines.

Account Settings

For my solution, I plan to allow users to update their username and/or password. This will involve modifying the corresponding data stored in the database. For security reasons, when a user attempts to change their password, the system will first verify their current password to confirm their identity before applying any changes. I will also include options for users to reset their accounts or permanently delete them. The reset feature will restore the account to its default state while keeping the user's login details, whereas the delete option will completely remove all associated data from the database. I will also store each user's preferred theme in the database. This will ensure that their chosen colour scheme and appearance settings are saved and automatically applied whenever they log in. By doing this, the program provides a more personalised and consistent user experience. I plan to allow users to choose whether to participate in the leaderboard. When a user opts in, their score will be added to the leaderboard after completing a level or challenge. If a user later decides to opt out, their entry will be removed from the leaderboard. This will involve updating or deleting the relevant records in the database to reflect the user's preference.

Key Variables

Variable Name	Data Type	Description	Justification
candidatePosition	List<int>	Positions available for placing weights	Determines where weights can be placed to balance beam
Solution	List<int,int>	Stores final placement of weights	Final output of the puzzle solver method
editingUsername, editingPassword	Bool	Tracks if username/password is being edited	Determines which UI elements should be visible and determines validation
lightMode	Bool	Theme State (True – Light, False – Dark)	Controls UI theme and colours
leaderboardOpt	Bool	Indicates if user is on leaderboard	Updates UI and database when user opts in and out

Testing

Input	Testing	Justification	Expected Result
Projectile Motion Puzzle Answer	Click Show Answer Button	To check if the stored correct answer is displayed correctly	Correct answer is displayed for the current puzzle
Torque Puzzle Answer	Click Show Answer Button	Multiple combinations may solve the puzzle, test if one valid solution is shown	A valid combination of weights and positions is displayed
Change Font to Comic Sans, Size 24	Adjust menu settings	Ensure font changes are applied to all relevant text areas	Text is updated to Comic Sans, 24
Switch Between Dark and Light Mode	Adjust settings	Ensure colours are applied consistently	Consistent theme across forms
Add Multiple lines to graph	Click to add lines	Test if multiple lines can be plotted without interference	Graph displays lines correctly
Change line colour/thickness	Adjust menu settings	Test if user can customise graph intuitively	Line colours/thickness update immediately

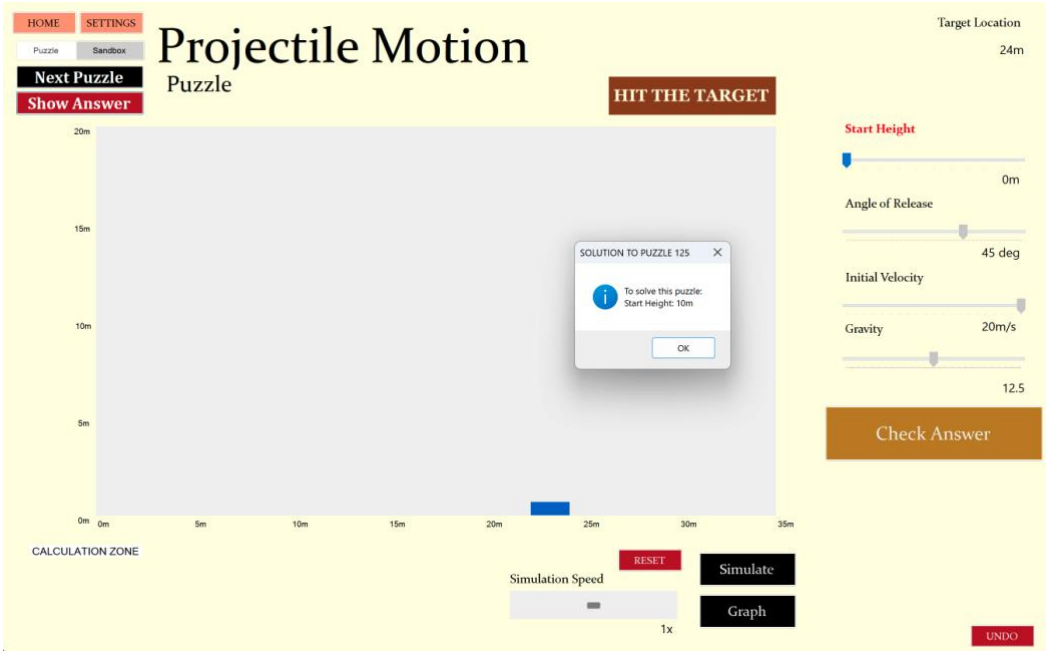
Update Username	Enter new username	Verify database updates correctly	Username changes and is saved for future login
Update Password	Enter current and new password	Ensure identity verification before update	Password changes and is saved for future login
Reset Account	Select reset option	Test if account returns to default while keeping login	Account resets scores but retains login info
Delete Account	Select delete option	Ensure full deletion of user daya	All account data removed from database, user cannot log in
Opt out / in from Leaderboard	Select opt in/out option	Check if user's score is recorded/hidden	User data shown/hidden on leaderboard.

Development

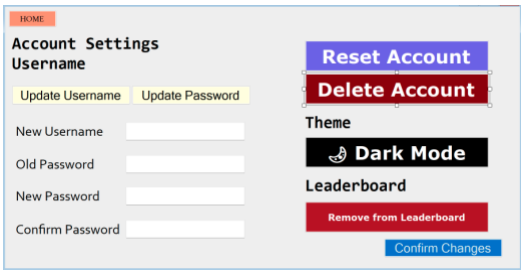
To begin, I developed the show-answer button for projectile motion. To do this, I took the value stored in the class and shared it with the user when they clicked it. Once the answer is shown, it will not allow the user to continue with that puzzle and will move on to the next puzzle.



```
if (puzzle.startingValues.startHeight == -1)
{
    unknowns.Add("startHeight");
    SetLabelBold(HeightTitle, true);
    answer += $"Start Height: {puzzle.startHeight}m\n";
    angle.Enabled = false;
    initialVelocity.Enabled = false;
}
else
{
    lbl_startHeight.Text = puzzle.startingValues.startHeight.ToString() + "m";
    startHeight.Value = puzzle.startingValues.startHeight;
    startHeight.Enabled = false;
}
if (puzzle.startingValues.velocity == -1)
{
    unknowns.Add("initVel");
    SetLabelBold(VelocityTitle, true);
    answer += $"Velocity: {puzzle.initVel}m/s\n";
}
else
{
    lbl_initialVelocity.Text = puzzle.startingValues.velocity.ToString() + "m/s";
    initialVelocity.Value = puzzle.startingValues.velocity;
    initialVelocity.Enabled = false;
}
if (puzzle.startingValues.angle == -1)
{
    unknowns.Add("angle");
    SetLabelBold(AngleTitle, true);
    answer += $"Angle: {puzzle.angle} deg\n";
}
else
{
    lbl_degree.Text = puzzle.startingValues.angle.ToString() + " deg";
    angle.Value = puzzle.startingValues.angle;
    angle.Enabled = false;
}
```



Following this, I began to build the settings page



```
private void UIChange()
{
    Point point = new Point(441,900);
    int start = 400;
    confirm_BUT.Visible = false;
    foreach (Label label in labels)
    {
        label.Visible = false;
    }
    foreach (TextBox textbox in textBoxes)
    {
        textbox.Visible = false;
    }
    if (editingPassword && editingUsername)
    {
        foreach (Label label in labels)
        {
            label.Visible = true;
        }
        foreach (TextBox textbox in textBoxes)
        {
            textbox.Visible = true;
        }
    }
    if (editingUsername && !editingPassword)
    {
        point = new Point(441,520);
        newUsername.Visible = true;
        NewUsername_lbl.Visible = true;
        newUsername.Visible = true;
        NewUsername_lbl.Visible = true;
    }
    if(!editingUsername && editingPassword)
    {
        point = new Point(441, 760);
        start = 280;
        foreach (Label label in labels)
        {
            label.Visible = true;
        }
        foreach (TextBox textbox in textBoxes)
        {
            textbox.Visible = true;
        }
        newUsername.Visible = false;
        NewUsername_lbl.Visible = false;
    }
    if (editingUsername || editingPassword) { confirm_BUT.Visible = true; }
    confirm_BUT.Location = point;
    int startCopy = start;
    foreach (Label label in labels)
    {
        label.Location = new Point(27,start);
        start += 120;
    }
    foreach (TextBox textbox in textBoxes)
    {
        textbox.Location = new Point(420,startCopy);
        startCopy += 120;
    }
}
```

The UIChange method dynamically updates the layout and visibility of form elements depending on whether the user is editing their username, password, or both. It first hides all labels, text boxes, and the confirm button to ensure a clean interface. Then, based on the variables editingUsername and editingPassword, it selectively displays the relevant input fields and positions them appropriately to define their on-screen coordinates. The confirm button is only shown when at least one of the edit options is active, and its position changes according to what is being edited.

I then created the database methods to ensure that the changes are reflected in the backend

```
public void UpdateLeaderboardOptIn(int userID, bool optIn)
{
    string sql = "UPDATE Scores SET LeaderboardOpt = @optIn WHERE UserID = @userID;";

    ExecuteNonQuery(sql,
        new SQLiteParameter("@optIn", optIn ? 1 : 0),
        new SQLiteParameter("@userID", userID)
    );
}

2 references
public void Themes(int userID, string theme)
{
    string sql = "UPDATE Users SET Theme = @theme WHERE UserID = @userID;";

    ExecuteNonQuery(sql,
        new SQLiteParameter("@theme", theme),
        new SQLiteParameter("@userID", userID)
    );
}
```

```
private void UIDark()
{
    foreach (Control control in this.Controls)
    {
        if (control is Label label)
        {
            label.ForeColor = Color.White;
        }
    }
    this.BackColor = Color.Black;

}

1 reference
private void UILight()
{
    foreach (Control control in this.Controls)
    {
        if (control is Label label)
        {
            label.ForeColor = Color.Black;
        }
    }
    this.BackColor = Color.White;

}
```

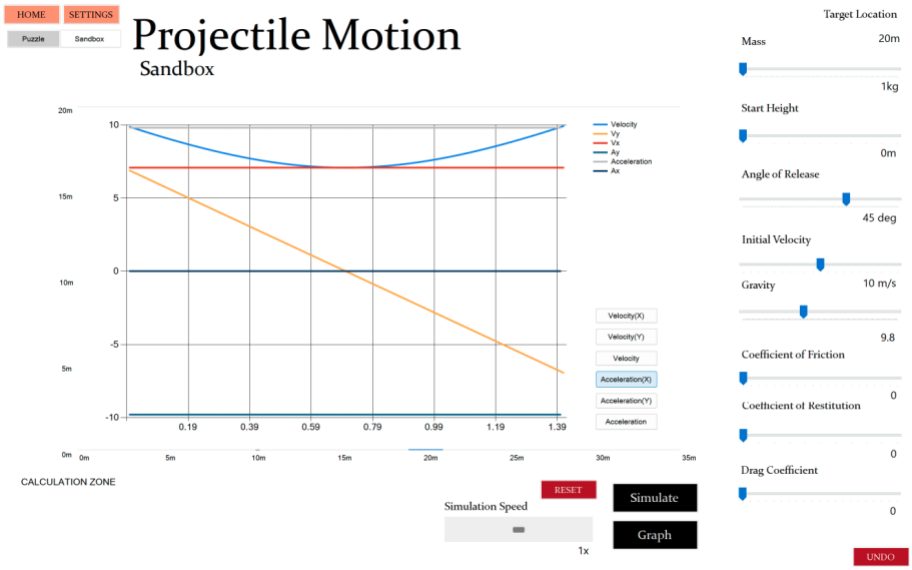
Following this, I implemented the database logic for the changing username and password. When this occurs validation checks are performed to ensure that new usernames are unique and passwords meet minimum length requirements. If the input is valid, the new details are saved to the database using the Database class and the interface is updated accordingly.

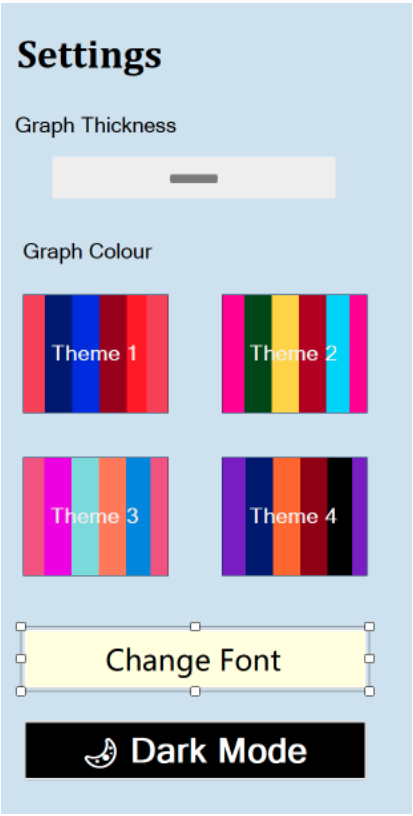
```
public void setUsername(int userID, string Username)
{
    string sql = "UPDATE Users SET Username = @username WHERE UserID = @userID";
    ExecuteNonQuery(sql,
        new SQLiteParameter("@username", Username),
        new SQLiteParameter("@userID", userID)
    );
}

1 reference
public void setPassword(int userID, string Password)
{
    string sql = "SELECT Salt FROM Users WHERE UserID = @userID";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@userID", userID));
    if (result.Rows.Count > 0)
    {
        string salt = result.Rows[0]["Salt"].ToString();
        string pwd = HashPassword(Password, salt);
        string sql2 = "UPDATE Users SET HashedPassword = @pwd WHERE UserID = @userID";
        ExecuteNonQuery(sql2,
        new SQLiteParameter("@pwd", pwd),
        new SQLiteParameter("@userID", userID)
    );
    }

}
```

In order to allow the users to see multiple graphs at once, I switched from radio buttons to normal buttons and used Boolean flags to set whether they were being shown or hidden.





Then in my settings tab, I allowed the user to change the thickness and colour of the lines.

```
private void graphThickness_Scroll(object sender, ScrollEventArgs e)
{
    VxSeries.BorderWidth = graphThickness.Value;
    VySeries.BorderWidth = graphThickness.Value;
    VSeries.BorderWidth = graphThickness.Value;
    AxSeries.BorderWidth = graphThickness.Value;
    AySeries.BorderWidth = graphThickness.Value;
    ASeries.BorderWidth = graphThickness.Value;
}

1 reference
private void theme1_Click(object sender, EventArgs e)
{
    VxSeries.Color = Color.DarkRed;
    VySeries.Color = Color.IndianRed;
    VSeries.Color = Color.OrangeRed;
    AxSeries.Color = Color.Navy;
    AySeries.Color = Color.DeepSkyBlue;
    ASeries.Color = Color.Blue;
}
```

I then implemented a Font Dialogue to allow users to personalise the appearance of text across the interface. When the user clicks the Change Font button, a FontDialog window opens, enabling them to choose the

typeface, size, and style they prefer. Once the user confirms their choice, the selected font is applied to all labels on the form, including those contained inside panels, group boxes, or other nested controls.

```
private void ChangeFont_Click(object sender, EventArgs e)
{
    using (FontDialog fontDialog = new FontDialog())
    {
        fontDialog.ShowColor = true;
        fontDialog.Font = this.Font;

        if (fontDialog.ShowDialog() == DialogResult.OK)
        {

            ApplyFont(this, fontDialog.Font);
        }
    }
}

2 references
private void ApplyFont(Control parent, Font newFont)
{
    foreach (Control control in parent.Controls)
    {
        if (control is Label label)
        {
            label.Font = newFont;
        }


        if (control.HasChildren)
        {
            ApplyFont(control, newFont);
        }
    }
}
```

```
public List<(int position, int mass)> SolvePuzzle()
{
    if (pivots.Count == 0) return null;
    int pivotIndex = 0;
    int PivotPosition = Convert.ToInt32(pivots[pivotIndex]);
    double torque = CalculateTorque(PivotPosition, 0);
    if (pivots.Count > 1)
    {
        int pivot2 = Convert.ToInt32(pivots[1]);
        double torque2 = CalculateTorque(pivot2, 0);

        if (Math.Abs(torque2) > Math.Abs(torque))
        {
            pivotIndex = 1;
            PivotPosition = pivot2;
            torque = torque2;
        }
    }
    List<(int position, int mass)> solution = new List<(int position, int mass)>();


    if (torque < 0)
    {
        for (int pos = PivotPosition + 2; pos <= 20; pos += 2)
            candidatePositions.Add(pos);
    }
    else
    {
        for (int pos = PivotPosition - 2; pos >= 0; pos -= 2)
            candidatePositions.Add(pos);
        candidatePositions.Reverse();
    }

    // Remove positions already taken
    foreach (var weight in weights)
    { candidatePositions.Remove(Convert.ToInt32(weight.position)); }

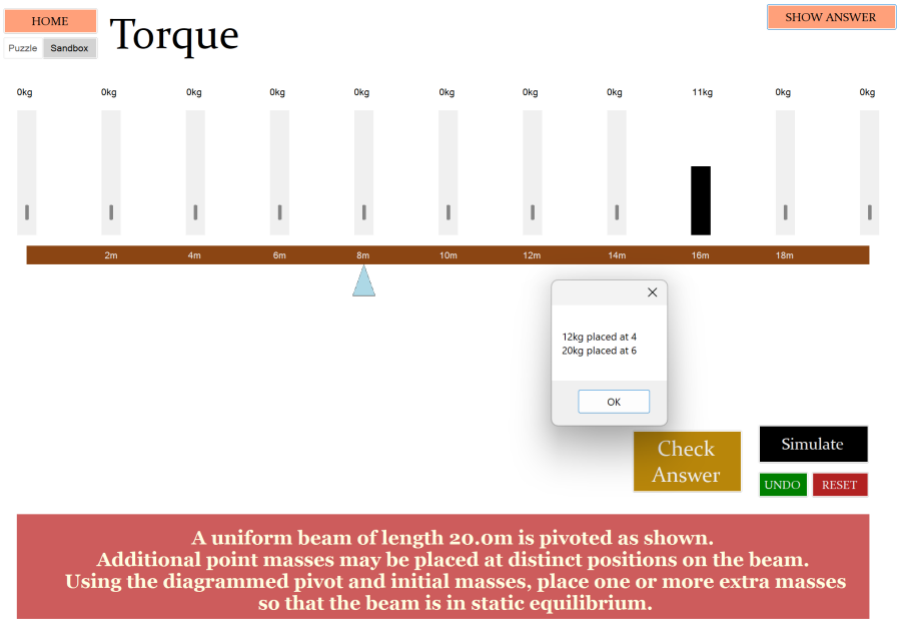
    foreach (int pos in candidatePositions)
    {
        distances.Add(Math.Abs(PivotPosition - pos));
    }
    double targetMoment = Math.Abs(torque);
    solution = TrySolve(0, 0, targetMoment);
    return solution;
}
```

This method automatically selects the pivot with the largest tipping torque, identifies the side of the beam to balance, calculates distances from the pivot for available positions, and uses the recursive TrySolve() method to perform a depth-first search over integer masses at candidate positions, returning a list of (position, mass) pairs whose combined torque exactly balances the beam.

```
public List<(int position, int mass)> TrySolve(int index, double currentMoment, double targetMoment)
{
    if (Math.Abs(currentMoment - targetMoment) < 0.0001)
    {
        return new List<(int position, int mass)>();
    }
    if (index >= distances.Count || currentMoment > targetMoment)
    {
        return null;
    }
    for (int w = 0; w<=20; w++)
    {
        double newMoment = currentMoment + w * distances[index];
        var result = TrySolve(index + 1, newMoment, targetMoment);
        if (result != null)
        {
            if (w > 0)
                result.Insert(0, (candidatePositions[index], w));
            return result;
        }
    }
    return null;

}
```

However when testing this program, I found that the solution found was incorrect as seen in the image below. This calculated the correct absolute torque however it failed to take into account the direction properly



When I first made the Torque Puzzle, the code tried to handle puzzles with one or two pivots using the same functions. This caused a few problems, for two-pivot puzzles, the solver often said there was no solution even when there was one and for single pivot solutions the masses where placed on the wrong side. Sometimes the puzzle generated was already balanced, which meant the generator could get stuck in an infinite loop.

To fix this, I decided to separate the logic for single-pivot and double-pivot puzzles. For a single pivot, I calculated the torque at the pivot and then figured out candidate positions for anti-weights on the opposite side to counteract the torque. These positions are turned into distances relative to the pivot, and a recursive function checks if there's a combination of weights that balances it.

For two pivots, I handle each pivot separately. I ignored any masses that are between the pivots when calculating torque, so the two pivots don't interfere with each other. I also make sure that positive torque is clockwise and that distances are flipped when necessary, so the extra weights actually balance the lever rather than make it tip further.

I then wrote two solving functions, SolveOnePivot() and SolveTwoPivots(), which take the torque, generate the candidate positions and distances, and use the recursive solver to find a solution. For two pivots, I combined

the solutions from both pivots and remove duplicates. I also added a check so that if the pivot is already balanced, it returns an empty solution straight away, which stops the puzzle generator from getting stuck in a loop.

```
public List<(int position, int mass)> SolvePuzzle()
{
    if (pivots.Count == 1) return SolveOnePivot();
    if (pivots.Count == 2) return SolveTwoPivots();
    return new List<(int, int)>();

}

1 reference
public List<(int position, int mass)> SolveOnePivot()
{
    int pivot = Convert.ToInt32(pivots[0]);
    double torque = CalculateTorque(pivot, 0);

    if (Math.Abs(torque) < 1e-6) return new List<(int, int)>();

    List<int> candidatePositions = new List<int>();
    if (torque > 0) // clockwise torque
    {
        for (int pos = pivot - 2; pos >= 0; pos -= 2)
            if (!weights.Any(w => Math.Abs(w.position - pos) < 1e-6))
                candidatePositions.Add(pos);
    }
    else // anticlockwise torque
    {
        for (int pos = pivot + 2; pos <= 20; pos += 2)
            if (!weights.Any(w => Math.Abs(w.position - pos) < 1e-6))
                candidatePositions.Add(pos);
    }

    // flip distances so adding masses counteracts torque
    List<int> distances = candidatePositions.Select(p => -(p - pivot)).ToList();
    var solution = TrySolve(candidatePositions, distances, torque);

    return solution ?? new List<(int, int)>();
}
```

```
private List<(int position, int mass)> TrySolve(List<int> positions, List<int> distances, double targetTorque, int index = 0, double currentTorque = 0)
{
    if (Math.Abs(currentTorque - targetTorque) < 1e-6)
        return new List<(int position, int mass)>();

    if (index >= distances.Count)
        return null;

    for (int mass = 1; mass <= 20; mass++)
    {
        double newTorque = currentTorque + mass * distances[index];
        var result = TrySolve(positions, distances, targetTorque, index + 1, newTorque);
        if (result != null)
        {
            result.Insert(0, (positions[index], mass));
            return result;
        }
    }

    return null;
}
```

```
public List<(int position, int mass)> SolveTwoPivots()
{
    if (pivots.Count < 2) return new List<(int, int)>();

    int pivot1 = Convert.ToInt32(pivots[0]);
    int pivot2 = Convert.ToInt32(pivots[1]);
    if (pivot1 > pivot2) (pivot1, pivot2) = (pivot2, pivot1);

    double torque1 = CalculateTorque(pivot1, 0, 1); // exclude masses between pivots
    double torque2 = CalculateTorque(pivot2, 0, 0);


    List<int> candidatePivot1 = new List<int>();
    if (torque1 > 0) // clockwise
    {
        for (int p = pivot1 - 2; p >= 0; p -= 2)
            if (!weights.Any(w => Math.Abs(w.position - p) < 1e-6))
                candidatePivot1.Add(p);
    }
    else // anticlockwise
    {
        for (int p = pivot2 + 2; p <= 20; p += 2)
            if (!weights.Any(w => Math.Abs(w.position - p) < 1e-6))
                candidatePivot1.Add(p);
    }
    var distances1 = candidatePivot1.Select(p => -(p - pivot1)).ToList();


    List<int> candidatePivot2 = new List<int>();
    if (torque2 > 0) // clockwise
    {
        for (int p = 0; p <= pivot1 - 2; p += 2)
            if (!weights.Any(w => Math.Abs(w.position - p) < 1e-6))
                candidatePivot2.Add(p);
    }
    else // anticlockwise
    {
        for (int p = pivot2 + 2; p <= 20; p += 2)
            if (!weights.Any(w => Math.Abs(w.position - p) < 1e-6))
                candidatePivot2.Add(p);
    }
    var distances2 = candidatePivot2.Select(p => -(p - pivot2)).ToList();

    var solution1 = TrySolve(candidatePivot1, distances1, torque1);
    var solution2 = TrySolve(candidatePivot2, distances2, torque2);

    if (solution1 == null && solution2 == null) return new List<(int, int)>();

    var solution = new List<(int, int)>();
    if (solution1 != null) solution.AddRange(solution1);
    if (solution2 != null) solution.AddRange(solution2);

    return solution.Distinct().ToList();
}
```

Testing

For the testing I will be using the following account

	UserID	Name	Username	Salt	HashedPassword	IsNewUse	Theme
1	23	Pranav	pran23	62l1vCbMLtYOt3zAeEBO0Q==	qfrWpZKxpFY+d4WOk0VNITKxA6Docectuf5K17J6kXw=	1	Light

	ScoreID	UserID	ProjectileScore	ProjectileLevel	TorqueScore	TorqueLevel	Lead	TotalScore	LeaderboardOpt
1	20	23	33	1	5	2	0	38	1

Input	Testing	Justification	Expected Result	Result
Projectile Motion Puzzle Answer	Click Show Answer Button	To check if the stored correct answer is displayed correctly	Correct answer is displayed for the current puzzle	**SUCCESS** *See Figure 9.1*
Torque Puzzle Answer	Click Show Answer Button	Multiple combinations may solve the puzzle, test if one valid solution is shown	A valid combination of weights and positions is displayed	**SUCCESS** *See Figure 9.2*
Change Font to Comic Sans, Size 24	Adjust menu settings	Ensure font changes are applied to all relevant text areas	Text is updated to Comic Sans, 24	**SUCCESS** *See Figure 9.3*
Switch Between Dark and Light Mode	Adjust settings	Ensure colours are applied consistently	Consistent theme across forms	**SUCCESS** *See Prototype 5 – Video 1*

Add Multiple lines to graph	Click to add lines	Test if multiple lines can be plotted without interference	Graph displays lines correctly	**SUCCESS** *See Prototype 5 – Video 2 [0:00 – 0:08]*
Change line colour/thickness	Adjust menu settings	Test if user can customise graph intuitively	Line colours/thickness update immediately	**SUCCESS** *See Prototype 5 – Video 2 [0:08 – 0:15]*
Update Username	Enter new username	Verify database updates correctly	Username changes and is saved for future login	**SUCCESS** *See Figure 9.4*
Update Password	Enter current and new password	Ensure identity verification before update	Password changes and is saved for future login	**SUCCESS** *See Figure 9.5*
Reset Account	Select reset option	Test if account returns to default while keeping login	Account resets scores but retains login info	**SUCCESS** *See Figure 9.6*
Delete Account	Select delete option	Ensure full deletion of user data	All account data removed from database, user cannot log in	**SUCCESS** *See Figure 9.7*
Opt out / in from Leaderboard	Select opt in/out option	Check if user's score is recorded/hidden	User data shown/hidden on leaderboard.	**SUCCESS** *See Prototype 5 – Video 3*

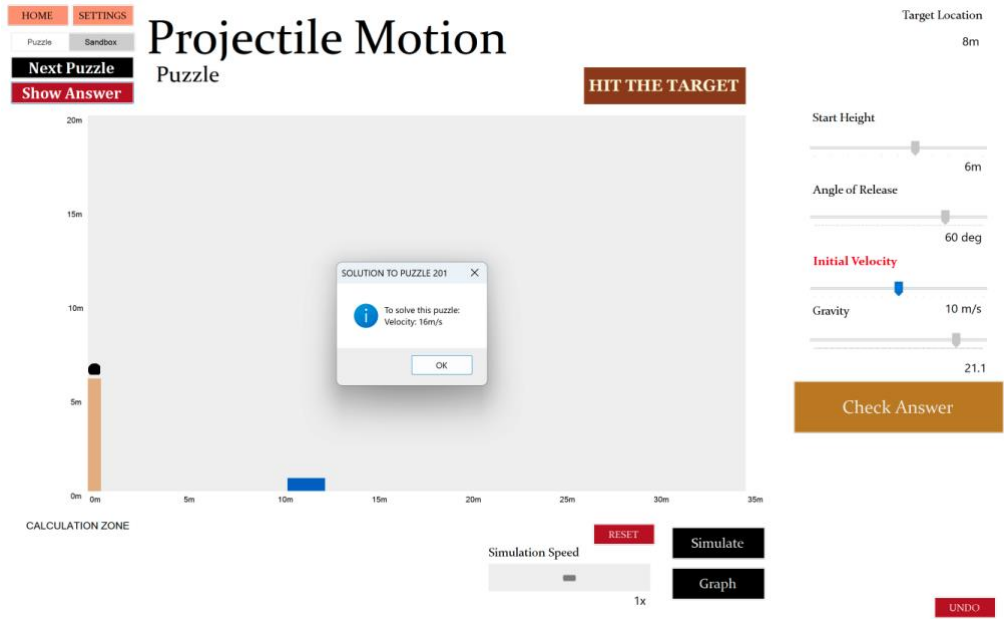


Figure 9. 1

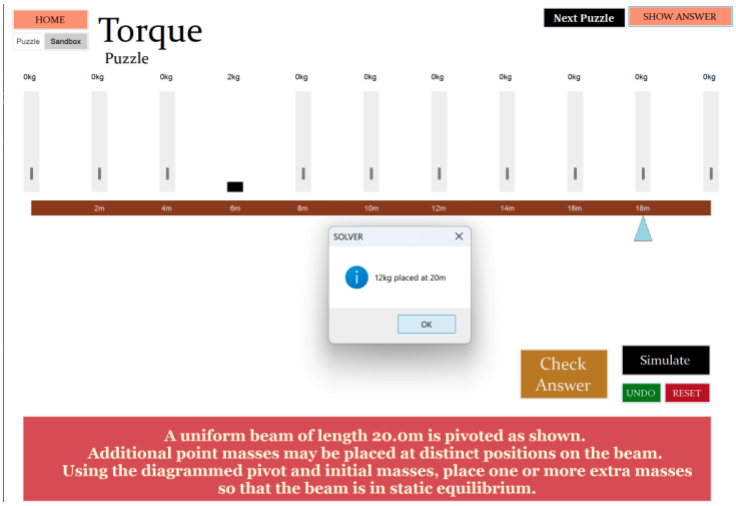


Figure 9. 2

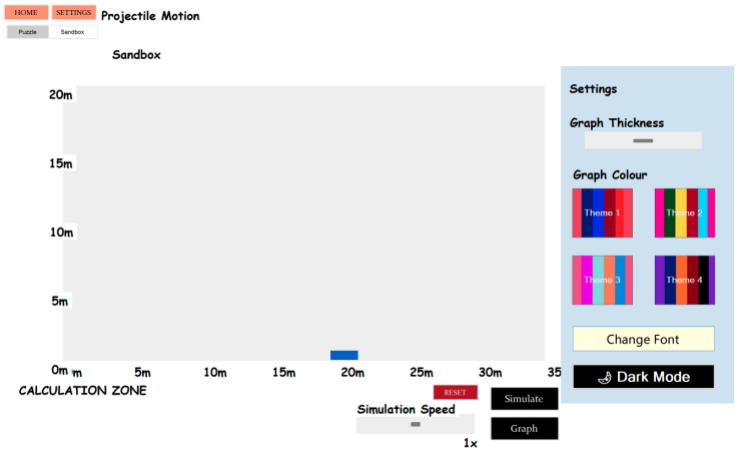
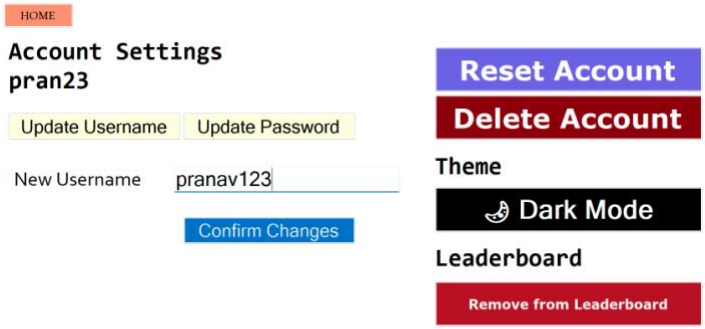


Figure 9. 3



	UserID	Name	Username	Salt	HashedPassword	IsNewUse	Theme
1	23	Pranav	pranav123	62l1vCbMLtYOt3zAeEBO0Q==	qfrWpZKxpFY+d4WOk0VNlTKxA6Docectuf5K17J6kXw=	1	Light

Figure 9. 4

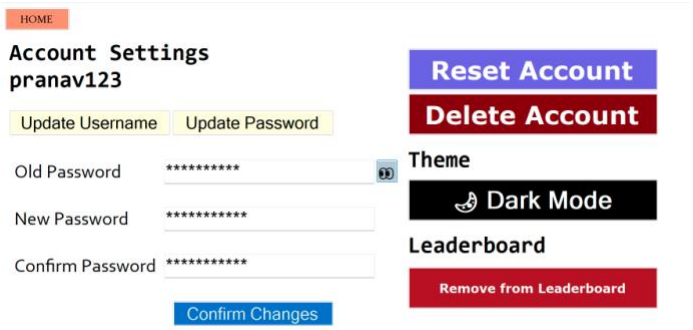


Figure 9. 5

	ScoreID	UserID	ProjectileScore	ProjectileLevel	TorqueScore	TorqueLevel	Lead	TotalScore	LeaderboardOpt
1	20	23	0	1	0	1	0	0	1

Figure 9. 6

	UserID	Name	Username	Salt	HashedPassword	IsNewUse	Theme
1	24	dhwani	drajesh	tUDygwMxotvllkcegzz9Vw==	Piz2BMuvgc9wzNoU39OYJ7BSq5k22c7klzO0/OrFyhk=	1	Light
2	25	Ian	McLennan	+Thir8oj0nCCRfQH1xoCcQ==	tlBrHbCyMUXol26wFiNNVauCuEqaOQdE4dTta0vmrsQ=	1	Light
3	26	Nik	nik123	TTWQdCjfmvRH2tBE31fNpA==	eJnl++gtyJxNaz9Omg+1os9ihCBxA5uMzHZS2HS4NWo=	1	Light
4	27	vb	vb.203	QIMYG37nZuD+KhkazElsAQ==	+IjM5pjM9lihmfwpfr0kQCZa/X6pSbFJVWVZWjUG6fQ=	1	Light
5	28	Name	JRod	ooqbCq82A5aWcvwpAzn2rQ==	UK7cKzCmC1jj3tvStyZ8QyRexytF3ZUijRmShS5085Q=	1	Light
6	29	Musa	I LOVE EQUALITY	HeQFeRQNrTec045Fm+8j9Q==	RMVqM/SI44JXJq8P1SfUm4vxXMR/BWkiz+UauhCVLdo=	1	Light
7	30	Joseph	Hosef	hRnmDUr332ixUbhIGLdxTQ==	sWTloyqMPoiTB3T46+onqXX/B5x9UNohnh1FHGn/xes=	1	Light
8	31	tanya	hunt	J60pVO+3whyIk8FVH8wMvA==	E+NGxfzaxeutUWC+0A+gYU++Qo86VB+5Miw6y0SJavU=	1	Light
9	32	jack	jack_wl	rZ1i+gGT7NcWPiFBUUTx5w==	xprFJQgAmtyBCs8H1BY+OMFzgegRM+1jMAjsIggSB9I=	1	Light
10	33	Tom	KING	OsRAStBTpR5mju+DCVLIJw==	ZNX1CAK3RPSgKai7mwhgQoN9JNt+6WFBw7m5pWvmEbg=	1	Light

Figure 9. 7 (No longer in database)

Validation

In this prototype, most of what the user does comes from buttons, which naturally limits invalid input scenarios. As a result, the majority of inputs are controlled and do not require extensive validation logic. However the username/password change feature required explicit validation to ensure that the inputs met the requirements of length to maintain data integrity. Additionally, during development, I identified a case where the simulate button in the Torque Sandbox could be pressed before any pivots were placed, resulting in an error as the system attempted to simulate empty data. To address this, I will add validation in the remedial action to ensure simulation only runs when pivots are present. This prevents unexpected crashes and ensures the system behaves predictably even under edge-case user actions.

```
private void newUsername_TextChanged(object sender, EventArgs e)
{
    if (newUsername.Text.Length <= 4 || newUsername.Text.Length >=12)
    {
        errormessage_username.Visible = true;
        valid = false;
    }
    else
    {
        errormessage_username.Visible = false;
    }

    if (db.UsernameExists(newUsername.Text))
    {
        errormessage_alreadyexists.Visible = true;
        valid = false;
    }
    else
    {
        errormessage_alreadyexists.Visible = false;
        if (newUsername.Text.Length > 4 && newUsername.Text.Length < 12) { valid = true; errormessage_username.Visible = false; }

    }
}

2 references
private void newPassword_TextChanged(object sender, EventArgs e)
{
    if (newPassword.Text.Length <= 8 || newPassword.Text.Length >= 16)
    {
        errormessage_password.Visible = true;
        valid = false;
    }
    else if (newPassword.Text.Length > 8 && newPassword.Text.Length < 16)
    {
        errormessage_password.Visible = false;
        valid = true;
    }
}
```

Remedial Action

All tests were successful. The only action needed is adding validation to the simulate button to ensure the Torque Sandbox does not attempt to run a simulation without any pivots placed. This will prevent runtime errors and ensure the feature behaves reliably.

```
private void Simulate_Click(object sender, EventArgs e)
{
    if (!currentlySimulating && PlacedPivots.Count > 0)
    {
        currentlySimulating = true;
        engine.weights.Clear();
        CleanList();
        int pivot1pos = -1;
        int pivot2pos = -1;
        pivot1pos = Convert.ToInt32(engine.pivots[0]);
        try { pivot2pos = Convert.ToInt32(engine.pivots[1]);  } catch { }
        foreach (var weight in placedMasses)
        {
```

Review

Following testing I can conclude that I was successfully able to implement

- Log In /Registration System
- Torque Sandbox
- Projectile Motion Sandbox
- Torque Puzzles
- Projectile Motion Puzzles
- Scoring
- Leaderboard
- Usability Features

Prototype 6 – Levels

In this prototype, the goal is to introduce a user-level system that adapts challenge difficulty based on performance. The system will track user success rates and progression, and assign an appropriate level that determines the complexity of puzzles they receive. This ensures users are consistently presented with tasks that match their ability.

In order to discern which level the user is at, I will look at the following metrics rather than relying on a single measure. This ensures that progression reflects genuine understanding and consistency.

- Number of puzzles solved
- Average number of attempts taken per puzzle
- Average time taken per puzzle
- The percentage of puzzles that they used the simulation
- Skip frequency
- Improvement trend

Together, these metrics offer a balanced evaluation of accuracy, speed, and independence, allowing the system to assign levels that genuinely represent the user's skill progression.

Level	Projectile	Torque
1 - Beginner	Users adjust only one variable to hit the target	Users need to balance a beam that contains a single pivot and a single mass
2 - Intermediate	Users adjust two variables to hit the target	Users need to balance a beam that contains a single pivot and multiple masses
3 - Advanced	Users must find any viable solution to hit the target with no restrictions imposed	Users need to balance a beam that contains a multiple pivots and multiple masses

This structure promotes gradual skill development, encourages strategic problem-solving, and provides a clear, measurable pathway for progression, while maintaining control over puzzle complexity appropriate to the user's current skill level.

Class Diagram

LevelDeterminer

Key Attributes
`INT CurrentLevel`
`INT TotalAttempts`
`INT PuzzlesSolved`
`DOUBLE TotalTime`
`INT SimulationsUsed`
`INT SkippedPuzzles`
`LIST RecentScores`

Key Methods
`RecordPuzzleResult()`
`AverageAttempts()`
`AverageTime()`
`SimulationUseRate()`
`SkipRate()`
`ImprovementTrend()`
`UpdateLevel()`

Furthermore, I will create a child class of my existing database class to encapsulate all functionality related to user levels. This subclass will be responsible for storing, retrieving, and updating level-specific metrics, ensuring a clear separation of concerns and maintaining a modular, maintainable design.

Database

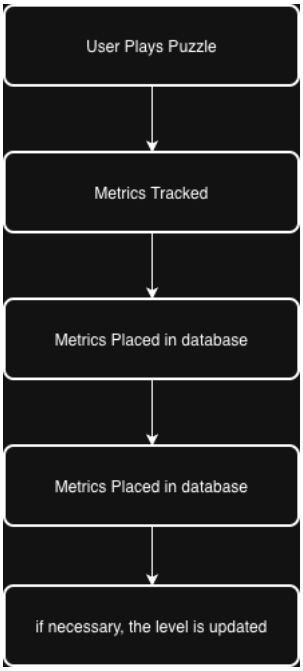


TABLE: UserPuzzleData

Column Name	Data Type	Description
DataID	PRIMARY KEY	Unique identifier for each attempt
UserID	FOREIGN KEY	Foreign key linking to the user
PuzzleType	STRING	“Torque” or “Projectile”
PuzzleLevel	INT	Level of the attempted puzzle
Attempts	INT	Number of attempts for this puzzle
TimeTaken	FLOAT	Time taken in seconds
SimulationUsed	BOOL	TRUE if simulation used FALSE if simulation not used
Skipped	BOOL	TRUE if skipped FALSE if not skipped
Score	FLOAT	Score = 100 – (Attempts - 1)*5 – TimeTaken / 2 – SimulationPenalty Set to 0 if skipped
AttemptDate	DATETIME	Timestamp of attempt

```
CREATE TABLE UserPuzzleData (
	DataID INTEGER PRIMARY KEY AUTOINCREMENT,
	UserID INTEGER NOT NULL,
	PuzzleType TEXT NOT NULL,
	PuzzleLevel INTEGER NOT NULL,
	Attempts INTEGER NOT NULL,
	TimeTaken REAL NOT NULL,
	SimulationUsed INTEGER NOT NULL CHECK (SimulationUsed IN (0,1)),
	SKIPPED INTEGER NOT NULL CHECK (SKIPPED IN (0,1)),
	SCORE REAL NOT NULL,
	AttemptDate DATETIME DEFAULT CURRENT_TIMESTAMP,
	FOREIGN KEY (UserID) REFERENCES Users(UserID) ON DELETE CASCADE
);
```

Algorithms

Score Calculation

Calculates a numeric score for each puzzle based on attempts, time, simulation usage, and whether it was skipped

```
CalculateScore()
	If skipped
		Return 0
	MaxScore = 100
	AttemptPenalty = 5
	TimePenalty = 2
	SimulationPenalty = 5
	Score = MaxScore
		- (Attempts - 1) * AttemptPenalty
		- (TimeTaken / TimePenalty)
		-If Simulation Used (-SimulationPenalty)
	if Score < 0
		Score = 0
	Return Score
```

Improvement Trend

This algorithm compares recent performance to earlier performances and outputs a numeric value. If it is positive, the user is improving; if it is negative, the user is declining, and if it is zero, the user's performance is unchanging. This will calculate the slope of score vs attempt number, which represents improvement over time through linear regression. Skipped puzzles (score = 0) can be ignored, so they don't skew the trend. Each score is also weighted by its level so that higher level puzzles count more towards the trend.

Equation for linear regression: $m = \frac{n\Sigma(x_i y_i) - \Sigma(x_i)\Sigma(y_i)}{n\Sigma x_i^2 - (\Sigma x_i)^2}$

```
ImprovementTrend()
N = number of data points
Check that there are enough data points to carry out analysis
Apply weighting to the scores
x = attempt
y = weighted score
Combine x and y element by element and multiply corresponding elements. Then add
up all the products and store as sumXY
Foreach xi in x square it and sum all the square. Store as sumX2
Slope = (n * SumXY – sumX * sumY) / (n * sumX2 – sumX * sumX)
Return slope
```

UpdateLevel

Each factor is weighted to produce a final score. If this score exceeds a certain threshold, the user will advance to the next level.

```
UpdateLevel()
      n = number of puzzles
      NormalisedScore = sum of scores / n * 100
      w1 = 0.5
      w2 = 0.2
      w3 = 0.1
      w4 = 0.2
      PerformanceScore = w1 * NormalisedScore + w2 * improvementTrend – w3*simRate – w4*skipRate
      If Level == 1 && performanceScore > 0.6
            Level = 2
      If Level == 2 && performanceScore > 0.8
            Level = 3
      If Level == 3 && performanceScore < 0.5
            Level = 2
```

Usability Features

This prototype focuses entirely on scoring and level progression. There are no usability features because the primary purpose is not user-facing. The only user-facing feedback is a message indicating when a user has moved up or down a level, as all other operations are handled in the backend.

Key Variables

Variable Name	Data Type	Description	Justification
currentLevel	INT	The user's current level	Used to determine puzzle complexity
TotalAttempts	INT	Total number of attempts across all puzzles	Used for analysis
PuzzlesSolved	INT	Number of puzzles successfully completed	Used to calculate averages and percentages
TotalTime	FLOAT	Cumalative time taken across puzzles	Used to calculate average
SimulationsUsed	INT	Number of puzzles where the simulation was used	Used to calculate average
SkippedPuzzles	INT	Number of puzzles skipped	Used to calculate skip rate
RecentScores	LIST<FLOAT>	Stores recent scores for trend calculation	Used for linear regression
PuzzleType	STRING	"Projectile" or "Torque"	Used to determine which data to take
PuzzleLevel	INT	Level of attempted puzzle	Used for weighting in trend calculation
Attempts	INT	Number of attempts for the puzzle	Used in score calculation
TimeTaken	FLOAT	Time in seconds taken to complete the puzzle	Used in score calculation
SimulationUsed	BOOL	TRUE if simulation used FALSE if simulation not used	Penalises performance score if true
Skipped	BOOL	TRUE if skipped FALSE if not skipped	Score set to 0 if true
Score	FLOAT	Calculated numeric score for the puzzle	Combines attempts, time, simulation penalty and skip status

Development

To begin with, I created the database table in order to store all the data

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	Default value
1	DataID	INTEGER	🔑							NULL
2	UserID	INTEGER		🔗						NULL
3	PuzzleType	TEXT								NULL
4	PuzzleLevel	INTEGER								NULL
5	Attempts	INTEGER								NULL
6	TimeTaken	REAL								NULL
7	SimulationUsed	INTEGER								NULL
8	SKIPPED	INTEGER								NULL
9	SCORE	REAL								NULL
10	AttemptDate	DATETIME								CURRENT_TIMESTAMP

```
public void AddData(int userID, string puzzleType, int puzzleLevel, int attempts, double timeTaken, bool simulation, bool skipFlag, double score)
{
    int simulationUsed = simulation ? 1 : 0;
    int Skipped = skipFlag ? 1 : 0;
    string sql = @"
        INSERT INTO UserPuzzleData (UserID, PuzzleType, PuzzleLevel, Attempts, TimeTaken, SimulationUsed, SKIPPED, SCORE)
        VALUES (@userID, @puzzletype, @puzzlelevel, @attempts, @timetaken, @simulationused, @skipped, @score);
        ";
    ExecuteNonQuery(sql,
        new SQLiteParameter("@userID", userID),
        new SQLiteParameter("@puzzletype", puzzleType),
        new SQLiteParameter("@puzzlelevel", puzzleLevel),
        new SQLiteParameter("@attempts", attempts),
        new SQLiteParameter("@timetaken", timeTaken),
        new SQLiteParameter("@simulationused", simulationUsed),
        new SQLiteParameter("@skipped", Skipped),
        new SQLiteParameter("@score", score)
        );
}

0 references
public int NumberofDataPoints(int userID)
{
    string sql = "SELECT COUNT(*) FROM UserPuzzleData WHERE UserID = @userID";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@userID", userID));
    return Convert.ToInt32(result.Rows[0][0]);

}
0 references
public double GetTotalScore(int userID)
{
    string sql = "SELECT SUM(SCORE) FROM UserPuzzleData WHERE UserID = @userID";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@userID", userID));
    return Convert.ToDouble(result.Rows[0][0]);
}
0 references
public int GetTotalSimulation(int userID)
{
    string sql = "SELECT SUM(SimulationUsed) FROM UserPuzzleData WHERE UserID = @userID";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@userID", userID));
    return Convert.ToInt32(result.Rows[0][0]);
}
0 references
```

```
public int GetTotalSkips(int userID)
{
    string sql = "SELECT SUM(SKIPPED) FROM UserPuzzleData WHERE UserID = @userID";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@userID", userID));
    return Convert.ToInt32(result.Rows[0][0]);
}
0 references
public List<(double score, int Level)> GetLast15Scores(int userID)
{
    string sql = @"
SELECT SCORE, PuzzleLevel
FROM UserPuzzleData
WHERE UserID = @userID
ORDER BY AttemptDate DESC
LIMIT 15";

    DataTable dt = ExecuteQuery(sql, new SQLiteParameter("@userID", userID));
    var result = new List<(double score, int Level)>();

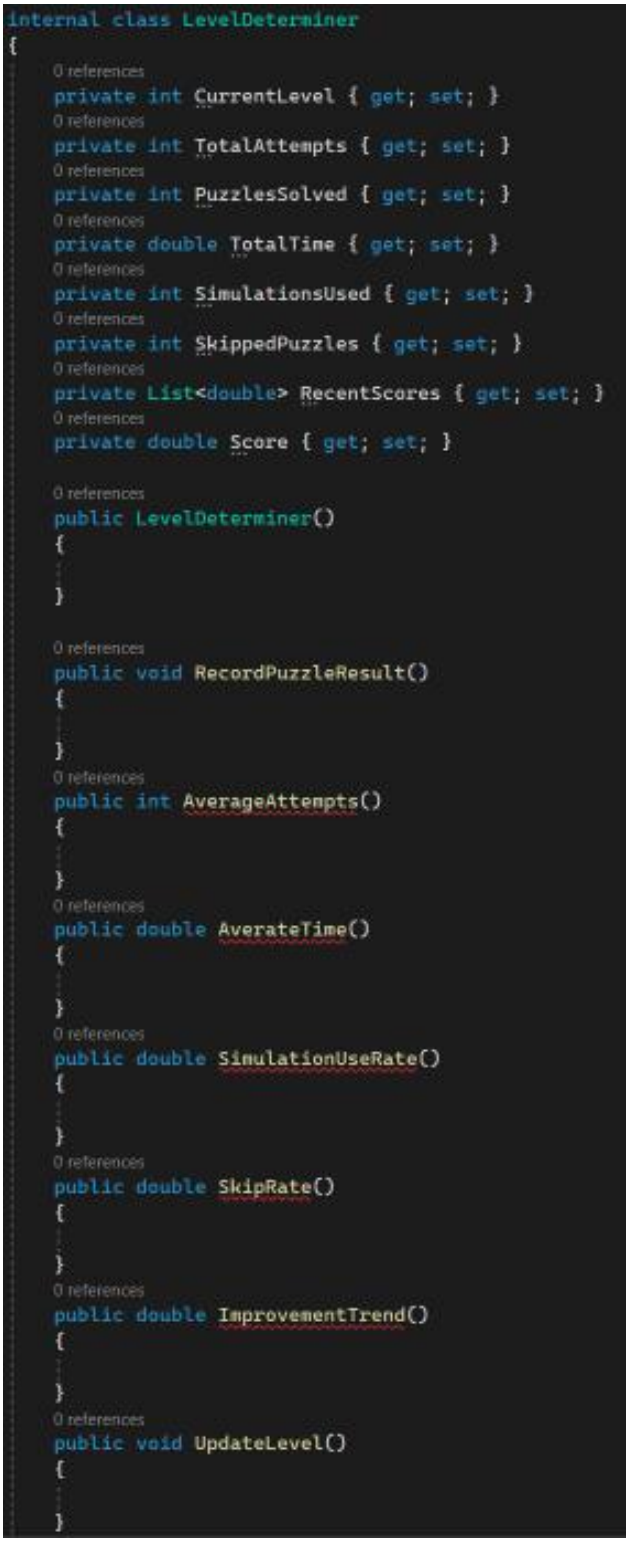
    foreach (DataRow row in dt.Rows)
    {
        double score = Convert.ToDouble(row["SCORE"]);
        int level = Convert.ToInt32(row["PuzzleLevel"]);
        result.Add((score, level));
    }

    return result;
}
```

The LevelDatabase class manages all interactions with the UserPuzzleData table. It allows new puzzle attempts to be recorded through the AddData method and also provides methods to retrieve user statistics.

NumberofDataPoints counts attempts, GetTotalScore sums all scores, and GetTotalSimulation and GetTotalSkips track simulations used and puzzles skipped. GetLast15Scores retrieves the most recent fifteen attempts, returning the score and corresponding puzzle level as a list of tuples. Using tuples allows the scores and levels to be paired together in a simple, type-safe structure, making it easy to process or display the data in the program.

Following this, I then created the LevelDeterminerClass



```
public void RecordPuzzleResult(int puzzleLevel, int Attempts, double TimeTaken, bool sim, bool skip)
{
    attempts = Attempts;
    timeTaken = TimeTaken;
    simulation = sim;
    CalculateScore();
    db.AddData(Session.UserID, puzzleType, puzzleLevel, attempts, timeTaken, sim, skip, Score);
    TotalN = db.NumberofDataPoints(Session.UserID);
}
1 reference
public void CalculateScore()
{
    int MaxScore = 100;
    int AttemptPenalty = 5;
    int TimePenalty = 2;
    int SimulationPenalty = 5;
    Score = MaxScore - (attempts - 1) * AttemptPenalty - (timeTaken / TimePenalty);
    if (simulation) { Score = Score - SimulationPenalty; }
    if (Score < 0) { Score = 0; }
}

0 references
public double AverageTime()
{
    TotalTime = db.GetTotalTime(Session.UserID);
    return TotalTime / TotalN;
}
1 reference
public void SimulationUseRate()
{
    SimulationsRate = db.GetTotalSimulation(Session.UserID)/TotalN;
}
1 reference
public void SkipRate()
{
    SkippedRate =  db.GetTotalSkips(Session.UserID) / TotalN;
}
```

```
public void ImprovementTrend()
{
    RecentScores = db.GetLast15Scores(Session.UserID);
    if ( TotalN < 4 ) { improvement = 0; }
    else
    {
        int[] x = new int[TotalN];
        double[] y = new double[TotalN];
        for (int i = 0; i < TotalN; i++)
        {
            x[i] = i + 1;
            y[i] = RecentScores[i].score * RecentScores[i].level;
        }
        double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;

        for (int i = 0; i < TotalN; i++)
        {
            sumX += x[i];
            sumY += y[i];
            sumXY += x[i] * y[i];
            sumX2 += x[i] * x[i];
        }
        double numerator = (TotalN * sumXY) - (sumX * sumY);
        double denominator = (TotalN * sumX2) - (sumX * sumX);
        if (denominator == 0 ) {improvement = 0; }
        else { improvement = numerator / denominator; }
    }
}
```

```
public void UpdateLevel()
{
    SimulationUseRate();
    SkipRate();
    ImprovementTrend();
    double NormalisedScore = db.GetTotalScore(Session.UserID) / (TotalN * 100);
    double w1 = 0.5;
    double w2 = 0.2;
    double w3 = 0.1;
    double w4 = 0.2;
    double PerformanceScore = w1 * NormalisedScore + w2 * improvement - w3 * SimulationsRate - w4 * SkippedRate;
    if (Session.Level == 1 && PerformanceScore > 0.6)
    {
        Session.Level = 2;
    }
    if (Session.Level == 2 && PerformanceScore > 0.8)
    {
        Session.Level = 3;
    }
    if (Session.Level == 3 && PerformanceScore < 0.5)
    {
        Session.Level = 2;
    }
}
```

This class manages the game's adaptive difficulty by monitoring the player's long-term performance and automatically adjusting their level. After each puzzle, the system records performance data and saves it to the database and analyses their full performance history.

I found that there was a logic error with the above code, being that the database does not distinguish between data from projectile and data from torque

In order to fix the first problem, I added an extra parameter to the database method

```
public int NumberofDataPoints(int userID, string pType)
{
    string sql = "SELECT COUNT(*) FROM UserPuzzleData WHERE UserID = @userID AND PuzzleType = @pType";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@userID", userID), new SQLiteParameter("@pType", pType));
    return Convert.ToInt32(result.Rows[0][0]);

}
```

```
public double AverageTime()
{
    TotalTime = db.GetTotalTime(Session.UserID,puzzleType);
    return TotalTime / TotalN;
}
```

In order to fix the ensure the data is saved to the session for other forms, I also needed to ensure that the level was being updated for the relevant puzzle type

```
if (db.GetLevel(Session.UserID, puzzleType) == 1 && PerformanceScore > 0.6)
{

    db.setLevel(Session.UserID, puzzleType, 2);
}
if ((db.GetLevel(Session.UserID, puzzleType) == 2 && PerformanceScore > 0.8))
{

    db.setLevel(Session.UserID, puzzleType, 3);
}
if ((db.GetLevel(Session.UserID, puzzleType) == 3 && PerformanceScore < 0.5))
{

    db.setLevel(Session.UserID, puzzleType, 2);
}
Session.ProjectileLevel = db.GetLevel(Session.UserID, "Projectile");
Session.TorqueLevel = db.GetLevel(Session.UserID, "Torque");
```

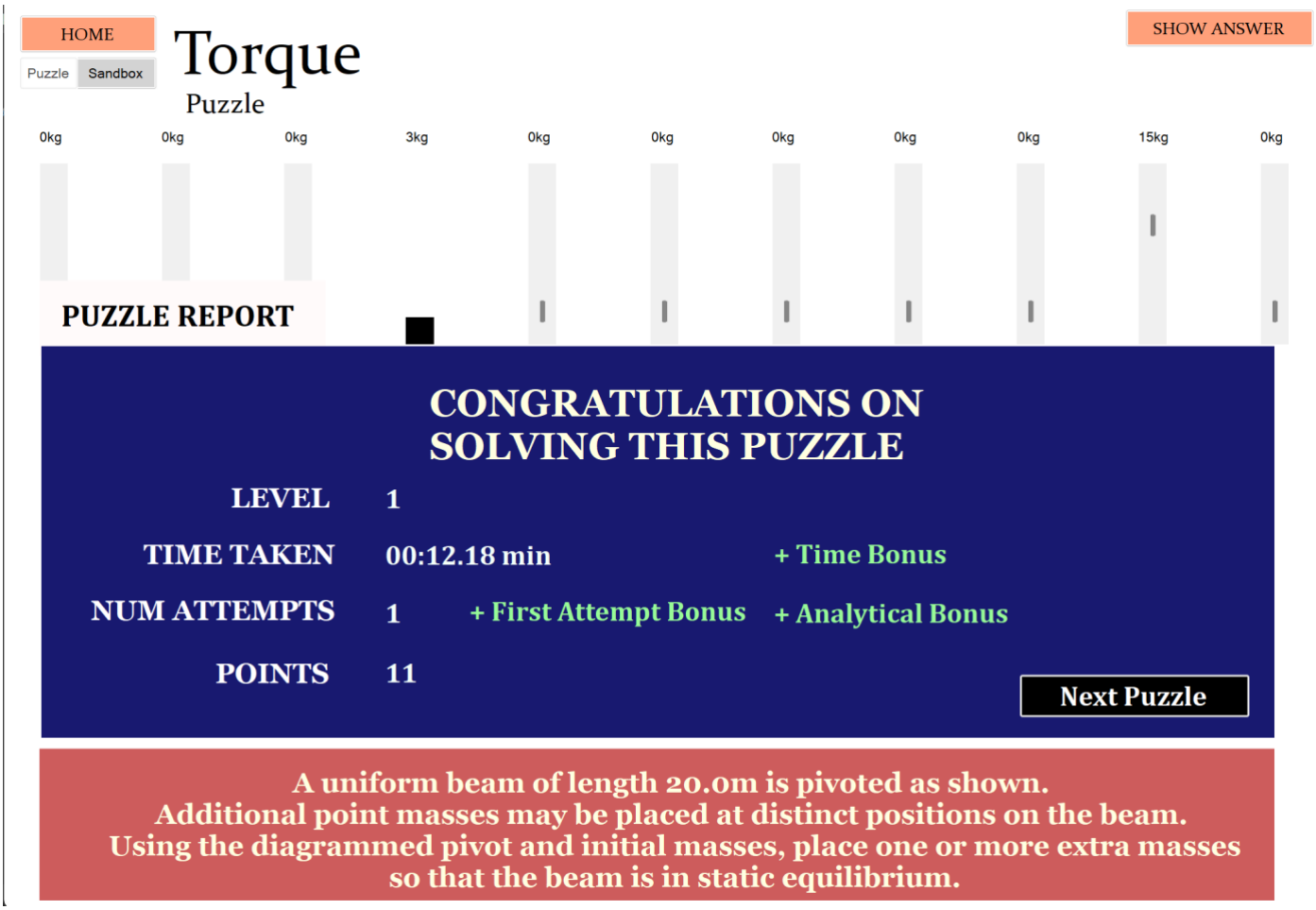
I then used my captured data (Prototype 3) and sent it to the LevelDeterminer class

```
private void puzzleSolved()
{
    Toggle(this, true, "Solved");
    SolvedAttempts.Text = puzzleAttempts.ToString();
    stopwatch.Stop();
    TimeSpan elapsed = stopwatch.Elapsed;
    string timeElapsed = $"{elapsed.Minutes:D2}:{elapsed.Seconds:D2}.{elapsed.Milliseconds / 10:D2} min";
    int seconds = (int)stopwatch.Elapsed.TotalSeconds;
    nextPuzzleBUT.Visible = false;
    SolvedLevel.Text = puzzle.level.ToString();
    SolvedTime.Text = timeElapsed;
    if (seconds < 60)
    {
        TimeBonus.Visible = true;
        points += 2;
    }
    if (puzzleAttempts == 1)
    {
        FirstAttemptBonus.Visible = true;
        points += 2;
        if (!usedSimulation)
        {
            AnalyticalBonus.Visible = true;
            points += 2;
        }
    }
    SolvedPoints.Text = points.ToString();
    db.AddScore(Session.UserID, points, "Projectile");
    analysis.RecordPuzzleResult(puzzle.level, puzzleAttempts, seconds, usedSimulation, false);
    nextPuzzleBUT.Visible = false;
}
```

	DataID	UserID	PuzzleType	PuzzleLeve	Attempts	TimeTaken	SimulationI	SKIPPED	SCORE	AttemptDate
1	1	34	Projectile	1	2	11	0	0	89.5	2025-11-04 09:09:28
2	2	34	Projectile	1	0	0	0	1	0	2025-11-04 09:10:11
3	3	34	Torque	1	3	31	1	0	69.5	2025-11-04 10:51:27
4	4	34	Torque	1	0	0	0	1	0	2025-11-04 10:53:41

When testing this, I found that in my torque puzzle the level was not clearly displayed to the user. To fix this, I added the following line

```
SolvedLevel.Text = Session.TorqueLevel.ToString();
```



I know need to create the code to allow users to easily find their current level and be informed on when they move up a level.

```
lbl_mode.Text = "Puzzle - Level  " + Session.ProjectileLevel.ToString();
```

```
LblMode.Text = "Puzzle - Level" + Session.TorqueLevel.ToString();
```

To allow the user to know when they have moved up or down a level, I updated the levelDeterminer class to return an integer value where

0 – No Change in level

1 – Change from Level 1 to 2

2 – Change from Level 2 to 3

3 – Change from Level 3 to 2

```
public int RecordPuzzleResult(int puzzleLevel, int Attempts, double TimeTaken, bool sim, bool skip)
{
    attempts = Attempts;
    timeTaken = TimeTaken;
    simulation = sim;
    CalculateScore();

    if (skip) { db.AddData(Session.UserID, puzzleType, puzzleLevel, 0, 0, false, skip, 0); }
    else { db.AddData(Session.UserID, puzzleType, puzzleLevel, attempts, timeTaken, sim, skip, Score); }

    TotalN = db.NumberofDataPoints(Session.UserID, puzzleType);
    return UpdateLevel();
}
```

```
public int UpdateLevel()
{
    SimulationUseRate();
    SkipRate();
    ImprovementTrend();
    double NormalisedScore = db.GetTotalScore(Session.UserID,puzzleType) / (TotalN * 100);
    double w1 = 0.5;
    double w2 = 0.2;
    double w3 = 0.1;
    double w4 = 0.2;
    int change = 0;
    double PerformanceScore = w1 * NormalisedScore + w2 * improvement - w3 * SimulationsRate - w4 * SkippedRate;
    if (db.GetLevel(Session.UserID, puzzleType) == 1 && PerformanceScore > 0.6)
    {

        db.setLevel(Session.UserID, puzzleType, 2);
        change = 1;
    }
    if ((db.GetLevel(Session.UserID, puzzleType) == 2 && PerformanceScore > 0.8))
    {

        db.setLevel(Session.UserID, puzzleType, 3);
        change = 2;
    }
    if ((db.GetLevel(Session.UserID, puzzleType) == 3 && PerformanceScore < 0.5))
    {

        db.setLevel(Session.UserID, puzzleType, 2);
        change = 3;
    }
    Session.ProjectileLevel = db.GetLevel(Session.UserID, "Projectile");
    Session.TorqueLevel = db.GetLevel(Session.UserID, "Torque");
    return change;
}
```

```
private void LevelClose_Click(object sender, EventArgs e)
{
    LevelUIToggle(false);
    change = 0;
}

0 references
private void LevelChange()
{
    LevelMessage1.Text = "You Levelled UP!";
    if (change == 1)
    {
        LevelMessage2.Text = "Welcome to Level 2";
    }
    if (change == 2)
    {
        LevelMessage2.Text = "Welcome to Level 3";
    }
    if (change == 3)
    {
        LevelMessage1.Text = "OH NO! You've been DEMOTED";
        LevelMessage2.Text = "Back to Level 2";
    }
    if (change != 0)
    {
        LevelUIToggle(true);
    }
}

2 references
private void LevelUIToggle(bool flag)
{
    LevelMessage1.Visible = flag;
    LevelMessage2.Visible = flag;
    LevelChangePNL.Visible = flag;
    LevelClose.Visible = flag;
}
```

While testing the program, I found that my thresholds where to high so I changed them to 0.4 (1-> 2), 0.6 (2 -> 3) and 0.25(3->2). Additionally, I found that there were errors in calculating the performance score due to errors occurring because integer division zeroes out key parts.

To fix this, I made the following changes:

```
double NormalisedScore = (double)db.GetTotalScore(Session.UserID, puzzleType) / (TotalN * 100);
```

```
public void SimulationUseRate()
{
    SimulationsRate = (double)db.GetTotalSimulation(Session.UserID, puzzleType)/TotalN;
}
1 reference
public void SkipRate()
{
    SkippedRate =  (double)db.GetTotalSkips(Session.UserID,puzzleType) / TotalN;
}
```

This however did not fully solve the problem as the use zero values in the linear regression meant that the slope was giving a negative value even for high quality data. To fix this, I made the following changes

```
RecentScores = db.GetLast15Scores(Session.UserID, puzzleType).Where(s => s.score > 0).ToList();
RecentScores.Reverse();
```

In addition, to prevent against little spikes that would cause 1 correct answer to promote, I implemented a minimum number of puzzles for each level.

```
int n = TotalN - db.GetTotalSimulation(Session.UserID, puzzleType);

if (db.GetLevel(Session.UserID, puzzleType) == 1 && PerformanceScore > 0.4 && n > 8)
{

    db.setLevel(Session.UserID, puzzleType, 2);
    change = 1;
}
if ((db.GetLevel(Session.UserID, puzzleType) == 2 && PerformanceScore > 0.6 && n > 21))
{

    db.setLevel(Session.UserID, puzzleType, 3);
    change = 2;
}
if ((db.GetLevel(Session.UserID, puzzleType) == 3 && PerformanceScore < 0.25) && n > 26)
{

    db.setLevel(Session.UserID, puzzleType, 2);
    change = 3;
}
Session.ProjectileLevel = db.GetLevel(Session.UserID, "Projectile");
Session.TorqueLevel = db.GetLevel(Session.UserID, "Torque");
return change;
```

Validation

No validation is required as there is no data coming from the user

Review

Following testing I can conclude that I was successfully able to implement

- Log In /Registration System
- Torque Sandbox
- Projectile Motion Sandbox
- Torque Puzzles
- Projectile Motion Puzzles
- Scoring
- Leaderboard
- Usability Features
- Levels

Prototype 7 – Save Feature

In my final prototype, I will be creating the save feature for my program. This feature will allow users to save both their custom sandbox configurations and any puzzles they have attempted, ensuring they can revisit and analyse their previous work. Saved setups will only be accessible through Sandbox mode, and each saved file will include a user-defined name as well as an automatic timestamp, allowing users to organise and identify their sessions more easily. In this prototype I will also add the following usability features

- Tutorial
- Sound Effects
- A notes feature

The notes feature is a toggleable textbox that lets users jot down calculations or reminders while solving puzzles. It can be shown or hidden as needed, keeping the interface clear while still supporting problem-solving and user convenience.

Database

Table: ProjectileSave

Column Name	Data Type	Description
SaveID	PRIMARY KEY	Unique ID for each saved puzzle
UserID	FOREIGN KEY	ID of user who saved the setup
SaveName	STRING	Name given to the save
Timestamp	DATETIME	Timestamp of save
Mass	INT	Mass of projectile
StartHeight	INT	Starting Height
Angle	INT	Angle of release (deg)
InitialVelocity	INT	Initial Velocity Value
Gravity	FLOAT	Gravity Value
Friction	FLOAT	Coefficient of friction
Restitution	FLOAT	Coefficient of restitution
Drag	FLOAT	Drag coefficient
Target	INT	Position of target

```
CREATE TABLE ProjectileSave
(
	SaveID INTEGER PRIMARY KEY AUTOINCREMENT
	UserID INTEGER NOT NULL,
	SaveName TEXT NOT NULL,
	Timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
	Mass INTEGER,
	StartHeight INTEGER,
	Angle INTEGER,
	InitVel INTEGER,
	Gravity REAL,
	Friction REAL,
	Restitution REAL,
	Drag REAL,
	Target INT,
	FOREIGN KEY (UserID) REFERENCES Users(UserID) ON DELETE CASCADE
)
```

Table: TorqueSave

Column Name	Data Type	Description
SaveID	PRIMARY KEY	Unique ID for each saved puzzle
UserID	FOREIGN KEY	ID of user who saved the setup
SaveName	STRING	Name given to the save
Timestamp	DATETIME	Timestamp of save
PivotPositions	TEXT	JSON array representing where pivots are

Masses	TEXT	JSON array representing mass on each position

```
CREATE TABLE TorqueSave
(
	SaveID INTEGER PRIMARY KEY AUTOINCREMENT,
	UserID INTEGER NOT NULL,
	SaveName TEXT NOT NULL,
	Timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
	PivotPositions TEXT NOT NULL,
	Masses TEXT NOT NULL,
	FOREIGN KEY (UserID) REFERENCES Users(UserID) ON DELETE CASCADE
)
```

My code will perform both retrieval and insertion operations on the rows in this database.

Algorithms

Unique Name Generation

This algorithm will ensure that each name inputted is unique to prevent clashes.

```
UniqueName(baseName, UserID)
	existingName = QUERY db for SetupnAME where UserID
	if baseName NOT IN existing names
		return baseName
else
	Count = 1
	newname = baseName + count
	while newname in existingName
		count ++
		newname = baseName + count
return newname
```

Load from DB

```
LoadSetupMenu
        setupRow = QUERY database for SetupName, DateSaved WHERE UserID = userID AND
        PuzzleType = puzzleType ORDER BY DateSaved DESC
        menuList = []
        for each row in setupRows:
                menuList.ADD(row.SetupName + " (" + row.DateSaved + ")")
        return menuList
```

Key Variables

Variable Name	Data Type	Description	Justification
setupName	string	Name of saved setup	Must be unique to allow selection
jsonData	String	Serialized JSON representation of the setup	Stores the entire setup in a single, structured format. JSON is lightweight, extensible, and allows nested structures
baseName	string	Default base for unique naming	Handles cases when users don't input a name. A default base name allows automatic naming with uniqueness checks.

Validation

Each saved setup requires a unique name to prevent overwriting existing data. The prototype checks the user-provided name against all previously saved setups for the same puzzle type and user ID. If a duplicate is detected, the system automatically appends a numeric suffix to create a unique name. Additionally, a maximum name length is enforced to prevent excessively long strings that could disrupt the database or UI. This approach ensures clarity for the user while maintaining data integrity.

Testing

Input	Testing	Justification	Expected Result
Save a puzzle	Save puzzle	To check if the data is being stored correctly	Puzzle Saved
Open a puzzle	Load puzzle	To check if the data can be used to recreate a setup	Puzzle opened as expected
Add Notes	Notes feature	To check if notes can be added using the textbox	Notes textbox visible

Development

I first began by creating the unique name algorithm

```
public string UniqueName(string baseName, int userID)
{
    string sql = $"SELECT SaveName FROM {mode}Save WHERE UserID = @userID";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@userID", userID));
    List<string> saveNames = result.AsEnumerable()
                        .Select(row => row.Field<string>("SaveName"))
                        .ToList();
    if (!saveNames.Contains(baseName))
    {
        return baseName;
    }
    int count = 1;
    string newName = baseName + count.ToString();
    while (saveNames.Contains(newName))
    {
        count++;
        newName = baseName + count.ToString();
    }
    return newName;


}
```

The Unique Name algorithm is designed to ensure that every saved game name for a particular user is unique. When the method is called, it first retrieves all the existing save names associated with that user from the database and stores them in a list. It then checks whether the base name entered by the user already exists in that list. If the name is not found, it is returned immediately as it is already unique. However, if the name is taken, the algorithm concatenates a number to the end of the base name, starting from one, and continues to increment this number until it finds a name that does not appear in the list. This new name is then returned and can be used as the unique identifier for the save file. For example, if the user already has saves called “Game”, “Game1”, and “Game2”, the algorithm will automatically generate “Game3”.

```
string text = SaveTextbox.Text;
validation = true;
ErrorMsgLength.Visible = false;
ErrorMsgLength.Text = "";


if (text.Contains("{") || text.Contains("}"))
{
    ErrorMsgLength.Text += "Name cannot contain { or }\n";
    validation = false;
}


if (text.Length < 4 || text.Length > 16)
{
    ErrorMsgLength.Text += "Setup Name must be between 4 and 16 characters inclusive\n";
    validation = false;
}
else if (text.Length == 16)
{
    ErrorMsgLength.Text += "Max Characters reached\n";

}


if (ErrorMsgLength.Text.Length > 0)
{
    ErrorMsgLength.Visible = true;
}
else
{
    ErrorMsgLength.Visible = false;
}
```

At the beginning of the method, the program retrieves the text currently entered in the textbox and clears any previous error messages. It then performs two key validation checks, character and length validation.

The program verifies that the text does not contain the characters { or }. These symbols are considered invalid for a setup name. If either of these characters is detected, an appropriate error message is appended to the label ErrorMsgLength, and the validation flag is set to false to indicate that the input is not acceptable. The program also ensures that the setup name is between 4 and 16 characters in length. If the input is shorter than 4 characters or longer than 16 characters, the message “Setup Name must be between 4 and 16 characters inclusive” is displayed, and validation is set to false. If the input length is exactly 16 characters, the message “Max Characters reached” is shown to inform the user that the upper limit has been reached, although the input is still considered valid.

Following this, I created the database methods to save and load puzzles to and from the database

```
public void StoreProjectile(int UserID, string saveName, int mass, int startheight, int angle, int vel, double gravity, double friction, double restitution, double drag, in
{
    string sql = @"
        INSERT INTO ProjectileSave(UserID, SaveName, Mass, StartHeight, Angle, InitVel, Gravity, Friction, Restitution,Drag,Target)
        VALUES(@userID,@saveName,@mass,@height,@angle,@velocity,@grav, @mu, @e, @drag, @target);
        ";
    ExecuteNonQuery(sql,
        new SQLiteParameter("@userID", UserID),
        new SQLiteParameter("@saveName", saveName),
        new SQLiteParameter("@mass", mass),
        new SQLiteParameter("@height", startheight),
        new SQLiteParameter("@angle", angle),
        new SQLiteParameter("@velocity", vel),
        new SQLiteParameter("@grav", gravity),
        new SQLiteParameter("@mu", friction),
        new SQLiteParameter("@e", restitution),
        new SQLiteParameter("@drag", drag),
        new SQLiteParameter("@target", target)
        );
}
1 reference
public (int mass, int height, int angle, int velocity, double gravity, double friction, double restitution, double drag, int target) GetProjectilePuzzle(int userID, string
{
    string sql = @"
SELECT Mass, StartHeight, Angle, InitVel, Gravity, Friction, Restitution, Drag, Target
FROM ProjectileSave
WHERE UserID = @userID AND SaveName = @saveName";

    DataTable result = ExecuteQuery(sql,
        new SQLiteParameter("@userID", userID),
        new SQLiteParameter("@saveName", saveName)
    );
```

The puzzle is returned as a tuple to make it easy to use when showing the user their previous setup

```
private void populatePuzzle((int mass, int height, int angle, int velocity, double gravity, double friction, double restitution, double drag, int target) puzzle)
{
    varmass.Value = puzzle.mass;
    varstartHeight.Value = puzzle.height;
    varangle.Value = puzzle.angle;
    varinitialVelocity.Value = puzzle.velocity;
    vargravity.Value = Convert.ToInt32(puzzle.gravity * 10);
    varCOF.Value = Convert.ToInt32(puzzle.friction * 10);
    varCOR.Value = Convert.ToInt32(puzzle.restitution * 10);
    vardragCoefficient.Value = Convert.ToInt32(puzzle.drag * 10);
    target.Location = new Point(Convert.ToInt32(50 * puzzle.target - 50), target.Location.Y);

    varlbl_target.Text = puzzle.target.ToString() + "m";
    varlbl_drag.Text = puzzle.drag.ToString();
    varlbl_cor.Text = puzzle.restitution.ToString();
    varlbl_mu.Text = puzzle.friction.ToString();
    varlbl_gravity.Text = puzzle.gravity.ToString();
    varlbl_initialVelocity.Text = puzzle.velocity.ToString() + "m/s";
    varlbl_degree.Text = puzzle.angle.ToString() + "deg";
    varlbl_startHeight.Text = puzzle.height.ToString() + "m";
    varlbl_mass.Text = puzzle.mass.ToString() + "kg";
```

```
public List<(string name, DateTime date)> GetAllName(int userID)
{
    string sql = $"SELECT SaveName, Timestamp FROM {mode}Save WHERE UserID = @userID";
    DataTable result = ExecuteQuery(sql, new SQLiteParameter("@userID", userID));
    var saves = new List<(string name, DateTime date)>();

    foreach (DataRow row in result.Rows)
    {
        string name = row["SaveName"].ToString();
        DateTime date = DateTime.Parse(row["Timestamp"].ToString());
        saves.Add((name, date));
    }

    return saves;
}
```

```
private void OpenSetupButton_Click(object sender, EventArgs e)
{
    if (open)
    {
        open = false;
        Toggle(this, true, "File");
        FilePNL.SendToBack();
        OpenSetupButton.Text = "☑";
        List<(string name, DateTime date)> Saves = database.GetAllName(Session.UserID);
        FileDropDown.Items.Clear();
        foreach (var save in Saves)
        {
            FileDropDown.Items.Add($"{save.name} ({save.date:dd/MM/yyyy})");
        }
    }
    else
    {
        open = true;
        string selectedText = FileDropDown.SelectedItem.ToString();
        if (selectedText.Length > 0)
        {
            string selectedName = selectedText.Split(new string[] { " (" }, StringSplitOptions.None)[0];
            var puzzle = database.GetProjectilePuzzle(Session.UserID, selectedName);
            populatePuzzle(puzzle);
            Toggle(this, false, "File");
            OpenSetupButton.Text = "🖫";
        }

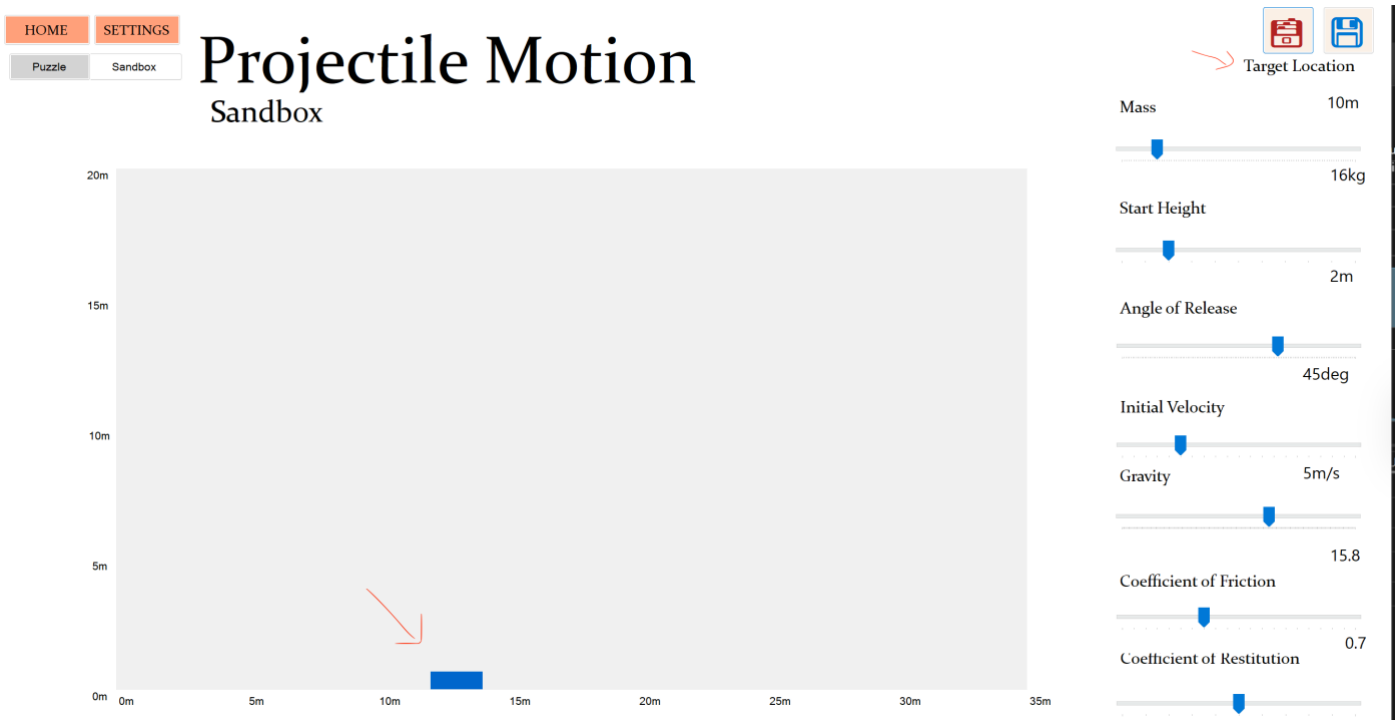
    }
}
```

When a user attempts to load a puzzle, the drop down menu of previous values is shown and the menu is populated in the format Name (date). When the user selects a value from the drop down menu, the value is split at the “ (” so that it can be used to load the required puzzle from the database.

When creating the code to show a previous setup, I added the following line

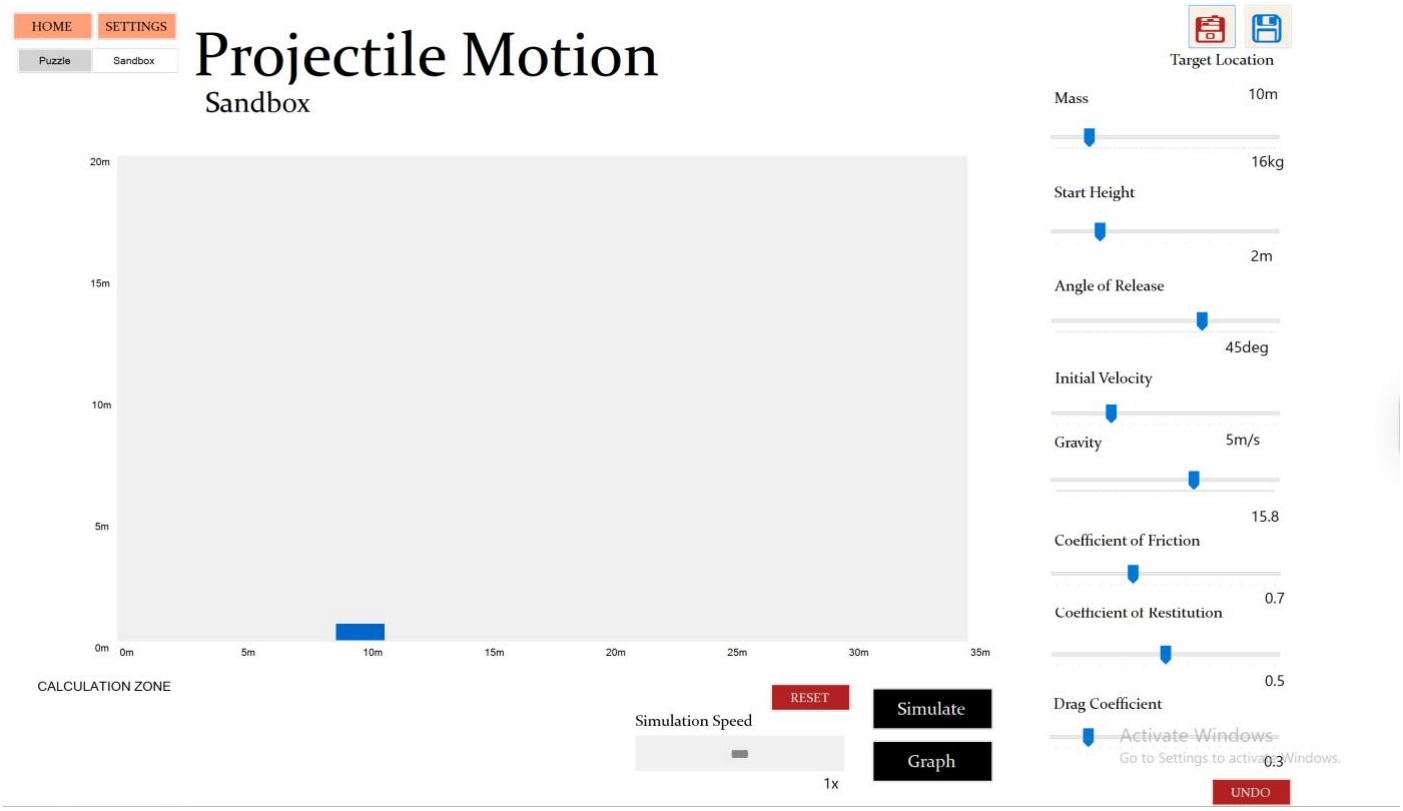
```
target.Location = new Point(Convert.ToInt32(36.75 * puzzle.target + 237), 965);
```

This however did not place the target in the correct x position as seen below



When I changed that line to this, it worked as expected

```
target.Location = new Point(Convert.ToInt32(50 * puzzle.target - 50), target.Location.Y);
```



At the moment, the setups are displayed with oldest at the top and newest at the bottom. As the user would be more likely to want to find newer saved setups, I have flipped the order that the values are inputted into the comboBox.

```
Saves.Reverse();
```

```
public void StoreTorque(List<int> pivotPositions, List<int> mass,int userID, string saveName)
{
    string pivotJSON = JsonConvert.SerializeObject(pivotPositions);
    string massJSON = JsonConvert.SerializeObject(mass);
    string sql = @"
        INSERT INTO TorqueSave(UserID, SaveName, PivotPositions,Masses)
        VALUES(@userID,@saveName,@pivots,@masses);
        ";
    ExecuteNonQuery(sql,
        new SQLiteParameter("@userID", userID),
        new SQLiteParameter("@saveName", saveName),
        new SQLiteParameter("@pivots", pivotJSON),
        new SQLiteParameter("@masses", massJSON)
        );
}
```

This method adds a user's torque setup to the database by converting lists of pivot positions and masses into JSON strings and inserting them into the database. Using JSON allows variable-length data to be stored in a single record, making the system flexible to changes in the number of pivots or masses.

```
public (List<int> pivotPositions, List<int> masses) LoadTorque(int userID, string saveName)
{
    string sql = @"
    SELECT PivotPositions, Masses
    FROM TorqueSave
    WHERE UserID = @userID AND SaveName = @saveName;
    ";

    DataTable result = ExecuteQuery(sql,
        new SQLiteParameter("@userID", userID),
        new SQLiteParameter("@saveName", saveName)
    );

    DataRow row = result.Rows[0];

    List<int> pivotPositions = JsonConvert.DeserializeObject<List<int>>(row["PivotPositions"].ToString()); //returns the positions of the active pivots
    List<int> masses = JsonConvert.DeserializeObject<List<int>>(row["Masses"].ToString()); // returns the mass at each position (0-10)

    return (pivotPositions, masses);
}
```

This method retrieves a user's saved torque configuration from the database for use in the sandbox. The pivot and mass data are stored as JSON in the database, so deserialization is used to convert them back into lists.

```
string selectedText = FileDropDown.SelectedItem.ToString();
if (selectedText.Length > 0)
{
    string selectedName = selectedText.Split(new string[] { " (" }, StringSplitOptions.None)[0];
    var puzzle = database.GetProjectilePuzzle(Session.UserID, selectedName);
    populatePuzzle(puzzle);
    Toggle(this, false, "File");
    OpenSetupButton.Text = "";
}
```

Initially, I attempted to validate the dropdown selection by checking the length of the string returned from SelectedItem.ToString(). However, this approach was flawed because SelectedItem could still be null, causing a NullReferenceException before the length check even occurred. To fix this, I first checked whether SelectedItem was null, and only if that was false would I access and process its string value.

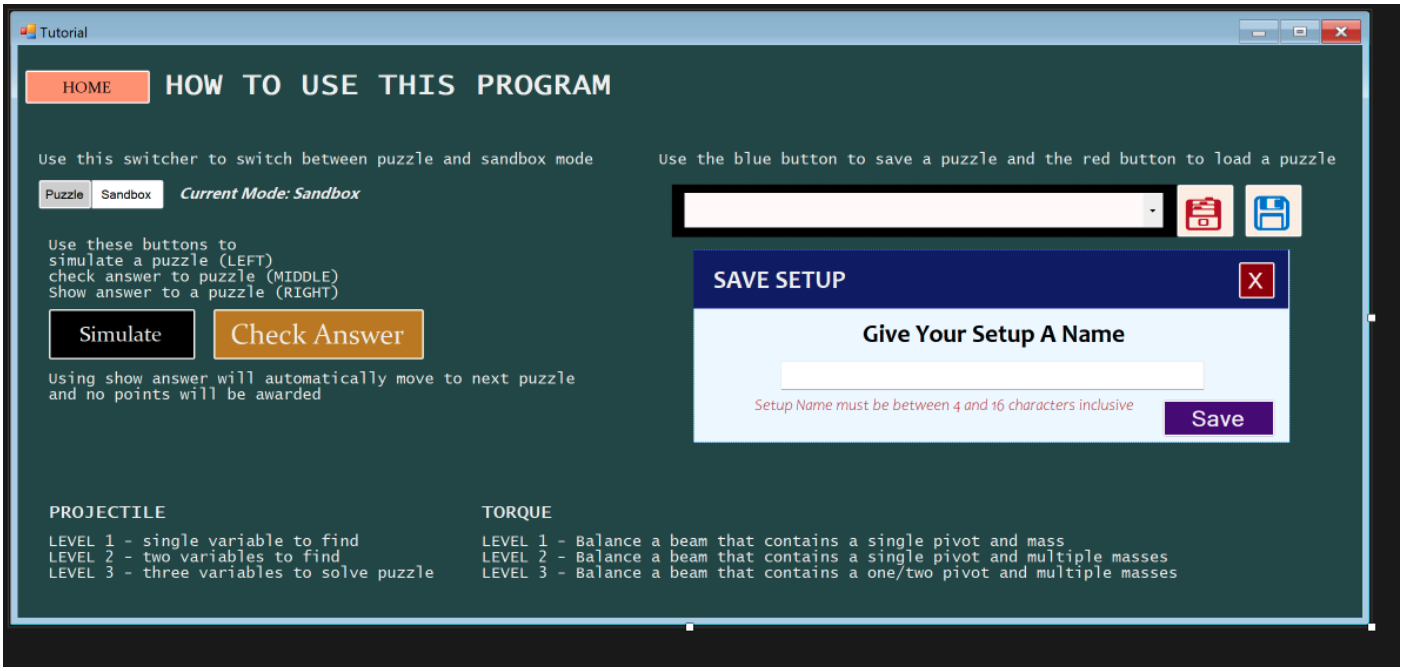
```
open = true;
if (FileDropDown.SelectedItem != null)
{
    string selectedText = FileDropDown.SelectedItem.ToString();
    string selectedName = selectedText.Split(new string[] { " (" }, StringSplitOptions.None)[0];

    var puzzle = database.GetProjectilePuzzle(Session.UserID, selectedName);
    populatePuzzle(puzzle);
    Toggle(this, false, "File");
    OpenSetupButton.Text = "";
}
else
{
    MessageBox.Show("No file selected. Please choose one from the list.");
}
```

```
private void populatePuzzle((List<int> pivotPositions, List<int> masses) puzzle)
{
    resetSandbox();
    for (int i = 0; i< puzzle.masses.Count; i++)
    {
        int massValue = puzzle.masses[i];
        Label lbl = this.Controls.Find("lbl_m" + (i + 1), true).FirstOrDefault() as Label;
        VScrollBar massSlider = this.Controls.Find("mass" + (i+1),true).FirstOrDefault() as VScrollBar;
        massSlider.Value = massValue;
        lbl.Text = (massSlider.Maximum = massValue).ToString() + "kg";
    }
    foreach(int pivotIndex in puzzle.pivotPositions)
    {
        Button pivotButton = this.Controls.Find("pivot" + pivotIndex, true).FirstOrDefault() as Button;
        PivotClick(pivotButton,EventArgs.Empty);
    }
    this.Invalidate();
}
```

This method restores a saved torque setup to the UI. It first clears any existing pivots or masses. It then iterates through the masses list, setting each mass slider directly to its saved value and updating the corresponding label to display the actual mass in kilograms. After that, it loops through the pivotPositions list and activates each pivot using PivotClick, ensuring both the UI and engine state reflect the saved setup. Finally, the form is refreshed, so all changes are visually updated.

In addition, in this prototype I also created a basic tutorial as seen below



During evaluation, my stakeholders commented that the program was intuitive and did not require extensive guidance to use. In response, the tutorial was made to focus on clarity rather than instruction. Most elements were kept static, however, limited interactivity was retained through the mode switcher and file handling buttons, allowing users to explore key features without affecting the program state.

```
private void PlayCelebrationTune()
{

    (int freq, int duration)[] melody = new (int, int)[]
    {
(523, 200), // C5
(659, 200), // E5
(784, 200), // G5
(1046, 400), // C6
(784, 200), // G5
(659, 200), // E5
(523, 400), // C5
    };

    foreach (var note in melody)
    {
        Console.Beep(note.freq, note.duration);
        Thread.Sleep(50);
    }
}
```

The PlayCelebrationTune method is designed to produce a short musical sequence using the system speaker. Within the method, a tuple array named melody defines the sequence of musical notes, where each tuple consists of a frequency, measured in Hertz, and a duration, measured in milliseconds. The specific notes selected, C5, E5, G5, C6, G5, E5, and C5, form a simple ascending and descending pattern, characteristic of a celebratory tune.

Testing

Input	Testing	Justification	Expected Result	Result
Save a puzzle	Save puzzle	To check if the data is being stored correctly	Puzzle Saved	**SUCCESS** *For Projectile: See Prototype 7 – Video 1 [0:00 - 0:04]*

				For Torque: See Prototype 7 – Video 2 [0:00 – 0:05]
Open a puzzle	Load puzzle	To check if the data can be used to recreate a setup	Puzzle opened as expected	**SUCCESS** *For Projectile: See Prototype 7 – Video 1 [0:04 – 0:09]* *For Torque: See Prototype 7 – Video 2 [0:05 - 0:09]*
Add Notes	Notes feature	To check if notes can be added using the textbox	Notes textbox visible	**SUCCESS** *See Figure 10.1*

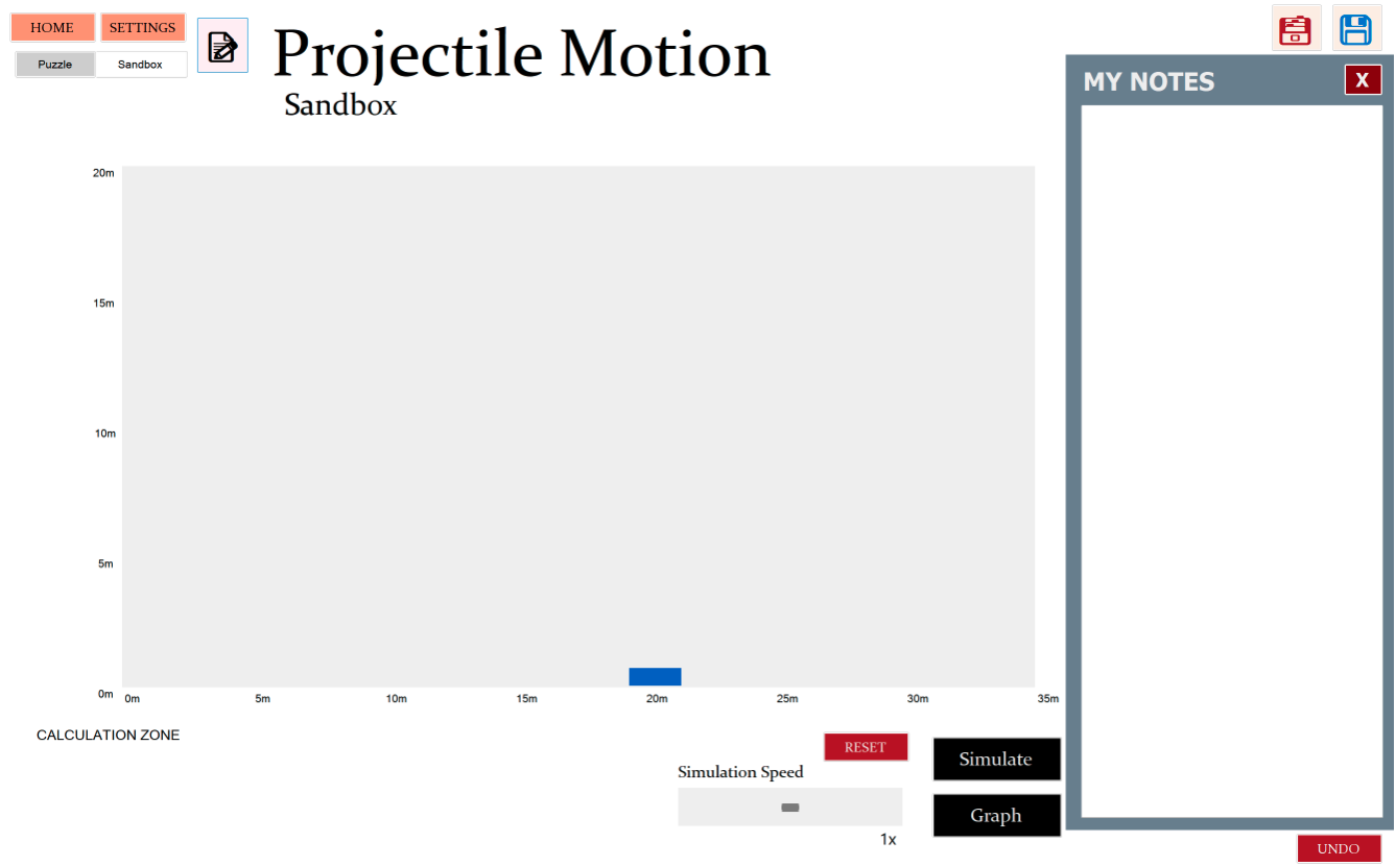


Figure 10. 1

Review

Following testing I can conclude that I was successfully able to implement

- Log In /Registration System
- Torque Sandbox
- Projectile Motion Sandbox
- Torque Puzzles
- Projectile Motion Puzzles
- Scoring
- Leaderboard
- Usability Features
- Multiple Levels with user tracking
- Save/Load Puzzles
- Tutorial
- Sound Effects
- Annotations/Notes Features

Testing & Evaluation

With consideration to time constraints, the solution has now been fully developed. The aim of this testing is to compare the developed solution to the success criteria in order to identify the strengths and limitations of the current solution as well as areas for improvement for future development

Each test below has been given a number taken from the success criteria's post-development test table and all justifications can be found in that table. If any extra information is required, it can be found in the below table. All video files can be found in the folder *NEA Videos / Final Testing Videos*

Function Testing

Success Criteria	Test No.	Description	Test Data	Type of Data	Expected Outcome	Outcome	Evidence
1.1	1	Ensures that the solution can accurately simulate the projectile motion	Parameters set to a GCSE / A-Level style question that does not consider friction, coefficient of restitution or resistance	Normal	The projectile is simulated as expected	**SUCCESS** *The projectile followed a parabolic path as expected*	See Test Video 1.mp4
	2		Consider drag and COR	Normal	The projectile is simulated as expected	**SUCCESS**	See Test Video 2.mp4
1.2-1.5	3	The user should be able to change these values in order to experiment and solve puzzles	**When in sandbox mode** Set Start Height to 5m Set angle to 75 deg Set velocity to 15m/s Set mass to 10kg	Normal	The values of height, angle, velocity and mass should change and show the user the new values	**SUCCESS**	See Test Video 3.mp4 0:00 – 0:26
	4		**When in puzzle mode** Can the user change the start	Normal		**SUCCESS** *The solution will only allow*	See Test Video 4

			height, angle, velocity and mass as necessary			*the user to change some values and not the preset values for the puzzle*	0:26 – 0:33
1.6	5	Allow the user to move the target and the user can see the position of the target.	The user drag and drops the target	Normal	The target can move and the position of the target can be seen by the user	**SUCCESS** *The user can drag and drop the target to place it on the ground of the sandbox*	See Test Video 4
1.7	6	The puzzles should be unique so that they can solve new problems without redoing the same problems The puzzles should be unique so that they can solve new problems without redoing the same problems	Go through ten puzzles and see if the puzzles repeat.	Normal	Each puzzle should be unique	**SUCCESS** *Each puzzle initial setup was different*	See Test Video 5
	7		Through the use of a stakeholder feedback form ask the user if they feel that the puzzles repeat	Normal	Each puzzle should be unique	**SUCCESS** *Average score 8.3/10 therefore the stakeholders found that they were unique, however they were quite similar*	See Chart 1
1.8	8	Check that the user is solving problems that are suitable for the level that they are at	Stakeholder Questionnaire	Normal	The puzzles should match user level	**SUCCESS** *Average score 8.6/10 therefore stakeholders felt that overall the puzzles were mostly suitable*	See Chart 2
1.9, 1.10,1.15	9	The solution should display a graph when requested	Parameters set to a GCSE / A-Level style question that does not consider friction, coefficient of restitution or resistance	Normal	A graph that shows the velocity and acceleration of the projectile should be shown	**SUCCESS**	See Test Video 6 0:13 – 0:29
	10		Consider drag and COR	Normal		**SUCCESS**	See Test Video 6 0:00 – 0:13
1.11	11	The graph should display the key values on the graph	Parameters are all non-zero	Normal	The graph should have labelled key values such as stationary points	**Feature not implemented**	
1.12	12	The motion of the projectile should be	The simulation speed is set to 0.5x	Normal	The solution should show an	**SUCCESS**	See Test Video 6

		animated with the user able to change the speed of the simulation			animation of the projectile		0:52 – 1:04
	13		The simulation speed is set to 1.5x	Normal	The solution should show an animation of the projectile	**SUCCESS**	See Test Video 6 1:04 – 1:18
1.13	14	The motion displayed should be accurate to the physics it is meant to be simulating	Parameters set to a GCSE / A-Level style question that does not consider friction, coefficient of restitution or resistance	Normal	The motion displayed should be a concave parabolic curve	**SUCCESS**	See Test Video 1
	15	The motion displayed should be accurate to the physics it is meant to be simulating	Stakeholder Questionnaire	Normal	The motion displayed should be accurate	**SUCCESS** *Average score 9/10 therefore stakeholders found the simulation to be accurate*	See Chart 3
1.14	16	The solution should display the calculations taking place to simulate the motion	Parameters are all non zero	Normal	The solution should show the calculation that is accurate	**SUCCESS**	See Test Video 6 1:25 – 1:30
1.16	17	The user should be able to zoom into sections of the graph	Parameters are all non zero and the user should attempt to increase the scale on the axis	Normal	The solution zooms into the graph	**Feature not implemented**	
1.17	18	The user should be able to see a vector representation of the setup	Parameters are all non zero	Normal	The user can see that vectors	**Feature not implemented** *I instead allowed the user to see the vertical and horizontal components of the graph rather than drawing them as vectors at every point along the graph*	
1.18	19	The user should be able to change what graphs are visible to them as well as the colour and thickness of those lines	Parameters are all non zero User hides/shows velocity and acceleration graphs	Normal	The graphs should show and hide by buttons	**SUCCESS**	See Test Video 6 0:17 – 0:35
	20		Parameters are all non zero User changes colours of lines	Normal	The colour of the lines should change	**SUCCESS**	See Test Video 6 0:42 – 0:48

	21		Parameters are all non zero User changes thickness of lines	Normal	The thickness of the lines should change	**SUCCESS**	See Test Video 6 0:35 – 0:42
1.19	22	The solution should allow the user to undo and redo changes that they have made to the setup	The user clicks undo	Normal	The previous state should appear	**FAILED** *The undo button did not work as expected as instead of reverting back to the previous state it instead created a new state*	See Test Video 6 1:30 – 1:34
	23		The user clicks redo	Normal	Should go back to original state	**Feature not implemented**	
1.20	24	Is the correct answer to the puzzle shown when the user request it	The user should be in puzzle mode and click a button to show the answer	Normal	The answer to the puzzle should be displayed	**SUCCESS** *The answer provided when show answer is clicked will according to the SUVAT equations mean that the projectile hits the target*	See Figure 11.1
2.1	24	Ensures that the solution can accurately simulate the motion	Single pivot and mass	Normal	The motion is simulated as expected	**SUCCESS**	See Test Video 7 0:10 – 0:15
	25		Multiple mass and pivots	Normal	The motion is simulated as expected	**SUCCESS**	See Test Video 7 0:20 – 0:26
2.2	26	The user should be able to complete puzzles that contain multiple pivot points	The user should be in puzzle mode	Normal	A puzzle with multiple pivots should show	**SUCCESS**	See Figure 11.3
2.3	27	The user should be able to place multiple masses at each pivot point	Place 15kg of mass at the centre of the beam	Normal	The solution should display 15kg of mass at the centre of the beam	**SUCCESS** *The solution does not use drag and drop but instead uses sliders so any mass can be placed at any point*	See Test Video 7 0:00 – 0:05
2.4	28	The solution should allow the user to undo and redo changes that they have made to the setup	The user clicks undo	Normal	The previous state should appear	**SUCCESS**	See Test Video 7 0:00 – 0:10
	29		The user clicks redo	Normal	Should go back to original state	**Feature not implemented**	

2.5	30	The puzzles should be unique so that they can solve new problems without redoing the same problems	Go through ten puzzles and see if the puzzles repeat.	Normal	Each puzzle should be unique	**SUCCESS**	See Test Video 8
	31		Through the use of a stakeholder feedback form ask the user if they feel that the puzzles repeat	Normal	Each puzzle should be unique	**SUCCESS** *Average score 8.3/10 therefore stakeholders found the puzzles to be unique however noted that they were quite similar*	See Chart 4
2.6	32	Check that the user is solving problems that are suitable for the level that they are at	Stakeholder Questionnaire	Normal	The puzzles should match user level	**SUCCESS** *Average score 9.3/10 therefore stakeholders found the level to be suitable*	See Chart 5
2.7	33	The solution should display the calculations taking place to simulate the motion	Parameters are all non-zero	Normal	The solution should show the calculation that is accurate	**SUCCESS**	See Test Video 7 0:10 – 0:15
2.8	34	Is the correct answer to the puzzle shown when the user request it	The user should be in puzzle mode and click a button to show the answer			**SUCCESS** *The answer provided would balance the puzzle as the anticlockwise torque is 12 and the solution would produce a clockwise torque of (1*2) + (1 *4) + (1 * 6) = 12 thereby balancing the puzzle*	See Figure 11.2
3.1	35	Does the leaderboard automatically update	Find user leaderboard position and then complete a puzzle and find new position	Normal	The leaderboard should show a change in the score of the user	**SUCCESS** *The account pran23 started on rank 8 and moved to rank 6*	See Test Video 9
3.2	36	The leaderboard should show an accurate score for the player	Take the score before and after completing a puzzle	Normal	The change in score match the points awarded	**SUCCESS** *The account pran23 started on 20 points and completed a puzzle in*	See Test Video 9

						which 11 points where awarded. After that the score showed 31.	
3.3	37	The user should have the option to leave the leaderboard	Click button to leave leaderboard	Normal	User's score should not be visible on the leaderboard	**SUCCESS** *When the user pran23 clicked to remove from leaderboard their score was no longer visible*	See Test Video 10 0:00 – 0:15
3.4	38	The user should have the option to rejoin the leaderboard	Click button to join leaderboard	Normal	User's score should be visible on the leaderboard	**SUCCESS** *When pran23 rejoined the leaderboard their score was visible and was preserved*	See Test Video 10 0:15 – 0:26
4.1	39	The user should be able to change these values in order to experiment	Set Start Height to 5m Set angle to 75 deg Set velocity to 15m/s Set mass to 10kg And all other values to non zero	Normal	The values of height, angle, velocity and mass should change and show the user the new values	**SUCCESS**	See Figure 11.4
4.2	40	The user should be able to create setups in torque puzzle	Place pivots at 4m and 12m Place masses at 6m, 10m and 18m	Normal	These changes should be applied and shown to the user	**SUCCESS**	See Figure 11.5
5.1	41	The user should not be able to create an account with an existing username	Attempt to create an account with an existing username	Normal	This action should be forbidden and an error message should appear.	**SUCCESS**	See Test Video 11
5.4	42	Users should be able to create accounts	Create an account	Normal	The account should be seen in the database	**SUCCESS** *Account with username: testing and password: password123 Successfully created*	See Test Video 12
5.2	43	A user should be able to log in to the system if they input the correct details	Log in using correct credentials	Normal	The user should be given access to the solution	**SUCCESS**	See Test Video 13 0:13 – 0:20
	44		Attempt to log in using incorrect credentials	Normal	The user should not be given access to the solution	**SUCCESS** *Showed what would happen if you inputted the incorrect username and*	See Test Video 13 0:00 – 0:13

						incorrect password	
5.3	45	The user should be able to use sandbox mode without an account	Log in as a guest	Normal	The user should only be able to access sandbox features	**Feature not implemented**	
5.5	46	When a user is signing back into the system their preferences, score and level should be retrieved and applied	Log back into an account that has an alternative colour theme	Normal	The solution should be shown in that colour scheme and the user should be able to see their current score and level	**SUCCESS** *The testing account was set to dark mode and when it was logged back into the dark mode was still applied and it was not shown on the pran23 account which is in light mode*	Test Video 14
5.6	47	Does the level that the user is at match their stage of learning	Stakeholder Questionnaire	Normal	The level should match the user's ability	**PARTIAL SUCCESS** *Average score 5.6/10 for projectile motion and 7.3/10 for torque, therefore stakeholders that is only somewhat matched*	See Chart 6 and 7
5.7	48	The stated level of the puzzle should match the difficulty of the puzzle	Stakeholder Questionnaire	Normal	The level should match the difficulty	**SUCCESS** *Average score 8.3/10 therefore stakeholders found the level and difficulty matched*	See Chart 8 and 9
6.1	49	The tutorial should appear for a new user	Create a new account	Normal	The tutorial should automatically appear	**SUCCESS**	Test Video 12 0:19 – 0:22
6.2	50	The user should be able to load the tutorial when they need it from the main menu	Attempt to load the tutorial	Normal	The tutorial should load	**SUCCESS** *Users can access the tutorial from the main menu*	Test Video 15
6.3	51	The tutorial should be easy to understand	Stakeholder Questionnaire	Normal	The tutorial is easy to understand	**PARTIAL SUCCESS** *Average score 7/3/10 therefore*	See Chart 10

						stakeholders found the tutorial somewhat easy to understand	
7.1	52	The user should be able to save a puzzle setup	**Projectile Motion** All parameters set to a standard A-Level question level Save as TestSetup	Normal	The data should be saved in the database under the name TestSetup	**SUCCESS** *As a saved setup called TestSetup already exists it was saved as TestSetup_1*	Test Video 16 0:00 – 0:11
	53		**Torque** Pivot at 10m, Masses as 4m, 8m and 16m Save as "TorqueSetup"	Normal	The data should be saved in the database under the name TorqueSetup	**SUCCESS**	Test Video 16 0:15 – 0:26
7.2	54	The user should be able to open previously saved setups	**Projectile Motion** Open setup named "TestSetup" created in test 52	Normal	The setup should be displayed correctly	**SUCCESS**	Test Video 16 0:30 – 0:38
	55		**Torque** Open setup named "TorqueSetup" created in test 53	Normal	The setup should be displayed correctly	**SUCCESS**	Test Video 16 0:44 – 0:52
8.1	56	The user should be able to apply themes to the UI	Change the theme of the UI	Normal	The colour theme should change in all the forms	**SUCCESS**	Test Video 14 0:00 – 0:14
8.2	57	The UI should be easy to navigate and use	Stakeholder Questionnaire	Normal	The UI is easy to navigate and use	**SUCCESS** *Average score 8/10 therefore stakeholders found the UI easy to use*	See Chart 11
8.3	58	**Projectile Motion** Is the UI drag and drop	Change the position of the target using drag and drop	Normal	The position of the target should change	**SUCCESS** *As the target moves the position in metres is shown at the top right of the screen*	Test Video 17
	59		Change the initial variables such as start height, and angle of release using drag and drop	Normal	The initial variables of the projectile should change	**NOT IMPLEMENTED** *In the solution I chose to use sliders as I felt it was easier for a new user to work out what to do and*	
	60	**Torque** Is the UI drag and drop	Place mases on the beam using drag and drop	Normal	The masses should appear on the beam		

						easier to add validation to	
8.4	61	The user should be able to toggle the music	Toggle music off	Normal	Music should not be heard in the solution	**SUCCESS**	See Test Video 21 0:40-0:52
	62		Toggle music on	Normal	Music should be heard in the solution	**SUCCESS**	See Test Video 21 0:00 – 0:20
8.5	63	The user should be able to reset their account	Click a button to reset account	Normal	The account should be completely reset so that the database no longer holds any saved setups, all preferences are reset, scores and levels are reset and their solve history is cleared	**SUCCESS** *See figure 11.6 for database values for before account was reset and figure 11.7 for after. The account used for this was pran23*	See Test Video 18
8.6	64	The user should be able to change their username and password	User changes their username	Normal	The user should be able to log in with the new username	**SUCCESS** *As seen in Figure 11.8 the user pran23 has had their username changed to newUsername*	See Test Video 19
	65		User changes their password	Normal	The user should be able to log in with their new password	**SUCCESS** The video shows the user changing their password and successfully changing logging on with their new password	See Test Video 20

Test 7 On a scale from 1-10 where 1 is the lowest and 10 is the highest, did you feel that the puzzles in projectile motion where unique and that you weren't redoing the same puzzles

3 responses

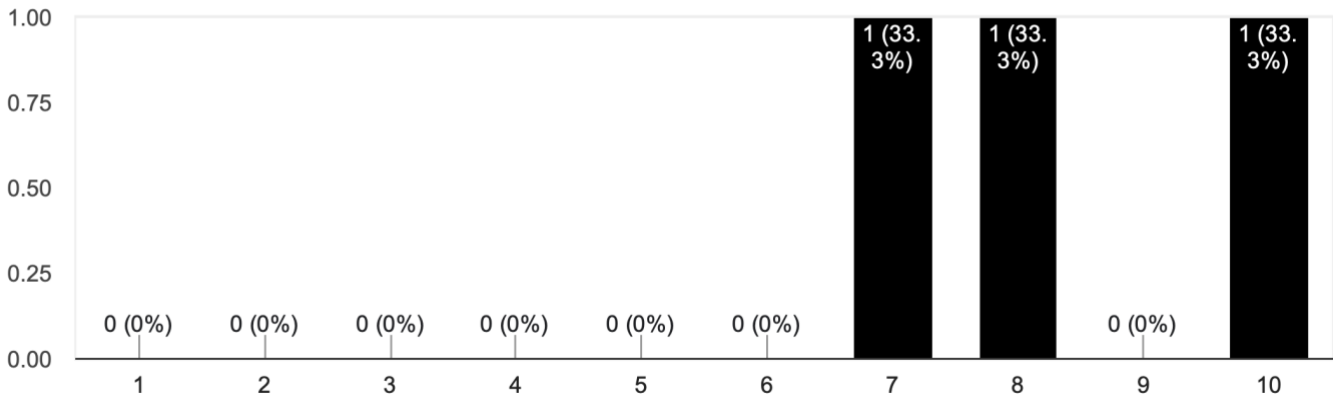


Chart 1

Test 8 How suitable was the projectile motion puzzle for the level that the user is on

3 responses

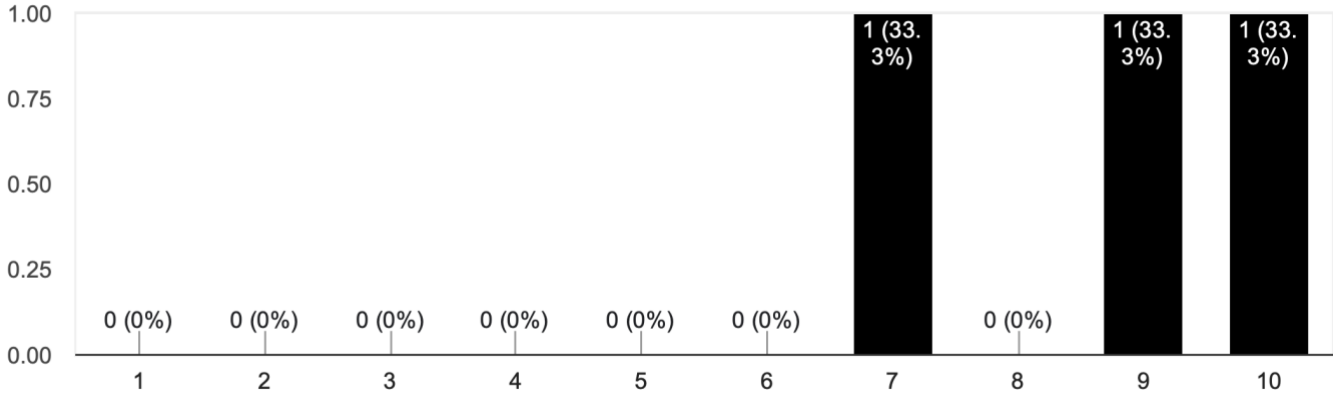


Chart 2

Test 15 How accurate was the simulation in projectile motion

3 responses

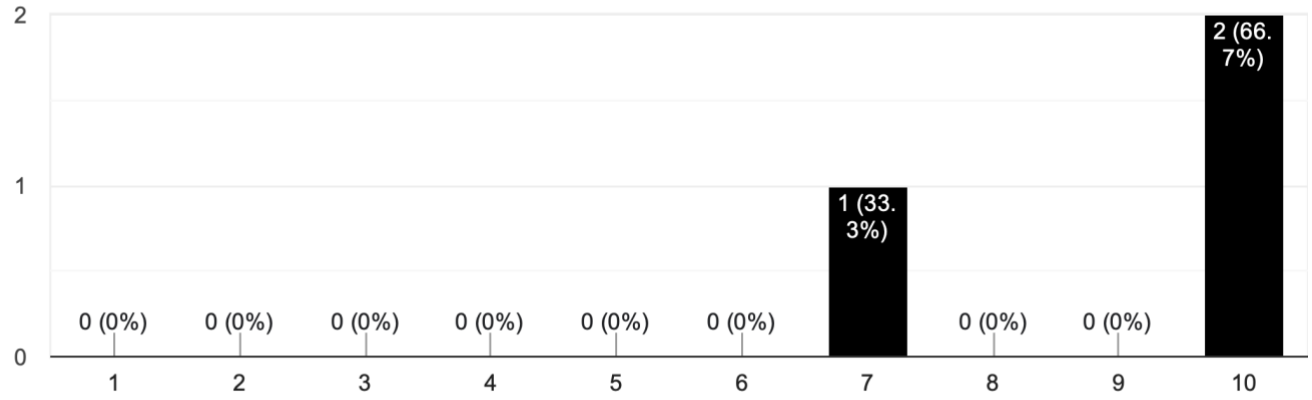


Chart 3

Test 31 On a scale from 1-10 where 1 is the lowest and 10 is the highest, did you feel that the puzzles in torque where unique and that you weren't redoing the same puzzles

3 responses

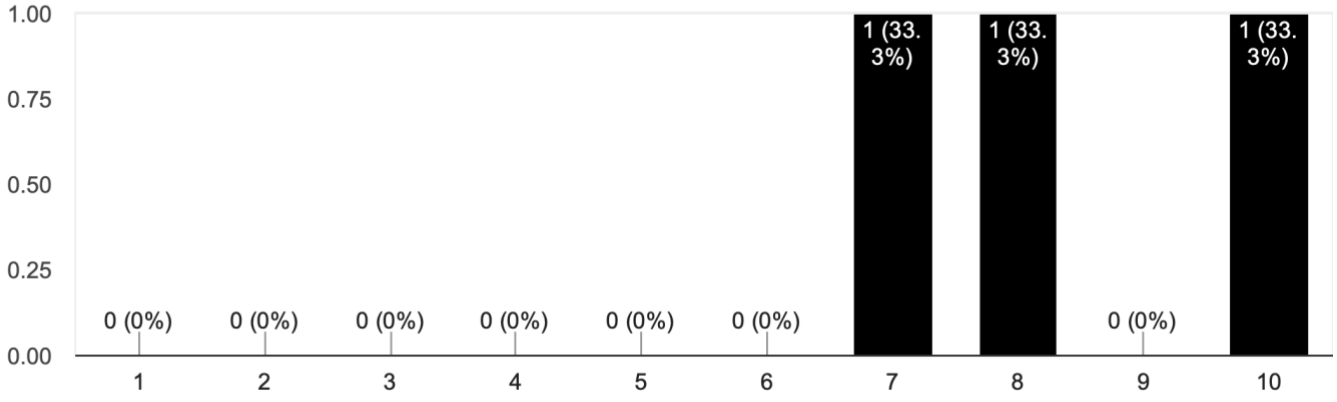


Chart 4

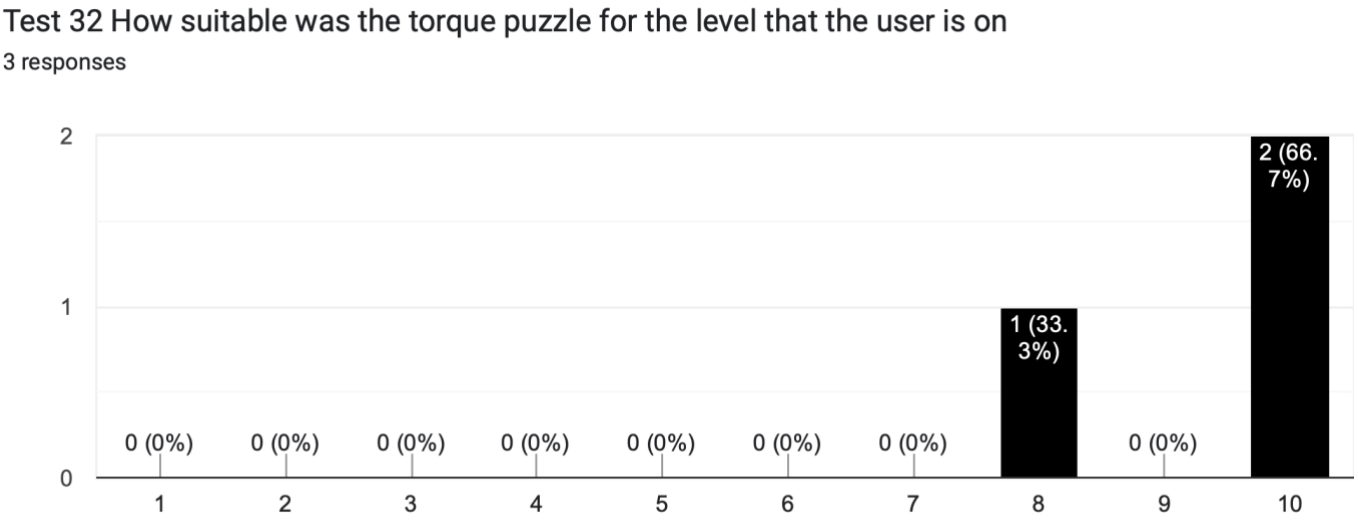


Chart 5

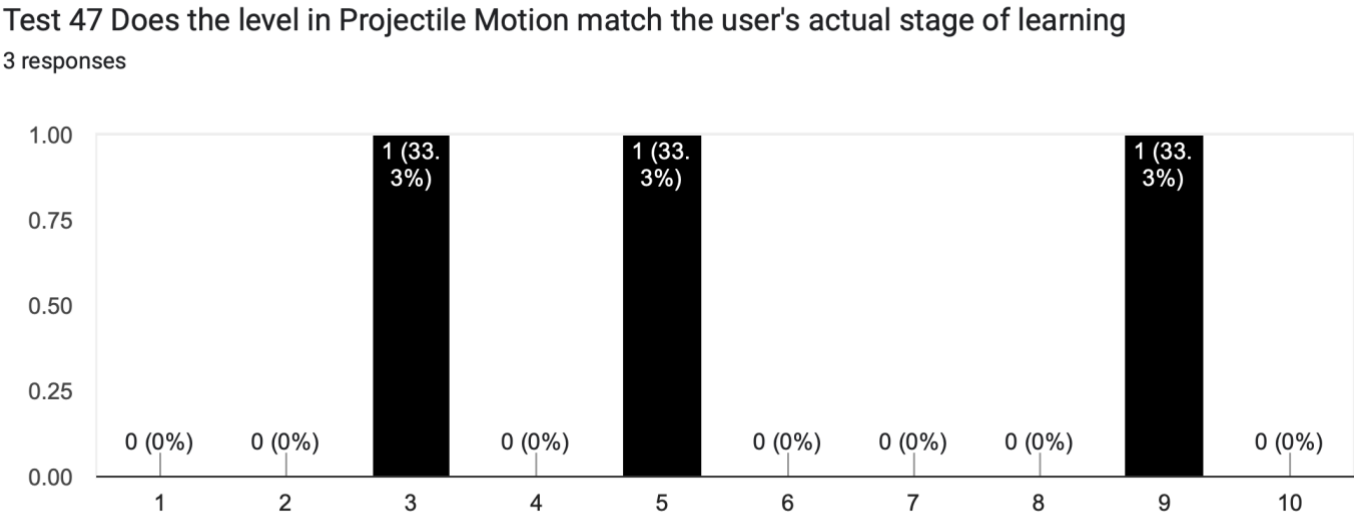


Chart 6

Test 47 Does the level in Torque match the user's actual stage of learning

3 responses

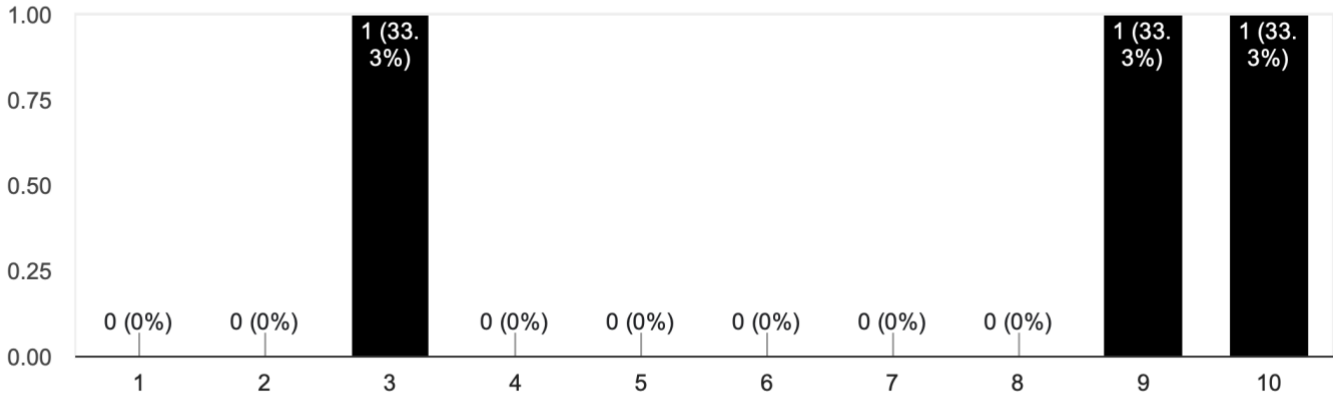


Chart 7

Test 48 Does the level in Torque match the difficulty of the puzzle

3 responses

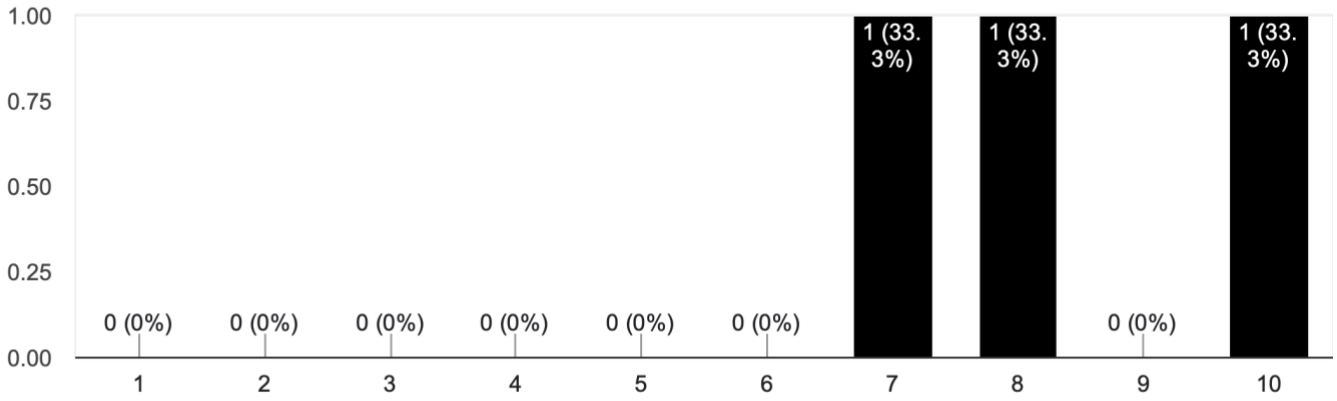


Chart 8

Test 48 Does the level in Projectile Motion match the difficulty of the puzzle

3 responses

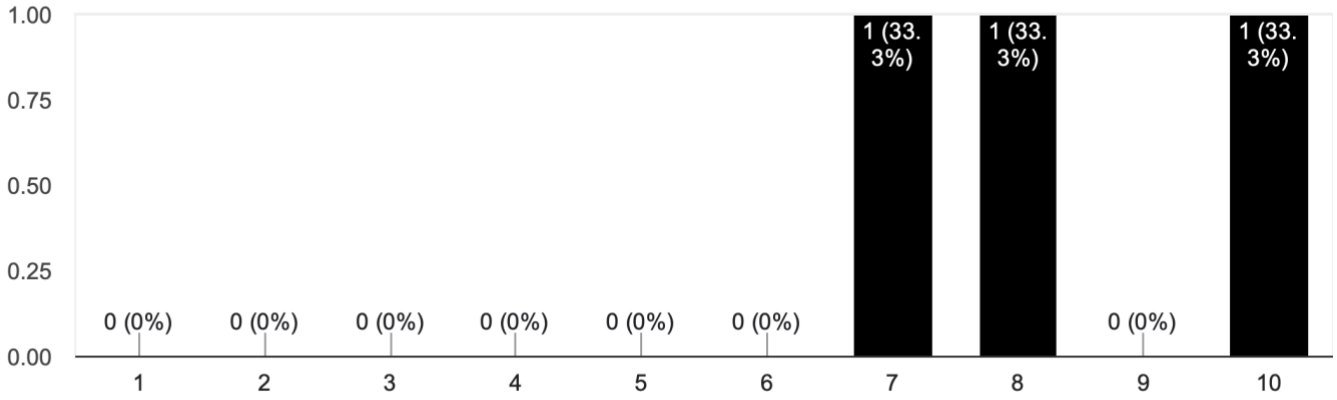


Chart 9

Test 51 How easy is the tutorial to understand

3 responses

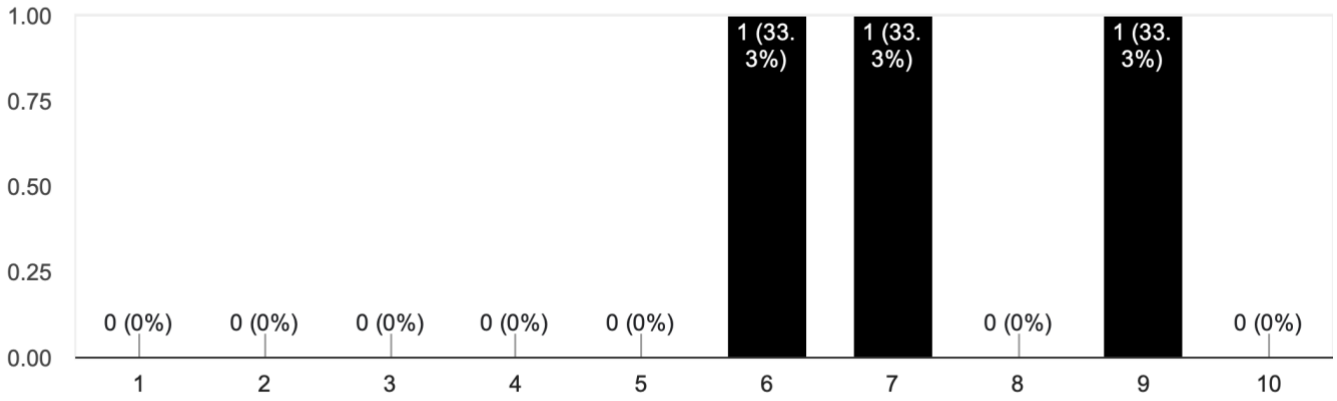


Chart 10

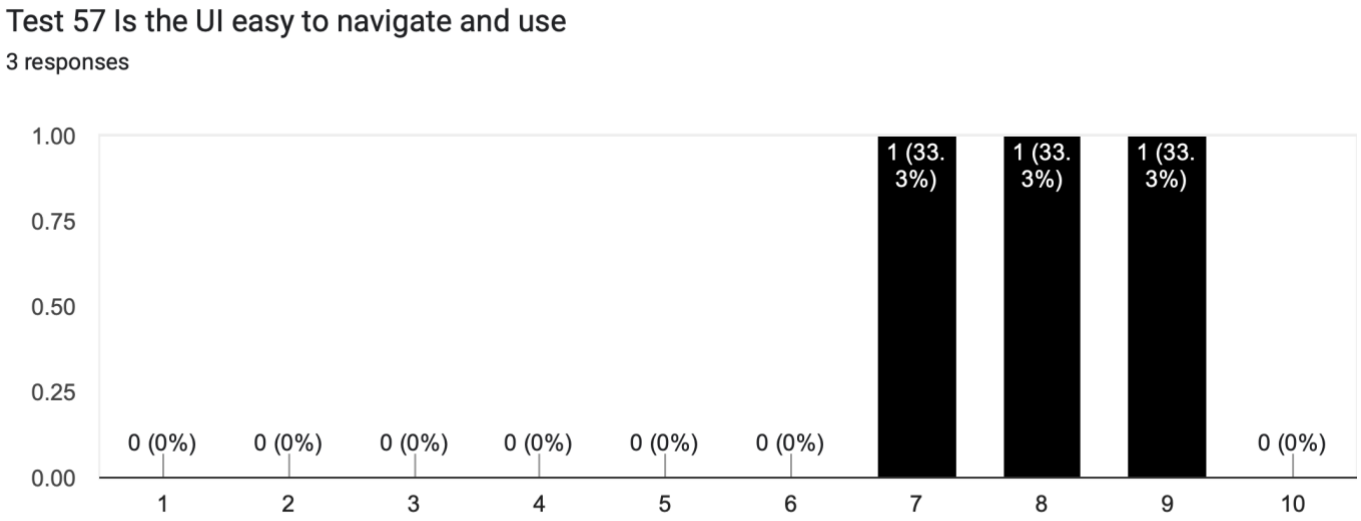


Chart 11

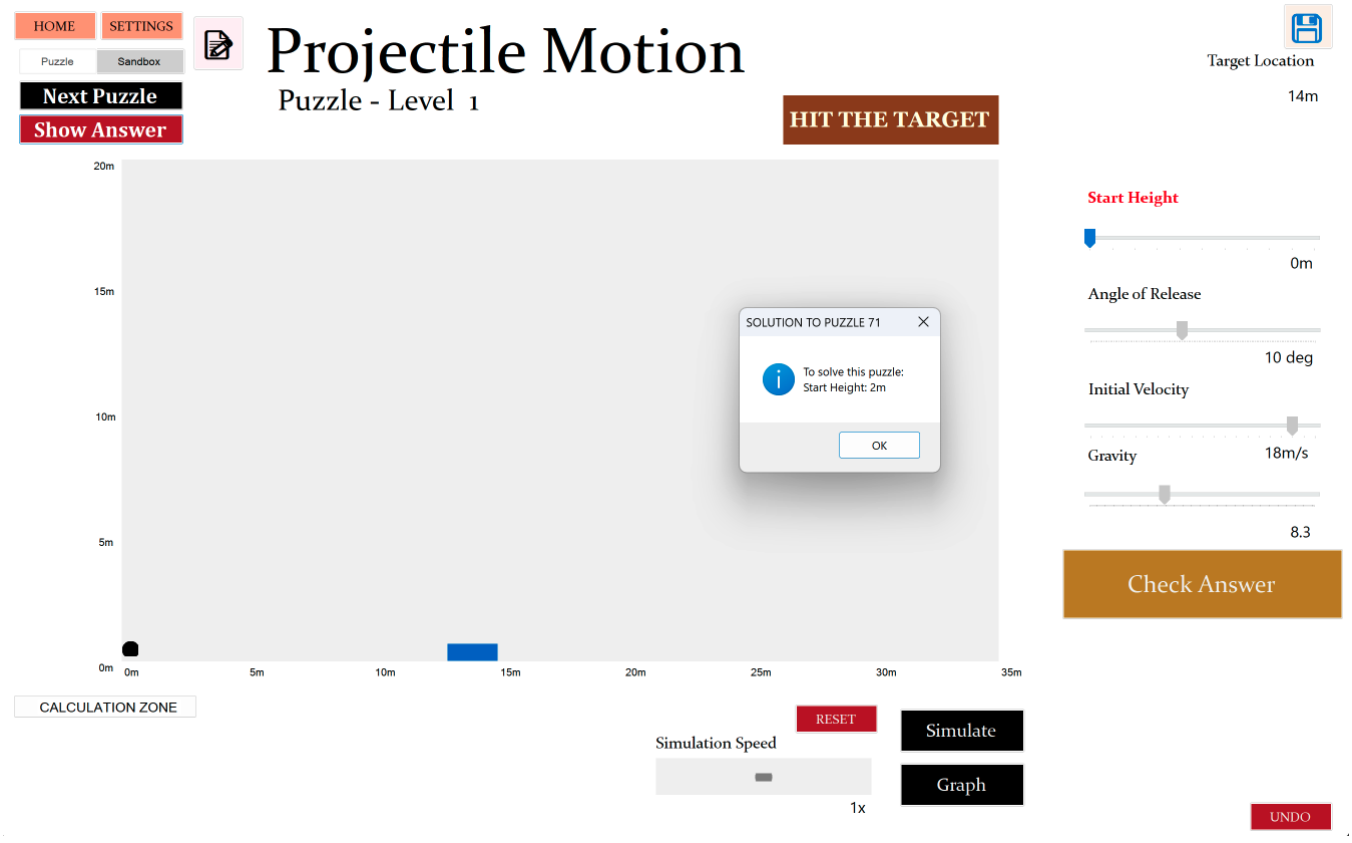


Figure 11. 1

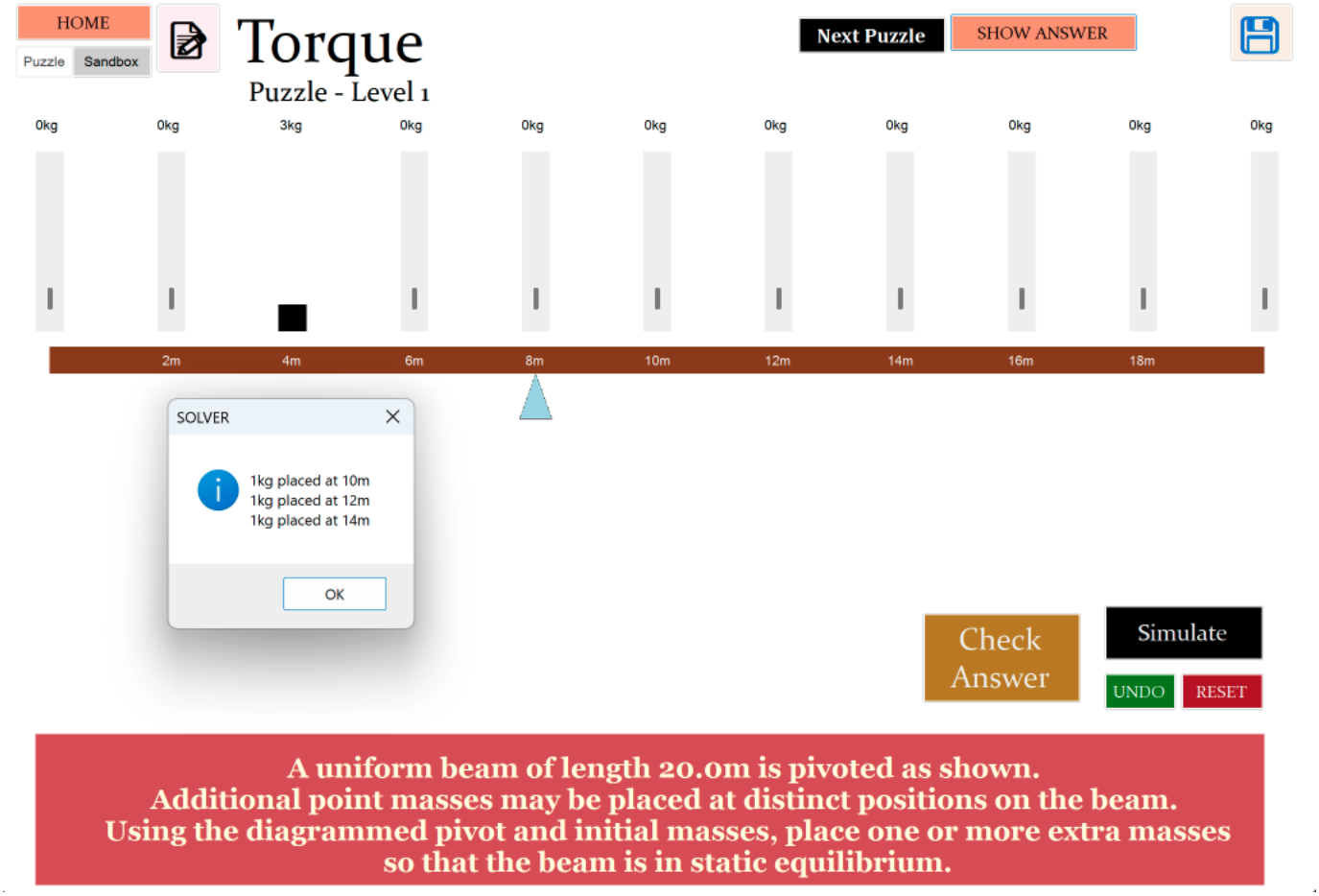

HOME
Puzzle
Sandbox
Torque
Puzzle - Level 1
Next Puzzle
SHOW ANSWER
0kg
0kg
3kg
0kg
0kg
0kg
0kg
0kg
0kg
0kg
0kg
2m
4m
6m
8m
10m
12m
14m
16m
18m
SOLVER
1kg placed at 10m
1kg placed at 12m
1kg placed at 14m
OK
Check Answer
Simulate
UNDO
RESET
A uniform beam of length 20.0m is pivoted as shown.
Additional point masses may be placed at distinct positions on the beam.
Using the diagrammed pivot and initial masses, place one or more extra masses so that the beam is in static equilibrium.

Figure 11. 2

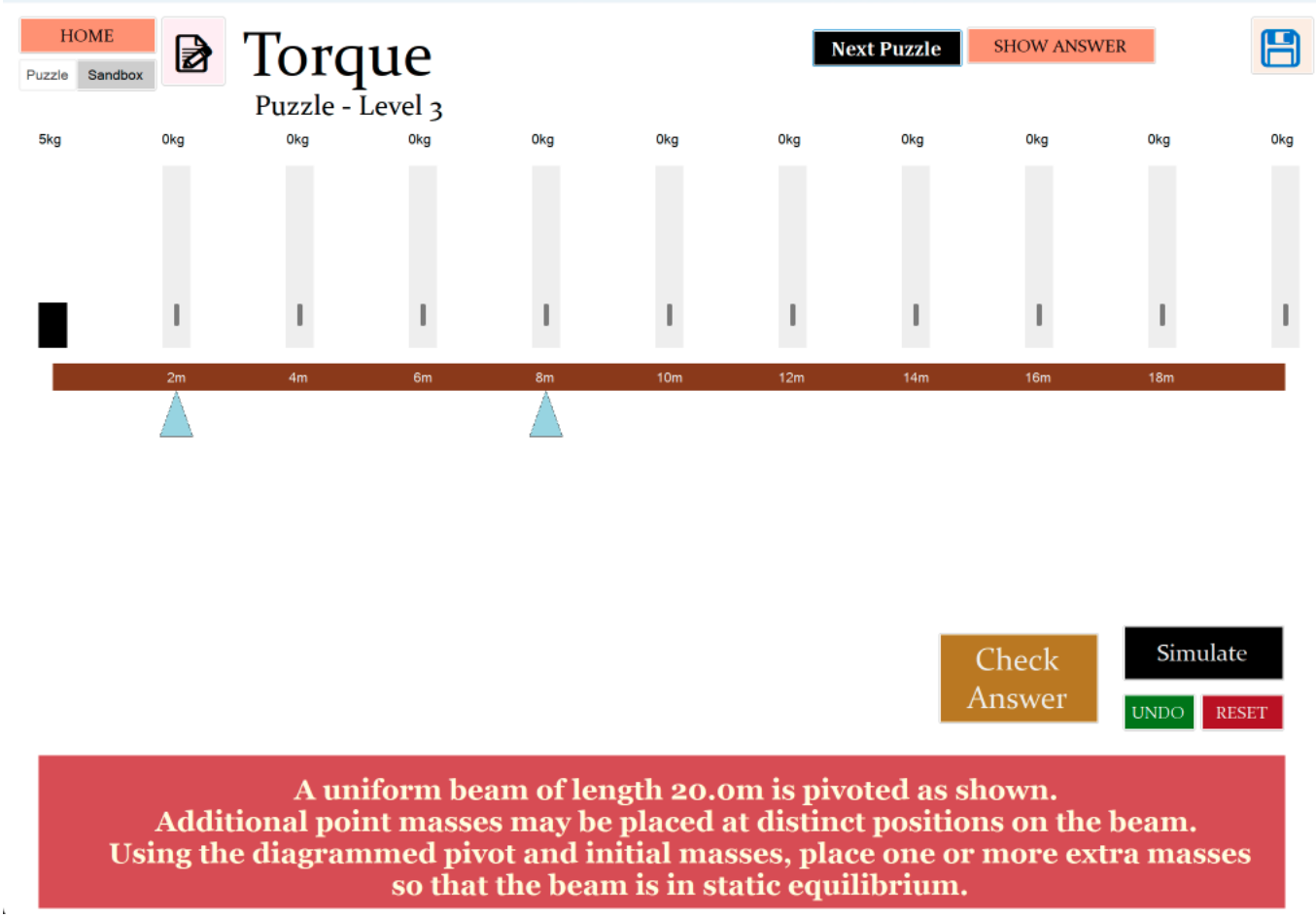


Figure 11. 3

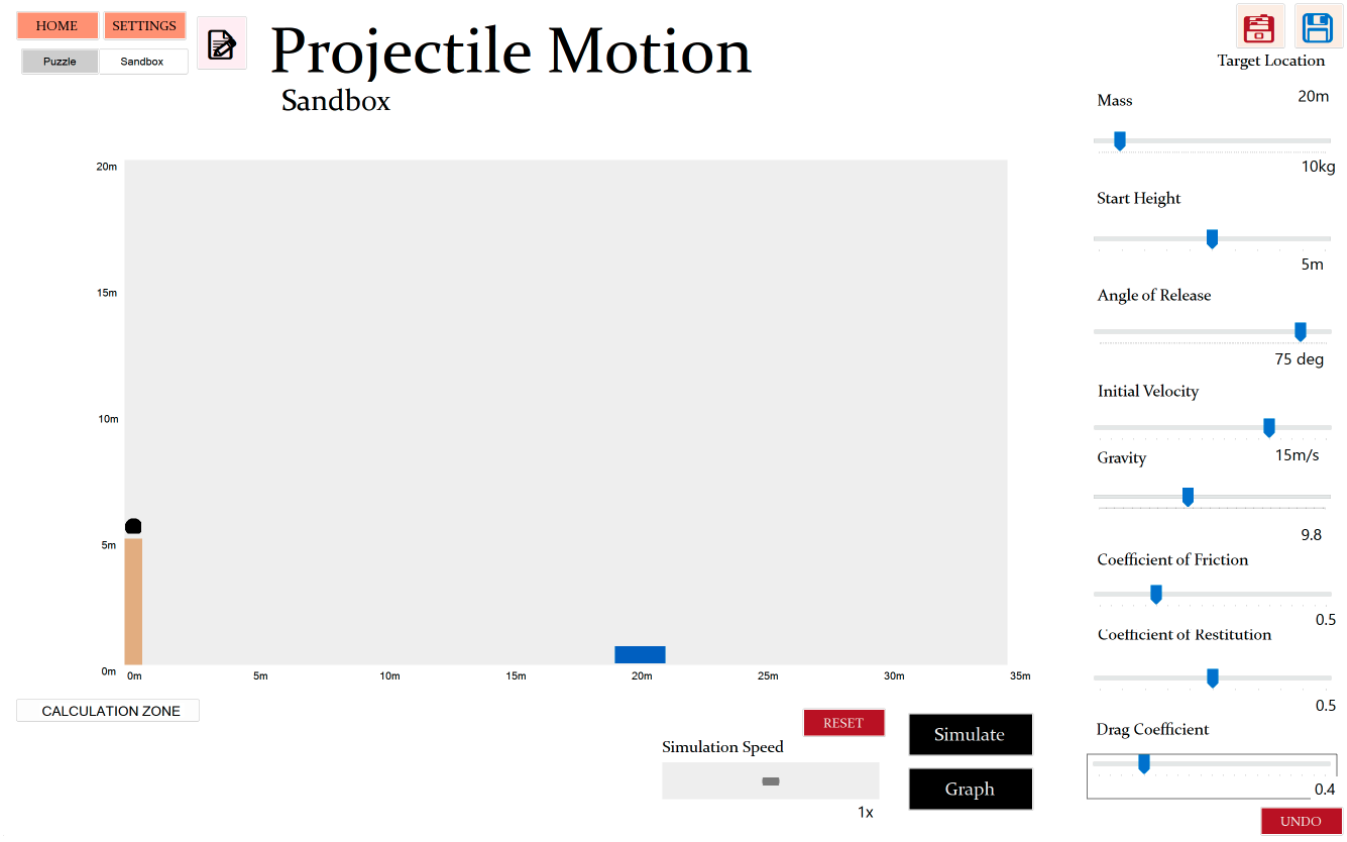


Figure 11. 4

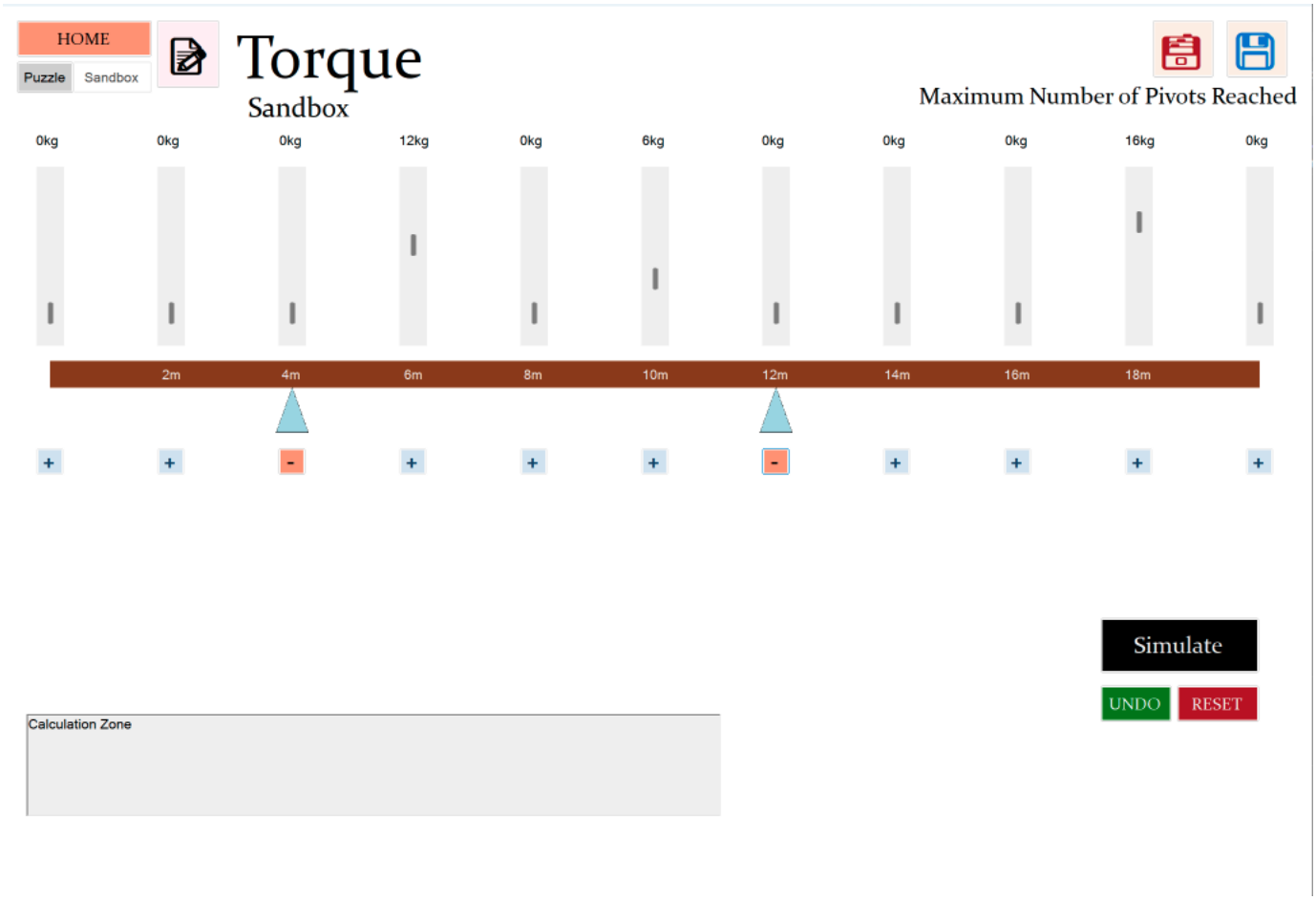


Figure 11. 5

	DataID	UserID	PuzzleTyp	PuzzleLev	Attempts	TimeTaker	Simulatior	SKIPPED	SCORE	AttemptDate
1	31	34	Projectile	1	2	17	0	0	86.5	2025-11-10 21:34:51
2	32	34	Projectile	1	1	9	0	0	95.5	2025-11-10 21:35:10
3	33	34	Projectile	1	0	0	0	0	105	2025-11-10 21:35:21
4	34	34	Projectile	1	0	0	0	0	105	2025-11-10 21:35:26

	SaveID	UserID	SaveName	Timestamp	PivotPositior	Masses
1	3	34	Setup	2025-11-16 21:08:42	[6]	[20,20,13,20,18,20,20,20,4,20,20]
2	4	34	Setup_1	2025-11-16 21:11:11	[6]	[20,20,13,20,4,20,20,20,7,20,20]

	ScoreID	UserID	ProjectileS	ProjectileL	TorqueScc	TorqueL	Leaderboa	TotalScore	Leaderboa
1	20	23	0	1	0	1	0	0	1
2	21	24	36	1	54	1	0	90	1
3	22	29	23	1	33	1	0	56	1
4	23	26	15	1	34	1	0	49	1
5	24	27	40	1	21	1	0	61	1
6	25	28	13	1	13	1	0	26	1
7	26	25	5	1	5	1	0	10	1
8	27	30	11	1	11	1	0	22	1
9	28	31	18	1	45	1	0	63	1
10	29	32	7	1	9	1	0	16	1
11	30	33	5	1	3	1	0	8	1
12	31	34	11	1	9	1	0	20	1
13	32	35	0	1	0	1	0	0	1
14	33	36	0	1	0	1	0	0	1

	SaveID	UserID	SaveName	Timestamp	Mass	StartHeigh	Angle	InitVel	Gravity	Friction	Restitution	Drag	Target
1	15	34	Setup	2025-11-11 09:47:13	41	2	45	10	9.8	0	0	0	20
2	16	34	Setup_1	2025-11-16 21:08:14	31	3	15	5	18.8	0.5	0.3	0.5	10

Figure 11. 6

	UserID	Name	Username	Salt	HashedPassword	IsNewUse	Theme
1	24	dhwani	drajesh	tUDygwMxotvllkcegzz9Vw==	Piz2BMuvgc9wzNoU39OYJ7BSq5k22c7klzO0/OrFyhk=	1	Light
2	25	Ian	McLennan	+Thir8oj0nCCRfQH1xoCcQ==	tlBrHbCyMUXol26wFiNNVauCuEqaOQdE4dTta0vmrsQ=	1	Light
3	26	Nik	nik123	TTWQdCjfmvRH2tBE31fNpA==	eJnl++gtyJxNaz9Omg+1os9ihCBxA5uMzHZS2HS4NWo=	1	Light
4	27	vb	vb.203	QIMYG37nZuD+KhkazElsAQ==	+IjM5pjM9lihmfwpfr0kQCZa/X6pSbFJVWVZWjUG6fQ=	1	Light
5	28	Gem	JRod	ooqbCq82A5aWcvwpAzn2rQ==	UK7cKzCmC1jj3tvStyZ8QyRexytF3ZUijRmShS5085Q=	1	Light
6	29	Musa	I LOVE EQUALITY	HeQFeRQNrTec045Fm+8j9Q==	RMVqM/Sl44JXJq8P1SfUm4vxXMR/BWkiz+UauhCVLdo=	1	Dark
7	30	Joseph	Hosef	hRnmDUr332ixUbhIGLdxTQ==	sWTloyqMPoiTB3T46+onqXX/B5x9UNohnh1FHGn/xes=	1	Light
8	31	tanya	hunt	J60pVO+3whyIk8FVH8wMvA==	E+NGxfzaxeutUWC+0A+gYU++Qo86VB+5Miw6y0SJavU=	1	Light
9	32	jack	jack_wl	rZ1i+gGT7NcWPiFBUUTx5w==	xprFJQgAmtyBCs8H1BY+OMFzgegRM+1jMAjsIggSB9I=	1	Light
10	33	Tom	KING	OsRAStBTpR5mju+DCVLlJw==	ZNX1CAK3RPSgKai7mwhgQoN9JNt+6WFBw7m5pWvmEbg=	1	Light
11	34	Pranav	pran23	8vWeRdLt3cU7ubcf2v2fRQ==	l4tT6E44Cdgs44pccru6LYuWuGDUZnmbZ0o08t120sY=	1	Light
12	35	joshua	joshue	gNAM3CUlFLDQlSuxSW0lbQ==	1BZ62levG2wEVm5xx0f/QZpXfKfgoAcEKBZXVUNjWcU=	1	Light
13	36	skk	skk12	i80bEKCClTOfXtLpYksvrw==	4KwP3ZgRQ9mFSw/JAmJRS5cDzScHYw1ybCVCMyxu+8g=	1	Light
14	38	Test	testing	LMmN4JPVJT5BW9nLJRY5aA==	SIU3LhqyPm7H3p7S9SDGZP6RPFcMp+xyjEYlzlKZ/2s=	1	Dark

	DataID	UserID	PuzzleTyp	PuzzleLev	Attempts	TimeTaker	Simulatior	SKIPPED	SCORE	AttemptDate
1	36	26	Torque	1	1	17	1	0	86.5	2025-11-11 09:44:26

SaveID	UserID	SaveName	Timestamp	PivotPositior	Masses

	ScoreID	UserID	ProjectileS	ProjectileI	TorqueScc	TorqueLev	Leaderboa	TotalScore	Leaderboa
1	20	23	0	1	0	1	0	0	1
2	21	24	36	1	54	1	0	90	1
3	22	29	23	1	33	1	0	56	1
4	23	26	15	1	34	1	0	49	1
5	24	27	40	1	21	1	0	61	1
6	25	28	13	1	13	1	0	26	1
7	26	25	5	1	5	1	0	10	1
8	27	30	11	1	11	1	0	22	1
9	28	31	18	1	45	1	0	63	1
10	29	32	7	1	9	1	0	16	1
11	30	33	5	1	3	1	0	8	1
12	31	34	0	1	0	1	0	0	1
13	32	35	0	1	0	1	0	0	1
14	33	36	0	1	0	1	0	0	1
15	34	34	0	1	0	1	0	0	1
16	35	38	0	1	0	1	0	0	1

SaveID	UserID	SaveName	Timestamp	Mass	StartHeigh	Angle	InitVel	Gravity	Friction	Restitution	Drag	Target

Figure 11. 7

	UserID	Name	Username	Salt	HashedPassword	IsNewUse	Theme
1	24	dhwani	drajesh	tUDygwMxotvllkcegzz9Vw==	Piz2BMuvgc9wzNoU39OYJ7BSq5k22c7klzO0/OrFyhk=	1	Light
2	25	Ian	McLennan	+Thir8oj0nCCRfQH1xoCcQ==	tlBrHbCyMUXol26wFiNNVauCuEqaOQdE4dTta0vmrsQ=	1	Light
3	26	Nik	nik123	TTWQdCjfmvRH2tBE31fNpA==	eJnl++gtyJxNaz9Omg+1os9ihCBxA5uMzHZS2HS4NWo=	1	Light
4	27	vb	vb.203	QIMYG37nZuD+KhkazElsAQ==	+IjM5pjM9lihmfwpfr0kQCZa/X6pSbFJVWVZWjUG6fQ=	1	Light
5	28	Gem	JRod	ooqbCq82A5aWcvwpAzn2rQ==	UK7cKzCmC1jj3tvStyZ8QyRexytF3ZUijRmShS5085Q=	1	Light
6	29	Musa	I LOVE EQUALITY	HeQFeRQNrTec045Fm+8j9Q==	RMVqM/SI44JXJq8P1SfUm4vxXMR/BWkiz+UauhCVLdo=	1	Dark
7	30	Joseph	Hosef	hRnmDUr332ixUbhIGLdxTQ==	sWTloyqMPoiTB3T46+onqXX/B5x9UNohnh1FHGn/xes=	1	Light
8	31	tanya	hunt	J60pVO+3whyIk8FVH8wMvA==	E+NGxfzaxeutUWC+0A+gYU++Qo86VB+5Miw6y0SJavU=	1	Light
9	32	jack	jack_wl	rZ1i+gGT7NcWPiFBUUTx5w==	xprFJQgAmtyBCs8H1BY+OMFzgegRM+1jMAjsIggSB9I=	1	Light
10	33	Tom	KING	OsRAStBTpR5mju+DCVLlJw==	ZNX1CAK3RPSgKai7mwhgQoN9JNt+6WFBw7m5pWvmEbg=	1	Light
11	34	Pranav	newUsername	8vWeRdLt3cU7ubcf2v2fRQ==	l4tT6E44Cdgs44pccru6LYuWuGDUZnmbZ0o08t120sY=	1	Light
12	35	joshua	joshue	gNAM3CUlFLDQlSuxSW0IbQ==	1BZ62levG2wEVm5xx0f/QZpXfKfgoAcEKBZXVUNjWcU=	1	Light
13	36	skk	skk12	i80bEKCCITOfXtLpYksvrw==	4KwP3ZgRQ9mFSw/JAmJRS5cDzScHYw1ybCVCMyxu+8g=	1	Light
14	38	Test	testing	LMmN4JPVJT5BW9nLJRY5aA==	SIU3LhqyPm7H3p7S9SDGZP6RPFcMp+xyjEYlzIKZ/2s=	1	Dark

Figure 11. 8

Usability Features

As this was more subjective to test, I decided to ask my stakeholders to use my solution and decide whether each usability feature was effective through the use of a stakeholder questionnaire. Each test is presented as the question posed to the stakeholder. Each feature is named below, along with each stakeholder's rating out of 5 and their opinion.

Feature 1: Light and Dark Mode

This feature allows the user to go between light and dark mode. Their preferences are saved in the database to load the solution in their preferred theme the next time they open the solution

Test 66: How effective is the Light/Dark Mode feature at improving your experience?

Stakeholder Ratings:

- **Nik:** 5/5 It's very nice
- **Mr McLennan:** 5/5
- **Adam (Beginner Player Persona)** 5/5 Thoroughly enjoyed dark mode, very useful for my sensitive eyes

Evidence: See Test Video 14 0:00 – 0:14

Outcome: **SUCCESS 5/5**

Feature 2: Music

This solution plays a sound effect each time the user correctly solves a puzzle and moves up a level. The user has the option to toggle music on and off

Test 67: How well does the music/sound effect feature work when you solve a puzzle, and how easy is it to turn the sound on or off when you want to?

Stakeholder Ratings:

- **Nik:** 4/5 would have been better if the music didn't stop the UI
- **Mr McLennan:** 3/5
- **Adam (Beginner Player Persona):** 3/5 Would have been better to have more choice in music options

Evidence: See Test Video 21

Outcome: **PARTIAL SUCCESS 3.3/5**

Feature 3: Leaderboard Colours and Sections

The leaderboard uses gold, silver, and bronze highlighting to show the top three users. In addition, if the user is not in the top 3, their score is highlighted in blue, and if they are not in the top 10 then their score is still shown in the leaderboard. Furthermore, the leaderboard uses sections to separate the leaderboards of torque, projectile motion as well as showing the overall leaderboard.

Test 68: How clear and helpful are the leaderboard colours (gold, silver, bronze, blue), and do the different sections (torque, projectile motion, overall) make it easier to understand the rankings?

Stakeholder Ratings:

- **Nik:** 5/5 The leaderboard UI is very clear and intuitive
- **Mr McLennan:**5/5
- **Adam (Beginner Player Persona)** 4/5 Make it clearer what the blue highlighting means

Evidence: See Figure 11.9

Outcome: **SUCCESS 5/5**

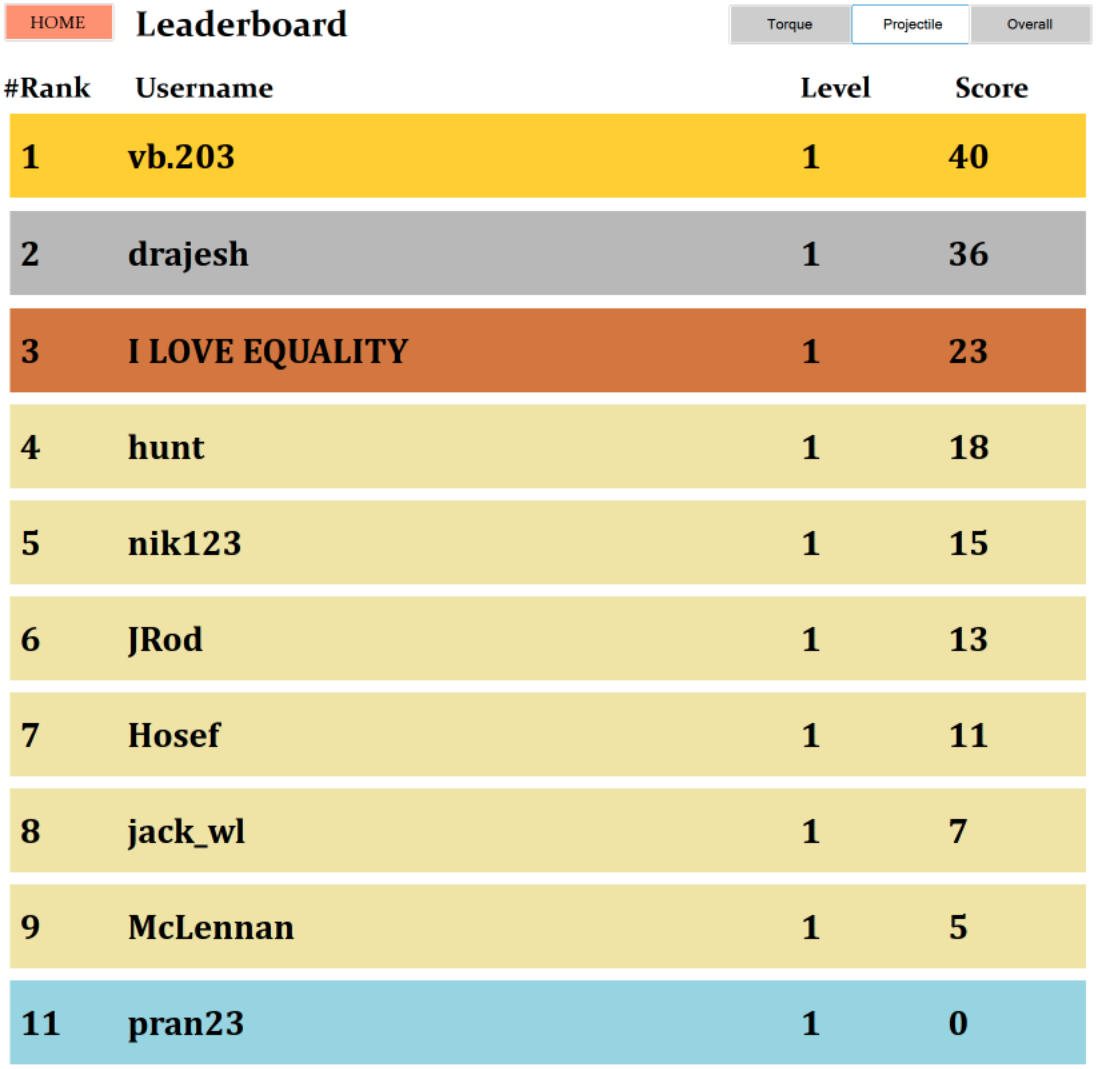


Figure 11. 9

Feature 4: Reset Account

The solution allows the user to reset all the saved data on their account and start afresh

Test 69: How useful is the Reset Account button when you want to clear your progress and start again, and does it feel easy to find and use?

Stakeholder Ratings:

- **Nik:** 5/5 works as expected
- **Mr McLennan:** 5/5
- **Adam (Beginner Player Persona)** 2/5 I don't see a point in resetting my progress

Evidence: See Test Video 18 and Test 63

Outcome: **SUCCESS 4/5**

Feature 5: Change username and password

This solution allows the user to change their username and password

Test 70: How easy is it for you to change your login credentials, and does the process feel straightforward and quick?

Stakeholder Ratings:

- **Nik:** 4/5 Would have been nice to have a confirmation message
- **Mr McLennan:** 5/5
- **Adam (Beginner Player Persona)** 5/5 Worked very well, quick and easy process, allowed me to get rid of my previous password

Evidence: See Test Video 19 and Test Video 20

Outcome: **SUCCESS 4.6/5**

Feature 6: Save and Load Setups

The solution allows the user to save puzzles that they created and load them when they wish

Test 71: How well does the feature for saving and loading your custom puzzles work, and does it feel reliable when you want to return to previous setups?

Stakeholder Ratings:

- **Nik:** 5/5
- **Mr McLennan:** 4/5
- **Adam (Beginner Player Persona)** 4/5 Very cool that I can save my past setups, which I worked so hard on. Although it would have been helpful to have the buttons labelled

Evidence: See Test Video 16

Outcome: **SUCCESS 4.3/5**

Feature 7: Graph Themes

The graph has 4 different themes that the user can pick from to make the graph look more appealing to them

Test 72: How helpful are the different graph themes in making the graph look clearer or more appealing to you?

Stakeholder Ratings:

- **Nik:** 5/5 The selection of graph themes is welcome because I disliked the previous colour scheme

- **Mr McLennan:** 5/5
- **Adam (Beginner Player Persona)** 5/5 Pretty lines and colours

Evidence: See Test Video 6

Outcome: **SUCCESS 5/5**

Feature 8: Graph Thickness

The user is able to change the thickness of the graphs to make it easier for them to see

Test 73: How helpful is the option to change the graph thickness, and does it make the graph easier for you to see and understand?

Stakeholder Ratings:

- **Nik:** 5/5
- **Mr McLennan:** 4/5
- **Adam (Beginner Player Persona)** 5/5 Makes graph easier to see

Evidence: See Test Video 6

Outcome: **SUCCESS 4.7/5**

Feature 9: Graph

This solution allows the user to select which graphs they want visible to them

Test 74: How easy is it for you to choose which graphs you want to see, and does the feature make it simple to focus on what you need?

Stakeholder Ratings:

- **Nik:** 4/5
- **Mr McLennan:** 4/5
- **Adam (Beginner Player Persona)** 5/5 Very easy to choose and focus on a specific line

Evidence: See Test Video 6

Outcome: **SUCCESS 4.3/5**

Feature 10: Menu

This solution has a menu from which you can access each section of the program. If you wish to go to a different section you must return to the menu screen and choose which section you wish to go to

Test 75: How easy is it to use the main menu to move between different sections of the program, and does the layout feel simple to follow?

Stakeholder Ratings:

- **Nik:** 4/5
- **Mr McLennan:** 4/5
- **Adam (Beginner Player Persona)** 5/5 Can move very easily, concise and condensed menu gives ease of access

Evidence: See Figure 11.10

Outcome: **SUCCESS 4.3/5**

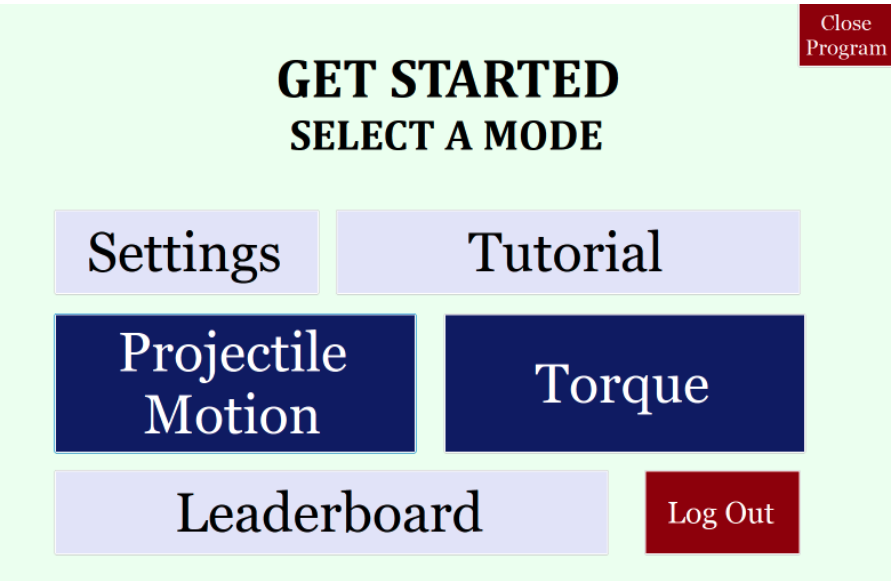


Figure 11. 10

Feature 11: Font

This solution allows the user to change the fonts of the program

Test 76: How useful is the option to change the font, and does switching fonts make the text easier or nicer for you to read?

Stakeholder Ratings:

- **Nik:** 3/5 When you try change the font the sizing is not preserved
- **Mr McLennan:** 3/5
- **Adam (Beginner Player Persona)** 5/5 I can use fonts such as comic sans that I find more readable

Evidence: See Test Video 26

Outcome: **PARTIAL SUCCESS 3.7/5**

Feature 12: Navigation

In order to make this solution easy to navigate, most of my buttons and sliders have text labels apart from the file handling button, which uses the commonly recognisable floppy disc icon. In addition, in the puzzles, any slider that the user cannot use is disabled to prevent accidental changes

Test 77: How clear are the buttons, text labels, and icons? Does it make things easier to understand and prevent mistakes?

Stakeholder Ratings:

- **Nik:** 4/5 Buttons are generally easy to understand with the exception of a switch for puzzle and sandbox mode as it is small and difficult to find
- **Mr McLennan:** 4/5
- **Adam (Beginner Player Persona)** 5/5 Big buttons make it easier to see, font size changes are also useful

Evidence: See Figure 11.11

Outcome: **SUCCESS 4.3/5**

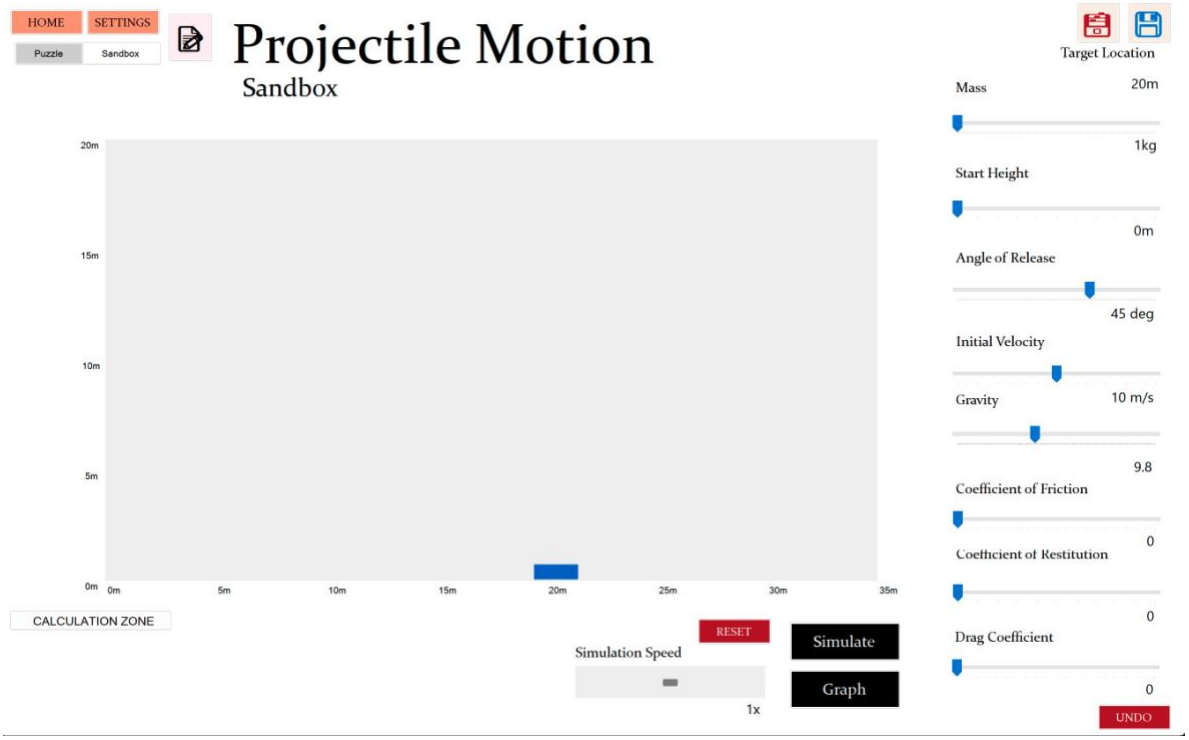


Figure 11. 11

Feature 13: Tutorial

This solution contains a tutorial which allows the user to see how to use some of the features

Test 78: How helpful is the tutorial in showing you how to use the features and understand the program, especially if you're new to it?

Stakeholder Ratings:

- **Nik:** 3/5 Videos would be easier to follow and understand than text
- **Mr McLennan:** 3/5
- **Adam (Beginner Player Persona)** 4/5 Very useful to understanding the system layout. Would be interesting to see tutorial integrated into puzzles rather than a separate menu

Evidence: See Figure 11.12

Outcome: **PARTIAL SUCCESS 3.3/5**

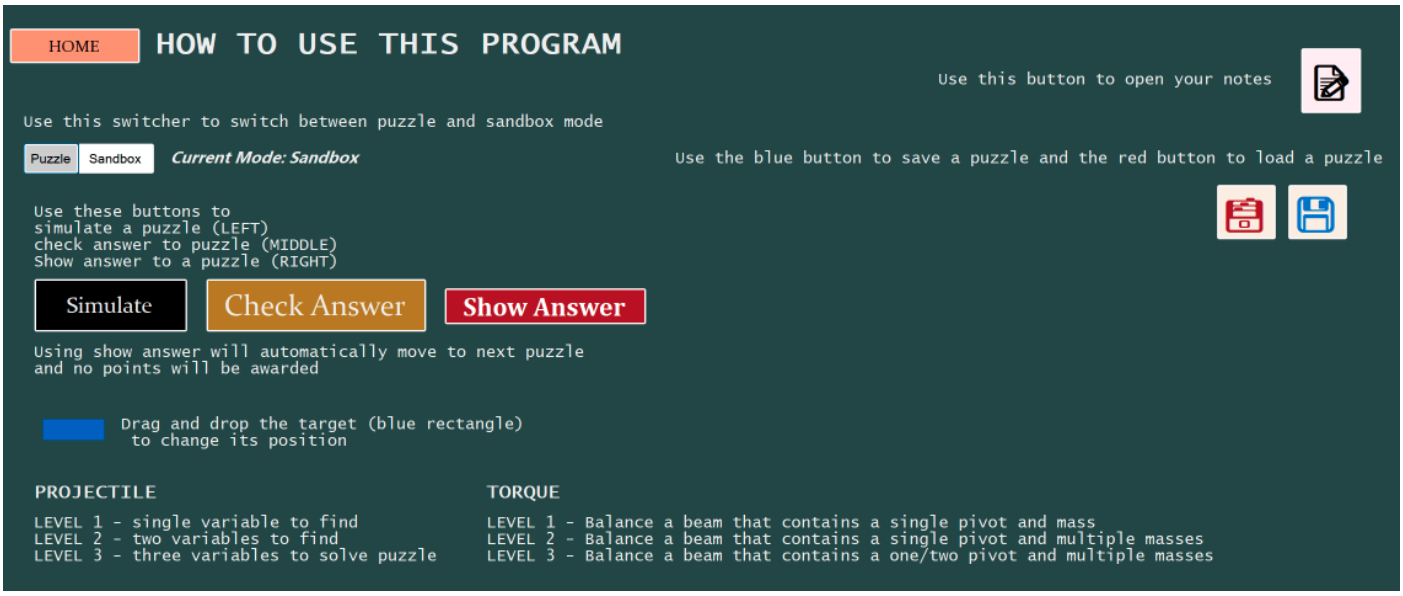


Figure 11. 12

Feature 14: Attractive UI

This feature is subjective; however, I made sure to use consistent themes to ensure that my solution is attractive, which I believe has succeeded in making an attractive UI

Test 79: How good does the overall design of the program look to you, and does the consistent theme make the interface feel more enjoyable to use?

Stakeholder Ratings:

- **Nik:** 4/5
- **Mr McLennan:** 4/5
- **Adam (Beginner Player Persona)** 4/5 Very pretty, could do with more themes for interface

Outcome: **SUCCESS 4/5**

Feature 15: Notes

This solution allows the user to make notes that they can show and hide

Test 80: How useful did you find the notes feature

Stakeholder Ratings:

- **Nik:** 4/5 A useful feature that could be improved if the user were able to resize it and drag it around the screen instead of it being in a static place
- **Mr McLennan:** 3/5
- **Adam (Beginner Player Persona)** 4/5 The notes are useful; ability to add maths symbols would be useful

Evidence: See Test Video 27

Outcome: **PARTIAL SUCCESS 3.7/5**

Overall Score

Nik: 64/75

Mr McLennan: 61/75

Adam (Beginner Player Persona): 61/75

Robustness

Test No.	Description	Test Data	Type of Data	Expected Outcome	Outcome	Evidence
81	Projectile Motion simulation	All values set to maximum	Boundary	The motion should animate accurately	**Partial Success** *Whilst some of the motion is simulated accurately it does not show the bounce that should be occurring*	Test Video 22
82		Value of coefficient of restitution set to negative	Invalid	The coefficient of restitution should be bound between 0 and 1	**SUCCESS** *As the solution uses sliders the parameters are always valid*	Test Video 24
83		Angle set to 0	Boundary	Valid simulation (flat-line trajectory)	**SUCCESS** *The simulation shows a non-parabolic trajectory*	Test Video 23
84		Angle set to 720°	Invalid	Should clamp at the maximum value of angle	**SUCCESS** *As the solution uses sliders the parameters are always valid*	Test Video 24
85		Velocity Negative	Invalid	Should clamp to zero	**SUCCESS** *As the solution uses sliders the parameters are always valid*	Test Video 24
86		Angle = 90°	Boundary	Should travel vertically upwards	**SUCCESS** *The projectile does travel vertically upwards as expected*	Test Video 22
87		Set mass to “hello”	Erroneous	This should not be allowed	**SUCCESS** *As the solution uses sliders the parameters cannot be typed into the system*	Test Video 24
88		Value of coefficient of	Invalid	The coefficient of restitution should	**SUCCESS** *As the solution uses sliders the*	Test Video 24

		friction set to negative		be bound between 0 and 2	*parameters are always valid*	
89		Change parameters whilst simulation is running	Invalid	The solution should block the user from changing any values during the simulation	**SUCCESS** *The sliders are disabled during simulation*	Test Video 25
90		Coefficient of restitution set to 1	Boundary	The projectile should not lose energy during the bounce	**SUCCESS** *The projectile continues to bounce*	Test Video 25
91		Continuous use of simulation	Boundary	The solution should continue to work as expected	**PARTIAL SUCCESS** *Mostly worked apart from trying to set both restitution and friction to high values where the program then froze.*	Test Video 41
92		Set target position to “hello”	Erroneous	This should not be allowed	**SUCCESS** *As the solution uses sliders the parameters cannot be typed into the system*	Test Video 24
93	Puzzle Mode	Attempts to change locked fields	Invalid	Should not change	**SUCCESS** *The solution only allows the user to change the allowed variable*	Test Video 28
94	Target Position	Try dragging target off screen to the left	Invalid	Should clamp to the left	**SUCCESS** *The solution does not allow you to place a target at < 5m so it clamped to 5m*	Test Video 29 0:00 – 0:05
95		Try dragging target off screen to the right	Invalid	Should clamp to the right	**SUCCESS** *The solution does not allow the user to take the target off screen to the right*	Test Video 29 0:05 – 0:14
96		Drop target mid air	Normal	Should clamp to ground	**SUCCESS** *The target always ends up on the ground level*	Test Video 29 0:18 – 0:20
97		Drop target on boundary	Boundary	Should be placed normally	**SUCCESS**	Test Video 29 0:20 – 0:25

98		Drag around screen repeatedly	Stress	No glitching	**SUCCESS** *The target position is able to be constantly changed and set with the location label also changing accordingly*	Test Video 29
99	Graph	Hide all lines	Boundary	Graph empty	**SUCCESS**	Test Video 30 0:08 – 0:13
100		Show all lines	Boundary	All visible	**SUCCESS** *All lines visible*	Test Video 30 0:00 – 0:07
101		Set line thickness to max	Boundary	Thick line drawn	**SUCCESS** *Thickness set to max*	Test Video 30 0:18 – 0:22
102		Change colour theme rapidly	Stress	Colours should change	**SUCCESS** *Colours change as theme is changed*	Test Video 30 0:22 – 0:30
103	Simulation Speed	Set speed 0.1x	Boundary	Projectile should simulate slowly	**SUCCESS**	Test Video 31 0:00 – 0:20
104		Set speed to 2x	Boundary	Projectile should simulate fast	**SUCCESS**	Test Video 31 0:27 – 0:32
105		Set speed to negative	Invalid	Should clamp to the minimum value	**SUCCESS** *Can't go below 0.1x speed*	Test Video 31 0:32 – 0:36
106		Change speed during simulation	Stress	The simulation should change speed	**SUCCESS** *Speed changes as the simSpeed changes*	Test Video 31 0:36 – 0:44
107	Torque	Pivot at max	Boundary	Simulates as expected	**SUCCESS**	Test Video 32 0:00 – 0:10
108		Pivot at 0	Boundary	Simulates as expected	**SUCCESS**	Test Video 32 0:10 – 0:18
109		Mass is text	Erroneous	Reject	**SUCCESS** *As the system uses sliders the mass can only be an integer value*	Test Video 32
110		No Pivot Placed	Invalid	No simulation	**SUCCESS** *System shows error message*	Test Video 32 0:25 – 0:34
111		No Mass	Boundary	No torque	**SUCCESS** *No torque message displayed*	Test Video 32 0:18 – 0:25
112	Account	Create account with empty name	Invalid	The solution should not allow the user to register	**SUCCESS** *Error message displayed and user not able to register*	Test Video 33

113		Create an account with the username being too short	Invalid	The solution should allow this and proceed as normal	**SUCCESS** *Error message displayed and user not able to register*	Test Video 33 0:00 – 0:04
114		Create an account with a 2 character password	Invalid	The solution should not allow the user to register	**SUCCESS** *Error message displayed and user not able to register*	Test Video 33 0:04 - 0:06
115		Create an account where the password is the minimum number of characters	Boundary	The solution should allow this and proceed as normal	**SUCCESS** *Username with 9 characters inputted and was accepted*	Test Video 33 0:06 – 0:10
116		Create an account where the password and the entered repeat password are different	Invalid	The solution should display an error message	**SUCCESS** *Error message displayed and user not able to register*	Test Video 33 0:17 – 0:22
117		Login with the wrong password	Invalid	The solution should display an error message	**SUCCESS** *The account used for this had the following credentials Username: testAccount Password: password123 In the test the password was entered as 'password' thus it showed an incorrect password message*	Test Video 34 0:00 – 0:10
118		Login with the wrong username	Invalid	The solution should display an error message	**SUCCESS** *The account used for this had the following credentials Username: testAccount Password: password123 In the test the username was entered as 'randomUserna me' which was a username that hadn't been*	Test Video 34 0:10 – 0:20

					created therefore it showed an error message	
119		Attempt to do an SQL Injection	Erroneous	The solution should not allow the database to be affected and should continue as normal	**SUCCESS** *The following SQL was injected via the username field '105 OR 1=1; DROP TABLE Users' however as the program uses parametrised queries the injected SQL would not run and there was no effect to the table*	Test Video 35
120	Save	Attempt to save a projectile setup with the same name as a previous save	Invalid	The solution should change the name of the save to Name_1	**SUCCESS** *There are two setups created with the name of TestSetup so the second one is automatically given the name TestSetup_1*	Test Video 36
121		Save setup with empty name – projectile	Invalid	The solution does not allow you to save unless the name fits the criteria	**SUCCESS** *Displays an error message*	Test Video 36
122		Save setup with empty name – torque	Invalid	The solution does not allow you to save unless the name fits the criteria	**SUCCESS** *Displays an error message*	Test Video 37
123	Change login credentials	Attempt to change username to an invalid username	Invalid	Error message displayed	**SUCCESS** *The username only changes if it is a valid username and an error message is displayed otherwise*	Test Video 38
124		Attempt to change password to a password that has the minimum number of characters	Boundary	The solution continues as normal	**SUCCESS** *The inputted password is 9 characters which is the minimum accepted*	Test Video 39

125		Repeatedly change username	Stress	The program remains stable	**SUCCESS**	Test Video 40

Success Criteria Evaluation

Below, my success criteria has been repeated and evaluated with test numbers in the above tests given as evidence. The outcomes have been colour-coded in order to make it easier to visually see the success of each of the success criteria, where green is fully met, orange is partially met and red is not met. In addition, a comment explaining each outcome is also provided.

Feature	Identified Success Criteria		Justification for Success Criteria	Outcome and explanation	Evidence
1. Projectile Motion	1	Does the projectile launch when a button is pressed	The solution should progress as expected	The projectile mostly launches when the simulate button is pressed however when stress testing the solution it was found that the program would freeze (see test 91)	Test 1,2, 81-91
	2	Is the starting height of the launch point adjustable	The user should be able to change these values in order to solve the problems. They may not all be adjustable for each problem	The starting height can be adjusted by the user through the use of a slider which visually shows the change on the UI as the slider value is adjusted.	Test 3,4,81
	3	Is the angle of launch adjustable		The angle can be adjusted through the use of a slider	Test 3, 4, 81, 83, 84, 86
	4	Is the starting velocity of the projectile variable		The velocity can be adjusted through the use of a slider, and this value is accurately used as the value for velocity in the simulation.	Test 3,4, 85
	5	Is the projectile mass variable for required questions		Not implemented. In	Test 4

				my puzzles the user cannot interact with drag, restitution and friction therefore there is no reason for the user to change the mass when solving puzzles	
	6	Is the target displayed with distances shown	Having distances shown allows the user to use the skills that they have learnt to solve the problems rather than just using trial and error.	The target is able to be moved across the sandbox and the label is accurately updated. In addition when a projectile is launched, it can correctly determine the position of the target. In puzzle mode the target is visually moved by the program as well to make it clearer to the user	Test 5, 94 - 98
	7	Are the puzzles unique	The puzzles should be unique so that they can solve new problems without redoing the same problems	The puzzles were found to be unique however stakeholders did not that the puzzles were very similar and thus felt repetitive	Test 6,7
	8	Do the puzzles stated difficulty match the actual difficulty	Ensures that the user is solving problems that are suitable for the level that they are at	Stakeholder feedback found that the puzzles mostly were the right difficulty for the level and that they were able to move up levels at the right rate	Test 8

	9	Is a graph shown when a button is pressed	The solution should progress as expected	The graph is shown when the graph button is selected. The shown graph matches what is expected of the graph that should appear for the related motion	Test 9,10
	10	Does the graph match the motion of the projectile	The graph should match the motion so that it can be used to assist the user		
	11	Are the key values on the graph labelled	Being able to inspect key data points encourages learners to connect motion to graphs	Whilst the graph contains a set of axis that are labelled no other points such as stationary points are explicitly labelled	Feature not implemented
	12	Is the motion of the projectile animated	Makes the solution more visually appealing as well as emphasising what is happening in the game	The projectile mostly launches when the simulate button is pressed however when stress testing the solution it was found that the program would freeze (see test 91)	Test 6
	13	Does the simulation accurately show what would happen with the setup	Essential to ensure the simulation's accuracy as it serves as the user's tool for understanding the concepts.	The physics is correctly simulated with the stakeholders also finding that the projectile was simulated in the way that they expected	Test 14,15
	14	Does the program show you the calculations required to solve the problem	Helps the user understand why the simulation has produced the result it has	If the user clicks the calculation zone button then the solution will show you the maths required to find where the	Test 16

				projectile will end up.	
	15	Can the graph be displayed when the button is clicked	By displaying the graph of motion, it will allow users to gain a deeper understanding into questions in GCSE / A-Level which are regarding the motion graphs as they will have visually seen what factors will cause the graph to change.	When a button is clicked the graph mode will appear, following this the user will be able to select which graphs they wish to see and which they wish to hide.	Test 6
	16	Can you zoom into the graph	It will allow users can see specific sections of the graph in greater detail which will increase the educational value as they can use the graph zoom functionality to find key values from the graph	Feature not implemented	
	17	Does the graph show the vector setup	Helps the user understand the vector formation as well as the graphs.	Instead of allowing the user to see the vertical and horizontal the components of the velocity as vectors at every point along the graph or as something that was animated across the graph, I instead chose during development to show the horizontal and vertical components as their own graphs	Test 18
	18	Is the graph customisable	This allows the user to make the graph easier	The user is able to change the line	Test 19,20, 21, 72,73,74

			to read for themselves	thickness, colour and which graphs are visible which from the usability testing we can see that the stakeholders found this easy to use and helped them understand the graphs better.	
	19	Does the undo/redo button take you to the previous and following state respectively.	When users are experimenting, they can undo moves without restarting the entire solution which is especially useful for puzzles that require trial and error	Whilst this feature was demonstrated working in prototype 1. In the final testing, the undo button did not work as expected as instead of reverting back to the previous state it instead created a new state. Furthermore, a redo function was not implemented	Test 22,23
	20	Is the correct answer displayed if the user requires it	Gives the user feedback that allows them to gain understanding on where they went wrong	The answer is provided to the puzzle when the user requests it. This answer will be correct considering the SUVAT equations	Test 24
2. Torque Problems	1	Does the solution animate the result of placing masses / moving the pivot point	Makes the solution more visually appealing as well as emphasising what is happening in the game	The simulation animates the torque movement accurately	Test 24,25, 107-111
	2	Are there puzzles with multiple pivot points	Allows the user to apply their knowledge to more complex systems that will	When a user is on a level 3 there are puzzles which contain multiple pivots. When a user reaches	Test 26

			challenge more advanced users	level 3, some puzzles are randomly decided to have multiple pivot points	
	3	Is there an option to stack masses	Gives the user more control in the total mass placed on certain points	The solution does not use drag and drop but instead uses sliders so any mass can be placed at any point	Test 27
	4	Does the undo/redo button take you to the previous and following state respectively.	When users are experimenting, they can undo moves without restarting the entire solution which is especially useful for puzzles that require trial and error	The undo button successfully takes you back to the state that you were in before by changing the masses placed and pivots placed. However a redo button was not implemented as I felt that this would not be necessary as it is unlikely to be used by users who would rather just place a new pivot or mass	Test 28, 29
	5	Are the puzzles unique	The puzzles should be unique so that they can solve new problems without redoing the same problems	Puzzles were found to be unique however stakeholder analysis did note that the puzzles were very repetitive and similar	Test 30,31
	6	Do the puzzles stated difficulty match the actual difficulty	Ensures that the user is solving problems that are suitable for the level that they are at	Average score 9.3/10 therefore stakeholders found the level to be suitable	Test 32
	7	Does the program show the calculation	By displaying the underlying maths, it	In torque mode, the program will	Test 33

			will help students using the solution to gain a better understanding of the principle of moments which will allow them to gain a deeper understanding that will aid them during formal schooling	show both the clockwise and negative torque and then show the overall torque to help the user understand. This is only shown in sandbox mode as it would defeat the purpose of the puzzle mode	
	8	Is the correct answer displayed if the user requires it	Gives the user feedback that allows them to gain understanding on where they went wrong	When the user clicks a button the answer is shown to the user. For a torque puzzle there are multiple possible answers and therefore my solution will only show one of these possible answers.	Test 34
3. Leaderboard	1	Is the leaderboard automatically updated to show which player is in the lead	The leaderboard is needed to maintain competitiveness. If the leaderboard didn't update automatically, it would feel irrelevant to users thus reducing engagement.	The leaderboard accurately shows the position of the users	Test 35
	2	Does the leaderboard correctly show the score of the player			Test 36
	3	Do users have the option to leave the leaderboard	Allows the user to decide whether their stats are visible to other users.	When a user leaves the leaderboard their scores are no longer visible to anyone	Test 37
	4	Do users have the option to rejoin the leaderboard		When a user rejoins the leaderboard their scores are once again visible	Test 38

4. Sandbox	1	Can the user change all the parameters such as height, velocity, target, projectile mass and air resistance	The sandbox allows for users to test certain scenarios rather than looking to just solve a problem	All the parameters are able to be changed by the user	Test 39
	2	Can users set up balance problems with their required parameters			Test 40
5. User	1	Can users enter a username and password	Needed so that users can create their own account so that progress is accurate and they don't overwrite other user's data	Users can create accounts that meet the validation requirements of being a unique username, and meeting the length requirements	Test 41
	2	Are the username and password checked against stored values to allow or deny entry	Needed to ensure that the user is accessing their own account	The user can log in if they input the correct username and the related password. Error messages are displayed if the information provided is incorrect	Test 43, 44
	3	Can users enter the sandbox mode without any details	Allows the user to play for short periods without having to sign in	Feature not implemented as the user would eventually have to create an account to be able to use all the features of the solution	Feature not implemented
	4	Does the solution allow for new users to be created	Allows new users to use the program with their data being saved	The solution allows for new user to be created and that data is saved in the database for future retrieval.	Test 42
	5	If a returning user signs in, will the solution retrieve their current level and score	Allows for continuity between sessions and ensures that the solution is useful thus	The solution correctly retrieves and applies the user's	Test 46

			improving user retention	score, level and theme preferences	
	6	Do users level match their stage of learning	To ensure that users get the most out of this solution, it will have to become more difficult as the user progresses so that they can learn how to do more difficult questions and better apply their knowledge.	Stakeholders found this to be partially successful as they found that whilst the users progression was linked to the questions they answered it did not factor in their stage of learning before using the solution	Test 47
	7	Are the puzzles generated right for that level		Stakeholders found that the level of the puzzles were correct for the level that the puzzle was designed to be	Test 48
6. Tutorial	1	Does the tutorial appear when the solution is first executed	Allows the user to learn while they are becoming familiar to the program	When a new registers the sign up button will take them to the tutorial once the sign up is validated	Test 49
	2	Can the tutorial be loaded from the main menu when necessary	In case the user forgets how to use the solution the tutorial can be loaded so that they can refresh their understanding	The tutorial can be loaded from a button in the main menu labelled 'Tutorial'	Test 50
	3	Is the tutorial easy to understand	The tutorial should be easy to understand so that the user can understand how to use the program in as little time as possible	The tutorial is understandable however stakeholders suggested that it may have been clearer had the tutorial been interactive	Test 51, 78

7. Save	1	Can a user save a puzzle setup	This feature is useful as it will allow the user to be able to save setups that they wish to look over in the future to aid their revision	The user can save puzzles with unique names, this data is saved to the database	Test 52,53,120-122
	2	Can a user load a setup they previously created		The user can select the setup they wish to load from a list that is ordered newest to oldest	Test 54,55
8. UI	1	Are there multiple themes that the user can apply to the UI	Allows the program to be more visually appealing to the user	The user can switch between dark and light mode. This is saved persistently and will show on every form in the solution	Test 56, 79
	2	Is the UI easy to navigate and use	People who have never used this program should be able to use the solution without problem from the very beginning thus increasing usability	Stakeholders found the UI easy to use	Test 57
	3	Is it a drag-and-drop interface	Users should be able to learn the controls quickly and be able to use the program with limited prior knowledge	In the solution I chose to use sliders as I felt it was easier for a new user to work out what to do and easier to add validation to	Test 58 - 60
	4	Can the music be toggled	Allows users to avoid distractions	The user can toggle the music/sound effect that plays when the user gets a puzzle correct	Test 61,62

	5	Can the user reset their account	Allows the user to delete all data stored on them	The user can reset their account. This will clear all related data with their account	Test 63
	6	Can the user change their username/password	Gives the user more control over their account	Users can change their username and password provided it follows the validation rules	Test 64,65
	7	Is there a place for users to write notes	Means that the user no longer has to do all the calculations in their head or find paper to do it on	There is a place for users to write notes. However stakeholders noted that it would be better if the notes tab could be moved around the screen and if it had the capabilities for maths markup as well as automatically filling in the result to numerical equations such as 8*7 = …	Test 80

Addressing Partially and Unmet Criteria

Usability Features and Robustness

When testing the following usability features and robustness features were identified to have only partially met their tests

Usability Feature 2 – Music (Partial Success)

Whilst this feature exists and works as currently expected. It currently causes the program to hold while the sound effect is being played and then only allows the program to continue once the sound effect has finished playing. One way to fix this would be to make the sound effect method asynchronous so that it will run in parallel with the UI changes. One other issue with the music feature that was brought up by my stakeholder was that currently the only music that played was the one sound effect and it would have been better to have more variation in the music where the user could choose what sound effect they wished the program to use

Usability Feature 11 – Fonts (Partial Success)
This feature successfully changes the font of the items on the page however as I am using a font box dialog, the font and size can currently be chosen by the user. This means that the relative sizing of the text to each other is not preserved. One way this could be fixed is to only allow the user to choose the font and then keep the font size constant. If the user wishes to change the font size they will then be able to apply a scale factor to the font size so that they all scale according to their relative sizing

Usability Feature 13 – Tutorial (Partial Success)

In my original design, the tutorial was intended to be interactive, demonstrating the use of most features and providing feedback to the user when they correctly used each feature mentioned by the tutorial. However, due to time constraints, I had to make it a text-based tutorial as according to my stakeholders the program as already easy to understand how to use. In future development, this could be expanded on by allowing users to go into different sections of the program and if they are marked as a new user, it will give you a tutorial that points at different controls and tells you what it does as well as asking the user to use it.

Usability Feature 15 – Notes (Partial Success)
This feature currently works as expected as it lets the user make notes that they can hide and show as they wish however stakeholders noted that it would have been better if the notes feature could be dragged around the screen and resized so that it didn't hide parts of the program. This could be implemented in a similar way to how the target can be moved. Making the notes tab resizable.

Test 91 - Continuous use of simulation (Partial Success)

When testing, I found that it mostly worked apart from trying to set both restitution and friction to high values where the program then froze. In future development, I will take steps to ensure that the simulation remains stable even when users push values to their extremes. Although the settings are technically valid, some combinations, such as very high restitution paired with very high friction, can create behaviour that the physics solver struggles to resolve. To improve reliability, I will implement internal handling for edge-case physics scenarios, such as adding extra solver iterations when extreme values are detected or applying automatic stabilisers like sleep thresholds and collision dampening.

Success Criteria

When testing the success criteria, the following criteria were noted to have partially met / not met the success criteria. Each criterion is labelled with its numerical code (e.g. 1.1) as well as the name of the success criterion.

1.1 Does a projectile launch when a button is pressed (Partial Success)

When testing, I found that it mostly worked apart from trying to set both restitution and friction to high values, where the program then froze. In future development, I will take steps to ensure that the simulation remains stable even when users push values to their extremes. Although the settings are technically valid, some combinations, such as very high restitution paired with very high friction, can create behaviour that the physics solver struggles to resolve. To improve reliability, I will implement internal handling for edge-case physics scenarios, such as adding extra solver iterations when extreme values are detected or applying automatic stabilisers like sleep thresholds and collision dampening.

1.5 Is the projectile mass variable for the required questions (Not Implemented)

I have not implemented this as in my puzzles, the user cannot interact with drag, restitution and friction; therefore, there is no reason for the user to change the mass when solving puzzles. In future development, I could include puzzles that involve drag and friction to allow the user to be able to vary the projectile mass. This would require more complicated computation to ensure that there are puzzles that would be solvable with the integer values for target position, velocity, mass, etc... and values with a 0.1 accuracy for gravity, drag and friction. I chose not to implement this at this point as it wasn't necessary for my stakeholders who were using this solution to prepare for GCSEs and A-Levels.

1.11 Are key values on the graph labelled (Not Implemented)

Whilst the graph contains a set of axis that are labelled no other points such as stationary points are explicitly labelled. In future development, to implement this feature, I would have to find the equation of the function rather than simply plotting the points. Once the function is defined mathematically, I could differentiate it to calculate key values such as stationary points, roots, and intercepts. These values could then be automatically annotated on the graph, allowing the user to hover over them to see the coordinates of the point.

1.12 Is the motion of the projectile animated (Partial Success)

When testing, I found that it mostly worked apart from trying to set both restitution and friction to high values where the program then froze. In future development, I will take steps to ensure that the simulation remains stable even when users push values to their extremes. Although the settings are technically valid, some combinations, such as very high restitution paired with very high friction, can create behaviour that the physics solver struggles to resolve. To improve reliability, I will implement internal handling for edge-case physics scenarios, such as adding extra solver iterations when extreme values are detected or applying automatic stabilisers like sleep thresholds and collision dampening.

1.16 Can you zoom into the graph (Not Implemented)

I have not implemented zooming functionality on the graph. Currently, the graph remains fixed, so users are unable to see specific sections of the graph in greater detail. In future development, I could add interactive camera controls, such as scroll-wheel zooming and click-and-drag panning to allow users to analyse the path more precisely. Additionally, I could provide buttons for zoom presets to ensure accessibility.

1.17 Does the graph show the vector setup (Not Implemented)

Instead of allowing the user to see the vertical and horizontal components of the velocity as vectors at every point along the graph or as something that was animated across the graph, I instead chose during development to show the horizontal and vertical components as their own graphs. In future development, I could extend the system so that when a projectile is animated, vectors are drawn at key points to show the direction and magnitude of the forces affecting motion. This feature would require additional rendering logic and performance optimisation to ensure that the animation remains smooth and responsive, especially at boundary parameters.

1.19 Does the undo/redo button take you to the previous and following state respectively (Not Implemented)

In future development, I would need to correctly manage state history using two stacks to ensure that every user action is stored and can be accurately restored. This would significantly improve usability, particularly in more complex puzzles where trial and error are expected. Adding redo support would also give users more confidence when experimenting with different values and configurations, helping them learn through exploration without fear of losing progress. To do this, when the user presses undo, the current state would be

pushed into the redo stack, and the last stored state from the undo stack would be restored, and when the user presses redo, the opposite would occur, the state on the redo stack would be reapplied and copied back into the undo stack.

2.3 Is there is an option to stack masses (Partial Success)

The current system allows the user to place multiple masses, but it does not use drag-and-drop stacking as originally intended. Instead, sliders are used to place each mass along the beam, which still technically enables stacking but is less visually representative of what is happening. In future development, I could implement a proper drag-and-drop interface so users can directly move masses on screen and stack them by placing one object onto another. This approach would provide clearer visual feedback

2.4 Does the undo/redo button take you to the previous and following state respectively (Partial Success)

The undo state stack is correctly implemented. However, the solution does not currently have a redo stack. In future development, I could implement a second redo stack to support redo functionality. To implement this, when the user presses undo, the current state would be pushed into the redo stack, and the last stored state from the undo stack would be restored, and when the user presses redo, the opposite would occur the state on the redo stack would be reapplied and copied back into the undo stack.

5.3 Can the user enter the sandbox mode without any details (Not Implemented)

Currently the user must have an account to use the solution. In future development, I could allow users to enter sandbox mode as a guest without providing any details. This would make the system more accessible for first-time users, whilst still keeping the option of an account for those who want features like progress tracking, puzzles, saved simulations, or leaderboards. Implementing this would involve allowing users to bypass authentication and still retain basic functionality.

5.6 Do users' level match their stage of learning (Partial Success)

Although the user's progression does adapt based on the questions they answer correctly, the system does not consider their existing knowledge before they begin using the solution. This means some learners may start at a level that is too easy for them, resulting in a less tailored experience. In future development, I could include a skill-assessment stage before the puzzles begin. This would help the system identify the user's baseline understanding and start them at an appropriate difficulty level from the very beginning.

6.3 Is the tutorial easy to understand (Partial Success)

In my original design, the tutorial was intended to be interactive, demonstrating the use of most features and providing feedback to the user when they correctly used each feature mentioned by the tutorial. However, due to time constraints, I had to make it a text-based tutorial as according to my stakeholders the program as already easy to understand how to use. In future development, this could be expanded on by allowing users to go into different sections of the program and if they are marked as a new user, it will give you a tutorial that points at different controls and tells you what it does as well as asking the user to use it.

8.3 Is it a drag-and-drop interface (Partial Success)

In the final solution, this was only implemented for the projectile target, which can be moved using drag-and-drop as intended. However, all other interactions, such as adjusting projectile properties and placing masses, rely on sliders and buttons instead. This approach made the interface easier to validate and more beginner friendly, but it does not fully meet the original goal of providing a drag and drop system. In future development, I

could extend drag-and-drop functionality to all forces and masses, so the interface becomes more intuitive, while still keeping sliders available for precise adjustments.

8.7 Is there a place for users to write notes (Partial Success)

This feature currently works as expected as it lets the user make notes that they can hide and show as they wish however stakeholders noted that it would have been better if the notes feature could be dragged around the screen and resized so that it didn't hide parts of the program. This could be implemented in a similar way to how the target can be moved. In addition, this could also be achieved by making the notes tab resizable.

Stakeholder Feedback

The table below shows the requirements from each of the stakeholder as described in the analysis section and whether they were met/partially met or not met

Requirement	Stakeholder	Outcome	Evidence
Smooth, clear animations	Nikodem Grabowski	**Fully Met**	Test No. 1, 12, 13, 14, 15, 103, 104
Customisable UI	Nikodem Grabowski	**Fully Met**	Test No. 56, 72, 76
Personalised hints to help him finish the problem	Nikodem Grabowski	**Partially Met** The solution only shows the answer to the puzzle and not a personalised hint on how to get the right answer	Test No. 24
Engaging and Fun Puzzles	Nikodem Grabowski	**Fully Met**	Test No. 6, 7, 30, 31
Match puzzle complexity to the stage of learning	Nikodem Grabowski	**Fully Met**	Test No. 8
Option to annotate and save puzzles so that they can be revisited	Nikodem Grabowski	**Partially Met** The user has the ability to save puzzles which can then be reloaded however the user is not able to annotate the puzzles	Test No. 52-55
Scoring system with leaderboards	Nikodem Grabowski	**Fully Met**	Test No. 35-38
Use of drag and drop rather than buttons	Nikodem Grabowski	**Partially Met** The solution uses a mix of buttons, sliders and drag and drop	Test No. 5
Teacher dashboard for tracking student performance	Mr McLennen	**Not Implemented**	N/A
Graph generation of the puzzles	Mr McLennen	**Fully Met**	Test No. 9, 10, 18–21, 72–74

Ability to change animation speed	Mr McLennen	**Fully Met**	Test No. 12, 13, 103 - 106
Ability to input custom scenarios / experiment in a sandbox environment	Mr McLennen	**Fully Met**	Test No. 3,39,40
Able to run on school computers	Mr McLennen	**Fully Met**	As school computers are windows based this criteria is met
Ability to display underlying maths	Mr McLennen	**Fully Met**	Test No. 16,33
Simple and clear tutorials that explain mechanics principles	Beginner Player Persona	**Partially Met** The tutorial (which stakeholders said they found easy to understand) only shows the key controls and does not explain the mechanics principles.	Test No. 49,50, 78
Instant visual feedback for actions and results	Beginner Player Persona	**Fully Met**	Test No. 1,5,14
Non-competitive practice mode	Beginner Player Persona	**Fully Met**	Test No. 3,39,40
Undo/redo system for experimentation	Beginner Player Persona	**Partially Met** The solution has an inconsistent undo system in the projectile motion and one that works as expected in the torque system. A redo system was not implemented	Test No. 22,23,28,29
A large bank of questions so that there is always a new question to complete	Beginner Player Persona	**Fully Met**	Test No, 6,7,30,31

Maintenance, Limitations & Future Improvements

Future Maintenance

Throughout the development of my solution, I have taken steps to ensure that my program is easily maintainable in the future. These features are listed below.

Modularity

The system has been structured using a modular design in which each logical component is implemented within its own class. This separation means that changes to one area do not impact others. For example, all projectile motion calculations are within the PhysicsEngine class, allowing future modifications to the physics model without altering any user interface code. As well as that, all saving and loading puzzle database queries

are within their own class. By making the code modular, it allows for more code reusability, rather than repeating logic in multiple parts of the code

Encapsulation

Through the use of encapsulation, the methods act as a black box where the rest of the code only needs to consider the inputs and outputs. Furthermore, by using encapsulation it ensures data integrity as it restricts direct access to internal class variables such as simulation variables in the PhysicsEngine class. This prevents unintended changes from external classes and ensures that any changes occur through intended methods. Encapsulation reduces the risk of logical errors and simplifies debugging. The database interaction is similarly encapsulated. Furthermore, all SQL queries are executed through methods such as ExecuteNonQuery and ExecuteQuery and uses parameterisation, meaning the UI code does not interact with SQL directly. If the database structure changes in the future, only the database class must be updated.

Variable Names

Throughout the solution, I have used descriptive variable names, e.g. *initialVelocity* to improve readability and reduce ambiguity. In addition, when developing my solution I ensured that all text labels had names with *lbl*. Furthermore, to ensure that the variable names are easy to understand. The variables names all have constant naming structure to make it easy for future maintenance without having to do extensive cross reference

```
projectileDrag = vardragCoefficient.Value / 10.0;
varlbl_drag.Text = projectileDrag.ToString();
```

However, some variables and method names use differing case styles. Adopting a unified convention (e.g., camelCase for variables and PascalCase for methods and classes) would improve consistency throughout the solution.

Comments

Comments have been used throughout the solution especially where key physics algorithms or other complex algorithms are occurring to make it clear to understand the intent behind the implementation making future maintenance easier as other developers will be able to clearly see what the purpose of the algorithms is and what each condition is checking

```
public List<(int position, int mass)> SolveOnePivot()
{
    int pivot = Convert.ToInt32(pivots[0]);
    double torque = CalculateTorque(pivot, 0);

    if (Math.Abs(torque) < 1e-6) return new List<(int, int)>();

    List<int> candidatePositions = new List<int>();
    if (torque > 0) // clockwise torque
    {
        for (int pos = pivot - 2; pos >= 0; pos -= 2)
            if (!weights.Any(w => Math.Abs(w.position - pos) < 1e-6))
                candidatePositions.Add(pos);
    }
    else // anticlockwise torque
    {
        for (int pos = pivot + 2; pos <= 20; pos += 2)
            if (!weights.Any(w => Math.Abs(w.position - pos) < 1e-6))
                candidatePositions.Add(pos);
    }

    // flip distances to counteracts torque
    List<int> distances = candidatePositions.Select(p => -(p - pivot)).ToList();
    var solution = TrySolve(candidatePositions, distances, torque);

    return solution ?? new List<(int, int)>();
}
```

Whitespace

I have used whitespace and indentation consistently to improve readability and maintain a clean visual structure. This includes spacing between logical code blocks and aligning parameters across multiple lines. For example:

```
return (
    Convert.ToInt32(row["Mass"]),
    Convert.ToInt32(row["StartHeight"]),
    Convert.ToInt32(row["Angle"]),
    Convert.ToInt32(row["InitVel"]),
    Convert.ToDouble(row["Gravity"]),
    Convert.ToDouble(row["Friction"]),
    Convert.ToDouble(row["Restitution"]),
    Convert.ToDouble(row["Drag"]),
    Convert.ToInt32(row["Target"])
);
```

By using whitespace it makes the program easier to read and understand especially when stepping through complex functions

Database

By using a SQL database, related data is stored in separate tables (Users, Scores, ProjectileSave, TorqueSave), reducing redundancy and simplifying queries In addition all database interactions are abstracted into dedicated methods with parameterisation included to prevent against SQL injection. By using this approach, it means that the underlying logic can be changed without altering UI logic.

Documentation

This report is a detailed explanation of the purpose of each section of the solution. Furthermore, the testing already shown in this report can be used by future developers to inform future development

Limitations

Below is a list of the known limitations in the current version of the solution. Additionally, the limitations stated in the analysis section will also be stated here. Based on the limitations listed, the following are ways these could be addressed with further development time. The limitations are ordered in order of priority. i.e. the features at the top of the list should be implemented first

1. Boundary settings can cause the simulation to freeze

See Addressing Partially and Unmet Criteria - Success Criteria 1.1

2. Music system bugged

See Addressing Partially and Unmet Criteria - Usability Feature 2

3. Fonts currently don't display dynamically

See Addressing Partially and Unmet Criteria - Usability Feature 15

4. No full undo/redo system

See Addressing Partially and Unmet Criteria – Success Criteria 1.19, 2.4

5. Difficulty level does not consider prior knowledge

See Addressing Partially and Unmet Criteria – Success Criteria 5.6

6. No personalised hints to guide users toward the correct solution

The current system only gives the user the correct answer to the puzzle when they request it and not how to get there. At a basic level, the puzzle could tell the user what they need to change to fix their answer to the puzzle. For example, a torque puzzle could tell the user that currently their answer will cause a moment in a certain direction and to counter it more/less mass needs to be placed on one side of the pivot. Additionally, for projectile motion puzzles the solution can tell the user whether they need to increase/decrease the variable they are changing. This could be further developed by analysing the user's parameter choices and the resulting outcome. By comparing them with the target conditions, the solution would be able to identify the likely mathematical misconception (incorrect trigonometric component, direction of torque error, wrong pivot distance, etc.), and then provide personalised feedback to improve learning.

7. Key graph points are not labelled

See Addressing Partially and Unmet Criteria – Success Criteria 1.11

8. The graph cannot be zoomed

See Addressing Partially and Unmet Criteria – Success Criteria 1.16

9. Vector components are not visualised on the graph

See Addressing Partially and Unmet Criteria – Success Criteria 1.17

10. Simplified Tutorial, which is text-based rather than interactive, that explains the mechanics principles

See Addressing Partially and Unmet Criteria – Success Criteria 6.3

11. No Annotations

In future development, an annotation system could be added to support visual working and help users better understand mechanics concepts. This would allow users to draw force arrows, mark angles, and highlight distances on top of the simulation using a custom drawing layer panel that captures mouse input (click, drag) to draw custom shapes and uses Graphics objects to render annotations in real time. A simple toolbar could be

implemented to let users switch between annotation modes such as arrows, lines and text labels, with Undo and Hide options to keep the interface clear. These annotations could be saved alongside puzzle setups, meaning students and teachers could return to their work later. To make the feature more useful, the drawing tools could be linked to the simulation so that arrows snap to pivots or launch points, and angle markers align with the projectile direction. Over time, this could be extended further by detecting incorrect annotations, such as wrong angle direction or the force applied on the wrong side of a pivot, enabling the solution to provide customised feedback. This would significantly improve the educational value of the simulation and help address common misconceptions by allowing students to see and correct their own thinking visually.

12. Notes panel cannot be moved or resized

See Addressing Partially and Unmet Criteria – Success Criteria 8.7

13. No Online features

In future, online functionality could be introduced to enhance collaboration and accessibility. For example, storing user profiles and puzzle progress in the cloud would allow learners to continue seamlessly across different devices at school and at home. A secure online leaderboard could also make the competitive elements more meaningful by connecting users beyond a single computer. This could be implemented by integrating a lightweight cloud database service. While this would require additional development time to learn networking protocols and ensure data security, it would significantly expand the flexibility of the system and improve its usefulness in a real classroom environment.

14. No teacher dashboard for monitoring student progress

A teacher dashboard could be added in the future to allow teachers to track individual student progress and better support learning. This dashboard could display key information such as the number of puzzles completed, success rates, the levels achieved in both projectile motion and torque, and any areas where the student is repeatedly struggling. It could also highlight common misconceptions identified by the system, enabling teachers to plan more effective lessons. This would be implemented by storing user performance data in a shared database and giving authenticated teacher accounts access to some user data. Additional features, such as homework assignments, would make the solution more valuable in a classroom environment. Although this was not implemented due to time constraints and additional networking requirements, adding it in the future would significantly improve the educational usefulness and stakeholder satisfaction of the final product.

15. No guest mode

See Addressing Partially and Unmet Criteria – Success Criteria 5.3

16. Masses cannot be stacked by drag-and-drop

See Addressing Partially and Unmet Criteria – Success Criteria 2.3

17. Drag and drop only implemented for the target object

See Addressing Partially and Unmet Criteria – Success Criteria 8.3

18. Windows Forms limits animation quality and modern UI features

Windows Forms places restrictions on the animation quality and the interface design because it is not hardware-accelerated and lacks built-in support for game-style rendering. In the future, the solution could be moved to a more suitable framework, such as Unity or .NET MAUI. This would allow the physics and animations to run using a dedicated render loop with GPU acceleration, resulting in smoother motion and more engaging simulations. Alternatively, .NET MAUI or WPF would provide a more modern UI with cleaner layouts, scalable styling, better font rendering, and improved accessibility features. Moving to one of these frameworks would therefore allow the interface to feel more contemporary and make the educational experience better

19. Windows Only

Currently, the solution was implemented in Windows Forms so the solution is only accessible to Windows users, and students using Linux and macOS will have to use a virtual machine to access the program. In future development, the solution could be moved to .NET MAUI, allowing the same codebase to run across Windows, macOS and Linux. This would remove the dependency on Windows Forms and make the system accessible to students regardless of device.

20. Projectile Mass cannot be changed in puzzles

See Addressing Partially and Unmet Criteria – Success Criteria 1.5

Future Development

Several areas for future improvement were identified throughout development that would increase both the educational value and the technical robustness of the system:

1. **Additional Mechanics Topics**
   Future versions could extend the system beyond projectile motion and moments by introducing topics such as simple harmonic motion, energy and power problems and circular motion to broaden the program's usefulness.
2. **Improved Error Handling**
   Stronger validation and error handling could be added to handle invalid inputs more gracefully
3. **More Visualisation Tools**
   At the moment, the user can only see the physics through a simulation and a graph for projectile motion. This can be improved by implementing vector diagrams showing resolved components that is overlayed over the simulation as well as real-time display of forces and accelerations
4. **Online Features**
   An extended version of this solution could include the ability to synchronise scores via a web server. This will allow for class wide leaderboard, and to be able to implement a teacher analytics dashboard which was requested by Mr McLennen. Furthermore, online features can be used for teachers to set puzzles for users to solve.
5. **Improved UI**
   The UI is currently not accommodative of different resolutions and does not allow the user to resize. By incorporating these features, it will allow for a more accessible program
6. **Expanding Feedback System and Tutorial**
   To make the solution more educationally valuable, in future development, to support weaker learners a step-by-step solution to the puzzles could be implemented. Furthermore, if a user gets a puzzle wrong the solution personalised feedback could be implemented.

Conclusion

In conclusion, the solution successfully achieves its core objective of providing an educational kinematics puzzle system that enhances users' understanding of projectile motion and moments. The program meets the majority of the success criteria and offers a mostly robust, engaging experience that meets the needs of all identified stakeholders. For students such as Nikodem, the system provides an environment where concepts can be explored through simulations and progressively challenging puzzles. This supports trial-and-error learning and strengthens understanding. For teachers such as Mr McLennan, the solution offers a tool for classroom demonstrations as well as graph generation. The beginner student persona also benefits from a simplified interface and the option to practise without competitive pressure.

From a technical perspective, my solution is usable. The physics engine produces consistent results across a wide range of scenarios and the puzzle generation adapts to the user's performance. Furthermore, the integration of the SQLite database enables reliable saving, loading, and long-term tracking of user progress. Some success criteria were only partially met. Certain robustness tests revealed edge cases (e.g., extreme parameter combinations or unexpected user behaviour) where additional error handling would be beneficial. The time constraints of the NEA meant that more advanced features, such as additional puzzle types or extended visualisation tools, could not be fully implemented.

Overall, while there are areas that could be expanded or refined, the program provides a strong foundation for future versions and demonstrates the value of computational methods in modelling and teaching kinematics.

Bibliography

Exchange, S., 2020. *Writing a Simple SHA256 Salted Hash Generator.* [Online]
Available at: https://security.stackexchange.com/questions/228944/writing-a-simple-sha256-salted-hash-generator
[Accessed 26 August 2025].

GeoGebra, n.d. *Projectile Motion Simulation.* [Online]
Available at: https://www.geogebra.org/m/JDxA5kPR
[Accessed 12 June 2025].

Hamxa, M., 2023. *Projectile Motion (With Air Resistance) In Python - Medium.* [Online]
Available at: https://medium.com/%40hamxa26/projectile-motion-with-air-resistance-in-python-fb43737226b7
[Accessed 21 August 2025].

Kalinda, C., 2025. *Sqlite C# .NET (How It Works For Developers).* [Online]
Available at: https://ironpdf.com/blog/net-help/sqlite-csharp-net/#:~:text=To%20create%20a%20table%20in,it%20using%20the%20ExecuteNonQuery%20method.
[Accessed 25 August 2025].

Martin, J., 2018. *InfoQ - Microsoft Open Sources WPF, WinForms, and WinUI.* [Online]
Available at: https://www.infoq.com/news/2018/12/msft-open-source-wpf-winforms/
[Accessed 25 June 2025].

Microsoft Learn, n.d. *CancellationTokenSource Class.* [Online]
Available at: https://learn.microsoft.com/en-us/dotnet/api/system.threading.cancellationtokensource?view=net-9.0
[Accessed 25 October 2025].

Microsoft, 2025. *Visual Studio 2022 Product Family System Requirements.* [Online]
Available at: https://learn.microsoft.com/en-us/visualstudio/releases/2022/system-requirements
[Accessed 25 June 2025].

Njoku, J., 2024. [Online]
Available at: https://code-maze.com/csharp-read-and-process-json-file/
[Accessed 5 September 2025].

PhET Interactive Simulations, 2025. *Projectile Motion.* [Online]
Available at: https://phet.colorado.edu/sims/html/projectile-motion/latest/projectile-motion_all.html
[Accessed 13 June 2025].

PhET Interactive Simulations, n.d. *Balancing Act.* [Online]
Available at: https://phet.colorado.edu/sims/html/balancing-act/latest/balancing-act_all.html
[Accessed 13 June 2025].

PhET Interactive Simulations, n.d. *Projectile Data Lab.* [Online]
Available at: https://phet.colorado.edu/sims/html/projectile-data-lab/latest/projectile-data-lab_all.html
[Accessed 13 June 2025].

Radečlč, D., 2021. *Medium - CSV Files for Storage? No Thanks. There's a Better Option.* [Online]
Available at: https://medium.com/data-science/csv-files-for-storage-no-thanks-theres-a-better-option-72c78a414d1d
[Accessed 25 June 2025].

Rawshdeh, S. N., 2007. *Gravity and Collision Simulation in C# - Code Project.* [Online]
Available at: https://www.codeproject.com/Articles/22438/Gravity-and-Collision-Simulation-in-C-?
[Accessed 25 August 2025].

SimPop, 2018. *Balance and Torque.* [Online]
Available at: https://simpop.org/balance/balance.htm
[Accessed 12 June 2025].

Sirols, S., 2024. *HP - Understanding CPU Speed: What Makes a Good Processor for Your Computer.* [Online]
Available at: https://www.hp.com/us-en/shop/tech-takes/what-is-processor-speed
[Accessed 25 June 2025].

W3Schools, n.d. *SQL Injection.* [Online]
Available at: https://www.w3schools.com/sql/sql_injection.asp
[Accessed 25 November 2025].

Whaley, S., 2024. *HP - How Much RAM Do I Need for My Laptop? A Complete Guide.* [Online]
Available at: https://www.hp.com/us-en/shop/tech-takes/how-much-ram-do-i-need-in-laptop
[Accessed 25 June 2025].